

[laysim-GR740-001]

laysim-GR740 Quad Core Processor Emulator User's Manual

Prepared by : Jong-Wook Choi (jwchoi@kari.re.kr)

Data : 2025-08-27

- Table of Contents -

1. Introduction	1
1.1 General	1
1.2 l1aysim-GR740 Overall Architecture	1
1.3 Architecture of l1aysim-gr740-dbt	2
1.3.1 Instruction Cycles	3
1.3.2 GPTIMER Emulation for Cycles	4
1.3.3 Time Event Operation	5
1.3.4 Translation Block Chaining	6
1.3.5 Self-Modifying Code	9
1.4 Architecture of l1aysim-gr740-mdbt	10
1.5 Multicore Scheduling in l1aysim-GR740	11
1.6 Other Features in l1aysim-GR740	13
1.6.1 Emulation of AMBA Plug & Play Information	13
1.6.2 Memory Map of l1aysim-GR740	14
1.6.3 Interrupts of l1aysim-GR740	16
1.7 Limitation of l1aysim-GR740	17
1.8 Applicable Documents	17
2. Installation	18
2.1 Installation for Linux x64	18
2.1.1 Preparation	18
2.1.2 Installation of l1aysim-GR740	18
2.1.2.1 Installation of additional packages for CentOS 7	18
2.1.2.2 Installation of additional packages for CentOS 8 and Ubuntu 18.04	18
2.1.2.3 Installation of additional packages for Debian 10.3	19
2.1.2.4 Installation of l1aysim-GR740	20
2.2 Installation for Windows	22
2.2.1 Preparation	22
2.2.2 Installation of l1aysim-GR740	22
2.3 Compilation of Examples	23
3. l1aysim-GR740 Operation	25
3.1 Emulators of l1aysim-GR740	25
3.1.1 l1aysim-gr740	25
3.1.2 l1aysim-gr740-cli	25
3.1.3 l1aysim-gr740-dbt	26
3.1.4 l1aysim-gr740-dbt-cli	26
3.1.5 l1aysim-gr740-mdbt	28
3.1.6 l1aysim-gr740-mdbt-cli	28
3.2 Starting l1aysim-GR740	29
3.3 Control Console of l1aysim-GR740	44
3.4 Directory Redirect for Debugging	61

4. Loadable Modules	63
4.1 Loadable PCI Module	63
4.1.1 RTEMS Shell for PCI Extension	64
4.1.2 RTEMS Actel 1553BRM for PCI Extension	65
4.1.3 RTEMS RMAP for PCI Extension	66
4.1.4 RTEMS TLIB for PCI Extension	67
4.1.5 RTEMS PCIDMA for PCI Extension	69
4.2 Loadable I/O Module	69
4.2.1 Actel Core 1553B on I/O Area	70
4.2.2 DMA Operation on I/O Area	72
4.2.3 Board Reset on I/O Area	72
4.3 Loadable SpW Module	74
4.4 Loadable 1553B RT Module	75
4.5 Loadable CAN Module	81
4.6 Loadable SPI Module	83
4.7 Loadable GPIO Module	83
4.8 Loadable UART Module	84
5. Shared Library	86
5.1 Shared Library Example	86
6. GDB Operation	90
6.1 Using Extended-Remote Protocol	90
6.2 Using Remote Protocol	95
6.3 Using Eclipse	96
7. Code Coverage	98
8. Emulation Characteristics of laysim-GR740	100
8.1 LEON4 Support	100
8.2 L2 Cache Support	101
8.3 MMCTRL Support	101
8.4 MEMSCRUB Support	101
8.5 IOMMU Support	101
8.6 SpaceWire Router Support	101
8.7 GRETH_GBIT Support	102
8.8 GRPCI2 Support	102
8.9 GR1553B Support	102
8.10 GRCAN Support	103
8.11 FTMCTRL Support	103
8.12 GPTIMER Support	103
8.13 IRQ(A)MP Support	103
8.14 GRGPIO Support	103
8.15 APBUART Support	103
8.16 SPICTRL Support	104
8.17 GRCLKGATE Support	104
8.18 L4STAT Support	104

8.19 AHBSTAT Support	104
8.20 BOOTSTRAP Support	104
8.21 GR740THSENS Support	104
8.22 GRGPRBANK Support	104
8.23 GRSPWTDTP Support	104

- Table of Figures -

Figure 1-1. Quick Comparison laysim-GR740	1
Figure 1-2. Architecture of laysim-GR740	2
Figure 1-3. Architecture of laysim-gr740-dbt	3
Figure 1-4. Instruction Cycle Counting in laysim-gr740-dbt	4
Figure 1-5. GPTIMER Emulation in laysim-gr740-dbt	4
Figure 1-6. Timed Operation for GR1553B Emulation in laysim-gr740-dbt	6
Figure 1-7. Translation Block Chaining	6
Figure 1-8. TB Chaining on Missing Successor for Unconditional Branch	7
Figure 1-9. TB Chaining on Finding Successor for Unconditional Branch	8
Figure 1-10. TB Chaining for Conditional Branch	8
Figure 1-11. General Handling Mechanism for Self-Modifying Code	9
Figure 1-12. Handling Mechanism for Self-Modifying Code in laysim-gr740-dbt	10
Figure 1-13. Architecture of laysim-gr740-mdbt	11
Figure 1-14. Time Quantum Scheduling of laysim-gr740-dbt	12
Figure 1-15. Basic Block Scheduling of laysim-gr740-dbt	12
Figure 1-16. Parallel Time Quantum Scheduling of laysim-gr740-mdbt	13
Figure 1-17. AMBA P&P Information of laysim-GR740	13
Figure 1-18. Memory Map of laysim-GR740	14
Figure 1-19. Interrupts of laysim-GR740	16
Figure 2-1. Install Cross Tool Chains for GR740 in Linux	18
Figure 2-2. Install libcanberra-gtk-module on Ubuntu 18.04	18
Figure 2-3. Install libgtk2.0 and libcanberra-gtk-module on Debian 10.3	19
Figure 2-4. Install laysim-GR740	20
Figure 2-5. Contents of laysim-GR740	21
Figure 2-6. Need License File	21
Figure 2-7. Install Cross Tool Chains for GR740 in Windows	22
Figure 2-8. Install laysim-GR740 in Windows	22
Figure 2-9. Contents of laysim-GR740	23
Figure 2-10. Need License File	23
Figure 2-11. Build BCC-2.2.2 Examples	24
Figure 2-12. Build RCC-1.3.1 Samples	24
Figure 3-1. laysim-gr740 Interfaces	25
Figure 3-2. laysim-gr740-cli Interfaces	25
Figure 3-3. laysim-gr740-dbt Interfaces	27
Figure 3-4. laysim-gr740-dbt-cli Interfaces	27
Figure 3-5. laysim-gr740-mdbt Interfaces	28
Figure 3-6. laysim-gr740-mdbt-cli Interfaces	28
Figure 3-7. Using '-fast_uart' start-up option	29
Figure 3-8. Run AMP example using '-core' start-up option	30
Figure 3-9. Run ROM example using '-rom' start-up option	31
Figure 3-10. Using '-r' start-up option	32
Figure 3-11. Using '-traplog' start-up option	32

Figure 3-12. Using '-batch' start-up option	33
Figure 3-13. Using '-cov' start-up option	34
Figure 3-14. Using '-nb' start-up option	35
Figure 3-15. Using '-ramsz' start-up option	36
Figure 3-16. Using '-freq' start-up option	37
Figure 3-17. Using '-baud' start-up option	38
Figure 3-18. Using '-quantum' start-up option	38
Figure 3-19. Using '-mod' start-up option	42
Figure 3-20. help command	44
Figure 3-21. display command	46
Figure 3-22. memory command	47
Figure 3-23. register command	47
Figure 3-24. wmem command	47
Figure 3-25. float command	48
Figure 3-26. run/stop command	49
Figure 3-27. step & next command	49
Figure 3-28. trace command	50
Figure 3-29. break command	50
Figure 3-30. break command	51
Figure 3-31. icache command	51
Figure 3-32. dcache command	52
Figure 3-33. l2cache command	52
Figure 3-34. core command	52
Figure 3-35. history command	53
Figure 3-36. perf command	53
Figure 3-37. mmu command	54
Figure 3-38. walk command	54
Figure 3-39. vdis command	55
Figure 3-40. rtems command	55
Figure 3-41. info sys & reg command	56
Figure 3-42. iommu lookup command	57
Figure 3-43. memdump command	58
Figure 3-44. rundelta command	58
Figure 3-45. load command	58
Figure 3-46. batch command	59
Figure 3-47. save & restore command	60
Figure 3-48. Set Directory Redirection	62
Figure 4-1. Overall Architecture of Loadable PCI Module	63
Figure 4-2. PCI Address Space for User PCI Module	63
Figure 4-3. rtems-pci-shell with loadable PCI module	64
Figure 4-4. rtems-pci-brm with loadable PCI module	65
Figure 4-5. Operation of GRSpW2 in user PCI	66
Figure 4-6. rtems-pci-rmap with loadable PCI module	66
Figure 4-7. rtems-pci-tlib with loadable PCI module	67
Figure 4-8. rtems-pci-pcidma with loadable PCI module	69
Figure 4-9. user-io-cl553brm with loadable I/O module	70

Figure 4-10. user-io-dma with loadable I/O module	72
Figure 4-11. smp08-reset with loadable I/O module	72
Figure 4-12. SpW Node Simulation using loadable SpW module	74
Figure 4-13. 1553B RT Simulation using loadable 1553B RT module	75
Figure 4-14. CAN Node Simulation using loadable CAN module	81
Figure 4-15. SPI Simulation using loadable SPI module	83
Figure 4-16. GPIO Simulation using loadable GPIO module	83
Figure 4-17. UART Simulation using loadable UART module	85
Figure 5-1. Source Code - app12-1.c & app12-2.c	86
Figure 5-2. app12-1 and app12-2 Result	87
Figure 6-1. Debugging with GDB #1 on laysim-GR740	90
Figure 6-2. Debugging with GDB #2 on laysim-GR740	95
Figure 6-3. Debug Configurations in Eclipse	97
Figure 6-4. Debugging with Eclipse on laysim-GR740	97
Figure 7-1. Code Coverage Test	98

- Revision History -

Version	Date	Description	Name
Rev #00	2020/04/22	Initial Version with laysim-GR740 v0.2	Jong-Wook Choi
Rev #01	2020/06/18	Updated according to laysim-GR740 v0.5	Jong-Wook Choi
Rev #02	2020/07/23	Updated according to laysim-GR740 v0.10	Jong-Wook Choi
Rev #03	2020/09/03	Updated according to laysim-GR740 v0.11	Jong-Wook Choi
Rev #04	2021/01/19	Updated according to laysim-GR740 v1.0	Jong-Wook Choi
Rev #05	2021/05/04	Updated according to laysim-GR740 v1.2 - Support openSUSE v12/15 - Support GR740 silicon revision 1 - Support loadable PCI and IO modules - Provide laysim-gr740-mdbt	Jong-Wook Choi
Rev #06	2021/06/30	Updated according to laysim-GR740 v1.3 - Support external SpW, 1553B and CAN node simulation - Support trap logging - Support running another cores on error mode - Support 512MB SDRAM for laysim-gr740-mdbt	Jong-Wook Choi
Rev #07	2021/08/30	Updated according to laysim-GR740 v1.4 - Support shared library in CLI version - Support save/restore in CLI version	Jong-Wook Choi
Rev #08	2021/10/18	Updated according to laysim-GR740 v1.5 - Support MMU TLB - Support GR740 reset API in loadable module	Jong-Wook Choi
Rev #09	2022/02/07	Updated according to laysim-GR740 v1.6 - Support Idle Loop detection - Support laysim log and UART redirection - Fix CASA instruction - Support GDB and code coverage	Jong-Wook Choi
Rev #10	2022/03/04	Updated according to laysim-GR740 v1.6.2 - Bug fix in GDB - Update GRETH/GRCAN model - Update support of loadable module	Jong-Wook Choi
Rev #11	2022/10/20	Updated according to laysim-GR740 v2.0 - Support Docker with cFS - Support IOMMU and GRLIB register information	Jong-Wook Choi
Rev #12	2023/05/30	Updated according to laysim-GR740 v2.2 - Improve Performance of FPU operation - Migration from MSYS/MinGW(32bit) to MSYS2/MinGW64(64bit) on Window Version - Support XNG setup - Support DWARF5 for debugging	Jong-Wook Choi
Rev #13	2025/08/27	Updated according to laysim-GR740 v3.0 - Merge MMU version - Support Linux guest again in DBT/MDBT - Remove internal 1553B/CAN/SpW simulation modules - Support laysim-gr740-mdbt GUI version - Support only GR740 silicon revision 1 (Drop rev 0) - Support DWARF4 for debugging - Support RAM/ROM/Freq./Baudrate change	Jong-Wook Choi

1. Introduction

1.1 General

This document is the user's manual for laysim-GR740 emulator. It describes the architecture and general usage of laysim-GR740. laysim-GR740 consists of three types of emulators: laysim-gr740, laysim-gr740-dbt and laysim-gr740-mdbt. These are provided both as stand-alone applications and as a set of libraries that can be integrated into an existing simulator.

laysim-gr740 is a GUI-based, cycle-accurate emulator for the GR740 quad core processor using an interpretation method. It includes an embedded source-level debugger and supports the MMU for XtratuM-SMP, AIR and Linux SMP. laysim-gr740-dbt is a GUI-based high-performance, cycle-approximate processor emulator that uses dynamic binary translation (DBT). laysim-gr740-mdbt (Multi-Thread DBT) is a parallel implementation of laysim-gr740-dbt. It adopts a multi-threaded DBT framework, enabling it to meet the real-time performance requirements of the GR740 on high-end PCs such as the Intel i9-12900K. laysim-GR740 supports both Windows and Linux x64 platforms, including CentOS 7/8, Ubuntu 18.04/22.04, openSUSE 12/15 and Debian 10.3.

Also the CLI (Command Line Interface) versions are provided separately. The CLI versions have command line interfaces without a GUI, so the performance of CLI version shows higher than the GUI version of laysim-GR740.

Figure 1-1 shows a brief comparison of the three versions of laysim-GR740.

Figure 1–1. Quick Comparison laysim-GR740

Core	laysim-gr740	laysim-gr740-dbt	laysim-gr740-mdbt
Emulator Type	Interpreter - For S/W development and debugging	DBT (Dynamic Binary Translation) - Aim for fast execution and simulation	Multi-Thread DBT (Parallel) - To meet the real-time performance of GR740 under the SMP environment - Require at least 5 x64 cores
Cycle Accuracy	Cycle accurate Max ±5% error rate	Cycle approximate Max ±10% error rate (But the error rate increases on FPU emulation)	Cycle approximate
L1/L2 Cache	Fully implemented	No L1/L2 cache for speed-up	
Multicore Emulation	Instruction & Cycle level - Execute a instruction from the lowest local cycles of core every time	Round-robin fashion on time quantum (Default time quantum is 500) - 1 means 4nsec on 250MHz, so 500 is 2usec The time quantum can be changed using '-quantum' start-up option	Parallel execution based on time quantum (Default is 500) - 1 means 4nsec on 250MHz, so 500 is 2usec
Control Console	Support - Can control emulation and check status of cores and memory - Can check the performance and information	Support - Can control emulation and check status of cores and memory - Can check the performance and information	Support - Can control emulation and check status of cores and memory - Can check the performance and information
GDB	Fully Supported - laysim-gr740 includes the embedded source-level debugger, so it is possible to debug GR740 application without GDB.	Fully Supported	Fully Supported
Real-Time Performance (RTP)	1) Single Core - 24.61% RTP on BCC2 dhrystone - 31.61% RTP on BCC2 whetstone SP 2) Multi Core - 4.33% RTP on smpmigration02 - 5.40% RTP on smpmrsp01	1) Single Core - 384.64% RTP on BCC2 dhrystone - 130.42% RTP on BCC2 whetstone SP 2) Multi Core - 74.09% RTP on smpmigration02 - 56.97% RTP on smpmrsp01	1) Single Core - 391.90% RTP on BCC2 dhrystone - 127.03% RTP on BCC2 whetstone SP 2) Multi Core - 154.24% RTP on smpmigration02 - 169.54% RTP on smpmrsp01
i9-12900K @5Gz			

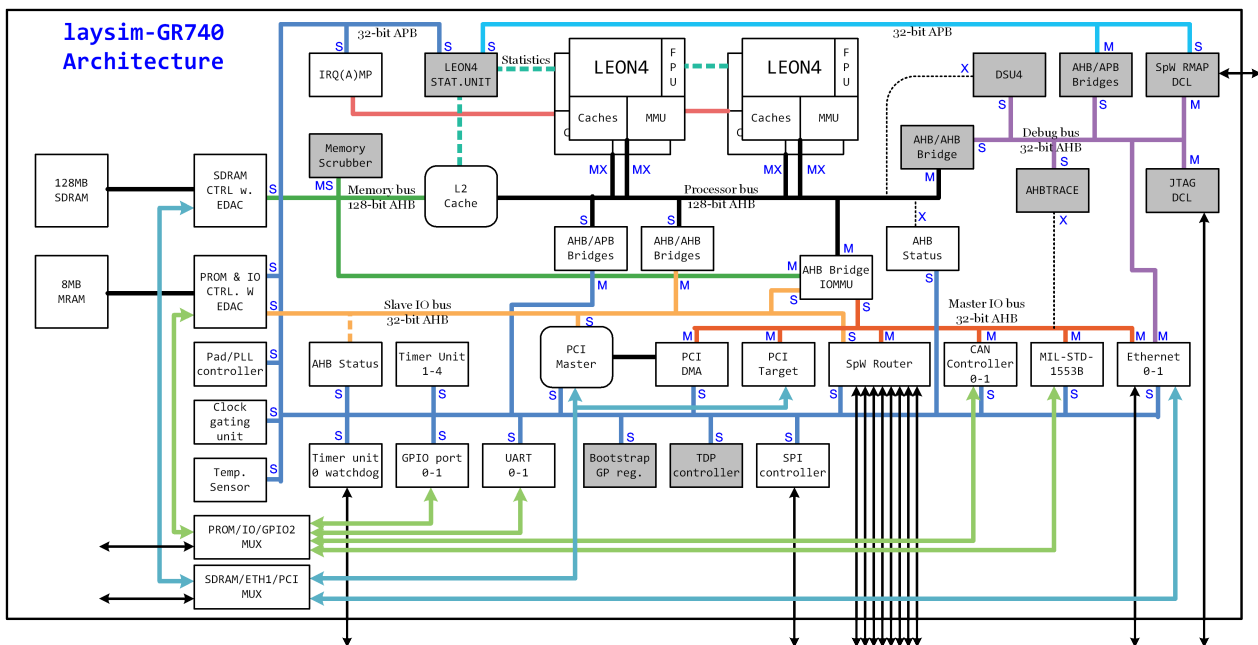
1.2 laysim-GR740 Overall Architecture

The architecture of laysim-GR740 is shown in Figure 1-2 and it emulates the GR740 quad core processor (silicon revision 1 only) with

GRLIB devices of GR740.

- LEON4 quad core processor (0x01:0x048) with 250MHz clock
 - Quad LEON4 with MMU and GRFPU
 - 16KB 4-way instruction cache and 16KB 4-way data cache and 2MB Level-2 cache only for **laysim-gr740**
 - Supported GRLIB devices of GR740
 - IRQAMP (0x01:0x00D), APBUART (0x01:0x00C), MMCTRL (0x01:0x05D), GPTIMER (0x01:0x011)
 - GR1553B (0x01:0x04D, BC-only), GRCLKGATE (0x01:0x02C), GRGPIO (0x01:0x01A), GRSPW2 (0x01:0x029)
 - GRSPWROUTER (0x01:0x08B without supporting time-code, port timer, distributed interrupt), FTMCTRL (0x01:0x054),
 - L2CACHE (0x01:0x4B, **laysim-gr740** only), GRIOMMU (0x01:0x04F), SPICTRL (0x01:0x02D)
 - GRCAN (0x01:0x03D), GRETH_GBIF (0x01:0x01D), AHBSTAT (0x01:0x052), GRPCI2 (0x01:0x07C)
 - Unsupported GRLIB devices of GR740
 - MEMSCRUB (0x01:0x057), GRSPWTDP (0x01:0x097), L4STAT (0x01:0x047),
 - Memory : 8MB MRAM and 128MB SDRAM as default
 - CPU Frequency, ROM size, RAM size, and APBUART#0/#1 baudrate can be changed using start-up option.

Figure 1-2. Architecture of laysim-GR740



1.3 Architecture of laysim-gr740-dbt

laysim-gr740-dbt uses only the dynamic binary translator without an interpreter, and it incorporates several performance enhancement techniques, including the reduction of generated machine codes, minimization of helper functions, and translation block chaining. It also adopts the shadow memory to handle the self-modifying code. The architecture of **laysim-gr740-dbt** is illustrated in Figure 1-3.

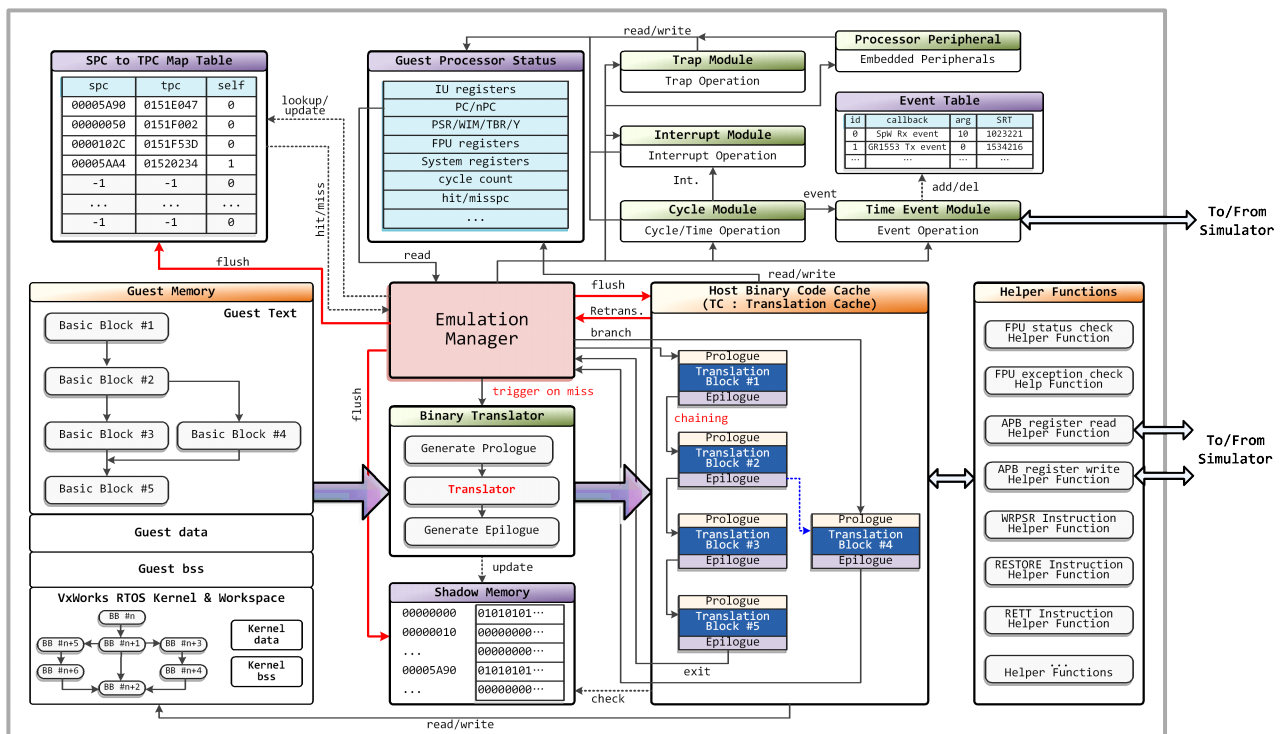
The emulation manager (EM) controls all operations of the **laysim-gr740-dbt**. It activates the DBT engine, which is responsible for generating native codes (x64) from guest codes (SPARC) when the TPC (target program counter) is not found in the SCP-to-TPC (source program counter to target program counter) map table. The SPC-to-TPC map table provides an index into the translation cache, which is

implemented as a direct mapping table, and the TC (translation cache) holds the translation blocks.

The shadow memory maintains the translation status of guest memory to check for self-modifying conditions. Helper functions, which are called from TB (translation block), support complex guest instructions and enable communication with the the simulator; these functions are minimized to improve performance.

The cycle module maintains the simulation time and accounts for instruction execution time. It also manages the processor's timer devices and invokes timer interrupts on expiration. The time event module services scheduled events in the event table. The interrupt module checks interrupt conditions and manipulates the interrupt controller of the processor. All synchronous traps and interrupts are handled by the trap module.

Figure 1–3. Architecture of laysim-gr740-dbt



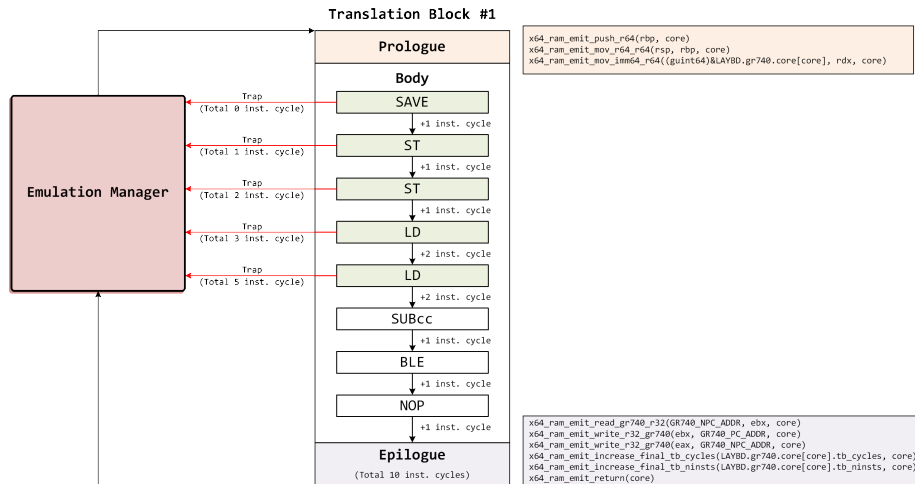
1.3.1 Instruction Cycles

Cycle accurate emulation is an important requirements for space processors in satellite simulator. The interpreter-based emulator, laysim-gr740, can achieve cycle accuracy by modeling annul operations, register dependency, L1/L2 cache hit/miss cycles, write buffer cycles, branch hit/miss cycles, and inter-processor effects. However, this approach approach results in significant performance degradation, and real-time performance cannot be guaranteed when the emulator is integrated into a satellite simulator.

Most dynamic binary translators (DBTs) do not provide instruction cycle counting mechanism due to the focus on achieving high performance. In a DBT system, the basic concept of instruction cycle counting is to accumulate instruction cycle counts as the basic block level, considering traps and branch conditions, whereas an interpreter updates instruction cycles for each instruction. A static cycle-approximate model can be applied in DBT systems to minimize performance penalties while maintaining acceptable cycle accuracy.

The instruction cycle counting method of `laysim-gr740-dbt` is illustrated in Figure 1-4. In this figure, five instructions are identified as trappable during the translation of a basic block. If a trap occurs during the execution of the final LD instruction, the emulator updates a total of five instruction cycles instead of ten, as shown in this figure. This mechanism improves the cycle accuracy of `laysim-gr740-dbt` and constitutes a key feature of the emulator.

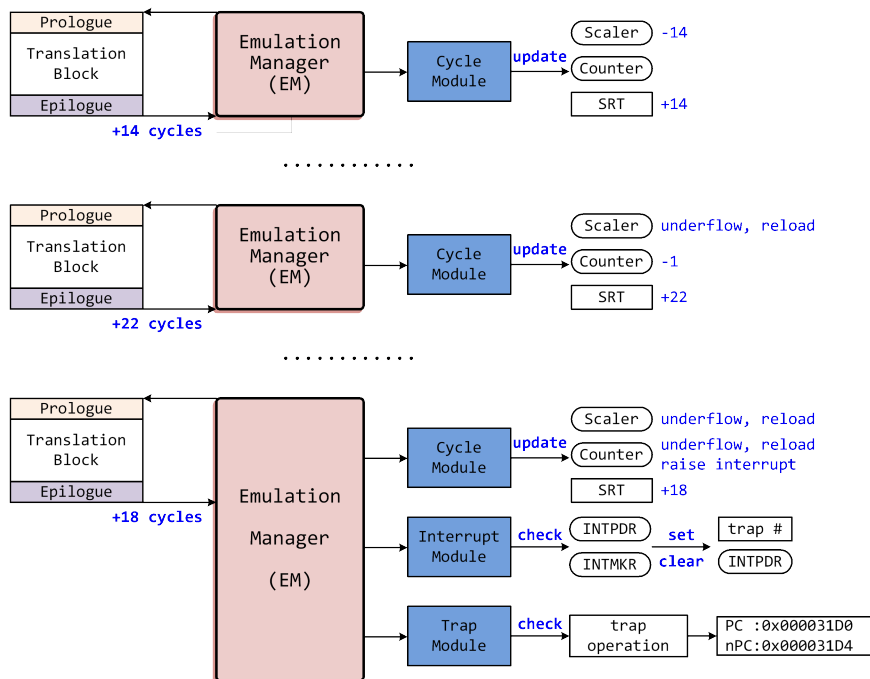
Figure 1-4. Instruction Cycle Counting in `laysim-gr740-dbt`



1.3.2 GPTIMER Emulation for Cycles

Cycle accuracy in processor emulation ensures that the emulator operates with precise timing. The simulated real time counter (SRT) is clocked according to the emulated processor clock and updated based on the number of executed instruction cycles. The SRT serves as the time source for emulated devices and simulated equipment, enabling synchronization with emulated processor.

Figure 1-5. GPTIMER Emulation in `laysim-gr740-dbt`



For example, the GR740 processor includes five GPTIMER units. GPTIMER#0 provides five timers, while the remaining GPTIMERs each provide four timers. Each timer unit contains its own shared scaler register and timer count register.

The scaler register is clocked by the system clock (250MHz, 4ns) and decrements on each cycle. When the scaler underflows, a tick pulse is generated for the timer, and the scaler is reloaded. The timer count register is decremented each time the scaler generates a tick pulse. When a timer underflows, a timer-specific interrupt is generated, and the counter is reloaded from reload register, as described in the GR740 User's Manual.

In the GR740, the interrupt from GPTIMER#0-0 is used as the RTOS system clock, typically set to 100Hz for the RTEMS system clock. In `laysim-gr740-dbt`, the cycle module (depicted in Figure 1-3) maintains the simulation time of emulator. It updates the scaler and timer count registers based on the number of elapsed cycles in each translation block, and invokes timer interrupt upon expiration.

After a timer interrupt is raised, the interrupt module checks the interrupt conditions, manipulates the processor's interrupt controller (IRQAMP), and sets the corresponding trap number, as shown in Figure 1-5. The trap module then handles the trap according to the processor's trap mechanism and updates the PC/nPC according to the trap base address. Finally, `laysim-gr740-dbt` either translates a new basic block when the TPC is not found in the SPC-to-TPC map table, or executes the corresponding translation block when TPC is found.

1.3.3 Time Event Operation

The time event module in `laysim-gr740-dbt`, shown in Figure 1-3, is based on the Simulated Real Time (SRT) and provides services for timed-events in the event table. This mechanism is used to simulate the operation of various devices and equipment according to the behavior of the FSW operation.

The simulated devices and equipment can be activated by either software or hardware operation, and their activities must be synchronized with the SRT of `laysim-gr740-dbt`. The event table consists of four members;

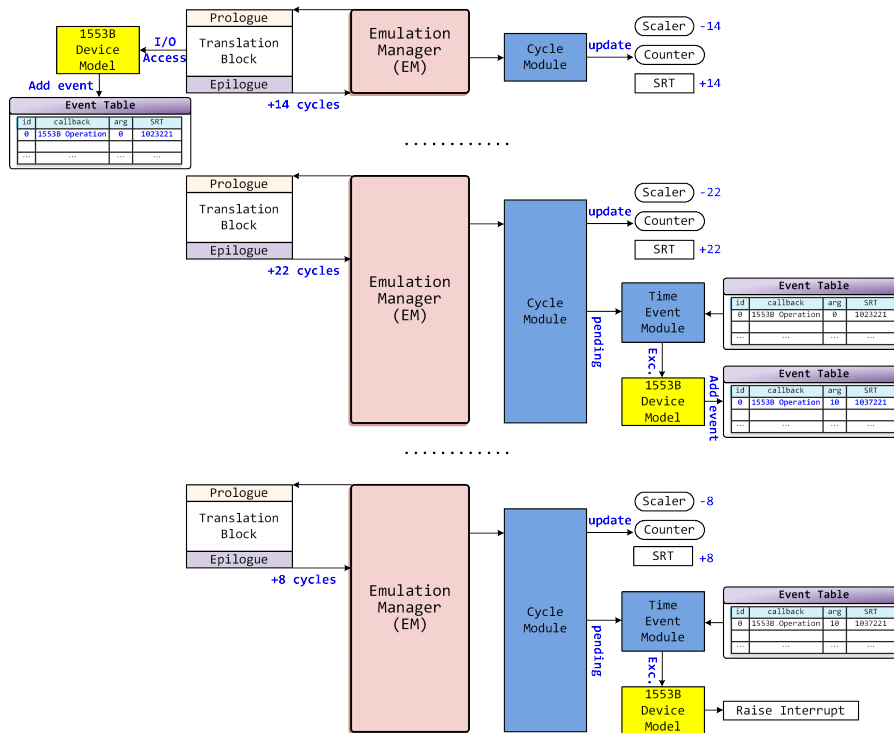
1. an event ID,
2. a callback function of the simulator,
3. an argument to be passed to the callback, and
4. the scheduled execution time based on the SRT.

When the FSW controls a device via APB operation, the corresponding simulation function is executed, and the next execution time is registered in the event table. After the execution of translation block, the cycle module of `laysim-gr740-dbt` updates the simulation time and checks pending events in the event table.

If a pending event exists, the time event module compares its scheduled execution time with the SRT. Once the scheduled time is reached, the registered callback function is executed. After execution, the simulation function registers the next execution time until the simulation operation is completed. During this process, it may raise a dedicated interrupt to signal the completion of the operation.

Figure 1-6 illustrates an example of timed operation for 1553B device emulation. This mechanism ensures that the simulation models remain synchronized with the SRT of `laysim-gr740-dbt`, thereby accurately emulating the behavior of the simulated devices and equipment.

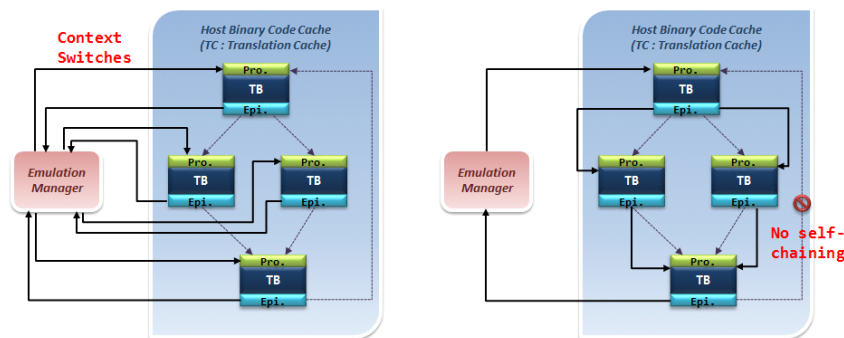
Figure 1–6. Timed Operation for GR1553B Emulation in laysim-gr740-dbt



1.3.4 Translation Block Chaining

Translation Block (TB) chaining is a well-known technique to improve the performance of dynamic binary translation. Normally, when the execution of a TB reaches its end, control is returned to the emulation manager (EM). This context switching introduces significant overhead. By reducing the number of returns to the EM and allowing direct jumps between associated basic blocks, TB chaining provides substantial performance improvement, as illustrated in Figure 1-7.

Figure 1–7. Translation Block Chaining



laysim-gr740-dbt also adopts TB chaining to enhance performance significantly, but does not allow self-chaining. Self-chaining is restricted because it would prevent returning to the EM, except in the case of a trap condition.

Each TB consists of a prologue and an epilogue. The epilogue updates the PC/nPC, increments the total accumulated instruction cycles of the TB, and returns control to the EM. After TB execution, the EM checks whether the successor basic block has already been translated. If

it has not yet been translated, the EM modifies the epilogue of the predecessor TB to insert a jump to the next TB body, which is allocated by the translator and preceded by its prologue. If the successor TB has already been translated, the EM retrieves its TPC from the map table and establishes a direct link by overwriting the epilogue with a jump to the successor TB.

The EM of `laysim-gr740-dbt` prevents self-chaining by using a self-flagging mechanism in the map table. This ensures that internal infinite loops are avoided while still benefiting from performance gains through TB chaining.

In `laysim-gr740-dbt`, only direct branch instructions such as `BiCC`, `FBfCC` and `CALL` where the destination address does not change are eligible for chaining by the EM. On the other hand, indirect branch instructions in the SPARC architecture, of which only `JMPL` and `RETT` exists, simply return control to the EM without chaining.

Direct branch instructions are divided into two categories:

1. Unconditional branch instructions:

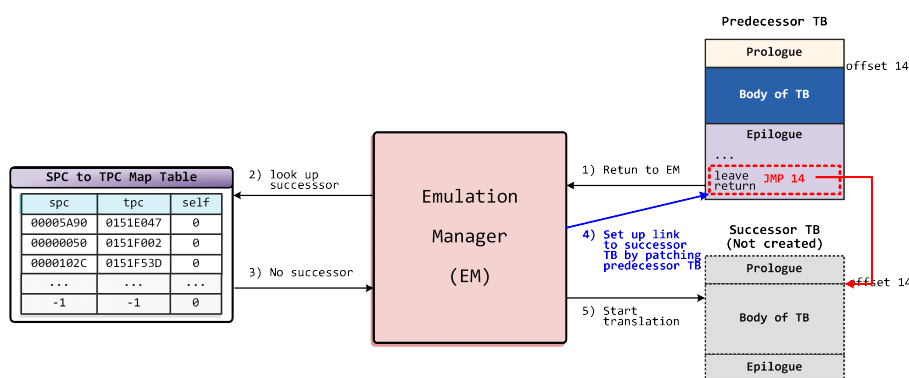
Examples include `CALL`, `BA` (branch always without `icc`) in `BiCC`, and `FBA` (branch always without `fcc`), `FBN` (branch never without `fcc`) in `FBfCC`. These instructions only have a single fixed branch destination address.

2. Conditional branch instructions:

These instructions can branch to one of two possible destination depending on the evaluation result of the condition. Therefore, the EM applies a different chaining strategy for these cases.

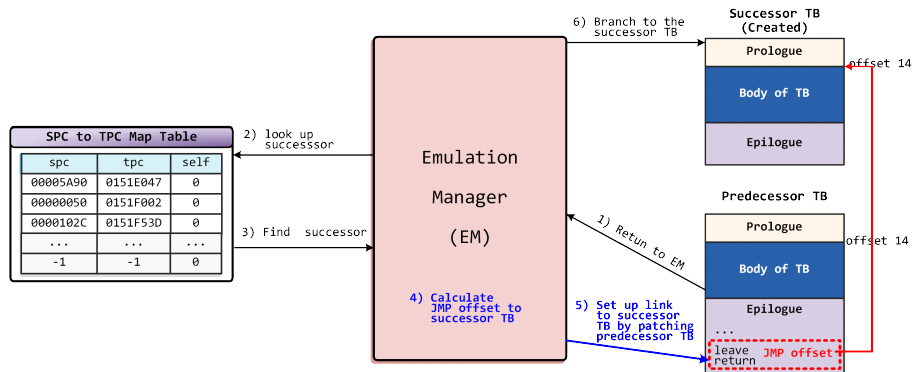
Figure 1-8 illustrates the chaining process for unconditional branches when a successor TB is not yet found in the SPC-to-TCP map table (i.e., the successor TB has not been translated). In this case, the EM patches the epilogue of the predecessor TB to insert a jump to the successor TB with a constant offset of 14, and then translates the successor basic block. When the predecessor TB is executed again, control flows directly to the successor TB without returning to the EM, thereby improving performance through TB chaining.

Figure 1–8. TB Chaining on Missing Successor for Unconditional Branch



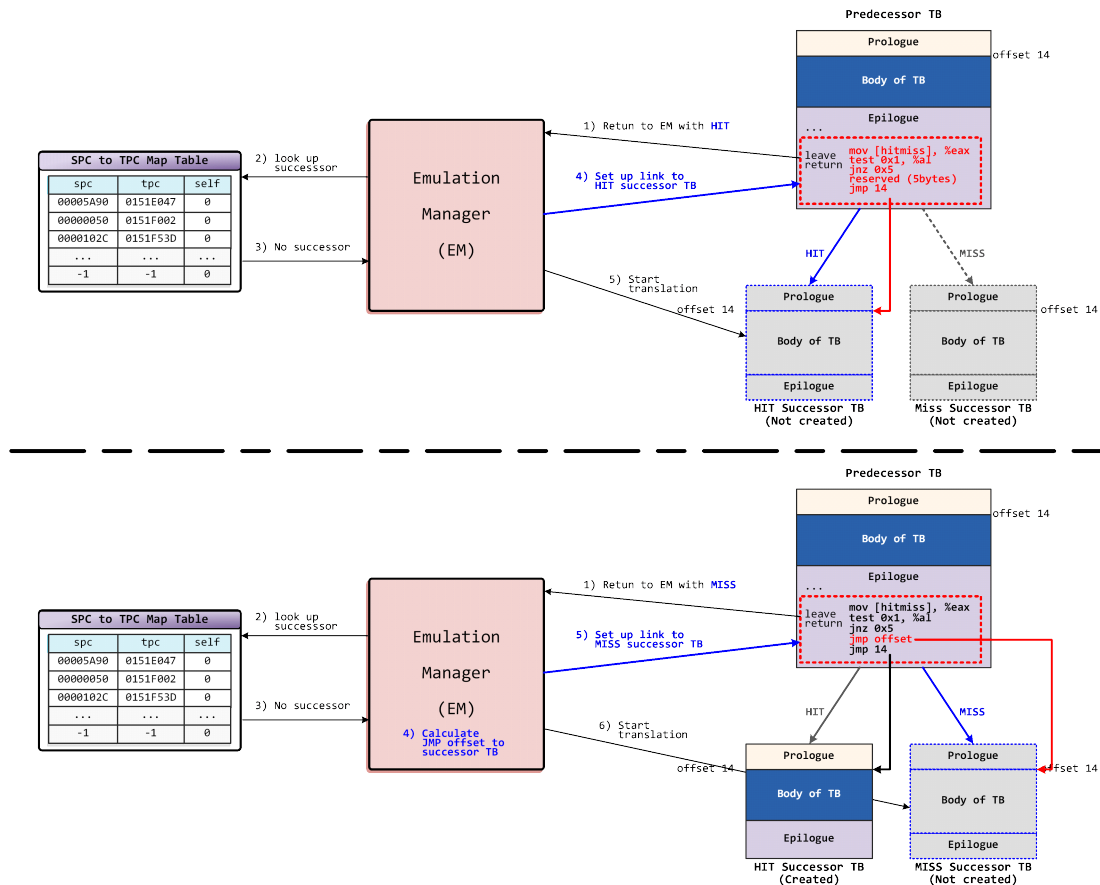
When the successor TB is already present in the SPC-to-TPC map table (i.e., the successor TB has been translated), the chaining step for an unconditional branch are shown in Figure 1-9. In this case, the EM overwrites the epilogue of the predecessor TB with a direct jump to the successor TB. Since the successor TB already exists in the translation cache, the EM can calculate the exact jump offset using the SPC-to-TPC map table. After updating the epilogue, control is transferred immediately to the successor TB for execution without returning to the EM.

Figure 1–9. TB Chaining on Finding Successor for Unconditional Branch



The steps of TB chaining for a conditional branch are more complex, as illustrated in Figure 1-10. When a TB returns to the EM due to a conditional branch hit, the EM consults the SPC-to-TPC map table to locate the TPC of the hit successor TB. If the hit TPC is not found in the map table, the EM patches the epilogue of the predecessor TB to link to the hit successor TB, which has not yet been translated.

Figure 1–10. TB Chaining for Conditional Branch



Unlike the unconditional branch, which requires only a single `jmp` instruction of x64, conditional branch chaining requires five x64 instructions. The first instruction moves the value of the hitmiss variable (updated during execution of the conditional branch in the predecessor TB) into the `eax` register. The second instruction tests whether the branch was taken using the `eax` register. The third instruction

is a `jmp` with a fixed offset of 5, which directs execution to the fifth instruction. The fourth instruction is reserved for the branch-miss case, since the miss successor TB cannot yet be identified. The fifth instruction is `jmp` to the hit successor TB, which will be translated next with a constant offset 14. After patching, the EM proceeds to translate the hit successor TB.

On subsequent execution, if the predecessor TB returns to the EM with a conditional branch miss, the EM again looks up the map table. If the miss TPC is not found, the EM links the miss successor TB by overwriting the reserved fourth instruction with a `jmp` to the miss successor TB using the calculated offset. The EM then begins translation of the miss successor TB.

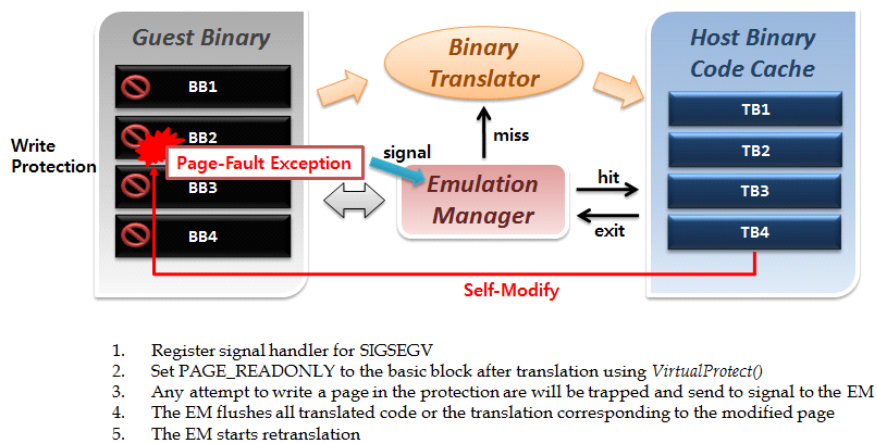
Once both the hit and miss successor TBs have been created, execution of the predecessor TB can directly transfer to either successor TB depending on the branch outcome, without returning to the EM.

1.3.5 Self-Modifying Code

Self-modifying code refers to guest code that modifies itself after being translated into a Translation Block (TB). This presents a potential issue for dynamic binary translation, because the code being executed is the translated TB, not the original guest code. As a result, the TB in the translation cache may no longer correspond to the modified guest code.

To address this issue, many DBT system marks the original guest code as write-protected at runtime using system calls such as `mprotect()` of Linux or `VirtualProtect()` of Windows, as shown in Figure 1-11. Any attempt to write to a protected page triggers a trap, sending a signal to the EM. Upon receiving this signal, all affected Translation Blocks, or only the TB corresponding to the modified page, are flushed from the translation cache. The EM then performs retranslation of the affected guest code.

Figure 1-11. General Handling Mechanism for Self-Modifying Code



However, the use of `VirtualProtect()/mprotect()` can have side effect when employing a linear Translation Cache (TC). On x64 architecture, memory protection attributes can only be applied per page, with a minimum size of 4KB. In VxWorks 5.4, the guest RTOS used here, the kernel's code and data are mixed within the Task Control Block (TCB) area. Any modification of data in the TCB by VxWorks can mistakenly trigger retranslation, being falsely recognized as self-modifying code.

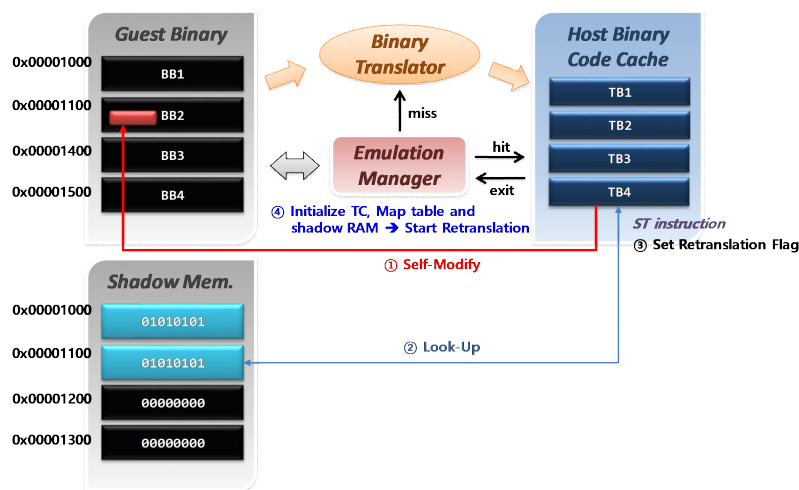
This issue is resolved by sing shadow memory, which mirrors the translated memory region without relying on system calls. By tracking

modifications in the shadow memory, `laysim-gr740-dbt` can correctly detect true self-modifying code while avoiding unnecessary retranslation.

To detect self-modifying code, `laysim-gr740-dbt` sets the corresponding address in the shadow memory during instruction translation. When a RAM write instruction is decoded, the DBT engine adds logic to check whether the target address has already been translated by consulting the shadow memory.

If a self-modifying case is detected, the EM reinitializes the TC, the SPC-to-TCP map table, and shadow memory, then begins retranslating the affected code. Figure 1-12 illustrates the overall workflow for handling self-modifying code in `laysim-gr740-dbt`.

Figure 1-12. Handling Mechanism for Self-Modifying Code in `laysim-gr740-dbt`



1.4 Architecture of `laysim-gr740-mdbt`

`laysim-gr740-mdbt` is a parallel execution emulator that reuses the mature, thread-safe core of `laysim-gr740-dbt`. Synchronization between cores is critical in multicore simulation with parallel execution. Without synchronization, each core runs at the maximum speed allowed by the host machine, which may maximize raw performance but introduces significant synchronization errors. Such errors generally prevent RTEMS SMP or XrtamtuM applications from functioning correctly.

`laysim-gr740-mdbt` employs quantum-based synchronization, also available in `laysim-gr740-dbt` via the '-quantum' option. Setting '-quantum 0' in `laysim-gr740-dbt` allows cores to synchronization at every TB instead of quantum, which can simulate more accurate but incurs considerable performance overhead.

In quantum synchronization, all cores execute their TBs and maintain local cycle counts. After each TB execution, local cycles are compared with the global cycle counter, which is incremented by the quantum length. If a core's local cycles exceed the global cycles, it pauses execution until all cores complete their current quantum. Afterwards, `laysim-gr740-dbt/mdbt` updates GPTIMERS, services time-dependent events, and resumes execution of all core after advancing the global cycles by one quantum.

This approach allows `laysim-gr740-mdbt` to achieve high performance while maintaining acceptable cycle accuracy in multicore

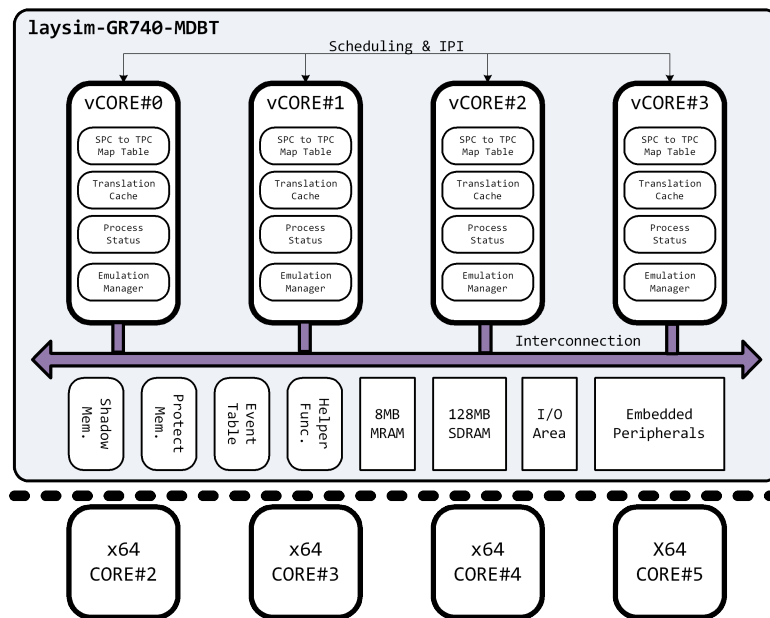
simulations.

The memory synchronization is a critical aspect in the parallel emulator. In `laysim-gr740-dbt`, which is a sequential emulator executing cores in round-robin order, memory synchronization among cores is unnecessary.

In contrast, `laysim-gr740-mdbt` must ensure memory consistency when multiple emulated cores access the same address simultaneously, including cases involving atomic instructions. Each core may write different data to the same memory location at the same time. To handle this, `laysim-gr740-mdbt` employs spin-locks using the x64 `cmpxchg` instruction. Spin-locks are also required for the IRQAMP, as each core may access it concurrently. Due to the overhead of memory synchronization, `laysim-gr740-mdbt` exhibits lower performance than `laysim-gr740-dbt` in single core applications.

Figure 1-13 illustrates the architecture of `laysim-gr740-mdbt`. Each of the GR740's four cores is mapped to a dedicated x64 core using pthread affinity: GR740 CORE#0 → x64 CORE#2, ..., GR740 CORE#3 → x64 CORE#5.

Figure 1-13. Architecture of `laysim-gr740-mdbt`



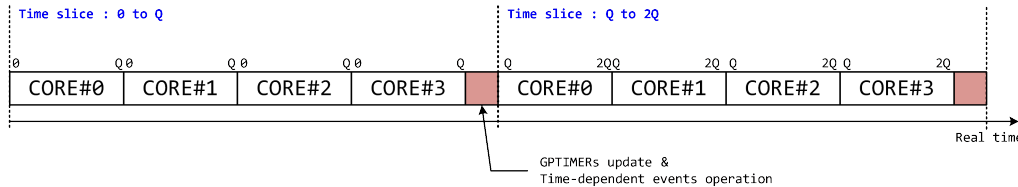
1.5 Multicore Scheduling in `laysim-GR740`

`laysim-gr740`, an interpreter-based emulator, schedules GR740 multicore execution at the instruction and cycle level. The EM selects the core with the lowest local cycles, fetches an instruction, and executes it while updating the core's local cycles according to L1/L2 cache hit/miss. The EM also updates GPTIMERS and processes time-dependent events registered by other units, such as APBUART or GR1553B. This fine-grained scheduling ensures high cycle accuracy, but performance is significantly reduced in SMP applications.

`laysim-gr740-dbt` and `laysim-gr740-mdbt` uses time quantum (TQ) scheduling for multicore execution and synchronization. Figure 1-14 illustrates the time quantum scheduling in `laysim-gr740-dbt`. By default, the time quantum is 500 cycles (2us) and can be modified using '`-quantum VALUE`' at startup..

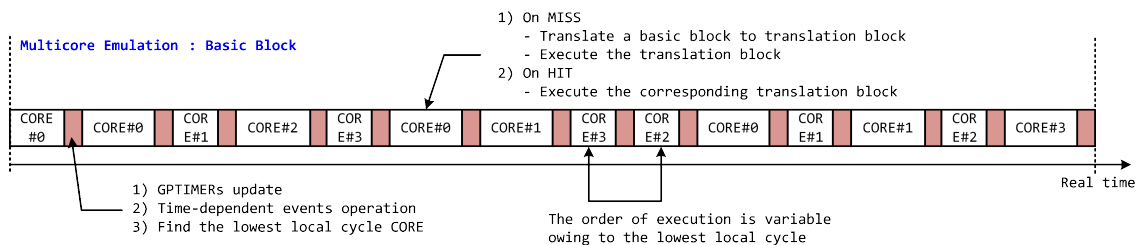
During multicore emulation, the EM executes CORE#0, CORE#1, CORE#2, and CORE#3 in round-robin order. After each quantum, it updates GPTIMERS (scaler and counter values, and raises GPTIMER interrupts) and processes timed events. This time quantum thus defines the minimum synchronization interval between all cores. This method increases performance, but may introduce synchronization violations in strict real-time systems.

Figure 1-14. Time Quantum Scheduling of laysim-gr740-dbt



laysim-gr740-dbt allows switching multicore emulation from time-quantum scheduling to basic block scheduling, as shown in Figure 1-15. By setting '-quantum 0' at startup, the EM changes the multicore scheduling method to the basic block scheduling. In this mode, the EM selects the core with the lowest local cycles, executes its TB, updates GPTIMERS, and processes time-dependent events. Consequently, the minimum synchronization interval between all cores corresponds to the executed cycles of the TB. This method ensures strict synchronization across all cores but inevitably reduces real-time performance.

Figure 1-15. Basic Block Scheduling of laysim-gr740-dbt



laysim-gr740-mdbt supports parallel execution based on the time quantum. The control thread manages multicore scheduling, as illustrated in Figure 1-16.

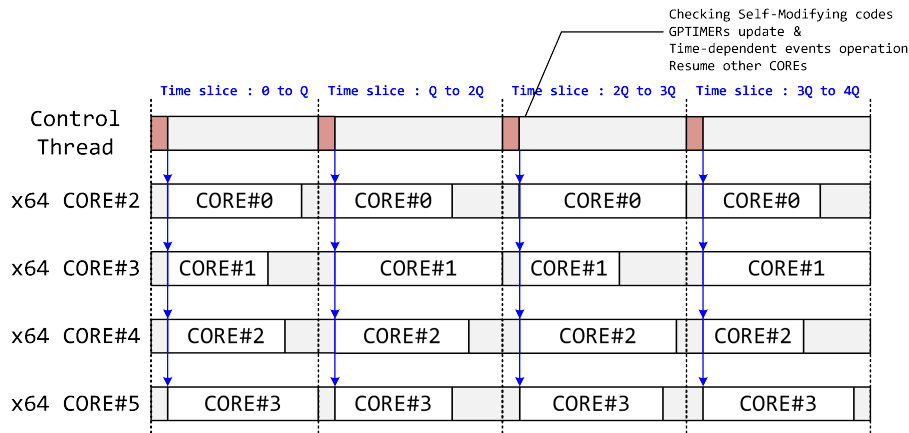
After executing their assigned Translation Blocks (TBs), all core maintain their local cycles and compares them with the global cycle counter, which is incremented by the quantum length. If a core's local cycles exceed the global cycles, it pauses execution and waits until all cores complete their current time quantum.

Once all core finish their allocated quantum, the control thread performs the following tasks:

1. checks for self-modifying code.
2. updates GPTIMERS
3. Services time-dependent events

After completing theses steps, the control thread increments the global cycles by the quantum length and resumes execution of all cores. This approach allows laysim-gr740-mdbt to achieve high performance while maintaining acceptable cycle accuracy.

Figure 1–16. Parallel Time Quantum Scheduling of laysim-gr740-mdbt



1.6 Other Features in laysim-GR740

1.6.1 Emulation of AMBA Plug & Play Information

laysim-GR740 provides the AMBA plug & play (P&P) information as implemented in the GR740 processor. Figure 1-17 shows the P&P information of laysim-GR740. This information is used in the BCC/RCC, and rtems-shell example of showing emulated H/W devices of laysim-GR740.

Figure 1–17. AMBA P&P Information of laysim-GR740

Core	Function	Vendor ID	Device ID	Emulation in laysim-GR740
LEON4-FT	LEON4 SPARC v8 32-bit processor	0x01	0x048	Supported by laysim-GR740
AHB2AHB	Uni-directional AHB/AHB bridge	0x01	0x020	Controlled by laysim-GR740
APBCTRL	AHB/APB bridge	0x01	0x006	Controlled by laysim-GR740
APBUART	8-bit UART with FIFO	0x01	0x00C	APBUART is supported with Tx/Rx FIFO and loadable UART Module
AHBJTAG	JTAG/AHB Debug interface	0x01	0x01C	Only P&P information
AHBSTAT	AHB Status Register	0x01	0x052	Supported by laysim-GR740
AHBTRACE	AHB trace buffer	0x01	0x017	Only P&P information
IRQ(A)MP	Multiprocessor interrupt controller	0x01	0x00D	Supported by laysim-GR740
DSU4	LEON4 Debug Support Unit	0x01	0x049	Only P&P information
MMCTRL	Memory controller	0x01	0x05D	Supported by laysim-GR740
GPTIMER	Modular timer unit with watchdog	0x01	0x011	Supported by laysim-GR740 (No watchdog)
GR1553B	MIL-STD-1553B/AS15531 interface	0x01	0x04D	Supported by laysim-GR740 (BC only)
GRCAN	CAN 2.0 controller with DMA	0x01	0x03D	Supported by laysim-GR740
GRCLKGATE	Clock gating unit	0x01	0x02C	Supported by laysim-GR740
GRETH_GBIF	10/100/1000 Ethernet MAC with DCL	0x01	0x01D	Supported by laysim-GR740
GRGPIO	General Purpose I/O Port	0x01	0x01A	Supported by laysim-GR740
GRGPRBANK	General Purpose Register Bank	0x01	0x08F	Supported by laysim-GR740
GRGPREG	General Purpose Register	0x01	0x087	Supported by laysim-GR740
GRIOMMU	AHB/AHB bridge with protection (IOMMU)	0x01	0x04F	Supported by laysim-GR740
SPICTRL	SPI controller	0x01	0x02D	SPICTRL is supported with loadable SPI Module
GRPCI2	Fast 32-bit PCI bridge	0x01	0x07C	Supported by laysim-GR740

				- Need loadable PCI module to extend
GRSPW2	SpaceWire codec with RMAP	0x01	0x029	Supported by laysim-GR740
GRSPWROUTER	SpaceWire Router Switch	0x01	0x08B	Supported by laysim-GR740
GRSPWTDP	SpaceWire - Time Distribution Protocol	0x01	0x097	Not supported in laysim-GR740
FTMCTRL	8/16/32-bit memory controller with EDAC	0x01	0x054	Supported by laysim-GR740 (No EDAC)
L2CACHE	Level 2 cache	0x01	0x04B	Only supported by laysim-gr740 (Not configurable)
L4STAT	LEON4 statistical unit	0x01	0x047	Not supported in laysim-GR740
MEMSCRUB	Memory scrubber	0x01	0x057	Not supported in laysim-GR740
GR740THSENS	GR740 Temperature sensor controller	0x01	0x099	Only register operation

1.6.2 Memory Map of laysim-GR740

laysim-GR740 provides 8MB MRAM, 128MB SDRAM as default and AHB/APB buses from GR740 processor as shown in Figure 1-18. CPU Frequency, ROM size, RAM size, and APBUART#0/#1 baudrate can be changed using start-up option.

Figure 1–18. Memory Map of laysim-GR740

Component		Address Range	Area	Bus	Emulation in laysim-GR740
L2CACHE		0x00000000-0x7FFFFFFF	L2 cache memory area Covers SDRAM memory area	Processor	• ELF/AOUT/binary file can be loaded to SDRAM • 128MB SDRAM is implemented as default, but it can be changed using -ramsz option
GRPCI2		0x80000000-0xBFFFFFFF	PCI memory area	Slave I/O	Supported in laysim-GR740 - Need loadable PCI module to extend
FTMCTRL		0xC0000000-0xCFFFFFFF 0xD0000000-0xDFFFFFFF	PROM area Memory mapped I/O area	Slave I/O	• ELF/binary file can be loaded to MRAM • 8MB MRAM is implemented as default, but it can be changed using -romsz option
L2CACHE		0xF0000000-0xF03FFFFF	L2 cache configuration registers	Processor	Three L2CACHE registers are supported
GRPCI2		0xFF800000-0xFF83FFFF	PCI I/O area	Slave I/O	Supported in laysim-GR740 - Need loadable PCI module to extend
GRIOMMU		0xFF840000-0xFF847FFF	IOMMU configuration registers	Slave I/O	Supported in laysim-GR740 - APV and TLB caches are not implemented
GRSPWROUTER		0xFF880000-0xFF880FFF	SpaceWire router configuration port	Slave I/O	Supports RMAP protocol with Nodes SpaceWire-D and SpaceWire Plug-and-Play are not supported
		0xFF8FF000-0xFF8FFFFF	Slave I/O bus plug&play area	Slave I/O	P&P information is supported
APBBRIDGE0		0xFF90F000-0xFF90FFFF	APB bridge 0	Processor	Supported in laysim-GR740
A	APBUART0	0xFF900000-0xFF9000FF	UART 0 registers	Processor	APBUART is supported with Tx FIFO
P	APBUART1	0xFF901000-0xFF9010FF	UART 0 registers	Processor	APBUART is supported with Tx FIFO
B	GRGPIO0	0xFF902000-0xFF9020FF	General purpose I/O port registers	Processor	GRGPIO is supported with interrupt generation
B	FTMCTRL	0xFF903000-0xFF9030FF	PROM/IO controller registers	Processor	FTMCTRL is supported without EDAC
R	IRQ(A)MP	0xFF904000-0xFF907FFF	Interrupt controller registers	Processor	IRQ(A)MP is supported
I	GPTIMER0	0xFF908000-0xFF9080FF	Timer unit 0 registers	Processor	GPTIMER is supported with latching
D	GPTIMER1	0xFF909000-	Timer unit 1 registers	Processor	GPTIMER is supported with latching

E 0		0xFF9090FF			
	GPTIMER2	0xFF90A000-0xFF90A0FF	Timer unit 2 registers	Processor	GPTIMER is supported with latching
	GPTIMER3	0xFF90B000-0xFF90B0FF	Timer unit 3 registers	Processor	GPTIMER is supported with latching
	GPTIMER4	0xFF90C000-0xFF90C0FF	Timer unit 4 registers	Processor	GPTIMER is supported with latching
	GRSPWRROUTER	0xFF90D000-0xFF90DFFF	SpaceWire router AMBA interface 0	Processor	Support RMAP protocol and router operation
	GRSPWRROUTER	0xFF90E000-0xFF90EFFF	SpaceWire router AMBA interface 1	Processor	Support RMAP protocol and router operation
	GRSPWRROUTER	0xFF90F000-0xFF90FFFF	SpaceWire router AMBA interface 2	Processor	Support RMAP protocol and router operation
	GRSPWRROUTER	0xFF910000-0xFF910FFF	SpaceWire router AMBA interface 3	Processor	Support RMAP protocol and router operation
	GRETH_GBITH0	0xFF940000-0xFF9400FF	Gigabit Ethernet MAC 0 registers	Processor	GRETH is supported
	GRETH_GBITH1	0xFF980000-0xFF9800FF	Gigabit Ethernet MAC 1 registers	Processor	GRETH is supported
APBBRIDGE0		0xFF9FF000-0xFF9FFFFF	APB bus 0 plug&play area	Processor	P&P information is supported
APBBRIDGE1		0xFFA00000-0xFFAFFFFFFF	APB bridge 1	Processor	Supported in laysim-GR740
A P B R I D G E 1	GRPCI2	0xFFA00000-0xFFA000FF	PCI controller registers	Processor	Supported in laysim-GR740 - Need loadable PCI module to extend
	GRCAN0	0xFFA01000-0xFFA013FF	CAN 2.0 controller 0	Processor	GRCAN is supported
	GRCAN1	0xFFA02000-0xFFA023FF	CAN 2.0 controller 1	Processor	GRCAN is supported
	SPICTRL	0xFFA03000-0xFFA030FF	SPI controller	Processor	SPI is supported with loadable SPI module
	GRCLKGATE	0xFFA04000-0xFFA040FF	Clock gating unit	Processor	Controlled by laysim-GR740
	GR1553B	0xFFA05000-0xFFA050FF	MIL-STD-1553B controller	Processor	GR1553B is supported (BC only)
	AHBSTAT0	0xFFA06000-0xFFA060FF	AHB status register monitoring Processor AHB bus	Processor	AHBSTAT is supported
	AHBSTAT1	0xFFA07000-0xFFA070FF	AHB status register monitoring Slave I/O AHB bus	Processor	AHBSTAT is supported
	GRGPIO1	0xFFA08000-0xFFA080FF	General purpose I/O register for multiplexed pins	Processor	GRGPIO is supported with interrupt generation
	GRGPREG	0xFFA09000-0xFFA090FF	Register for bootstrap signals	Processor	Controlled by laysim-GR740
	GR740THSENS	0xFFA0A000-0xFFA0A0FF	Temperature sensor	Processor	Controlled by laysim-GR740
	GRGPRBANK	0xFFA0B000-0xFFA0B0FF	General purpose register bank	Processor	Controlled by laysim-GR740
	GRSPWTDTP	0xFFA0C000-0xFFA0C1FF	CCSDS TDP controller	Processor	Not supported in laysim-GR740
	L4STAT	0xFFA0D000-0xFFA0D1FF	LEON4 Statistics Unit	Processor	Not supported in laysim-GR740
APBBRIDGE1		0xFFAFF000-0xFFAFFFFF	APB bus 1 plug&paly area	Processor	P&P information is supported
MMCTRL		0xFFE00000-0xFFE000FF	SDRAM controller register	Memory	MMCTRL is supported
MEMSCRUB		0xFFE01000-0xFFE010FF	Memory scrubber register	Memory	Not supported in laysim-GR740
		0xFFEFFF000-0xFFEFFFFFFF	Memory bus plug&play area	Memory	P&P information is supported
		0xFFFFF000-0xFFFFFFFFFFF	Processor bus plug&play area	Processor	P&P information is supported

1.6.3 Interrupts of laysim-GR740

laysim-GR740 supports interrupts according to the GR740 processor as shown in Figure 1-19. All interrupts are configured by the AMBA P&P information.

Figure 1–19. Interrupts of laysim-GR740

INT#	Peripheral	Function	Emulation in laysim-GR740
1	GPTIMER0	GPTIMER unit 0, timer 1	Supported by laysim-GR740
2		GPTIMER unit 0, timer 2	
3		GPTIMER unit 0, timer 3	
4		GPTIMER unit 0, timer 4	
5		GPTIMER unit 0, timer 5	
6	GPTIMER1	Shared interrupt for all timers on GPTIMER unit 1	Supported by laysim-GR740
7	GPTIMER2	Shared interrupt for all timers on GPTIMER unit 2	Supported by laysim-GR740
8	GPTIMER3	Shared interrupt for all timers on GPTIMER unit 3	Supported by laysim-GR740
9	GPTIMER4	Shared interrupt for all timers on GPTIMER unit 4	Supported by laysim-GR740
10	IRQ(A)MP	Extended interrupt line	Supported by laysim-GR740
11	GRPCI/PCIDMA	PCI master/target and PCI DMA	Supported by laysim-GR740
12	Unassigned	Suitable for use by software for inter-processor and inter-process synchronization	
13	Unassigned		
14	Unassigned		
15	Unassigned	Note: Not maskable by processor	
16	GRGPIO0/1/CAN	The GPIO port has configuration registers that determine the mapping between general purpose I/O lines and the four interrupt lines allocated to the GPIO port Interrupt lines 16-18 are shared between the GPIO port and CAN controllers Interrupt line 19 is shared between the GPIO port and SPI controller	Supported by laysim-GR740
17	GRGPIO0/1/CAN		GRGPIO is supported with interrupt generation
18	GRGPIO0/1/CAN		
19	GRGPIO0/1/SPICTRL		Supported by laysim-GR740
20	SPWROUTER AMBA I/F 0	SpaceWire router AMBA interface 0	Supported by laysim-GR740
21	SPWROUTER AMBA I/F 1	SpaceWire router AMBA interface 1	Supported by laysim-GR740
22	SPWROUTER AMBA I/F 2	SpaceWire router AMBA interface 2	Supported by laysim-GR740
23	SPWROUTER AMBA I/F 3	SpaceWire router AMBA interface 3	Supported by laysim-GR740
24	GRETH_GBITH0	Gigabit Ethernet MAC 0	Supported by laysim-GR740
25	GRETH_GBITH1	Gigabit Ethernet MAC 1	
26	GR1553B	MIL-STD-1553B interface controller	Supported by laysim-GR740
27	AHBSTAT/ST65THSENS	Shared by all AHB Status registers in design and by temperature sensor	Supported by laysim-GR740
28	MEMSCRUB/L2CACHE	Memory scrubber and L2 cache	MEMSCRUB is not supported by laysim-GR740
29	APBUART0	UART 0	Supported by laysim-GR740
30	APBUART1	UART 1	Supported by laysim-GR740
31	GRIOMMU/GRSPWTDP	IOMMU register interface interrupt CCSDS TDP controller interrupt	GRIOMMU is supported by laysim-GR740 GRSPWTDP is not supported by laysim-GR740

1.7 Limitation of laysim-GR740

The evaluation version of laysim-GR740 does not impose execution limitations, except that users must acquire a 1-year free license as described in Chapter 2.2. The user-defined modules such as PCI and I/O modules can be supported according to the separate license agreement. In the evaluation version, only pre-built loadable PCI and I/O modules are provided.

1.8 Applicable Documents

[laysim-GR740-001] laysim GR740 Quad Core Processor Emulator - User's Manual
[laysim-GR740-002] GR740 Examples from BCC/RCC/RTEMS5 Result on laysim-GR740
[laysim-GR740-003] RTEMS5 Testsuites Result on GR740 Board and laysim-GR740
[laysim-GR740-004] GR740 SpaceWire and Router Operation on laysim-GR740
[laysim-GR740-005] GR740 GR1553B Operation on laysim-GR740
[laysim-GR740-006] GR740 IRQ(A)MP Operation on laysim-GR740
[laysim-GR740-007] GR740 Synchronous Trap Test on laysim-GR740
[laysim-GR740-008] GR740 GRCAN Operation on laysim-GR740
[laysim-GR740-009] laysim-GR740 Loadable Modules - User's Manual
[laysim-GR740-010] laysim-GR740 Shared Library - User's Manual
[laysim-GR740-011] laysim-GR740 - Real-Time Performance and Cycle Accuracy
[laysim-GR740-012] Debugging with GDB on laysim-GR740
[laysim-GR740-013] Code coverage operation on laysim-GR740
[laysim-GR740-014] GR740 IOMMU Operation on laysim-GR740
[laysim-GR740-015] Detailed Comparison with laysim-GR740 and Simic LEON4
[laysim-AIR-001] Debugging AIR 5.6.0 on laysim-GR740
[laysim-AIR-002] AIR 5.6.0 Test Result with laysim-GR740

2. Installation

2.1 Installation for Linux x64

2.1.1 Preparation

In order to compile examples of BCC-2.2.2/RCC-1.3.1 in Linux 64bit platform, 5 components should be installed with designated directories as shown in Figure 2-1. It is recommended to install them sequentially from top to bottom. Optionally, RTEMS5 from git can be installed.

Figure 2-1. Install Cross Tool Chains for GR740 in Linux

Component	Directory	File	Remark
BCC-2.2.2	/opt/bcc-2.2.2-gcc	bcc-2.2.2-gcc-linux64.tar.xz	
BCC-2.2.2 source code	/opt/bcc-2.2.2-gcc/src/bcc-2.2.2	bcc-2.2.2-src.tar.bz2	debugging BCC2 codes
RCC-1.3.1	/opt/rcc-1.3.1-gcc	sparc-rtems-5-gcc-10.2.0-1.3.1-1 linux.txz	-
RCC-1.3.1 source code	/opt/rcc-1.3.1-gcc/src/rcc-1.3.1	rtems-5-1.3.1-src.txz	debugging RCC-1.3.1 codes
MKPR0M2	/opt/mkprom2	mkprom2-2.0.65.tar.gz	create boot-image

After installation, the toolchain executables can be invoked from the command prompt by adding the executable directory to the PATH environment variable in .bashrc.

2.1.2 Installation of laysim-GR740

laysim-GR740 was built with gtk+2.0 (64bit) on CentOS 7, so there are no additional software packages for CentOS 7. But it is necessary to install libcanberra-gtk-module on CentOS 8 and Ubuntu 18.04 to suppress warning of 'Failed to load module "canberra-gtk-module"'. In case of Debian 10.3, libgtk2.0 and libcanberra-gtk-module should be installed.

2.1.2.1 Installation of additional packages for CentOS 7

laysim-GR740 was built with gtk+2.0 (64bit) on CentOS 7, and CentOS 7 x64 provides gtk+2.0-2.24.31 as default when it selected GNOME Desktop during installation. So there are no additional software packages on CentOS 7 for laysim-GR740.

2.1.2.2 Installation of additional packages for CentOS 8 and Ubuntu 18.04

CentOS 8 x64 provides gtk+2.0-2.24.31 as default when it selected GNOME Desktop during installation, and Ubuntu 18.04 provides gtk+2.0-2.24.32-1 as default. But it is necessary to install libcanberra-gtk-module to suppress warning as shown in Figure 2-2.

Figure 2-2. Install libcanberra-gtk-module on Ubuntu 18.04

```
layright@ubuntu:~$ sudo apt-get install libcanberra-gtk-module
Reading package lists... Done
Building dependency tree
Reading state information... Done
The following NEW packages will be installed:
```

```
libcanberra-gtk-module libcanberra-gtk0
0 upgraded, 2 newly installed, 0 to remove and 280 not upgraded.
Need to get 17.9 kB of archives.
After this operation, 84.0 kB of additional disk space will be used.
Do you want to continue? [Y/n] y
Get:1 http://us.archive.ubuntu.com/ubuntu bionic/universe amd64 libcanberra-gtk0 amd64 0.30-5ubuntu1 [7,864 B]
Get:2 http://us.archive.ubuntu.com/ubuntu bionic/universe amd64 libcanberra-gtk-module amd64 0.30-5ubuntu1 [10.0 kB]
Fetched 17.9 kB in 1s (16.1 kB/s)
Selecting previously unselected package libcanberra-gtk0:amd64.
(Reading database ... 140366 files and directories currently installed.)
Preparing to unpack .../libcanberra-gtk0_0.30-5ubuntu1_amd64.deb ...
Unpacking libcanberra-gtk0:amd64 (0.30-5ubuntu1) ...
Selecting previously unselected package libcanberra-gtk-module:amd64.
Preparing to unpack .../libcanberra-gtk-module_0.30-5ubuntu1_amd64.deb ...
Unpacking libcanberra-gtk-module:amd64 (0.30-5ubuntu1) ...
Processing triggers for libc-bin (2.27-3ubuntu1) ...
Setting up libcanberra-gtk0:amd64 (0.30-5ubuntu1) ...
Setting up libcanberra-gtk-module:amd64 (0.30-5ubuntu1) ...
Processing triggers for libc-bin (2.27-3ubuntu1) ...
```

2.1.2.3 Installation of additional packages for Debian 10.3

Debian 10.3 x64 does not provide gtk+2.0 as default, so it is required to install libgtk2.0 and libcanberra-gtk-module as shown in Figure 2-3.

Figure 2-3. Install libgtk2.0 and libcanberra-gtk-module on Debian 10.3

```
layright@debian:~$ sudo apt-get install libgtk2.0
[sudo] password for layright:
Reading package lists... Done
Building dependency tree
Reading state information... Done
Note, selecting 'libgtk2.0-bin' for regex 'libgtk2.0'
Note, selecting 'libgtk2.0-0' for regex 'libgtk2.0'
Note, selecting 'libgtk2.0-common' for regex 'libgtk2.0'
The following additional packages will be installed:
  libgail-common libgail18
The following NEW packages will be installed:
  libgail-common libgail18 libgtk2.0-0 libgtk2.0-bin libgtk2.0-common
0 upgraded, 5 newly installed, 0 to remove and 0 not upgraded.
Need to get 0 B/4,767 kB of archives.
After this operation, 27.0 MB of additional disk space will be used.
Do you want to continue? [Y/n] y
Get:1 cdrom://[Debian GNU/Linux 10.3.0 _Buster_ - Official amd64 DVD Binary-1 20200208-12:08] buster/main amd64 libgtk2.0-common all 2.24.32-3 [2,698 kB]
Get:2 cdrom://[Debian GNU/Linux 10.3.0 _Buster_ - Official amd64 DVD Binary-1 20200208-12:08] buster/main amd64 libgtk2.0-0 amd64 2.24.32-3 [1,809 kB]
Get:3 cdrom://[Debian GNU/Linux 10.3.0 _Buster_ - Official amd64 DVD Binary-1 20200208-12:08] buster/main amd64 libgail18 amd64 2.24.32-3 [55.5 kB]
Get:4 cdrom://[Debian GNU/Linux 10.3.0 _Buster_ - Official amd64 DVD Binary-1 20200208-12:08] buster/main amd64 libgail-common amd64 2.24.32-3 [156 kB]
Get:5 cdrom://[Debian GNU/Linux 10.3.0 _Buster_ - Official amd64 DVD Binary-1 20200208-12:08] buster/main amd64 libgtk2.0-bin amd64 2.24.32-3 [48.5 kB]
Selecting previously unselected package libgtk2.0-common.
(Reading database ... 132907 files and directories currently installed.)
Preparing to unpack .../libgtk2.0-common_2.24.32-3_all.deb ...
Unpacking libgtk2.0-common (2.24.32-3) ...
Selecting previously unselected package libgtk2.0-0:amd64.
Preparing to unpack .../libgtk2.0-0_2.24.32-3_amd64.deb ...
Unpacking libgtk2.0-0:amd64 (2.24.32-3) ...
Selecting previously unselected package libgail18:amd64.
Preparing to unpack .../libgail18_2.24.32-3_amd64.deb ...
Unpacking libgail18:amd64 (2.24.32-3) ...
Selecting previously unselected package libgail-common:amd64.
Preparing to unpack .../libgail-common_2.24.32-3_amd64.deb ...
Unpacking libgail-common:amd64 (2.24.32-3) ...
Selecting previously unselected package libgtk2.0-bin.
Preparing to unpack .../libgtk2.0-bin_2.24.32-3_amd64.deb ...
Unpacking libgtk2.0-bin (2.24.32-3) ...
Setting up libgtk2.0-common (2.24.32-3) ...
Setting up libgtk2.0-0:amd64 (2.24.32-3) ...
Setting up libgail18:amd64 (2.24.32-3) ...
Setting up libgtk2.0-bin (2.24.32-3) ...
Setting up libgail-common:amd64 (2.24.32-3) ...
Processing triggers for man-db (2.8.5-2) ...
Processing triggers for libc-bin (2.28-10) ...

layright@debian:~$ sudo apt-get install libcanberra-gtk-module
Reading package lists... Done
Building dependency tree
Reading state information... Done
The following additional packages will be installed:
  libcanberra-gtk0
The following NEW packages will be installed:
  libcanberra-gtk-module libcanberra-gtk0
```

```

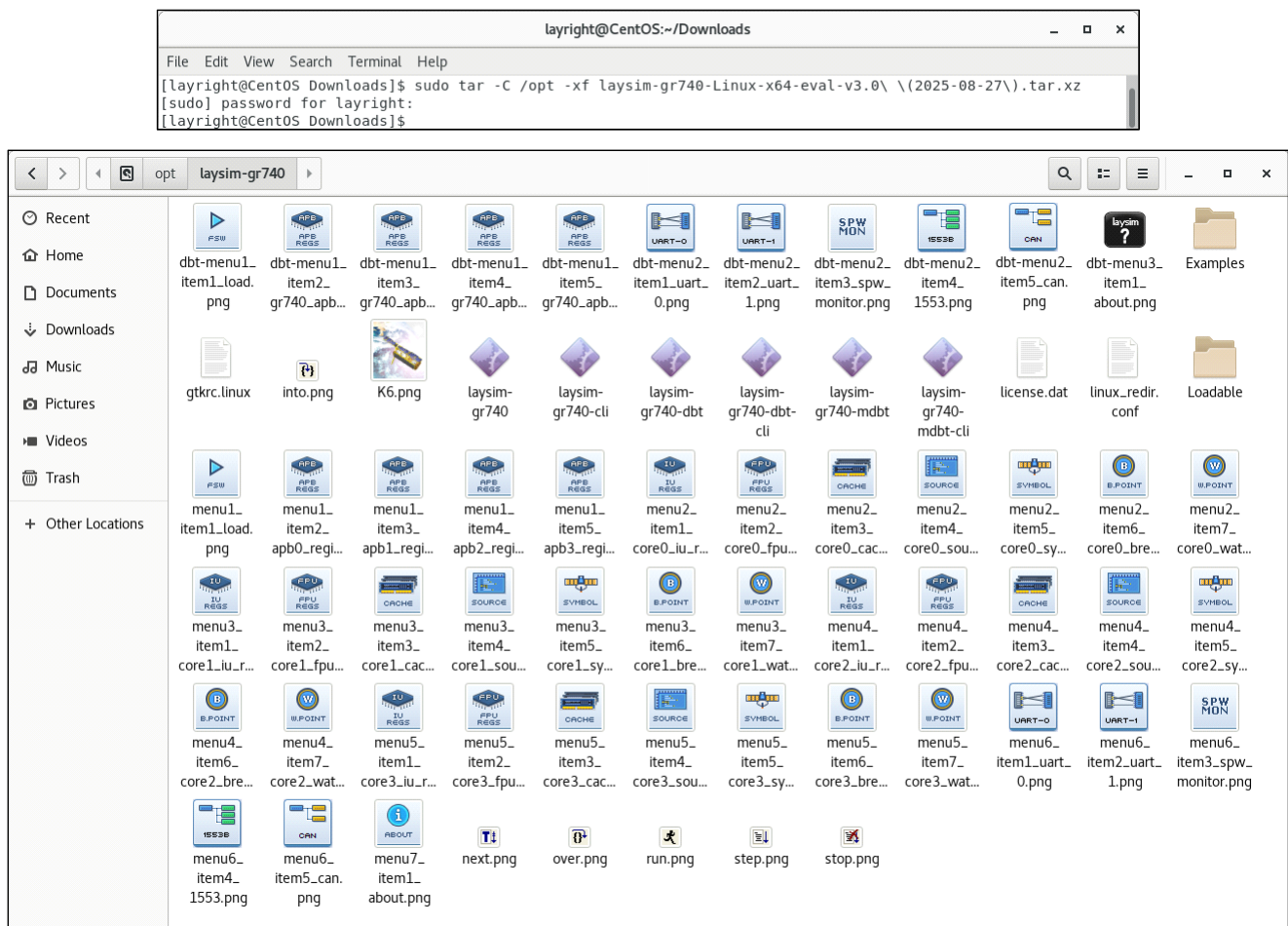
0 upgraded, 2 newly installed, 0 to remove and 0 not upgraded.
Need to get 0 B/27.0 kB of archives.
After this operation, 92.2 kB of additional disk space will be used.
Do you want to continue? [Y/n] y
Get:1 cdrom://[Debian GNU/Linux 10.3.0 _Buster_ - Official amd64 DVD Binary-1 20200208-12:08] buster/main amd64 libcanberra-gtk0 amd64 0.30-7 [12.5 kB]
Get:2 cdrom://[Debian GNU/Linux 10.3.0 _Buster_ - Official amd64 DVD Binary-1 20200208-12:08] buster/main amd64 libcanberra-gtk-module amd64 0.30-7 [14.5 kB]
Selecting previously unselected package libcanberra-gtk0:amd64.
(Reading database ... 133183 files and directories currently installed.)
Preparing to unpack .../libcanberra-gtk0_0.30-7_amd64.deb ...
Unpacking libcanberra-gtk0:amd64 (0.30-7) ...
Selecting previously unselected package libcanberra-gtk-module:amd64.
Preparing to unpack .../libcanberra-gtk-module_0.30-7_amd64.deb ...
Unpacking libcanberra-gtk-module:amd64 (0.30-7) ...
Setting up libcanberra-gtk0:amd64 (0.30-7) ...
Setting up libcanberra-gtk-module:amd64 (0.30-7) ...
Processing triggers for libc-bin (2.28-10) ...

```

2.1.2.4 Installation of laysim-GR740

laysim-GR740 should be installed to /opt/laysim-gr740 as shown in Figure 2-4, and it is needed to add execution path in .bashrc.

Figure 2-4. Install laysim-GR740



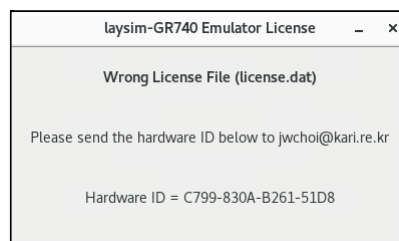
The contents of laysim-GR740 consist as below Figure 2-5.

Figure 2-5. Contents of laysim-GR740

Directory	Files	Description
/opt/laysim-gr740	laysim-gr740	GR740 quad core processor emulator based on interpretation
	laysim-gr740-cli	CLI version of laysim-gr740
	laysim-gr740-dbt	GR740 quad core processor emulator based on DBT
	laysim-gr740-dbt-cli	CLI version of laysim-gr740-dbt
	laysim-gr740-mdbt	GR740 quad core processor emulator based on MDBT
	laysim-gr740-mdbt-cli	CLI version of laysim-gr740-mdbt
./Examples	/BCC-2.2.2	BCC2 example files from BCC2 distribution
	/RCC-1.3.1	RCC example files from RCC-1.3.1 distribution
	/XtratuM	XtratuM 4.6 r1 for GR740-SMP
	/Linux	Linux SMP for GR740
	/cFS-GR740	NASA cFS for GR740
	/user-pci	user PCI examples - These examples can be used with loadable PCI module
	/user-io	user IO examples - These examples can be used with loadable IO module
./Loadable	/loadable-1553RT	loadable 1553B RT module
	/loadable-CAN-Nodes	loadable CAN node module
	/loadable-GPIO-Module	loadable GPIO module
	/loadable-IO	loadable I/O module
	/loadable-PCI	loadable PCI module
	/loadable-SPI-Module	loadable SPI module
	/loadable-SpW-Nodes	loadable SpW node module
	/loadable-UART-Module	loadable UART module

On first execution of laysim-GR740, it would request the license file as shown in Figure 2-6. The user should send the hardware ID to jwchoi@kari.re.kr for acquiring the license file. After the reception of the license file (license.dat), then the user copies it to /opt/laysim-gr740.

Figure 2-6. Need License File



2.2 Installation for Windows

2.2.1 Preparation

In order to compile BCC-2.2.2/RCC-1.3.1 examples in Windows 7 64bit platform, 5 components should be installed with designated directories as shown in Figure 2-7. It is recommended to install them sequentially from top to bottom and to use the 7zip extractor. Optionally, RTEMS5 from git can be installed.

Figure 2-7. Install Cross Tool Chains for GR740 in Windows

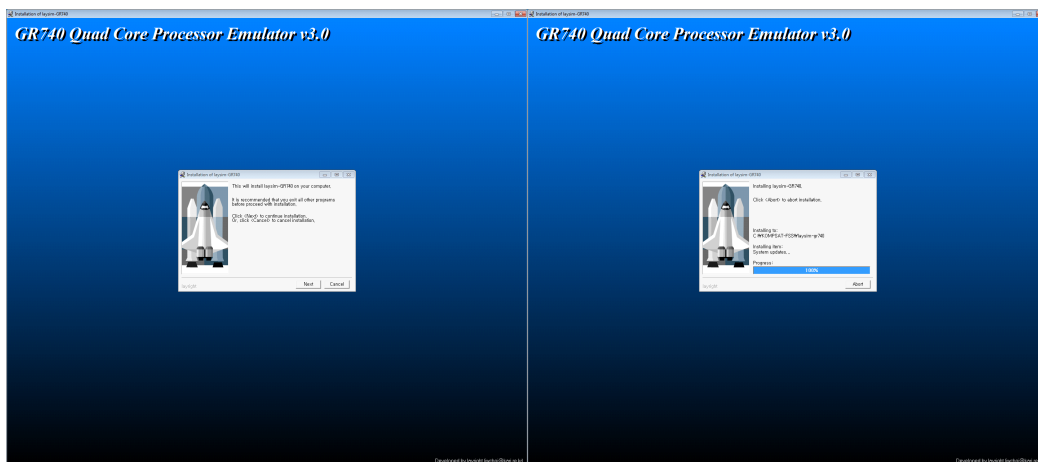
Component	Directory	File	Remark
BCC-2.2.2	C:\opt\bcc-2.2.2-gcc	bcc-2.2.2-gcc-mingw64.zip	
BCC-2.2.2 source code	C:\opt\bcc-2.2.2-gcc\src\bcc-2.2.2	bcc-2.2.2-src.tar.bz2	debugging BCC2 codes
RCC-1.3.1	C:\opt\rcc-1.3.1-gcc	sparc-rtems-5-gcc-10.2.0-1.3.1-mingw.zip	-
RCC-1.3.1 source code	C:\opt\rcc-1.3.1-gcc\src\rcc-1.3.1	rtems-5-1.3.1-src.txz	debugging RCC-1.3.1 codes
MKPROM2	C:\opt\mkprom2	mkprom2-2.0.65-mingw.tar.gz	create boot-image

After installation, the toolchain executables can be invoked from the command prompt by adding the executable directory to the PATH environment variable. (C:\opt\bcc-2.2.2-gcc\bin;C:\opt\rcc-1.3.1-gcc\bin;C:\opt\mkprom2)

2.2.2 Installation of laysim-GR740

Installation procedure of laysim-GR740 is started by executing the installer, laysim-GR740-installer.exe. The installer will install laysim-GR740 to C:\KOMPSAT-FSS\laysim-gr740, and laysim-gr740, laysim-gr740-dbt and laysim-gr740-mdbt emulators are registered in Windows Programs.

Figure 2-8. Install laysim-GR740 in Windows



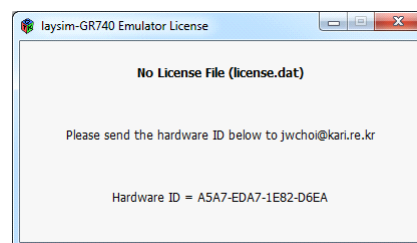
The contents of laysim-GR740 consist as below Figure 2-9. MinGW or MSYS2 may be needed to build examples of laysim-GR740.

Figure 2-9. Contents of laysim-GR740

Directory	Files	Description
C:\KOMPSAT-FSS\laysim-gr740	laysim-gr740.exe	GR740 quad core processor emulator based on interpretation
	laysim-gr740-cli.exe	CLI version of laysim-gr740
	laysim-gr740-dbt.exe	GR740 quad core processor emulator based on DBT
	laysim-gr740-dbt-cli.exe	CLI version of laysim-gr740-dbt
	laysim-gr740-mdbt.exe	GR740 quad core processor emulator based on MDBT
	laysim-gr740-mdbt-cli.exe	CLI version of laysim-gr740-mdbt
./Examples	/BCC-2.2.2	BCC2 example files from BCC2 distribution
	/RCC-1.3.1	RCC example files from RCC-1.3.1 distribution
	/XtratuM	XtratuM 4.6 r1 for GR740-SMP
	/Linux	Linux SMP for GR740
	/cFS-GR740	NASA cFS for GR740
	/user-pci	user PCI examples - These examples can be used with loadable PCI module
	/user-io	user IO examples - These examples can be used with loadable IO module
./Loadable	/loadable-1553RT	loadable 1553B RT module
	/loadable-CAN-Nodes	loadable CAN node module
	/loadable-GPIO-Module	loadable GPIO module
	/loadable-IO	loadable I/O module
	/loadable-PCI	loadable PCI module
	/loadable-SPI-Module	loadable SPI module
	/loadable-SpW-Nodes	loadable SpW node module
	/loadable-UART-Module	loadable UART module

On first execution of laysim-GR740, it would request the license file as shown in Figure 2-10. The user should send the hardware ID to jwchoi@kari.re.kr for acquiring the license file. After the reception of the license file (license.dat), then the user copies it to C:\KOMPSAT-FSS\laysim-gr740.

Figure 2-10. Need License File



2.3 Compilation of Examples

Once the BCC-2.2.2, RCC-1.3.1 and MKPROM2 are installed, examples provided by laysim-GR740 can be compiled and generated to PROM binary image as shown in Figure 2-11 and 2-12.

Figure 2–11. Build BCC–2.2.2 Examples

layright@CentOS:~/babo/Examples/BCC-2.2.2	layright@CentOS:~/babo/Examples/BCC-2.2.2
<pre>File Edit View Search Terminal Help [layright@CentOS BCC-2.2.2]\$ make sparc-gaisler-elf-gcc -g -O2 -c bdinit.c -o bdinit.o sparc-gaisler-elf-gcc -g -O3 -qbsp=gr740 -mcpu=leon3 i2cmst-eeeprom2.c -o i2cmst-eeeprom2.elf -ldrv mkp2 -g -qbsp=gr740 -mcpu=leon3 -stack 0x03FFFF00 -ramsize 65536 -romsize 8192 -sparcleon0 -memcfg1 0x0803c0ff -memcfg3 0x08000000 -dump -v -rstaddr 0xc0000000 -uart 0xFF900000 -freq 250 -baud 115200 -bdinit i2cmst-eeeprom2.elf -o i2cmst-eeeprom2.elf.mram MKPROM v2.0.65 - boot image generator for LEON applications Copyright Cobham Gaisler AB 2004-2017, all rights reserved. phead0: type: 1, off: 65536, vaddr: 0, paddr: 0, fsize: 71624, msize: 72152 section: .text at 0x0, size 66960 bytes Uncoded stream length: 66960 bytes Coded stream length: 39000 bytes Compression Ratio: 1.717 section: .rodata at 0x10590, size 3072 bytes Uncoded stream length: 3072 bytes Coded stream length: 1331 bytes Compression Ratio: 2.308 section: .data at 0x11190, size 1592 bytes Uncoded stream length: 1592 bytes Coded stream length: 680 bytes Compression Ratio: 2.341</pre>	<pre>File Edit View Search Terminal Help section: .text at 0x0, size 62704 bytes Uncoded stream length: 62704 bytes Coded stream length: 36902 bytes Compression Ratio: 1.699 section: .rodata at 0xf4f0, size 2032 bytes Uncoded stream length: 2032 bytes Coded stream length: 861 bytes Compression Ratio: 2.360 section: .data at 0xfce0, size 1592 bytes Uncoded stream length: 1592 bytes Coded stream length: 650 bytes Compression Ratio: 2.449 Creating LEON3 boot prom: spi-elf.mram sparc-gaisler-elf-gcc -O2 -g -N -T/opt/mkp2/linkprom -Ttext=0xc0000000 /opt/mkp2/lib/ut699/promcore.o /opt/mkp2/lib/ut699/prominit.o /opt/mkp2/lib/ut699/prominit_leon3.o /opt/mkp2/lib/ut699/promcrt0.o /opt/mkp2/lib/ut699/promload.o /opt/mkp2/lib/ut699/promdecomp.o -nostdlib /opt/mkp2/lib/ut699/prombdinit.o dump.s bdinit.o -o spi-elf.mram -g -qbsp=gr740 -mcpu=leon3 multidir:/lib/ut699 Success! sparc-gaisler-elf-objcopy -O binary spi-elf.mram spi-bin.mram rm xdump.s *.o dump.s [layright@CentOS BCC-2.2.2]\$</pre>

Figure 2–12. Build RCC–1.3.1 Samples

layright@CentOS:~/laysim-gr740/Examples/RCC-1.3-RC7/Samples	layright@CentOS:~/laysim-gr740/Examples/RCC-1.3-RC7/Samples
<pre>File Edit View Search Terminal Help [layright@CentOS Samples]\$ make sparc-gaisler-elf-gcc -O2 -c bdinit.c -o bdinit.o sparc-gaisler-rtms5-gcc -g -Wall -O2 -Wall -mcpu=leon3 -qbsp=gr740_smp rtms-hello.c -o rtms-hello.elf mkp2 -g -qbsp=gr740 -mcpu=leon3 -stack 0x03FFFF00 -ramsize 65536 -romsize 8192 -sparcleon0 -memcfg1 0x0803c0ff -memcfg3 0x08000000 -dump -v -rstaddr 0xc0000000 -uart 0xFF900000 -freq 250 -baud 115200 -bdinit rtms-hello.elf -o rtms-hello-elf.mram MKPROM v2.0.65 - boot image generator for LEON applications Copyright Cobham Gaisler AB 2004-2017, all rights reserved. phead0: type: 1, off: 128, vaddr: 0, paddr: 0, fsize: 164016, msize: 168128 phead1: type: 1, off: 164160, vaddr: 290c0, paddr: 290c0, fsize: 4864, msize: 20540 section: .text at 0x0, size 163920 bytes Uncoded stream length: 163920 bytes Coded stream length: 97727 bytes Compression Ratio: 1.677 section: .rtmsroset at 0x28050, size 96 bytes Uncoded stream length: 96 bytes Coded stream length: 94 bytes Compression Ratio: 1.021 section: .data at 0x290c0, size 4864 bytes Uncoded stream length: 4864 bytes</pre>	<pre>File Edit View Search Terminal Help section: .text at 0x0, size 194416 bytes Uncoded stream length: 194416 bytes Coded stream length: 115819 bytes Compression Ratio: 1.679 section: .rtmsroset at 0x2f770, size 96 bytes Uncoded stream length: 96 bytes Coded stream length: 98 bytes Compression Ratio: 0.980 section: .data at 0x30800, size 5600 bytes Uncoded stream length: 5600 bytes Coded stream length: 2573 bytes Compression Ratio: 2.176 Creating LEON3 boot prom: rtms-grcan-elf.mram sparc-gaisler-elf-gcc -O2 -g -N -T/opt/mkp2/linkprom -Ttext=0xc0000000 /opt/mkp2/lib/ut699/promcore.o /opt/mkp2/lib/ut699/prominit.o /opt/mkp2/lib/ut699/prominit_leon3.o /opt/mkp2/lib/ut699/promcrt0.o /opt/mkp2/lib/ut699/promload.o /opt/mkp2/lib/ut699/promdecomp.o -nostdlib /opt/mkp2/lib/ut699/prombdinit.o dump.s bdinit.o -o rtms-grcan-elf.mram -qbsp=gr740 -mcpu=leon3 multidir:/lib/ut699 Success! sparc-gaisler-rtms5-objcopy -O binary rtms-grcan-elf.mram rtms-grcan-bin.mram [layright@CentOS Samples]\$</pre>

3. laysim-GR740 Operation

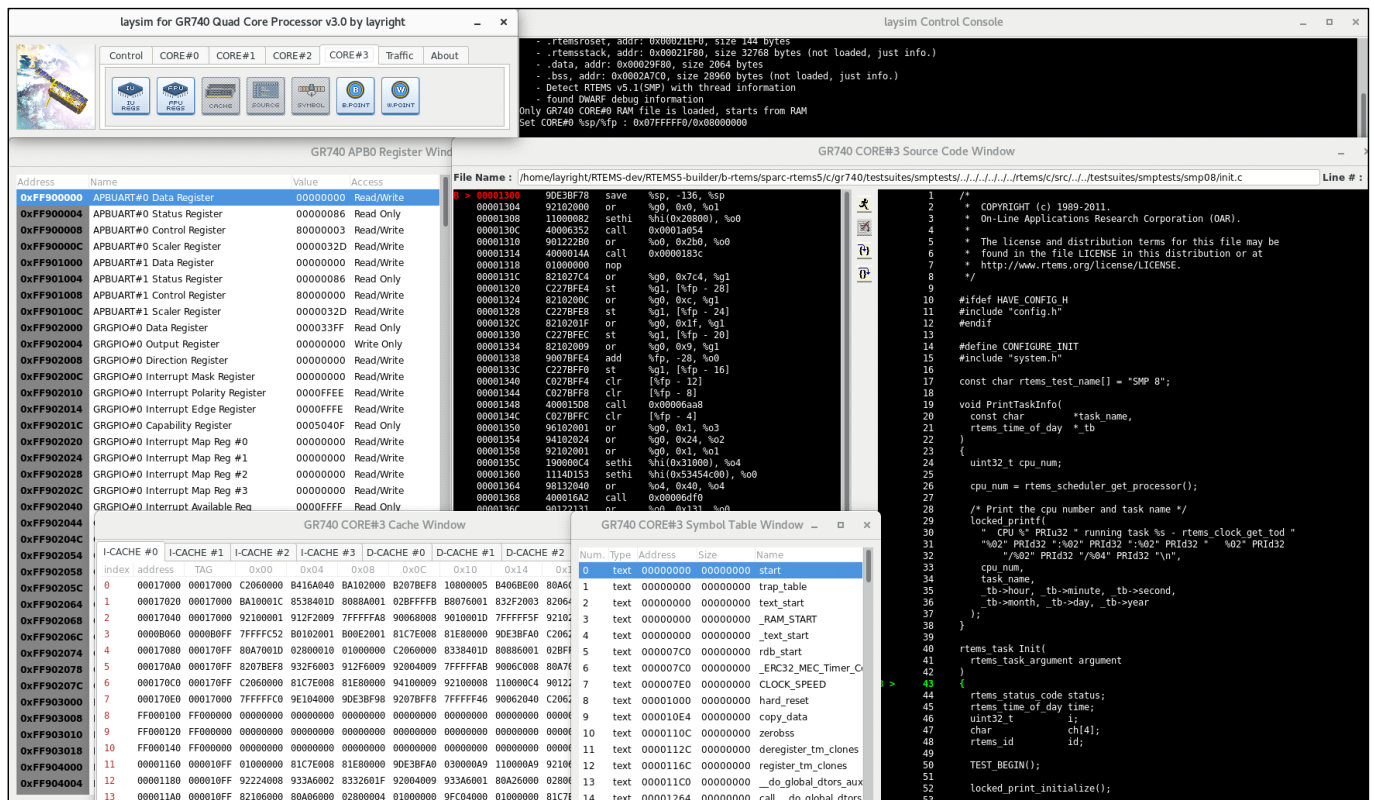
3.1 Emulators of laysim-GR740

laysim-GR740 provides six types of emulators: `laysim-gr740/laysim-gr740-cli`, `laysim-gr740-dbt/laysim-gr740-dbt-cli` and `laysim-gr740-mdbt/laysim-gr740-mdbt-cli`.

3.1.1 laysim-gr740

laysim-gr740 is a GUI-based, cycle-accurate emulator for the GR740 quad core processor using an interpretation method. It includes an embedded source-level debugger and supports the MMU of GR740. Figure 3-1 shows the overall execution of laysim-gr740.

Figure 3-1. laysim-gr740 Interfaces



3.1.2 laysim-gr740-cli

laysim-gr740-cli is a CLI version of laysim-gr740. It provides the same functionality of laysim-gr740 without a GUI, and it supports the debugging with GDB. laysim-gr740-cli only supports for the code coverage in the same format as TSIM3-LEON4. '-cov' start-up option should be enabled for the code coverage.

Figure 3-2. laysim-gr740-cli Interfaces

laysim-gr740-cli in Linux	laysim-gr740-cli in Windows
<pre>[layright@CentOS7 smptests]\$ laysim-gr740-cli -core0 smp08.elf laysim for GR740 Quad Core Processor CLI v3.0 by layright CORE#0 RAM file is loaded - smp08.elf - .text, addr: 0x00000000, size 138992 bytes - .rtmsroset, addr: 0x00021EF0, size 144 bytes</pre>	<pre>PS C:\laysim-gr740\RTMS5.1\smptests> laysim-gr740-cli -core0 smp08.elf laysim for GR740 Quad Core Processor CLI v3.0 by layright CORE#0 RAM file is loaded - smp08.elf - .text, addr: 0x00000000, size 138992 bytes - .rtmsroset, addr: 0x00021EF0, size 144 bytes</pre>

```
- .rtmsstack, addr: 0x0021F80, size 32768 bytes (not loaded, just info.)
- .data, addr: 0x0029F80, size 2064 bytes
- .bss, addr: 0x002A7C0, size 28960 bytes (not loaded, just info.)
- Detect RTEMS v5.1(SMP) with thread information
Only GR740 CORE#0 RAM file is loaded, starts from RAM
Set CORE#0 %sp/%fp : 0x07FFFFF0/0x08000000
laysim> run
```

```
*** BEGIN OF TEST SMP 8 ***
*** TEST VERSION: 5.1.0
*** TEST STATE: EXPECTED_PASS
*** TEST BUILD: RTEMS_NETWORKING RTEMS_POSIX_API RTEMS_SMP
*** TEST TOOLS: 7.5.0 20191114 (RTEMS 5, RSB 5.1, Newlib 7947581)
CPU 3 running task Init - rtems_clock_get_tod 09:00:00 12/31/1988
CPU 2 running task TA01 - rtems_clock_get_tod 09:00:00 12/31/1988
CPU 2 running task TA05 - rtems_clock_get_tod 09:00:00 12/31/1988
CPU 2 running task TA06 - rtems_clock_get_tod 09:00:00 12/31/1988
CPU 2 running task TA07 - rtems_clock_get_tod 09:00:00 12/31/1988
CPU 2 running task TA08 - rtems_clock_get_tod 09:00:00 12/31/1988
CPU 2 running task TA09 - rtems_clock_get_tod 09:00:00 12/31/1988
CPU 2 running task TA10 - rtems_clock_get_tod 09:00:00 12/31/1988
CPU 2 running task TA11 - rtems_clock_get_tod 09:00:00 12/31/1988
CPU 2 running task TA12 - rtems_clock_get_tod 09:00:00 12/31/1988
CPU 3 running task TA04 - rtems_clock_get_tod 09:00:00 12/31/1988
CPU 0 running task TA03 - rtems_clock_get_tod 09:00:00 12/31/1988
CPU 1 running task TA02 - rtems_clock_get_tod 09:00:00 12/31/1988
CPU 1 running task TA01 - rtems_clock_get_tod 09:00:05 12/31/1988
CPU 1 running task TA01 - rtems_clock_get_tod 09:00:10 12/31/1988
CPU 1 running task TA02 - rtems_clock_get_tod 09:00:10 12/31/1988
CPU 1 running task TA01 - rtems_clock_get_tod 09:00:15 12/31/1988
CPU 1 running task TA03 - rtems_clock_get_tod 09:00:15 12/31/1988
CPU 1 running task TA01 - rtems_clock_get_tod 09:00:20 12/31/1988
CPU 1 running task TA04 - rtems_clock_get_tod 09:00:20 12/31/1988
CPU 1 running task TA02 - rtems_clock_get_tod 09:00:20 12/31/1988
CPU 1 running task TA05 - rtems_clock_get_tod 09:00:25 12/31/1988
CPU 1 running task TA01 - rtems_clock_get_tod 09:00:25 12/31/1988
CPU 1 running task TA06 - rtems_clock_get_tod 09:00:30 12/31/1988
CPU 0 running task TA01 - rtems_clock_get_tod 09:00:30 12/31/1988
CPU 0 running task TA03 - rtems_clock_get_tod 09:00:30 12/31/1988
CPU 0 running task TA02 - rtems_clock_get_tod 09:00:30 12/31/1988

*** END OF TEST SMP 8 ***
```

```
CORE#0 halts because of Error Mode [8790298027cycles/9.11345sec]
laysim> perf
```

```
Performance statistics for CORE#0
Cycles : 8790298027
Instructions : 55004242
Overall CPI : 1.45
CORE#0 performance (250MHz) : 171.84 MOPS (171.84 MIPS, 0.00 MFLOPS)
Simulated time : 35.16 s
Processor utilization : 0.91 %

Performance statistics for CORE#1
Cycles : 8790298034
Instructions : 90760399
Overall CPI : 1.56
CORE#1 performance (250MHz) : 160.12 MOPS (160.12 MIPS, 0.00 MFLOPS)
Simulated time : 35.16 s
Processor utilization : 1.61 %

Performance statistics for CORE#2
Cycles : 8790298044
Instructions : 18225405
Overall CPI : 2.44
CORE#2 performance (250MHz) : 102.56 MOPS (102.56 MIPS, 0.00 MFLOPS)
Simulated time : 35.16 s
Processor utilization : 0.51 %

Performance statistics for CORE#3
Cycles : 8790298045
Instructions : 48215245
Overall CPI : 1.51
CORE#3 performance (250MHz) : 165.61 MOPS (165.61 MIPS, 0.00 MFLOPS)
Simulated time : 35.16 s
Processor utilization : 0.83 %

Performance of laysim-gr740
Real-time performance : 385.82 %
laysim-gr740 performance : 23.28 MIPS
Wall clock : 9.11 s
laysim>
```

```
- .rtmsstack, addr: 0x0021F80, size 32768 bytes (not loaded, just info.)
- .data, addr: 0x0029F80, size 2064 bytes
- .bss, addr: 0x002A7C0, size 28960 bytes (not loaded, just info.)
- Detect RTEMS v5.1(SMP) with thread information
Only GR740 CORE#0 RAM file is loaded, starts from RAM
Set CORE#0 %sp/%fp : 0x07FFFFF0/0x08000000
laysim> run
```

```
*** BEGIN OF TEST SMP 8 ***
*** TEST VERSION: 5.1.0
*** TEST STATE: EXPECTED_PASS
*** TEST BUILD: RTEMS_NETWORKING RTEMS_POSIX_API RTEMS_SMP
*** TEST TOOLS: 7.5.0 20191114 (RTEMS 5, RSB 5.1, Newlib 7947581)
CPU 3 running task Init - rtems_clock_get_tod 09:00:00 12/31/1988
CPU 2 running task TA01 - rtems_clock_get_tod 09:00:00 12/31/1988
CPU 2 running task TA05 - rtems_clock_get_tod 09:00:00 12/31/1988
CPU 2 running task TA06 - rtems_clock_get_tod 09:00:00 12/31/1988
CPU 2 running task TA07 - rtems_clock_get_tod 09:00:00 12/31/1988
CPU 2 running task TA08 - rtems_clock_get_tod 09:00:00 12/31/1988
CPU 2 running task TA09 - rtems_clock_get_tod 09:00:00 12/31/1988
CPU 2 running task TA10 - rtems_clock_get_tod 09:00:00 12/31/1988
CPU 2 running task TA11 - rtems_clock_get_tod 09:00:00 12/31/1988
CPU 2 running task TA12 - rtems_clock_get_tod 09:00:00 12/31/1988
CPU 3 running task TA04 - rtems_clock_get_tod 09:00:00 12/31/1988
CPU 0 running task TA03 - rtems_clock_get_tod 09:00:00 12/31/1988
CPU 1 running task TA02 - rtems_clock_get_tod 09:00:00 12/31/1988
CPU 1 running task TA01 - rtems_clock_get_tod 09:00:05 12/31/1988
CPU 1 running task TA01 - rtems_clock_get_tod 09:00:10 12/31/1988
CPU 1 running task TA02 - rtems_clock_get_tod 09:00:10 12/31/1988
CPU 1 running task TA01 - rtems_clock_get_tod 09:00:15 12/31/1988
CPU 1 running task TA03 - rtems_clock_get_tod 09:00:15 12/31/1988
CPU 1 running task TA01 - rtems_clock_get_tod 09:00:20 12/31/1988
CPU 1 running task TA04 - rtems_clock_get_tod 09:00:20 12/31/1988
CPU 1 running task TA02 - rtems_clock_get_tod 09:00:20 12/31/1988
CPU 1 running task TA05 - rtems_clock_get_tod 09:00:25 12/31/1988
CPU 1 running task TA01 - rtems_clock_get_tod 09:00:25 12/31/1988
CPU 1 running task TA06 - rtems_clock_get_tod 09:00:30 12/31/1988
CPU 0 running task TA01 - rtems_clock_get_tod 09:00:30 12/31/1988
CPU 0 running task TA03 - rtems_clock_get_tod 09:00:30 12/31/1988
CPU 0 running task TA02 - rtems_clock_get_tod 09:00:30 12/31/1988

*** END OF TEST SMP 8 ***
```

```
CORE#0 halts because of Error Mode [8790298027cycles/8.08081sec]
laysim> perf
```

```
Performance statistics for CORE#0
Cycles : 8790298027
Instructions : 55004242
Overall CPI : 1.45
CORE#0 performance (250MHz) : 171.84 MOPS (171.84 MIPS, 0.00 MFLOPS)
Simulated time : 35.16 s
Processor utilization : 0.91 %

Performance statistics for CORE#1
Cycles : 8790298034
Instructions : 90760399
Overall CPI : 1.56
CORE#1 performance (250MHz) : 160.12 MOPS (160.12 MIPS, 0.00 MFLOPS)
Simulated time : 35.16 s
Processor utilization : 1.61 %

Performance statistics for CORE#2
Cycles : 8790298044
Instructions : 18225405
Overall CPI : 2.44
CORE#2 performance (250MHz) : 102.56 MOPS (102.56 MIPS, 0.00 MFLOPS)
Simulated time : 35.16 s
Processor utilization : 0.51 %

Performance statistics for CORE#3
Cycles : 8790298045
Instructions : 48215245
Overall CPI : 1.51
CORE#3 performance (250MHz) : 165.61 MOPS (165.61 MIPS, 0.00 MFLOPS)
Simulated time : 35.16 s
Processor utilization : 0.83 %

Performance of laysim-gr740
Real-time performance : 435.12 %
laysim-gr740 performance : 26.26 MIPS
Wall clock : 8.08 s
laysim>
```

3.1.3 laysim-gr740-dbt

laysim-gr740-dbt is a GUI-based high-performance, cycle-approximate processor emulator that uses dynamic binary translation (DBT). It provides similar interfaces of laysim-gr740 without source-level debugger and L1/L2 cache for speed-up. Figure 3-3 shows the overall execution of laysim-gr740-dbt.

3.1.4 laysim-gr740-dbt-cli

laysim-gr740-dbt-cli is a CLI version of laysim-gr740-dbt. It provides the same functionality of laysim-gr740-dbt without a GUI, and it supports the debugging with GDB.

Figure 3–3. laysim-gr740-dbt Interfaces

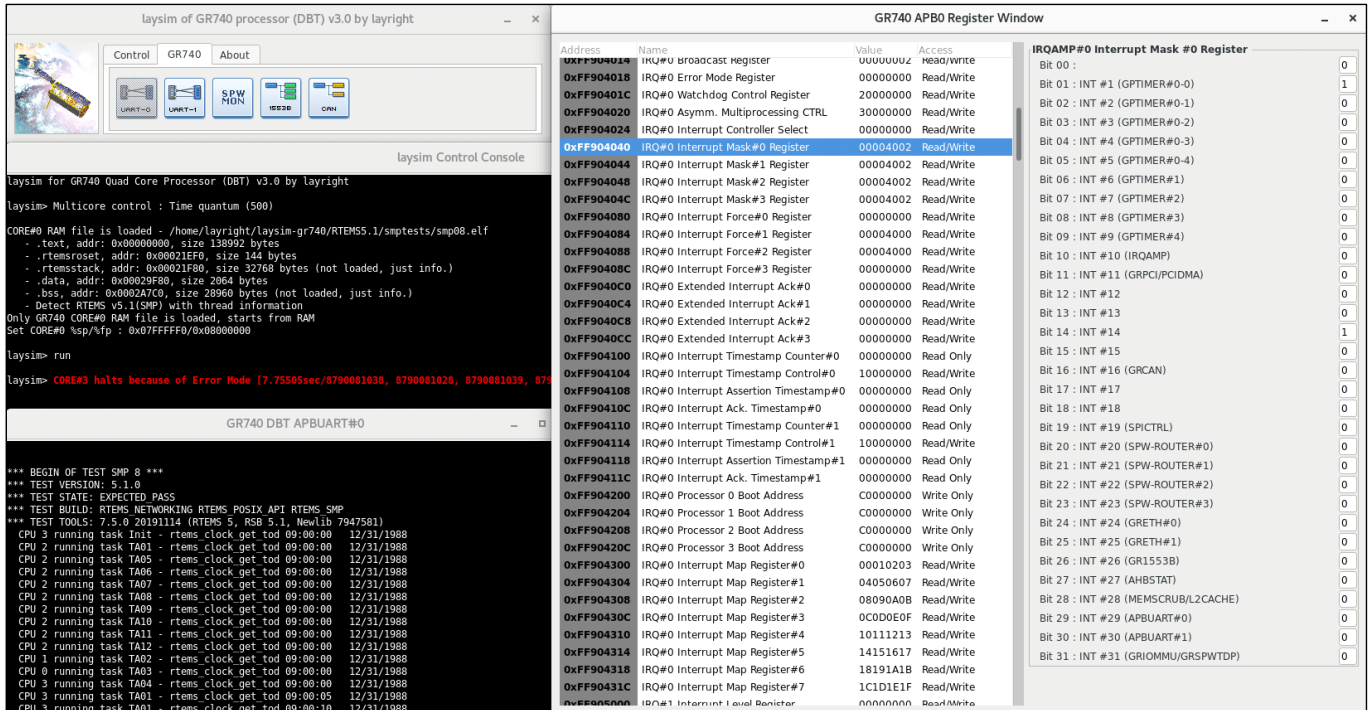


Figure 3–4. laysim-gr740-dbt-cli Interfaces

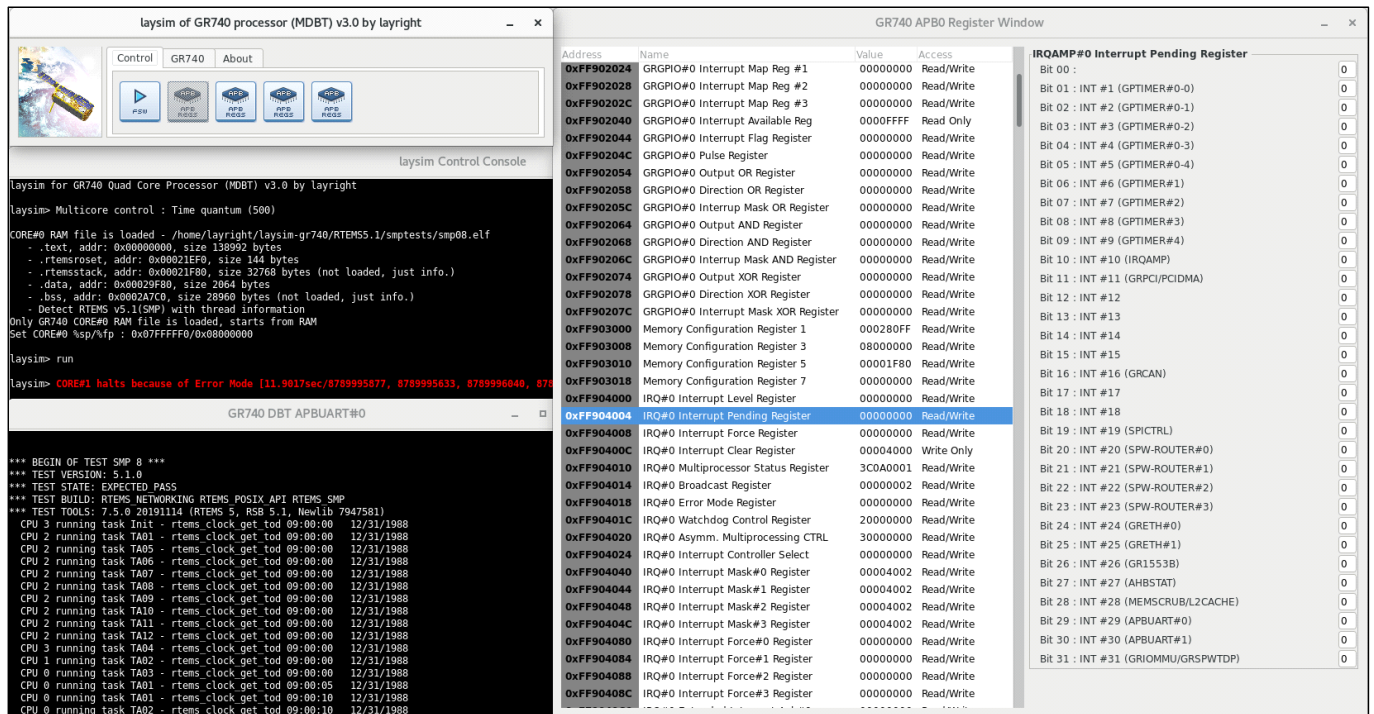
laysim-gr740-dbt-cli in Linux	laysim-gr740-dbt-cli in Windows
<pre>[layright@CentOS smp08]\$ laysim-gr740-dbt-cli -core0 smp08.elf laysim for GR740 Quad Core Processor (DBT) CLI v3.0 by layright Multicore control : Time quantum (500) CORE#0 RAM file is loaded - smp08.elf - .text, addr: 0x00000000, size 138992 bytes - .rtmsroset, addr: 0x00021EF0, size 144 bytes - .rtmsstack, addr: 0x00021F80, size 32768 bytes (not loaded, just info.) - .data, addr: 0x00029F80, size 2064 bytes - .bss, addr: 0x0002A7C0, size 28960 bytes (not loaded, just info.) - Detect RTEMS v5.1(SMP) with thread information Only GR740 CORE#0 RAM file is loaded, starts from RAM Set CORE#0 %sp/%fp : 0x07FFFFF0/0x08000000 laysim> run *** BEGIN OF TEST SMP 8 *** *** TEST VERSION: 5.1.0 *** TEST STATE: EXPECTED PASS *** TEST BUILD: RTEMS_NETWORKING RTEMS_POSIX_API RTEMS_SMP *** TEST TOOLS: 7.5.0 20191114 (RTEMS 5, RSB 5.1, NewLib 7947581) CPU 3 running task Init - rtems_clock_get_tod 09:00:00 12/31/1988 CPU 2 running task TA01 - rtems_clock_get_tod 09:00:00 12/31/1988 CPU 2 running task TA05 - rtems_clock_get_tod 09:00:00 12/31/1988 CPU 2 running task TA06 - rtems_clock_get_tod 09:00:00 12/31/1988 CPU 2 running task TA07 - rtems_clock_get_tod 09:00:00 12/31/1988 CPU 2 running task TA08 - rtems_clock_get_tod 09:00:00 12/31/1988 CPU 2 running task TA09 - rtems_clock_get_tod 09:00:00 12/31/1988 CPU 2 running task TA10 - rtems_clock_get_tod 09:00:00 12/31/1988 CPU 2 running task TA11 - rtems_clock_get_tod 09:00:00 12/31/1988 CPU 2 running task TA12 - rtems_clock_get_tod 09:00:00 12/31/1988 CPU 1 running task TA02 - rtems_clock_get_tod 09:00:00 12/31/1988 CPU 0 running task TA03 - rtems_clock_get_tod 09:00:00 12/31/1988 CPU 3 running task TA04 - rtems_clock_get_tod 09:00:00 12/31/1988 CPU 3 running task TA01 - rtems_clock_get_tod 09:00:05 12/31/1988 CPU 3 running task TA01 - rtems_clock_get_tod 09:00:10 12/31/1988 CPU 3 running task TA02 - rtems_clock_get_tod 09:00:10 12/31/1988 CPU 3 running task TA01 - rtems_clock_get_tod 09:00:15 12/31/1988 CPU 3 running task TA01 - rtems_clock_get_tod 09:00:20 12/31/1988 CPU 3 running task TA02 - rtems_clock_get_tod 09:00:20 12/31/1988 CPU 3 running task TA04 - rtems_clock_get_tod 09:00:20 12/31/1988 CPU 3 running task TA05 - rtems_clock_get_tod 09:00:25 12/31/1988 CPU 3 running task TA01 - rtems_clock_get_tod 09:00:25 12/31/1988 CPU 3 running task TA06 - rtems_clock_get_tod 09:00:30 12/31/1988 CPU 0 running task TA01 - rtems_clock_get_tod 09:00:30 12/31/1988 CPU 0 running task TA02 - rtems_clock_get_tod 09:00:30 12/31/1988 CPU 3 running task TA03 - rtems_clock_get_tod 09:00:30 12/31/1988 *** END OF TEST SMP 8 *** CORE#3 halts because of Error Mode [0.983817sec/8790081038, 8790081028, 8790081039, 8790081039] laysim> perf Performance statistics for CORE#0 Cycles : 8790081038</pre>	<pre>PS C:\laysim-gr740\RTM55.1\smp08> laysim-gr740-dbt-cli -core0 smp08.elf laysim for GR740 Quad Core Processor (DBT) CLI v3.0 by layright Multicore control : Time quantum (500) CORE#0 RAM file is loaded - smp08.elf - .text, addr: 0x00000000, size 138992 bytes - .rtmsroset, addr: 0x00021EF0, size 144 bytes - .rtmsstack, addr: 0x00021F80, size 32768 bytes (not loaded, just info.) - .data, addr: 0x00029F80, size 2064 bytes - .bss, addr: 0x0002A7C0, size 28960 bytes (not loaded, just info.) - Detect RTEMS v5.1(SMP) with thread information Only GR740 CORE#0 RAM file is loaded, starts from RAM Set CORE#0 %sp/%fp : 0x07FFFFF0/0x08000000 laysim> run *** BEGIN OF TEST SMP 8 *** *** TEST VERSION: 5.1.0 *** TEST STATE: EXPECTED_PASS *** TEST BUILD: RTEMS_NETWORKING RTEMS_POSIX_API RTEMS_SMP *** TEST TOOLS: 7.5.0 20191114 (RTEMS 5, RSB 5.1, NewLib 7947581) CPU 3 running task Init - rtems_clock_get_tod 09:00:00 12/31/1988 CPU 2 running task TA01 - rtems_clock_get_tod 09:00:00 12/31/1988 CPU 2 running task TA05 - rtems_clock_get_tod 09:00:00 12/31/1988 CPU 2 running task TA06 - rtems_clock_get_tod 09:00:00 12/31/1988 CPU 2 running task TA07 - rtems_clock_get_tod 09:00:00 12/31/1988 CPU 2 running task TA08 - rtems_clock_get_tod 09:00:00 12/31/1988 CPU 2 running task TA09 - rtems_clock_get_tod 09:00:00 12/31/1988 CPU 2 running task TA10 - rtems_clock_get_tod 09:00:00 12/31/1988 CPU 2 running task TA11 - rtems_clock_get_tod 09:00:00 12/31/1988 CPU 2 running task TA12 - rtems_clock_get_tod 09:00:00 12/31/1988 CPU 1 running task TA02 - rtems_clock_get_tod 09:00:00 12/31/1988 CPU 0 running task TA03 - rtems_clock_get_tod 09:00:00 12/31/1988 CPU 3 running task TA04 - rtems_clock_get_tod 09:00:00 12/31/1988 CPU 3 running task TA01 - rtems_clock_get_tod 09:00:05 12/31/1988 CPU 3 running task TA01 - rtems_clock_get_tod 09:00:10 12/31/1988 CPU 3 running task TA02 - rtems_clock_get_tod 09:00:10 12/31/1988 CPU 3 running task TA01 - rtems_clock_get_tod 09:00:15 12/31/1988 CPU 3 running task TA01 - rtems_clock_get_tod 09:00:20 12/31/1988 CPU 3 running task TA02 - rtems_clock_get_tod 09:00:20 12/31/1988 CPU 3 running task TA04 - rtems_clock_get_tod 09:00:20 12/31/1988 CPU 3 running task TA05 - rtems_clock_get_tod 09:00:25 12/31/1988 CPU 3 running task TA01 - rtems_clock_get_tod 09:00:25 12/31/1988 CPU 3 running task TA06 - rtems_clock_get_tod 09:00:30 12/31/1988 CPU 0 running task TA01 - rtems_clock_get_tod 09:00:30 12/31/1988 CPU 0 running task TA02 - rtems_clock_get_tod 09:00:30 12/31/1988 CPU 3 running task TA03 - rtems_clock_get_tod 09:00:30 12/31/1988 *** END OF TEST SMP 8 *** CORE#3 halts because of Error Mode [1.1388sec/8790081038, 8790081028, 8790081039, 8790081039] laysim> perf Performance statistics for CORE#0 Cycles : 8790081038</pre>

Instructions : 55223902 CORE#0 performance (250MHz) : 1.57 MIPS Simulated time : 35.16 s Performance statistics for CORE#1 Cycles : 8790081028 Instructions : 44268401 CORE#1 performance (250MHz) : 1.26 MIPS Simulated time : 35.16 s Performance statistics for CORE#2 Cycles : 8790081039 Instructions : 18241461 CORE#2 performance (250MHz) : 0.52 MIPS Simulated time : 35.16 s Performance statistics for CORE#3 Cycles : 8790080631 Instructions : 86783857 CORE#3 performance (250MHz) : 2.47 MIPS Simulated time : 35.16 s Performance of laysim-gr740 Real-time performance : 3573.81 % laysim-gr740 performance : 207.88 MIPS Wall clock : 0.98 s laysim>	Instructions : 55223902 CORE#0 performance (250MHz) : 1.57 MIPS Simulated time : 35.16 s Performance statistics for CORE#1 Cycles : 8790081028 Instructions : 44268401 CORE#1 performance (250MHz) : 1.26 MIPS Simulated time : 35.16 s Performance statistics for CORE#2 Cycles : 8790081039 Instructions : 18241461 CORE#2 performance (250MHz) : 0.52 MIPS Simulated time : 35.16 s Performance statistics for CORE#3 Cycles : 8790080631 Instructions : 86783857 CORE#3 performance (250MHz) : 2.47 MIPS Simulated time : 35.16 s Performance of laysim-gr740 Real-time performance : 3087.48 % laysim-gr740 performance : 179.59 MIPS Wall clock : 1.14 s laysim>
--	--

3.1.5 laysim-gr740-mdbt

laysim-gr740-mdbt (Multi-Thread DBT) is a parallel implementation of laysim-gr740-dbt, where it adopts the multi-thread framework with DBT so it can satisfy the real-time performance of GR740 on a high-end PC such as Intel i9-12900K. laysim-gr740-mdbt provides the similar interfaces of laysim-gr740-dbt. Figure 3-5 shows the overall execution of laysim-gr740-mdbt.

Figure 3-5. laysim-gr740-mdbt Interfaces



3.1.6 laysim-gr740-mdbt-cli

laysim-gr740-mdbt-cli is a CLI version of laysim-gr740-mdbt. It provides the same functionality of laysim-gr740-mdbt without a GUI, and it supports the debugging with GDB.

Figure 3-6. laysim-gr740-mdbt-cli Interfaces

laysim-gr740-mdbt-cli in Linux	laysim-gr740-mdbt-cli in Windows

<pre> [layright@CentOS smptests]\$ laysim-gr740-mdbt-cli -core0 smp08.elf laysim for GR740 Quad Core Processor (MDBT) CLI v3.0 by layright Multicore control : Time quantum (500) CORE#0 RAM file is loaded - smp08.elf - .text, addr: 0x00000000, size 138992 bytes - .rtmsroset, addr: 0x00021EF0, size 144 bytes - .rtmsstack, addr: 0x00021F80, size 32768 bytes (not loaded, just info.) - .data, addr: 0x00029F80, size 2064 bytes - .bss, addr: 0x0002A7C0, size 28960 bytes (not loaded, just info.) - Detect RTEMS v5.1(SMP) with thread information Only GR740 CORE#0 RAM file is loaded, starts from RAM Set CORE#0 %sp/%fp : 0x07FFFFFF0/0x08000000 laysim> run *** BEGIN OF TEST SMP 8 *** *** TEST VERSION: 5.1.0 *** TEST STATE: EXPECTED_PASS *** TEST BUILD: RTEMS_NETWORKING RTEMS_POSIX_API RTEMS_SMP *** TEST TOOLS: 7.5.0 20191114 (RTEMS 5, RSB 5.1, Newlib 7947581) CPU 3 running task Init - rtems_clock_gettod 09:00:00 12/31/1988 CPU 2 running task TA01 - rtems_clock_gettod 09:00:00 12/31/1988 CPU 2 running task TA05 - rtems_clock_gettod 09:00:00 12/31/1988 CPU 2 running task TA06 - rtems_clock_gettod 09:00:00 12/31/1988 CPU 2 running task TA07 - rtems_clock_gettod 09:00:00 12/31/1988 CPU 2 running task TA08 - rtems_clock_gettod 09:00:00 12/31/1988 CPU 2 running task TA09 - rtems_clock_gettod 09:00:00 12/31/1988 CPU 2 running task TA10 - rtems_clock_gettod 09:00:00 12/31/1988 CPU 2 running task TA11 - rtems_clock_gettod 09:00:00 12/31/1988 CPU 2 running task TA12 - rtems_clock_gettod 09:00:00 12/31/1988 CPU 0 running task TA03 - rtems_clock_gettod 09:00:00 12/31/1988 CPU 1 running task TA02 - rtems_clock_gettod 09:00:00 12/31/1988 CPU 3 running task TA04 - rtems_clock_gettod 09:00:00 12/31/1988 CPU 3 running task TA01 - rtems_clock_gettod 09:00:05 12/31/1988 CPU 3 running task TA01 - rtems_clock_gettod 09:00:10 12/31/1988 CPU 3 running task TA02 - rtems_clock_gettod 09:00:10 12/31/1988 CPU 3 running task TA01 - rtems_clock_gettod 09:00:15 12/31/1988 CPU 3 running task TA03 - rtems_clock_gettod 09:00:15 12/31/1988 CPU 3 running task TA01 - rtems_clock_gettod 09:00:20 12/31/1988 CPU 3 running task TA02 - rtems_clock_gettod 09:00:20 12/31/1988 CPU 3 running task TA04 - rtems_clock_gettod 09:00:20 12/31/1988 CPU 3 running task TA05 - rtems_clock_gettod 09:00:25 12/31/1988 CPU 3 running task TA01 - rtems_clock_gettod 09:00:25 12/31/1988 CPU 3 running task TA06 - rtems_clock_gettod 09:00:30 12/31/1988 CPU 1 running task TA01 - rtems_clock_gettod 09:00:30 12/31/1988 CPU 1 running task TA03 - rtems_clock_gettod 09:00:30 12/31/1988 CPU 1 running task TA02 - rtems_clock_gettod 09:00:30 12/31/1988 *** END OF TEST SMP 8 *** CORE#1 halts because of Error Mode [8.09717sec/8789994746, 8789994632, 8789995040, 8789 995069cycles] laysim> perf Performance statistics for CORE#0 Cycles : 8789995563 Instructions : 42275871 CORE#0 performance (250MHz) : 1.20 MIPS Simulated time : 35.16 s Performance statistics for CORE#1 Cycles : 8789994632 Instructions : 52842559 CORE#1 performance (250MHz) : 1.50 MIPS Simulated time : 35.16 s Performance statistics for CORE#2 Cycles : 8789995451 Instructions : 18183743 CORE#2 performance (250MHz) : 0.52 MIPS Simulated time : 35.16 s Performance statistics for CORE#3 Cycles : 8789995450 Instructions : 118098260 CORE#3 performance (250MHz) : 3.36 MIPS Simulated time : 35.16 s Performance of laysim-gr740 Real-time performance : 434.22 % laysim-gr740 performance : 28.58 MIPS Wall clock : 8.10 s laysim> </pre>	<pre> PS C:\laysim-gr740\RTM55.1\smptests> laysim-gr740-mdbt-cli -core0 smp08.elf laysim for GR740 Quad Core Processor (MDBT) CLI v3.0 by layright Multicore control : Time quantum (500) CORE#0 RAM file is loaded - smp08.elf - .text, addr: 0x00000000, size 138992 bytes - .rtmsroset, addr: 0x00021EF0, size 144 bytes - .rtmsstack, addr: 0x00021F80, size 32768 bytes (not loaded, just info.) - .data, addr: 0x00029F80, size 2064 bytes - .bss, addr: 0x0002A7C0, size 28960 bytes (not loaded, just info.) - Detect RTEMS v5.1(SMP) with thread information Only GR740 CORE#0 RAM file is loaded, starts from RAM Set CORE#0 %sp/%fp : 0x07FFFFFF0/0x08000000 laysim> run *** BEGIN OF TEST SMP 8 *** *** TEST VERSION: 5.1.0 *** TEST STATE: EXPECTED_PASS *** TEST BUILD: RTEMS_NETWORKING RTEMS_POSIX_API RTEMS_SMP *** TEST TOOLS: 7.5.0 20191114 (RTEMS 5, RSB 5.1, Newlib 7947581) CPU 3 running task Init - rtems_clock_gettod 09:00:00 12/31/1988 CPU 2 running task TA01 - rtems_clock_gettod 09:00:00 12/31/1988 CPU 2 running task TA05 - rtems_clock_gettod 09:00:00 12/31/1988 CPU 2 running task TA06 - rtems_clock_gettod 09:00:00 12/31/1988 CPU 2 running task TA07 - rtems_clock_gettod 09:00:00 12/31/1988 CPU 2 running task TA08 - rtems_clock_gettod 09:00:00 12/31/1988 CPU 2 running task TA09 - rtems_clock_gettod 09:00:00 12/31/1988 CPU 2 running task TA10 - rtems_clock_gettod 09:00:00 12/31/1988 CPU 2 running task TA11 - rtems_clock_gettod 09:00:00 12/31/1988 CPU 2 running task TA12 - rtems_clock_gettod 09:00:00 12/31/1988 CPU 1 running task TA02 - rtems_clock_gettod 09:00:00 12/31/1988 CPU 0 running task TA03 - rtems_clock_gettod 09:00:00 12/31/1988 CPU 3 running task TA04 - rtems_clock_gettod 09:00:00 12/31/1988 CPU 3 running task TA01 - rtems_clock_gettod 09:00:05 12/31/1988 CPU 3 running task TA01 - rtems_clock_gettod 09:00:10 12/31/1988 CPU 3 running task TA02 - rtems_clock_gettod 09:00:10 12/31/1988 CPU 3 running task TA01 - rtems_clock_gettod 09:00:15 12/31/1988 CPU 3 running task TA03 - rtems_clock_gettod 09:00:15 12/31/1988 CPU 3 running task TA01 - rtems_clock_gettod 09:00:20 12/31/1988 CPU 3 running task TA02 - rtems_clock_gettod 09:00:20 12/31/1988 CPU 3 running task TA04 - rtems_clock_gettod 09:00:20 12/31/1988 CPU 3 running task TA05 - rtems_clock_gettod 09:00:25 12/31/1988 CPU 3 running task TA01 - rtems_clock_gettod 09:00:25 12/31/1988 CPU 3 running task TA06 - rtems_clock_gettod 09:00:30 12/31/1988 CPU 0 running task TA01 - rtems_clock_gettod 09:00:30 12/31/1988 CPU 0 running task TA02 - rtems_clock_gettod 09:00:30 12/31/1988 CPU 3 running task TA03 - rtems_clock_gettod 09:00:30 12/31/1988 *** END OF TEST SMP 8 *** CORE#3 halts because of Error Mode [9.75002sec/8789994270, 8789994514, 8789994540, 8789 994132cycles] laysim> perf Performance statistics for CORE#0 Cycles : 8789995068 Instructions : 41659517 CORE#0 performance (250MHz) : 1.18 MIPS Simulated time : 35.16 s Performance statistics for CORE#1 Cycles : 8789994950 Instructions : 31629314 CORE#1 performance (250MHz) : 0.90 MIPS Simulated time : 35.16 s Performance statistics for CORE#2 Cycles : 8789994951 Instructions : 18183462 CORE#2 performance (250MHz) : 0.52 MIPS Simulated time : 35.16 s Performance statistics for CORE#3 Cycles : 8789994132 Instructions : 84302289 CORE#3 performance (250MHz) : 2.40 MIPS Simulated time : 35.16 s Performance of laysim-gr740 Real-time performance : 360.61 % laysim-gr740 performance : 18.03 MIPS Wall clock : 9.75 s laysim> </pre>
--	--

3.2 Starting laysim-GR740

When launching laysim-GR740, many functions can be enabled through the start-up options described below.

- **-fast_uart** : laysim-GR740 sets the baudrate of APBUART#0/#1 as 38400bps for executing RAM executable. When '-fast_uart' option is used as start-up option, laysim-GR740 runs APBUART#0/#1 at infinite speed, rather than correct baudrate.

Figure 3–7. Using '-fast_uart' start-up option

laysim-gr740-dbt-cli in Linux	laysim-gr740-dbt-cli in Windows
<pre> [layright@CentOS smptests]\$ laysim-gr740-dbt-cli -r -core0 smp08.elf -fast_uart laysim for GR740 Quad Core Processor (DBT) CLI v3.0 by layright Multicore control : Time quantum (500) CORE#0 RAM file is loaded - smp08.elf </pre>	<pre> PS C:\laysim-gr740\RTM55.1\smptests> laysim-gr740-dbt-cli -r -core0 smp08.elf -fast_u rt laysim for GR740 Quad Core Processor (DBT) CLI v3.0 by layright Multicore control : Time quantum (500) CORE#0 RAM file is loaded - smp08.elf </pre>

<pre> - .text, addr: 0x00000000, size 138992 bytes - .rtmsroset, addr: 0x00021EF0, size 144 bytes - .rtmsstack, addr: 0x00021F80, size 32768 bytes (not loaded, just info.) - .data, addr: 0x00029F80, size 2064 bytes - .bss, addr: 0x0002A7C0, size 28960 bytes (not loaded, just info.) - Detect RTEMS v5.1(SMP) with thread information Only GR740 CORE#0 RAM file is loaded, starts from RAM Set CORE#0 %sp/%fp : 0x07FFFFFF0/0x08000000 *** BEGIN OF TEST SMP 8 *** *** TEST VERSION: 5.1.0 *** TEST STATE: EXPECTED_PASS *** TEST BUILD: RTEMS_NETWORKING RTEMS_POSIX_API RTEMS_SMP *** TEST TOOLS: 7.5.0 20191114 (RTEMS 5, RSB 5.1, Newlib 7947581) CPU 3 running task Init - rtems_clock_get_tod 09:00:00 12/31/1988 CPU 2 running task TA01 - rtems_clock_get_tod 09:00:00 12/31/1988 CPU 0 running task TA03 - rtems_clock_get_tod 09:00:00 12/31/1988 CPU 2 running task TA04 - rtems_clock_get_tod 09:00:00 12/31/1988 CPU 0 running task TA05 - rtems_clock_get_tod 09:00:00 12/31/1988 CPU 2 running task TA06 - rtems_clock_get_tod 09:00:00 12/31/1988 CPU 3 running task TA08 - rtems_clock_get_tod 09:00:00 12/31/1988 CPU 3 running task TA10 - rtems_clock_get_tod 09:00:00 12/31/1988 CPU 0 running task TA07 - rtems_clock_get_tod 09:00:00 12/31/1988 CPU 0 running task TA12 - rtems_clock_get_tod 09:00:00 12/31/1988 CPU 3 running task TA11 - rtems_clock_get_tod 09:00:00 12/31/1988 CPU 2 running task TA09 - rtems_clock_get_tod 09:00:00 12/31/1988 CPU 1 running task TA02 - rtems_clock_get_tod 09:00:00 12/31/1988 CPU 1 running task TA01 - rtems_clock_get_tod 09:00:04 12/31/1988 CPU 2 running task TA01 - rtems_clock_get_tod 09:00:09 12/31/1988 CPU 1 running task TA02 - rtems_clock_get_tod 09:00:09 12/31/1988 CPU 1 running task TA01 - rtems_clock_get_tod 09:00:14 12/31/1988 CPU 2 running task TA03 - rtems_clock_get_tod 09:00:14 12/31/1988 CPU 2 running task TA02 - rtems_clock_get_tod 09:00:19 12/31/1988 CPU 3 running task TA01 - rtems_clock_get_tod 09:00:19 12/31/1988 CPU 1 running task TA04 - rtems_clock_get_tod 09:00:19 12/31/1988 CPU 1 running task TA01 - rtems_clock_get_tod 09:00:24 12/31/1988 CPU 3 running task TA05 - rtems_clock_get_tod 09:00:24 12/31/1988 CPU 3 running task TA01 - rtems_clock_get_tod 09:00:29 12/31/1988 CPU 2 running task TA03 - rtems_clock_get_tod 09:00:29 12/31/1988 CPU 0 running task TA02 - rtems_clock_get_tod 09:00:29 12/31/1988 CPU 1 running task TA06 - rtems_clock_get_tod 09:00:29 12/31/1988 CPU 1 running task TA01 - rtems_clock_get_tod 09:00:34 12/31/1988 CPU 0 running task TA07 - rtems_clock_get_tod 09:00:34 12/31/1988 *** END OF TEST SMP 8 *** *** END OF TEST SMP 8 *** *** END OF TEST SMP 8 *** *** END OF TEST SMP 8 *** CORE#2 halts because of Error Mode [0.785385sec/10016487955, 10016487953, 10016487898, 10016487457cycles] Performance statistics for CORE#0 Cycles : 10016487955 Instructions : 11468423 CORE#0 performance (250MHz) : 0.29 MIPS Simulated time : 40.07 s Performance statistics for CORE#1 Cycles : 10016487953 Instructions : 10234846 CORE#1 performance (250MHz) : 0.26 MIPS Simulated time : 40.07 s Performance statistics for CORE#2 Cycles : 10016487898 Instructions : 10148881 CORE#2 performance (250MHz) : 0.25 MIPS Simulated time : 40.07 s Performance statistics for CORE#3 Cycles : 10016487457 Instructions : 10129957 CORE#3 performance (250MHz) : 0.25 MIPS Simulated time : 40.07 s Performance of laysim-gr740 Real-time performance : 5101.30 % laysim-gr740 performance : 53.45 MIPS Wall clock : 0.79 s [layright@CentOS smptests]\$ </pre>	<pre> - .text, addr: 0x00000000, size 138992 bytes - .rtmsroset, addr: 0x00021EF0, size 144 bytes - .rtmsstack, addr: 0x00021F80, size 32768 bytes (not loaded, just info.) - .data, addr: 0x00029F80, size 2064 bytes - .bss, addr: 0x0002A7C0, size 28960 bytes (not loaded, just info.) - Detect RTEMS v5.1(SMP) with thread information Only GR740 CORE#0 RAM file is loaded, starts from RAM Set CORE#0 %sp/%fp : 0x07FFFFFF0/0x08000000 *** BEGIN OF TEST SMP 8 *** *** TEST VERSION: 5.1.0 *** TEST STATE: EXPECTED_PASS *** TEST BUILD: RTEMS_NETWORKING RTEMS_POSIX_API RTEMS_SMP *** TEST TOOLS: 7.5.0 20191114 (RTEMS 5, RSB 5.1, Newlib 7947581) CPU 3 running task Init - rtems_clock_get_tod 09:00:00 12/31/1988 CPU 2 running task TA01 - rtems_clock_get_tod 09:00:00 12/31/1988 CPU 0 running task TA03 - rtems_clock_get_tod 09:00:00 12/31/1988 CPU 2 running task TA04 - rtems_clock_get_tod 09:00:00 12/31/1988 CPU 0 running task TA05 - rtems_clock_get_tod 09:00:00 12/31/1988 CPU 2 running task TA06 - rtems_clock_get_tod 09:00:00 12/31/1988 CPU 3 running task TA08 - rtems_clock_get_tod 09:00:00 12/31/1988 CPU 3 running task TA10 - rtems_clock_get_tod 09:00:00 12/31/1988 CPU 0 running task TA07 - rtems_clock_get_tod 09:00:00 12/31/1988 CPU 0 running task TA12 - rtems_clock_get_tod 09:00:00 12/31/1988 CPU 3 running task TA11 - rtems_clock_get_tod 09:00:00 12/31/1988 CPU 2 running task TA09 - rtems_clock_get_tod 09:00:00 12/31/1988 CPU 1 running task TA02 - rtems_clock_get_tod 09:00:00 12/31/1988 CPU 1 running task TA01 - rtems_clock_get_tod 09:00:04 12/31/1988 CPU 2 running task TA01 - rtems_clock_get_tod 09:00:09 12/31/1988 CPU 1 running task TA02 - rtems_clock_get_tod 09:00:09 12/31/1988 CPU 1 running task TA01 - rtems_clock_get_tod 09:00:14 12/31/1988 CPU 2 running task TA03 - rtems_clock_get_tod 09:00:14 12/31/1988 CPU 2 running task TA02 - rtems_clock_get_tod 09:00:19 12/31/1988 CPU 3 running task TA01 - rtems_clock_get_tod 09:00:19 12/31/1988 CPU 1 running task TA04 - rtems_clock_get_tod 09:00:19 12/31/1988 CPU 1 running task TA01 - rtems_clock_get_tod 09:00:24 12/31/1988 CPU 3 running task TA05 - rtems_clock_get_tod 09:00:24 12/31/1988 CPU 3 running task TA01 - rtems_clock_get_tod 09:00:29 12/31/1988 CPU 2 running task TA03 - rtems_clock_get_tod 09:00:29 12/31/1988 CPU 0 running task TA02 - rtems_clock_get_tod 09:00:29 12/31/1988 CPU 1 running task TA06 - rtems_clock_get_tod 09:00:29 12/31/1988 CPU 1 running task TA01 - rtems_clock_get_tod 09:00:34 12/31/1988 CPU 0 running task TA07 - rtems_clock_get_tod 09:00:34 12/31/1988 *** END OF TEST SMP 8 *** *** END OF TEST SMP 8 *** *** END OF TEST SMP 8 *** *** END OF TEST SMP 8 *** CORE#2 halts because of Error Mode [1.0296sec/10016487955, 10016487953, 10016487898, 10016487457cycles] Performance statistics for CORE#0 Cycles : 10016487955 Instructions : 11468423 CORE#0 performance (250MHz) : 0.29 MIPS Simulated time : 40.07 s Performance statistics for CORE#1 Cycles : 10016487953 Instructions : 10234846 CORE#1 performance (250MHz) : 0.26 MIPS Simulated time : 40.07 s Performance statistics for CORE#2 Cycles : 10016487898 Instructions : 10148881 CORE#2 performance (250MHz) : 0.25 MIPS Simulated time : 40.07 s Performance statistics for CORE#3 Cycles : 10016487457 Instructions : 10129957 CORE#3 performance (250MHz) : 0.25 MIPS Simulated time : 40.07 s Performance of laysim-gr740 Real-time performance : 3833.32 % laysim-gr740 performance : 40.17 MIPS Wall clock : 1.05 s PS C:\laysim-gr740\RTEMS5.1\smptests> </pre>
---	---

- core0/1/2/3 : Load an RAM executable for CORE#0/#1/#2/#3. This option is only available in the CLI version of laysim-GR740. It is needed to set the entry point and stack point in order to run AMP example in real hardware with GRMON3. laysim-GR740 sets CORE#0/ CORE#1 %sp/%fp and %pc automatically according to loaded program address.

Figure 3–8. Run AMP example using '-core' start-up option

laysim-gr740-cli in Linux	laysim-gr740-cli in Windows
<pre> [layright@CentOS BCC-2.2.2]\$ laysim-gr740-cli -r -core0 bcc_amp0.elf -core1 bcc_amp1.elf laysim for GR740 Quad Core Processor CLI v3.0 by layright CORE#0 RAM file is loaded - bcc_amp0.elf - .text, addr: 0x00001000, size 57264 bytes - .rodata, addr: 0x0000EFB0, size 1904 bytes - .data, addr: 0x0000F720, size 1568 bytes - .bss, addr: 0x0000FD40, size 416 bytes (not loaded, just info.) </pre>	<pre> PS C:\laysim-gr740\BCC-2.2.2> laysim-gr740-cli -r -core0 bcc_amp0.elf -core1 bcc_amp1.elf laysim for GR740 Quad Core Processor CLI v3.0 by layright CORE#0 RAM file is loaded - bcc_amp0.elf - .text, addr: 0x00001000, size 57264 bytes - .rodata, addr: 0x0000EFB0, size 1904 bytes - .data, addr: 0x0000F720, size 1568 bytes - .bss, addr: 0x0000FD40, size 416 bytes (not loaded, just info.) </pre>

```

CORE#1 RAM file is loaded - bcc_aml.elf
- .text, addr: 0x00100000, size 57184 bytes
- .rodata, addr: 0x00100F60, size 1856 bytes
- .data, addr: 0x0010E6A0, size 1568 bytes
- .bss, addr: 0x0010EC00, size 416 bytes (not loaded, just info.)
Set CORE#0 %sp/%fp : 0x000FFFF0/0x00100000
Set CORE#1 %sp/%fp : 0x07FFFFFF0/0x00000000
GR740 is configured as AMP, starts from RAM
amp0:main:25: ENTRY
amp0:main:27: bcc_get_cpu_count() => 4
amp0:main:28:
amp0:main:30: I am supposed to have cpuid=0
amp0:main:32: bcc_get_cpuid() => 0
amp0:main:39: I will now call bcc_start_processor(1)
amp0:main:40:
amp1:main:25: ENTRY
amp1:main:27: bcc_get_cpu_count() => 4
amp1:main:28:
amp1:main:30: I am supposed to have cpuid=1
amp1:main:32: bcc_get_cpuid() => 1
amp1:main:45: I am the node with the highest CPUID for this AMP application.

CORE#1 halts because of Error Mode [30149954cycles/0.212817sec]
amp1:main:46:

Performance statistics for CORE#0
Cycles : 30150028
Instructions : 4382753
Overall CPI : 3.14
CORE#0 performance (250MHz) : 79.67 MOPS (79.67 MIPS, 0.00 MFLOPS)
Simulated time : 120.60 ms
Processor utilization : 45.62 %

Performance statistics for CORE#1
Cycles : 30149954
Instructions : 5222983
Overall CPI : 5.77
CORE#1 performance (250MHz) : 43.31 MOPS (43.31 MIPS, 0.00 MFLOPS)
Simulated time : 120.60 ms
Processor utilization : 100.00 %

Performance of laysim-gr740
Real-time performance : 56.67 %
laysim-gr740 performance : 45.14 MIPS
Wall clock : 0.21 s
[layright@CentOS BCC-2.2.2]$

CORE#1 RAM file is loaded - bcc_aml.elf
- .text, addr: 0x00100000, size 57184 bytes
- .rodata, addr: 0x00100F60, size 1856 bytes
- .data, addr: 0x0010E6A0, size 1568 bytes
- .bss, addr: 0x0010EC00, size 416 bytes (not loaded, just info.)
Set CORE#0 %sp/%fp : 0x000FFFF0/0x00100000
Set CORE#1 %sp/%fp : 0x07FFFFFF0/0x00000000
GR740 is configured as AMP, starts from RAM
amp0:main:25: ENTRY
amp0:main:27: bcc_get_cpu_count() => 4
amp0:main:28:
amp0:main:30: I am supposed to have cpuid=0
amp0:main:32: bcc_get_cpuid() => 0
amp0:main:39: I will now call bcc_start_processor(1)
amp0:main:40:
amp1:main:25: ENTRY
amp1:main:27: bcc_get_cpu_count() => 4
amp1:main:28:
amp1:main:30: I am supposed to have cpuid=1
amp1:main:32: bcc_get_cpuid() => 1
amp1:main:45: I am the node with the highest CPUID for this AMP application.

CORE#1 halts because of Error Mode [30149954cycles/0.2808sec]
amp1:main:46:

Performance statistics for CORE#0
Cycles : 30150028
Instructions : 4382753
Overall CPI : 3.14
CORE#0 performance (250MHz) : 79.67 MOPS (79.67 MIPS, 0.00 MFLOPS)
Simulated time : 120.60 ms
Processor utilization : 45.62 %

Performance statistics for CORE#1
Cycles : 30149954
Instructions : 5222983
Overall CPI : 5.77
CORE#1 performance (250MHz) : 43.31 MOPS (43.31 MIPS, 0.00 MFLOPS)
Simulated time : 120.60 ms
Processor utilization : 100.00 %

Performance of laysim-gr740
Real-time performance : 42.95 %
laysim-gr740 performance : 34.21 MIPS
Wall clock : 0.28 s
PS C:\laysim-gr740\BCC-2.2.2>

```

- -rom : Load a ROM executable to MRAM. This option is only available in the CLI version of laysim-GR740.

Figure 3–9. Run ROM example using '-rom' start-up option

laysim-gr740-cli in Linux	laysim-gr740-cli in Windows
<pre> [layright@CentOS BCC-2.2.2]\$ laysim-gr740-dbt-cli -r -rom dhry-bin.mram laysim for GR740 Quad Core Processor (DBT) CLI v3.0 by layright Multicore control : Time quantum (500) ROM file is loaded - dhry-bin.mram - binary file size : 43664 bytes GR740 ROM file is loaded, starts from ROM MKPROM2 boot loader v2.0.65 Copyright Cobham Gaisler AB - all rights reserved system clock : 250.0 MHz baud rate : 115313 baud prom : 8192 K, (15/15) ws (r/w) sram : 65536 K, 1 bank(s), 3/3 ws (r/w) decompressing .text to 0x00000000 decompressing .rodata to 0x0000e4b0 decompressing .data to 0x0000ecb0 starting dhry.elf Execution starts, 1000000 runs through Dhrystone Total execution time: 1.8 s Microseconds for one run through Dhrystone: 1.8 Dhrystones per Second: 570776.2 Dhrystones MIPS : 324.9 CORE#0 halts because of Error Mode [0.553932sec/494707307, 0, 0, 0cycles] Performance statistics for CORE#0 Cycles : 494707307 Instructions : 391839767 CORE#0 performance (250MHz) : 198.02 MIPS Simulated time : 1.98 s Performance of laysim-gr740 Real-time performance : 357.23 % laysim-gr740 performance : 707.38 MIPS Wall clock : 0.55 s [layright@CentOS BCC-2.2.2]\$ </pre>	<pre> PS C:\laysim-gr740\BCC-2.2.2> laysim-gr740-dbt-cli -r -rom dhry-bin.mram laysim for GR740 Quad Core Processor (DBT) CLI v3.0 by layright Multicore control : Time quantum (500) ROM file is loaded - dhry-bin.mram - binary file size : 43664 bytes GR740 ROM file is loaded, starts from ROM MKPROM2 boot loader v2.0.65 Copyright Cobham Gaisler AB - all rights reserved system clock : 250.0 MHz baud rate : 115313 baud prom : 8192 K, (15/15) ws (r/w) sram : 65536 K, 1 bank(s), 3/3 ws (r/w) decompressing .text to 0x00000000 decompressing .rodata to 0x0000e4b0 decompressing .data to 0x0000ecb0 starting dhry.elf Execution starts, 1000000 runs through Dhrystone Total execution time: 1.8 s Microseconds for one run through Dhrystone: 1.8 Dhrystones per Second: 570776.2 Dhrystones MIPS : 324.9 CORE#0 halts because of Error Mode [0.608401sec/494707307, 0, 0, 0cycles] Performance statistics for CORE#0 Cycles : 494707307 Instructions : 391839767 CORE#0 performance (250MHz) : 198.02 MIPS Simulated time : 1.98 s Performance of laysim-gr740 Real-time performance : 325.25 % laysim-gr740 performance : 644.05 MIPS Wall clock : 0.61 s PS C:\laysim-gr740\BCC-2.2.2> </pre>

- -r : Set auto-start & auto-exit with reporting the performance statistics. This option is only available in the CLI version of laysim-GR740.

Figure 3–10. Using '-r' start-up option

laysim-gr740-mdbt-cli in Linux	laysim-gr740-mdbt-cli in Windows
<pre>[layright@CentOS smptests]\$ laysim-gr740-mdbt-cli -r -core0 smp09.elf laysim for GR740 Quad Core Processor (MDBT) CLI v3.0 by layright Multicore control : Time quantum (500) CORE#0 RAM file is loaded - smp09.elf - .text, addr: 0x00000000, size 114336 bytes - .rtmsroset, addr: 0x0001BEA0, size 144 bytes - .rtmsstack, addr: 0x0001BF40/0x0001BF30, size 32768 bytes (not loaded, just inf o.) - .data, addr: 0x00023F40, size 1328 bytes - .bss, addr: 0x00024480, size 17952 bytes (not loaded, just info.) - Detect RTEMS v5.1(SMP) with thread information Only GR740 CORE#0 RAM file is loaded, starts from RAM Set CORE#0 %sp/%fp : 0x07FFFFFF0/0x08000000 *** BEGIN OF TEST SMP 9 *** *** TEST VERSION: 5.1.0 *** TEST STATE: EXPECTED_PASS *** TEST BUILD: RTEMS_NETWORKING RTEMS_POSIX_API RTEMS_SMP *** TEST TOOLS: 7.5.0 20191114 (RTEMS 5, RSB 5.1, Newlib 7947581) CPU 3 start task TA1 CPU 3 start task TA2 CPU 3 start task TA3 kill 10 clock ticks ----- CPU USAGE BY THREAD ----- ID NAME SECONDS PERCENT ----- 0x09010001 IDLE 1.112367 91.404 0x09010002 IDLE 1.106123 89.344 0x09010003 IDLE 1.099876 87.353 0x09010004 IDLE 1.000013 78.114 0x0a010001 UI1 0.301256 23.150 0x0a010002 TA1 0.222469 16.823 0x0a010003 TA2 0.237299 17.663 0x0a010004 TA3 0.252131 18.477 ----- TIME SINCE LAST CPU USAGE RESET IN SECONDS: 1.364498 ----- *** END OF TEST SMP 9 *** CORE#3 halts because of Error Mode [0.206374sec/130924394, 130924457, 130924454, 130924 127cycles] Performance statistics for CORE#0 Cycles : 130924684 Instructions : 260888 CORE#0 performance (250MHz) : 0.50 MIPS Simulated time : 523.70 ms Performance statistics for CORE#1 Cycles : 130924591 Instructions : 182809 CORE#1 performance (250MHz) : 0.35 MIPS Simulated time : 523.70 ms Performance statistics for CORE#2 Cycles : 130924588 Instructions : 183326 CORE#2 performance (250MHz) : 0.35 MIPS Simulated time : 523.70 ms Performance statistics for CORE#3 Cycles : 130924127 Instructions : 58597633 CORE#3 performance (250MHz) : 111.89 MIPS Simulated time : 523.70 ms Performance of laysim-gr740 Real-time performance : 253.72 % laysim-gr740 performance : 286.93 MIPS Wall clock : 0.21 s [layright@CentOS smptests]\$</pre>	<pre>PS C:\laysim-gr740\RTEMS5.1\smptests> laysim-gr740-mdbt-cli -r -core0 smp09.elf laysim for GR740 Quad Core Processor (MDBT) CLI v3.0 by layright Multicore control : Time quantum (500) CORE#0 RAM file is loaded - smp09.elf - .text, addr: 0x00000000, size 114336 bytes - .rtmsroset, addr: 0x0001BEA0, size 144 bytes - .rtmsstack, addr: 0x0001BF40/0x0001BF30, size 32768 bytes (not loaded, just inf o.) - .data, addr: 0x00023F40, size 1328 bytes - .bss, addr: 0x00024480, size 17952 bytes (not loaded, just info.) - Detect RTEMS v5.1(SMP) with thread information Only GR740 CORE#0 RAM file is loaded, starts from RAM Set CORE#0 %sp/%fp : 0x07FFFFFF0/0x08000000 *** BEGIN OF TEST SMP 9 *** *** TEST VERSION: 5.1.0 *** TEST STATE: EXPECTED_PASS *** TEST BUILD: RTEMS_NETWORKING RTEMS_POSIX_API RTEMS_SMP *** TEST TOOLS: 7.5.0 20191114 (RTEMS 5, RSB 5.1, Newlib 7947581) CPU 3 start task TA1 CPU 3 start task TA2 CPU 3 start task TA3 kill 10 clock ticks ----- CPU USAGE BY THREAD ----- ID NAME SECONDS PERCENT ----- 0x09010001 IDLE 1.112370 91.405 0x09010002 IDLE 1.106126 89.344 0x09010003 IDLE 1.099879 87.353 0x09010004 IDLE 1.000013 78.113 0x0a010001 UI1 0.301260 23.151 0x0a010002 TA1 0.222469 16.823 0x0a010003 TA2 0.237299 17.663 0x0a010004 TA3 0.252131 18.477 ----- TIME SINCE LAST CPU USAGE RESET IN SECONDS: 1.364501 ----- *** END OF TEST SMP 9 *** CORE#3 halts because of Error Mode [0.390001sec/130926330, 130926396, 130926357, 130926 126cycles] Performance statistics for CORE#0 Cycles : 130926686 Instructions : 261716 CORE#0 performance (250MHz) : 0.50 MIPS Simulated time : 523.71 ms Performance statistics for CORE#1 Cycles : 130926594 Instructions : 183737 CORE#1 performance (250MHz) : 0.35 MIPS Simulated time : 523.71 ms Performance statistics for CORE#2 Cycles : 130926591 Instructions : 183778 CORE#2 performance (250MHz) : 0.35 MIPS Simulated time : 523.71 ms Performance statistics for CORE#3 Cycles : 130926126 Instructions : 58599005 CORE#3 performance (250MHz) : 111.89 MIPS Simulated time : 523.70 ms Performance of laysim-gr740 Real-time performance : 134.28 % laysim-gr740 performance : 151.87 MIPS Wall clock : 0.39 s PS C:\laysim-gr740\RTEMS5.1\smptests></pre>

- traplog : Log a synchronous trap information on occurred except Window underflow/overflow and interrupts.

Figure 3–11. Using '-traplog' start-up option

laysim-gr740-dbt-cli in Linux	laysim-gr740-dbt-cli in Windows
<pre>[layright@CentOS Samples]\$ laysim-gr740-dbt-cli -r -core0 rtems-synctrp.elf -traplog laysim for GR740 Quad Core Processor (DBT) CLI v3.0 by layright Multicore control : Time quantum (500) CORE#0 RAM file is loaded - rtems-synctrp.elf - .text, addr: 0x00000000, size 155600 bytes - .rtmsroset, addr: 0x00025FD0, size 160 bytes - .rtmsstack, addr: 0x00026080, size 8192 bytes (not loaded, just info.) - .data, addr: 0x00028080, size 4720 bytes - .bss, addr: 0x00029300, size 14384 bytes (not loaded, just info.) - Detect RTEMS v5.1(SMP) with thread information Only GR740 CORE#0 RAM file is loaded, starts from RAM Set CORE#0 %sp/%fp : 0x07FFFFFF0/0x08000000 Unaligned Memory (tt=0x07) occurred at 471537 cycles ----- CORE#0 ----- INS LOCALS OUTS GLOBALS 0: 0002C8A0 F3000FC2 00000000 00000000 1: 0002DA58 0001344C 00000190 00000001 2: 00028DC4 00013450 0002DA0C 00000000 3: 00000001 F30000C2 00000000 00200000</pre>	<pre>PS C:\laysim-gr740\RCC-1.3.1\Samples> laysim-gr740-dbt-cli -r -core0 rtems-synctrp.elf -ltraplog laysim for GR740 Quad Core Processor (DBT) CLI v3.0 by layright Multicore control : Time quantum (500) CORE#0 RAM file is loaded - rtems-synctrp.elf - .text, addr: 0x00000000, size 155600 bytes - .rtmsroset, addr: 0x00025FD0, size 160 bytes - .rtmsstack, addr: 0x00026080, size 8192 bytes (not loaded, just info.) - .data, addr: 0x00028080, size 4720 bytes - .bss, addr: 0x00029300, size 14384 bytes (not loaded, just info.) - Detect RTEMS v5.1(SMP) with thread information Only GR740 CORE#0 RAM file is loaded, starts from RAM Set CORE#0 %sp/%fp : 0x07FFFFFF0/0x08000000 Unaligned Memory (tt=0x07) occurred at 471537 cycles ----- CORE#0 ----- INS LOCALS OUTS GLOBALS 0: 0002C8A0 F3000FC2 00000000 00000000 1: 0002DA58 0001344C 00000190 00000001 2: 00028DC4 00013450 0002DA0C 00000000 3: 00000001 F30000C2 00000000 00200000</pre>

<pre> 4: 00027D90 00000000 00000000 00000000 5: 00001000 00000020 0002A8D0 00000000 6: 0002DA10 FFFFFFFF 0002D9A8 0002B800 7: 000101C0 FFFFFFFF 00001310 00000000 psr:F34000E2 wim:00000008 tbr:000008A0 y:FFFFFFFE [n:0 z:1 v:0 c:0 ef:0 pil:0 s:1 ps:1 et:1 cwp:2] pc: 00001348 : D0004000 ld [%g1], %o0 npc: 0000134C : 91D02010 ta 0x10 Caught synchronous trap with vector 0x07 Caught synchronous trap with vector 0x90 CORE#0 halts because of Error Mode [0.010725sec/6008248, 0, 0, 0cycles] Performance statistics for CORE#0 Cycles : 6008248 Instructions : 2581371 CORE#0 performance (250MHz) : 107.41 MIPS Simulated time : 24.03 ms Performance of laysim-gr740 Real-time performance : 224.08 % laysim-gr740 performance : 240.69 MIPS Wall clock : 0.01 s [layright@CentOS Samples]\$ </pre>	<pre> 4: 00027D90 00000000 00000000 00000000 5: 00001000 00000020 0002A8D0 00000000 6: 0002DA10 FFFFFFFF 0002D9A8 0002B800 7: 000101C0 FFFFFFFF 00001310 00000000 psr:F34000E2 wim:00000008 tbr:000008A0 y:FFFFFFFE [n:0 z:1 v:0 c:0 ef:0 pil:0 s:1 ps:1 et:1 cwp:2] pc: 00001348 : D0004000 ld [%g1], %o0 npc: 0000134C : 91D02010 ta 0x10 Caught synchronous trap with vector 0x07 Caught synchronous trap with vector 0x90 CORE#0 halts because of Error Mode [0.0624001sec/6008248, 0, 0, 0cycles] Performance statistics for CORE#0 Cycles : 6008248 Instructions : 2581371 CORE#0 performance (250MHz) : 107.41 MIPS Simulated time : 24.03 ms Performance of laysim-gr740 Real-time performance : 38.51 % laysim-gr740 performance : 41.37 MIPS Wall clock : 0.06 s PS C:\laysim-gr740\RCC-1.3.1\Samples> </pre>
--	--

- batch : Run commands in a batch file. This option is only available in the CLI version of laysim-GR740.

Figure 3-12. Using '-batch' start-up option

laysim-gr740-dbt-cli in Linux	laysim-gr740-dbt-cli in Windows
<pre> [layright@CentOS Samples]\$ cat gr740.bat load rtems-hello.elf b Init t 10 run reg perf quit [layright@CentOS Samples]\$ laysim-gr740-dbt-cli -batch gr740.bat laysim for GR740 Quad Core Processor (DBT) CLI v3.0 by layright Multicore control : Time quantum (500) --> No CORE#0 RAM file is loaded Read batch file from gr740.bat CORE#0 RAM file is loaded - rtems-hello.elf - .text, addr: 0x00000000, size 165344 bytes - .rtemsroset, addr: 0x000285E0, size 160 bytes - .rtemsstack, addr: 0x00028680, size 8192 bytes (not loaded, just info.) - .data, addr: 0x0002A680, size 4848 bytes - .bss, addr: 0x0002B900, size 15792 bytes (not loaded, just info.) - Detect RTEMS v5.1(SMP) with thread information Only GR740 CORE#0 RAM file is loaded, starts from RAM Set CORE#0 %sp/%fp : 0x07FFFFFF0/0x00000000 [CORE#0] Set breakpoint at Init(0x000012C8) 0 (0) E CORE#0 00000000 : A0100000 or %g0, %g0, %l0 ! start 1 (1) E CORE#0 00000004 : 29000004 sethi %hi(0x1000), %l4 2 (2) E CORE#0 00000008 : 81C52000 jmp1 %l4, %g0 5 (5) E CORE#0 0000000C : A6102000 or %g0, 0x0, %l3 6 (6) E CORE#0 00001000 : 03000000 sethi %hi(0x0), %g1 ! hard_reset 7 (7) E CORE#0 00001004 : 8210C000 or %g1, 0x0, %g1 8 (8) E CORE#0 00001008 : 81980001 wr %g0, %g1, %tbr 9 (9) E CORE#0 0000100C : 83480000 rd %psr, %g1 10 (10) E CORE#0 00001010 : 84006001 add %g1, 0x1, %g2 11 (11) E CORE#0 00001014 : 8408A007 and %g2, 0x7, %g2 [CORE#0] Breakpoint hit at 0x000012C8(Init) ----- CORE#0 ----- INS LOCALS OUTS GLOBALS 0: 0002B840 00030658 0002ECA0 00000000 1: 0001317C 0002658C 00030658 000012C8 2: 00000000 0002B460 0002B44C 0A010001 3: 0002B44C 0002B400 00000001 0A010001 4: 0002B840 0002B464 0002A390 0003065C 5: 000265D4 0002B428 000263A0 00000000 6: 00030670 0002BE10 00030610 0002E700 7: 0000ED18 0002BB9C 000100EC 00000000 psr:F34000E3 wim:00000040 tbr:000008A0 y:FFFFFFFE [n:0 z:1 v:0 c:0 ef:0 pil:0 s:1 ps:1 et:1 cwp:3] pc: 000012C8 : 9DE3BFA0 save %sp, -96, %sp npc: 000012CC : 11000098 sethi %hi(0x26000), %o0 Performance statistics for CORE#0 Cycles : 473414 Instructions : 327942 CORE#0 performance (250MHz) : 173.18 MIPS Simulated time : 1.89 ms Performance of laysim-gr740 Real-time performance : 21.78 % laysim-gr740 performance : 37.72 MIPS Wall clock : 0.01 s [layright@CentOS Samples]\$ </pre>	<pre> PS C:\laysim-gr740\RCC-1.3.1\Samples> cat gr740.bat load rtems-hello.elf b Init t 10 run reg perf quit PS C:\laysim-gr740\RCC-1.3.1\Samples> laysim-gr740-dbt-cli -batch gr740.bat laysim for GR740 Quad Core Processor (DBT) CLI v3.0 by layright Multicore control : Time quantum (500) --> No CORE#0 RAM file is loaded Read batch file from gr740.bat CORE#0 RAM file is loaded - rtems-hello.elf - .text, addr: 0x00000000, size 165344 bytes - .rtemsroset, addr: 0x000285E0, size 160 bytes - .rtemsstack, addr: 0x00028680, size 8192 bytes (not loaded, just info.) - .data, addr: 0x0002A680, size 4848 bytes - .bss, addr: 0x0002B900, size 15792 bytes (not loaded, just info.) - Detect RTEMS v5.1(SMP) with thread information Only GR740 CORE#0 RAM file is loaded, starts from RAM Set CORE#0 %sp/%fp : 0x07FFFFFF0/0x00000000 [CORE#0] Set breakpoint at Init(0x000012C8) 0 (0) E CORE#0 00000000 : A0100000 or %g0, %g0, %l0 ! start 1 (1) E CORE#0 00000004 : 29000004 sethi %hi(0x1000), %l4 2 (2) E CORE#0 00000008 : 81C52000 jmp1 %l4, %g0 5 (5) E CORE#0 0000000C : A6102000 or %g0, 0x0, %l3 6 (6) E CORE#0 00001000 : 03000000 sethi %hi(0x0), %g1 ! hard_reset 7 (7) E CORE#0 00001004 : 8210C000 or %g1, 0x0, %g1 8 (8) E CORE#0 00001008 : 81980001 wr %g0, %g1, %tbr 9 (9) E CORE#0 0000100C : 83480000 rd %psr, %g1 10 (10) E CORE#0 00001010 : 84006001 add %g1, 0x1, %g2 11 (11) E CORE#0 00001014 : 8408A007 and %g2, 0x7, %g2 [CORE#0] Breakpoint hit at 0x000012C8(Init) ----- CORE#0 ----- INS LOCALS OUTS GLOBALS 0: 0002B840 00030658 0002ECA0 00000000 1: 0001317C 0002658C 00030658 000012C8 2: 00000000 0002B460 0002B44C 0A010001 3: 0002B44C 0002B400 00000001 0A010001 4: 0002B840 0002B464 0002A390 0003065C 5: 000265D4 0002B428 000263A0 00000000 6: 00030670 0002BE10 00030610 0002E700 7: 0000ED18 0002BB9C 000100EC 00000000 psr:F34000E3 wim:00000040 tbr:000008A0 y:FFFFFFFE [n:0 z:1 v:0 c:0 ef:0 pil:0 s:1 ps:1 et:1 cwp:3] pc: 000012C8 : 9DE3BFA0 save %sp, -96, %sp npc: 000012CC : 11000098 sethi %hi(0x26000), %o0 Performance statistics for CORE#0 Cycles : 473414 Instructions : 327942 CORE#0 performance (250MHz) : 173.18 MIPS Simulated time : 1.89 ms Performance of laysim-gr740 Real-time performance : 4.05 % laysim-gr740 performance : 7.01 MIPS Wall clock : 0.05 s PS C:\laysim-gr740\RCC-1.3.1\Samples> </pre>

- -cov : Enable code coverage. laysim-gr740-cli only supports for the code coverage in the same format as TSIM3-LEON4. '-cov' start-up option should be enabled for the code coverage. Detailed information can be found in [laysim-GR740-013] Code coverage operation on laysim-GR740.

Figure 3-13. Using '-cov' start-up option

laysim-gr740-cli in Linux	laysim-gr740-cli in Windows
<pre>[layright@CentOS smptests]\$ laysim-gr740-cli -r -core0 smp09.elf -cov laysim for GR740 Quad Core Processor CLI v3.0 by layright CORE#0 RAM file is loaded - smp09.elf - .text, addr: 0x00000000, size 114336 bytes - .rtmsrset, addr: 0x0001BEA0, size 144 bytes - .rtmsstack, addr: 0x0001BF40/0x0001BF30, size 32768 bytes (not loaded, just info.) - .data, addr: 0x00023F40, size 1328 bytes - .bss, addr: 0x00024480, size 17952 bytes (not loaded, just info.) - Detect RTEMS v5.1(SMP) with thread information Only GR740 CORE#0 RAM file is loaded, starts from RAM Set CORE#0 %sp/%fp : 0x07FFFFFF0/0x08000000 *** BEGIN OF TEST SMP 9 *** *** TEST VERSION: 5.1.0 *** TEST STATE: EXPECTED_PASS *** TEST BUILD: RTEMS_NETWORKING RTEMS_POSIX_API RTEMS_SMP *** TEST TOOLS: 7.5.0 20191114 (RTEMS 5, RSB 5.1, Newlib 7947581) CPU 3 start task TA1 CPU 3 start task TA2 CPU 3 start task TA3 kill 10 clock ticks ----- CPU USAGE BY THREAD ----- ID NAME SECONDS PERCENT ----- 0x09010001 IDLE 1.116508 91.433 0x09010002 IDLE 1.110264 89.380 0x09010003 IDLE 1.104028 87.395 0x09010004 IDLE 1.000046 77.865 0x0a010001 UI1 0.305362 23.392 0x0a010002 TA1 0.222456 16.770 0x0a010003 TA2 0.237297 17.609 0x0a010004 TA3 0.252129 18.421 ----- TIME SINCE LAST CPU USAGE RESET IN SECONDS: 1.368638 *** END OF TEST SMP 9 *** CORE#3 halts because of Error Mode [134715700cycles/1.53253sec] Performance statistics for CORE#0 Cycles : 134715707 Instructions : 259785 Overall CPI : 2.77 CORE#0 performance (250MHz) : 90.30 MOPS (90.30 MIPS, 0.00 MFLOPS) Simulated time : 538.86 ms Processor utilization : 0.53 % Performance statistics for CORE#1 Cycles : 134715700 Instructions : 260270 Overall CPI : 2.65 CORE#1 performance (250MHz) : 94.26 MOPS (94.26 MIPS, 0.00 MFLOPS) Simulated time : 538.86 ms Processor utilization : 0.51 % Performance statistics for CORE#2 Cycles : 134715708 Instructions : 261074 Overall CPI : 2.64 CORE#2 performance (250MHz) : 94.60 MOPS (94.60 MIPS, 0.00 MFLOPS) Simulated time : 538.86 ms Processor utilization : 0.51 % Performance statistics for CORE#3 Cycles : 134715700 Instructions : 58659970 Overall CPI : 2.30 CORE#3 performance (250MHz) : 108.86 MOPS (108.86 MIPS, 0.00 MFLOPS) Simulated time : 538.86 ms Processor utilization : 100.00 % Performance of laysim-gr740 Real-time performance : 35.16 % laysim-gr740 performance : 38.79 MIPS Wall clock : 1.53 s [layright@CentOS smptests]\$ ls *.cov smp09.elf.cov [layright@CentOS smptests]\$</pre>	<pre>PS C:\laysim-gr740\RTMS5.1\smptests> laysim-gr740-cli -r -core0 smp09.elf -cov laysim for GR740 Quad Core Processor CLI v3.0 by layright CORE#0 RAM file is loaded - smp09.elf - .text, addr: 0x00000000, size 114336 bytes - .rtmsrset, addr: 0x0001BEA0, size 144 bytes - .rtmsstack, addr: 0x0001BF40/0x0001BF30, size 32768 bytes (not loaded, just info.) - .data, addr: 0x00023F40, size 1328 bytes - .bss, addr: 0x00024480, size 17952 bytes (not loaded, just info.) - Detect RTEMS v5.1(SMP) with thread information Only GR740 CORE#0 RAM file is loaded, starts from RAM Set CORE#0 %sp/%fp : 0x07FFFFFF0/0x08000000 *** BEGIN OF TEST SMP 9 *** *** TEST VERSION: 5.1.0 *** TEST STATE: EXPECTED_PASS *** TEST BUILD: RTEMS_NETWORKING RTEMS_POSIX_API RTEMS_SMP *** TEST TOOLS: 7.5.0 20191114 (RTEMS 5, RSB 5.1, Newlib 7947581) CPU 3 start task TA1 CPU 3 start task TA2 CPU 3 start task TA3 kill 10 clock ticks ----- CPU USAGE BY THREAD ----- ID NAME SECONDS PERCENT ----- 0x09010001 IDLE 1.116508 91.433 0x09010002 IDLE 1.110264 89.380 0x09010003 IDLE 1.104028 87.395 0x09010004 IDLE 1.000046 77.865 0x0a010001 UI1 0.305362 23.392 0x0a010002 TA1 0.222456 16.770 0x0a010003 TA2 0.237297 17.609 0x0a010004 TA3 0.252129 18.421 ----- TIME SINCE LAST CPU USAGE RESET IN SECONDS: 1.368638 *** END OF TEST SMP 9 *** CORE#3 halts because of Error Mode [134715700cycles/1.6068sec] Performance statistics for CORE#0 Cycles : 134715707 Instructions : 259785 Overall CPI : 2.77 CORE#0 performance (250MHz) : 90.30 MOPS (90.30 MIPS, 0.00 MFLOPS) Simulated time : 538.86 ms Processor utilization : 0.53 % Performance statistics for CORE#1 Cycles : 134715700 Instructions : 260270 Overall CPI : 2.65 CORE#1 performance (250MHz) : 94.26 MOPS (94.26 MIPS, 0.00 MFLOPS) Simulated time : 538.86 ms Processor utilization : 0.51 % Performance statistics for CORE#2 Cycles : 134715708 Instructions : 261074 Overall CPI : 2.64 CORE#2 performance (250MHz) : 94.60 MOPS (94.60 MIPS, 0.00 MFLOPS) Simulated time : 538.86 ms Processor utilization : 0.51 % Performance statistics for CORE#3 Cycles : 134715700 Instructions : 58659970 Overall CPI : 2.30 CORE#3 performance (250MHz) : 108.86 MOPS (108.86 MIPS, 0.00 MFLOPS) Simulated time : 538.86 ms Processor utilization : 100.00 % Performance of laysim-gr740 Real-time performance : 33.54 % laysim-gr740 performance : 36.99 MIPS Wall clock : 1.61 s PS C:\laysim-gr740\RTMS5.1\smptests> ls *.cov Directory: C:\laysim-gr740\RTMS5.1\smptests Mode LastWriteTime Length Name ---- - -a--- 8/27/2025 3:09 PM 55473 smp09.elf.cov PS C:\laysim-gr740\RTMS5.1\smptests></pre>
<pre>00000000 : 7 3 3 3 6 2 2 2 6 2 2 2 6 2 2 2 6 2 2 2 19 19 19 0 1 1 1 11 6 2 2 2 00000080 : 6 2 2 6 2 2 2 6 2 2 2 6 2 2 2 6 2 2 2 6 2 2 2 6 2 2 2 6 2 2 2 00000100 : 6 6 2 7 1b 1b 1b 6 2 2 2 6 2 2 2 6 2 2 2 6 2 2 2 6 2 2 2 6 2 2 2 00000180 : 6 2 2 6 2 2 2 6 2 2 2 6 2 2 2 6 2 2 2 6 2 2 2 6 2 2 2 6 2 2 2 00000200 : 6 2 2 6 2 2 2 6 2 2 2 6 2 2 2 6 2 2 2 6 2 2 2 6 2 2 2 6 2 2 2 00000280 : 6 2 2 6 2 2 2 6 2 2 2 6 2 2 2 6 2 2 2 6 2 2 2 6 2 2 2 6 2 2 2 00000300 : 6 2 2 6 2 2 2 6 2 2 2 6 2 2 2 6 2 2 2 6 2 2 2 6 2 2 2 6 2 2 2 00000380 : 6 2 2 6 2 2 2 6 2 2 2 6 2 2 2 6 2 2 2 6 2 2 2 6 2 2 2 6 2 2 2 00000400 : 6 2 2 6 2 2 2 6 2 2 2 6 2 2 2 6 2 2 2 6 2 2 2 6 2 2 2 6 2 2 2 00000480 : 6 2 2 6 2 2 2 6 2 2 2 6 2 2 2 6 2 2 2 6 2 2 2 6 2 2 2 6 2 2 2</pre>	<pre>00000000 : 7 3 3 3 6 2 2 2 6 2 2 2 6 2 2 2 6 2 2 2 19 19 19 0 1 1 1 11 6 2 2 2 00000080 : 6 2 2 6 2 2 2 6 2 2 2 6 2 2 2 6 2 2 2 6 2 2 2 6 2 2 2 6 2 2 2 00000100 : 6 6 2 7 1b 1b 1b 6 2 2 2 6 2 2 2 6 2 2 2 6 2 2 2 6 2 2 2 6 2 2 2 00000180 : 6 2 2 6 2 2 2 6 2 2 2 6 2 2 2 6 2 2 2 6 2 2 2 6 2 2 2 6 2 2 2 00000200 : 6 2 2 6 2 2 2 6 2 2 2 6 2 2 2 6 2 2 2 6 2 2 2 6 2 2 2 6 2 2 2 00000280 : 6 2 2 6 2 2 2 6 2 2 2 6 2 2 2 6 2 2 2 6 2 2 2 6 2 2 2 6 2 2 2 00000300 : 6 2 2 6 2 2 2 6 2 2 2 6 2 2 2 6 2 2 2 6 2 2 2 6 2 2 2 6 2 2 2 00000380 : 6 2 2 6 2 2 2 6 2 2 2 6 2 2 2 6 2 2 2 6 2 2 2 6 2 2 2 6 2 2 2 00000400 : 6 2 2 6 2 2 2 6 2 2 2 6 2 2 2 6 2 2 2 6 2 2 2 6 2 2 2 6 2 2 2 00000480 : 6 2 2 6 2 2 2 6 2 2 2 6 2 2 2 6 2 2 2 6 2 2 2 6 2 2 2 6 2 2 2</pre>

[illegible]

- **-nb** : Run another cores on error mode of any core. By default, when any core transitions into error mode, **laysim-GR740** halts execution. However if the **-nb** option is enabled, other cores continue execution even if one core enters error mode, and execution halts only when all cores have transitioned into error mode.

Figure 3-14. Using '-nb' start-up option

laysim-gr740-mdbt-cli in Linux	laysim-gr740-mdbt-cli in Windows
<pre>[layright@CentOS0 AMP]\$ laysim-gr740-mdbt-cli -r -core0 dhry_core0_test4.elf -core1 dhry_core1.elf -core2 dhry_core2.elf -core3 dhry_core3.elf -nb laysim for GR740 Quad Core Processor (MDBT) CLI v3.0 by layright Multicore control : Time quantum (500) CORE#0 RAM file is loaded - dhry_core0_test4.elf - .text, addr: 0x00000000, size 24288 bytes - .rodata, addr: 0x00005FE0, size 256 bytes - .data, addr: 0x00005FE0, size 1216 bytes - .bss, addr: 0x000064A0, size 10680 bytes (not loaded, just info.) CORE#1 RAM file is loaded - dhry_core1.elf - .text, addr: 0x01000000, size 24272 bytes - .rodata, addr: 0x01005ED0, size 256 bytes - .data, addr: 0x01005FD0, size 1216 bytes - .bss, addr: 0x01006490, size 10680 bytes (not loaded, just info.) CORE#2 RAM file is loaded - dhry_core2.elf - .text, addr: 0x02000000, size 24272 bytes - .rodata, addr: 0x02005ED0, size 256 bytes - .data, addr: 0x02005FD0, size 1216 bytes - .bss, addr: 0x02006490, size 10680 bytes (not loaded, just info.) CORE#3 RAM file is loaded - dhry_core3.elf - .text, addr: 0x03000000, size 24272 bytes - .rodata, addr: 0x03005ED0, size 256 bytes - .data, addr: 0x03005FD0, size 1216 bytes - .bss, addr: 0x03006490, size 10680 bytes (not loaded, just info.) Set CORE#0 %sp/%fp : 0x00FFFFFFF0/0x01000000 Set CORE#1 %sp/%fp : 0x01FFFFFFF0/0x02000000 Set CORE#2 %sp/%fp : 0x02FFFFFFF0/0x03000000 Set CORE#3 %sp/%fp : 0x07FFFFFFF0/0x08000000 GR740 is configured as AMP, starts from RAM CORE#0 halts because of Error Mode [3.75232sec/2190014443, 2190014608, 2190014608, 2190014608]cycles] CORE#1 halts because of Error Mode [3.75241sec/2190014443, 2190028047, 2190027827, 2190027922]cycles] CORE#3 halts because of Error Mode [3.75244sec/2190014443, 2190028047, 2190028019, 2190028047]cycles] CORE#2 halts because of Error Mode [3.75244sec/2190014443, 2190028047, 2190028047, 2190028047]cycles] laysim> core GR740 CORE#0 is error mode GR740 CORE#1 is error mode GR740 CORE#2 is error mode GR740 CORE#3 is error mode GR740 CORE#0 is selected for debugging laysim></pre>	<pre>P5 C:\laysim-gr740\Cycle_Accuracy\AMP> laysim-gr740-mdbt-cli -r -core0 dhry_core0_test4.elf -core1 dhry_core1.elf -core2 dhry_core2.elf -core3 dhry_core3.elf -nb laysim for GR740 Quad Core Processor (MDBT) CLI v3.0 by layright Multicore control : Time quantum (500) CORE#0 RAM file is loaded - dhry_core0_test4.elf - .text, addr: 0x00000000, size 24288 bytes - .rodata, addr: 0x00005FE0, size 256 bytes - .data, addr: 0x00005FE0, size 1216 bytes - .bss, addr: 0x000064A0, size 10680 bytes (not loaded, just info.) CORE#1 RAM file is loaded - dhry_core1.elf - .text, addr: 0x01000000, size 24272 bytes - .rodata, addr: 0x01005ED0, size 256 bytes - .data, addr: 0x01005FD0, size 1216 bytes - .bss, addr: 0x01006490, size 10680 bytes (not loaded, just info.) CORE#2 RAM file is loaded - dhry_core2.elf - .text, addr: 0x02000000, size 24272 bytes - .rodata, addr: 0x02005ED0, size 256 bytes - .data, addr: 0x02005FD0, size 1216 bytes - .bss, addr: 0x02006490, size 10680 bytes (not loaded, just info.) CORE#3 RAM file is loaded - dhry_core3.elf - .text, addr: 0x03000000, size 24272 bytes - .rodata, addr: 0x03005ED0, size 256 bytes - .data, addr: 0x03005FD0, size 1216 bytes - .bss, addr: 0x03006490, size 10680 bytes (not loaded, just info.) Set CORE#0 %sp/%fp : 0x00FFFFFFF0/0x01000000 Set CORE#1 %sp/%fp : 0x01FFFFFFF0/0x02000000 Set CORE#2 %sp/%fp : 0x02FFFFFFF0/0x03000000 Set CORE#3 %sp/%fp : 0x07FFFFFFF0/0x08000000 GR740 is configured as AMP, starts from RAM CORE#0 halts because of Error Mode [4.75801sec/2190014443, 2190014608, 2190014608, 2190014608]cycles] CORE#1 halts because of Error Mode [4.78921sec/2190014443, 2190028047, 2190028022, 2190027916]cycles] CORE#2 halts because of Error Mode [4.78921sec/2190014443, 2190028047, 2190028047, 2190027941]cycles] CORE#3 halts because of Error Mode [4.78921sec/2190014443, 2190028047, 2190028047, 2190028047]cycles] laysim> core GR740 CORE#0 is error mode GR740 CORE#1 is error mode GR740 CORE#2 is error mode GR740 CORE#3 is error mode GR740 CORE#0 is selected for debugging laysim></pre>

- **-ramsz** : laysim-GR740 provides 128MB of SDRAM by default, and the SDRAM size can be modified using the **-ramsz** option. The updated SDRAM size is reflected in `MMCTRL.SDCFG1`, and if RAM code is loaded `%sp/%fp` are updated according to the new SDRAM size.

- 8 : 8MB, 16 : 16MB, 32 : 32MB, 64 : 64MB, 128 : 128MB (default), 256 : 256MB, 512 : 512MB, 1024 : 1024MB

Figure 3–15. Using '-ramsz' start-up option

laysim-gr740-mdbt-cli in Linux	laysim-gr740-mdbt-cli in Windows
<pre>[layright@CentOS Linux]\$ laysim-gr740-mdbt-cli -r -core0 Linux4.9-GR740-SMP.elf -ramsz 1024 laysim for GR740 Quad Core Processor (MDBT) CLI v3.0 by layright Multicore control : Time quantum (500) CORE#0 RAM file is loaded - Linux4.9-GR740-SMP.elf - .text, addr: 0x00000000, size 4320 bytes - .data, addr: 0x000010E0, size 80 bytes - .bss, addr: 0x00001130, size 8 bytes (not loaded, just info.) - .mmu_tables, addr: 0x00002000, size 2560 bytes (not loaded, just info.) - .vmlinux, addr: 0x00004000, size 7418304 bytes - .startup_prom, addr: 0x007171C0, size 40464 bytes Only GR740 CORE#0 RAM file is loaded, starts from RAM Set CORE#0 %sp/%fp : 0x3FFFFFF0/0x40000000 [prom_meminit_v0:26] bank[0] base=0x00000000, size=0x3FFFA970 PROMLIB: Sun Boot Prom Version 0 Revision 0 Linux version 4.9.54 (emanwon@Centos7SDE) (gcc version 4.9.4 (Cobham Gaisler Linux 4.9 Toolchain 1.1)) #6 SMP Thu Jan 2 15:34:50 KST 2020 bootconsole [earlyprom0] enabled ARCH: LEON TYPE: Leon3 System-on-a-Chip Ethernet address: 00:00:07:cc:01:45 CACHE: 4-way associative cache, set size 4k [bootmem_init:127] bank[i] base=0x00000000, size=0x3FFFA000 831MB HIGHMEM available. OF stdout device is: /a:a PROM: Built device tree with 68142 bytes of memory. Booting Linux... percpu: Embedded 9 pages/cpu @f145b000 s12736 r0 d24128 u36864 Built 1 zonelists in Zone order, mobility grouping on. Total pages: 259900 Kernel command line: console=ttyS0,115200 init=/sbin/init log_buf_len individual max cpu contribution: 4096 bytes log_buf_len total cpu_extra contributions: 12288 bytes log_buf_len min size: 16384 bytes log_buf_len: 32768 bytes early log buf free: 14608(89%) PID hash table entries: 1024 (order: 0, 4096 bytes) Dentry cache hash table entries: 32768 (order: 5, 131072 bytes) Inode-cache hash table entries: 16384 (order: 4, 65536 bytes) Sorting _ex_table... Memory: 1027348K/1041136K available (4150K kernel code, 150K rwdma, 712K rodata, 2220K init, 143K bss, 13788K reserved, 0K cma-reserved, 851944K highmem) Hierarchical RCU implementation. Build-time adjustment of leaf fanout to 32. NR_IRQS:64 Console: colour dummy device 80x25 console [ttyS0] enabled console [ttyS0] enabled bootconsole [earlyprom0] disabled bootconsole [earlyprom0] disabled clocksource: timer_cs: mask: 0xffffffffffffffff max_cycles: 0x1d854df40, max_idle_ns: 3 526361616960 ns Calibrating delay loop... 249.44 BogoMIPS (lpj=1247232) pid_max: default: 32768 minimum: 301 Mount-cache hash table entries: 1024 (order: 0, 4096 bytes) Mountpoint-cache hash table entries: 1024 (order: 0, 4096 bytes) Entering SMP Mode... leon: SMP IPis at IRQ 13 0:(4:4) cpus mpirq at 0xff904010 Starting CPU 1 : (irqmp: 0xff904010) Started CPU 1 Starting CPU 2 : (irqmp: 0xff904010) Started CPU 2 Starting CPU 3 : (irqmp: 0xff904010) Started CPU 3 Brought up 4 CPUs Total of 4 processors activated (997.78 BogoMIPS). devtmpfs: initialized clocksource: jiffies: mask: 0xffffffff max_cycles: 0xffffffff, max_idle_ns: 19112604462 750000 ns futex hash table entries: 1024 (order: 3, 32768 bytes) NET: Registered protocol family 16 SCSI subsystem initialized clocksource: Switched to clocksource timer_cs FS-Cache: Loaded CacheFiles: Loaded NET: Registered protocol family 2 TCP established hash table entries: 2048 (order: 1, 8192 bytes) TCP bind hash table entries: 2048 (order: 2, 16384 bytes) TCP: Hash tables configured (established 2048 bind 2048) UDP hash table entries: 256 (order: 1, 8192 bytes) UDP-Lite hash table entries: 256 (order: 1, 8192 bytes) NET: Registered protocol family 1 RPC: Registered named UNIX socket transport module. RPC: Registered udp transport module. RPC: Registered tcp transport module. RPC: Registered tcp NFSv4.1 backchannel transport module. random: fast init done workingset: timestamp_bits=30 max_order=18 bucket_order=0 FS-Cache: Netfs 'nfs' registered for caching NFS: Registering the id_resolver key type Key type id_resolver registered Key type id_legacy registered jitterentropy: Initialization failed with host not compliant with requirements: 2 bounce: pool size: 64 pages io scheduler noop registered io scheduler deadline registered io scheduler cfq registered (default) Serial: GRLIB APBUART driver ff0e870: ttyS0 at MMIO 0xff900000 (irq = 10, base_baud = 15625000) is a GRLIB/APBUART grib-abuart at 0xff900000, irq 10 ff0e7a8: ttyS1 at MMIO 0xff901000 (irq = 11, base_baud = 15625000) is a GRLIB/APBUART grib-abuart at 0xff901000, irq 11 brd: module loaded loop: module loaded NET: Registered protocol family 10 sit: IPv6, IPv4 and MPLS over IPv4 tunneling driver NET: Registered protocol family 17 Key type dns_resolver registered leon: power management initialized Freeing unused kernel memory: 2220K This architecture does not have kernel memory protection. Starting logging: OK Initializing random number generator... done. Starting network: OK</pre>	<pre>PS C:\laysim-gr740\Linux> laysim-gr740-mdbt-cli -r -core0 Linux4.9-GR740-SMP.elf -ramsz 1024 laysim for GR740 Quad Core Processor (MDBT) CLI v3.0 by layright Multicore control : Time quantum (500) CORE#0 RAM file is loaded - Linux4.9-GR740-SMP.elf - .text, addr: 0x00000000, size 4320 bytes - .data, addr: 0x000010E0, size 80 bytes - .bss, addr: 0x00001130, size 8 bytes (not loaded, just info.) - .mmu_tables, addr: 0x00002000, size 2560 bytes (not loaded, just info.) - .vmlinux, addr: 0x00004000, size 7418304 bytes - .startup_prom, addr: 0x007171C0, size 40464 bytes Only GR740 CORE#0 RAM file is loaded, starts from RAM Set CORE#0 %sp/%fp : 0x3FFFFFF0/0x40000000 [prom_meminit_v0:26] bank[0] base=0x00000000, size=0x3FFFA970 PROMLIB: Sun Boot Prom Version 0 Revision 0 Linux version 4.9.54 (emanwon@Centos7SDE) (gcc version 4.9.4 (Cobham Gaisler Linux 4.9 Toolchain 1.1)) #6 SMP Thu Jan 2 15:34:50 KST 2020 bootconsole [earlyprom0] enabled ARCH: LEON TYPE: Leon3 System-on-a-Chip Ethernet address: 00:00:07:cc:01:45 CACHE: 4-way associative cache, set size 4k [bootmem_init:127] bank[i] base=0x00000000, size=0x3FFFA000 831MB HIGHMEM available. OF stdout device is: /a:a PROM: Built device tree with 68142 bytes of memory. Booting Linux... percpu: Embedded 9 pages/cpu @f145b000 s12736 r0 d24128 u36864 Built 1 zonelists in Zone order, mobility grouping on. Total pages: 259900 Kernel command line: console=ttyS0,115200 init=/sbin/init log_buf_len individual max cpu contribution: 4096 bytes log_buf_len total cpu_extra contributions: 12288 bytes log_buf_len min size: 16384 bytes log_buf_len: 32768 bytes early log buf free: 14608(89%) PID hash table entries: 1024 (order: 0, 4096 bytes) Dentry cache hash table entries: 32768 (order: 5, 131072 bytes) Inode-cache hash table entries: 16384 (order: 4, 65536 bytes) Sorting _ex_table... Memory: 1027348K/1041136K available (4150K kernel code, 150K rwdma, 712K rodata, 2220K init, 143K bss, 13788K reserved, 0K cma-reserved, 851944K highmem) Hierarchical RCU implementation. Build-time adjustment of leaf fanout to 32. NR_IRQS:64 Console: colour dummy device 80x25 console [ttyS0] enabled console [ttyS0] enabled bootconsole [earlyprom0] disabled bootconsole [earlyprom0] disabled clocksource: timer_cs: mask: 0xffffffffffffffff max_cycles: 0x1d854df40, max_idle_ns: 3 526361616960 ns Calibrating delay loop... 249.44 BogoMIPS (lpj=1247232) pid_max: default: 32768 minimum: 301 Mount-cache hash table entries: 1024 (order: 0, 4096 bytes) Mountpoint-cache hash table entries: 1024 (order: 0, 4096 bytes) Entering SMP Mode... leon: SMP IPis at IRQ 13 0:(4:4) cpus mpirq at 0xff904010 Starting CPU 1 : (irqmp: 0xff904010) Started CPU 1 Starting CPU 2 : (irqmp: 0xff904010) Started CPU 2 Starting CPU 3 : (irqmp: 0xff904010) Started CPU 3 Brought up 4 CPUs Total of 4 processors activated (997.78 BogoMIPS). devtmpfs: initialized clocksource: jiffies: mask: 0xffffffff max_cycles: 0xffffffff, max_idle_ns: 19112604462 750000 ns futex hash table entries: 1024 (order: 3, 32768 bytes) NET: Registered protocol family 16 SCSI subsystem initialized clocksource: Switched to clocksource timer_cs FS-Cache: Loaded CacheFiles: Loaded NET: Registered protocol family 2 TCP established hash table entries: 2048 (order: 1, 8192 bytes) TCP bind hash table entries: 2048 (order: 2, 16384 bytes) TCP: Hash tables configured (established 2048 bind 2048) UDP hash table entries: 256 (order: 1, 8192 bytes) UDP-Lite hash table entries: 256 (order: 1, 8192 bytes) NET: Registered protocol family 1 RPC: Registered named UNIX socket transport module. RPC: Registered udp transport module. RPC: Registered tcp transport module. RPC: Registered tcp NFSv4.1 backchannel transport module. random: fast init done workingset: timestamp_bits=30 max_order=18 bucket_order=0 FS-Cache: Netfs 'nfs' registered for caching NFS: Registering the id_resolver key type Key type id_resolver registered Key type id_legacy registered jitterentropy: Initialization failed with host not compliant with requirements: 2 bounce: pool size: 64 pages io scheduler noop registered io scheduler deadline registered io scheduler cfq registered (default) Serial: GRLIB APBUART driver ff0e870: ttyS0 at MMIO 0xff900000 (irq = 10, base_baud = 15625000) is a GRLIB/APBUART grib-abuart at 0xff900000, irq 10 ff0e7a8: ttyS1 at MMIO 0xff901000 (irq = 11, base_baud = 15625000) is a GRLIB/APBUART grib-abuart at 0xff901000, irq 11 brd: module loaded loop: module loaded NET: Registered protocol family 10 sit: IPv6, IPv4 and MPLS over IPv4 tunneling driver NET: Registered protocol family 17 Key type dns_resolver registered leon: power management initialized Freeing unused kernel memory: 2220K This architecture does not have kernel memory protection. Starting logging: OK Initializing random number generator... done. Starting network: OK</pre>

Welcome to Buildroot
buildroot login:

Welcome to Buildroot
buildroot login:

- **-romsz** : laysim-GR740 provides 8MB of MRAM by default, and the MRAM size can be modified using the **-romsz** option. The updated ROM size is reflected in FTMCTRL.MCFG1.
 - 16 : 16KB, 32 : 32KB, 64 : 64KB, 128 : 128KB, 256 : 256KB, 512 : 512KB, 1024 : 1MB, 2048 : 2MB, 4096 : 4MB,
 - 8192 : 8MB (default), 16384 : 16MB, 32768 : 32MB, 65536 : 64MB, 131072 : 128MB, 262144 : 256MB
- **-freq** : laysim-GR740 operates at 250MHz by default, and the operating frequency (in MHz) can be modified using the **-freq** option. The frequency can be set from a minimum of 50MHz up to a maximum of 250MHz, and performance differences can be observed depending on the selected frequency. The configured frequency is also applied to communication interfaces such as SpW, 1553B, CANm and UART.

Figure 3–16. Using '-freq' start-up option

laysim-gr740-cli in Linux	laysim-gr740-cli in Windows
<pre>[layright@CentOS BCC-2.2.2]\$ laysim-gr740-cli -r -core0 dhry.elf -freq 50 laysim for GR740 Quad Core Processor CLI v3.0 by layright CORE#0 RAM file is loaded - dhry.elf - .text, addr: 0x00000000, size 58544 bytes - .rodata, addr: 0x0000E4B0, size 2048 bytes - .data, addr: 0x0000ECB0, size 1576 bytes - .bss, addr: 0x0000F2D8, size 10680 bytes (not loaded, just info.) Only GR740 CORE#0 RAM file is loaded, starts from RAM Set CORE#0 %sp/%fp : 0x07FFFFFFF/0x08000000 Execution starts, 1000000 runs through Dhrystone Total execution time: 8.6 s Microseconds for one run through Dhrystone: 8.6 Dhrystones per Second: 116008.7 Dhrystones MIPS : CORE#0 halts because of Error Mode [434128415cycles/7.7824sec] 66.0 Performance statistics for CORE#0 Cycles : 434128415 Instructions : 343996263 Overall CPI : 1.26 CORE#0 performance (50MHz) : 39.62 MOPS (39.62 MIPS, 0.00 MFLOPS) Simulated time : 8.68 s Processor utilization : 100.00 % Performance of laysim-gr740 Real-time performance : 111.57 % laysim-gr740 performance : 44.20 MIPS Wall clock : 7.78 s [layright@CentOS BCC-2.2.2]\$ laysim-gr740-cli -r -core0 dhry.elf -freq 100 laysim for GR740 Quad Core Processor CLI v3.0 by layright CORE#0 RAM file is loaded - dhry.elf - .text, addr: 0x00000000, size 58544 bytes - .rodata, addr: 0x0000E4B0, size 2048 bytes - .data, addr: 0x0000ECB0, size 1576 bytes - .bss, addr: 0x0000F2D8, size 10680 bytes (not loaded, just info.) Only GR740 CORE#0 RAM file is loaded, starts from RAM Set CORE#0 %sp/%fp : 0x07FFFFFFF/0x08000000 Execution starts, 1000000 runs through Dhrystone Total execution time: 4.3 s Microseconds for one run through Dhrystone: 4.3 Dhrystones per Second: 232017.3 Dhrystones MIPS : CORE#0 halts because of Error Mode [437186565cycles/7.18135sec] 132.1 Performance statistics for CORE#0 Cycles : 437186565 Instructions : 344969338 Overall CPI : 1.27 CORE#0 performance (100MHz) : 78.91 MOPS (78.91 MIPS, 0.00 MFLOPS) Simulated time : 4.37 s Processor utilization : 100.00 % Performance of laysim-gr740 Real-time performance : 60.88 % laysim-gr740 performance : 48.04 MIPS Wall clock : 7.18 s [layright@CentOS BCC-2.2.2]\$</pre>	<pre>PS C:\laysim-gr740\BCC-2.2.2> laysim-gr740-cli -r -core0 dhry.elf -freq 50 laysim for GR740 Quad Core Processor CLI v3.0 by layright CORE#0 RAM file is loaded - dhry.elf - .text, addr: 0x00000000, size 58544 bytes - .rodata, addr: 0x0000E4B0, size 2048 bytes - .data, addr: 0x0000ECB0, size 1576 bytes - .bss, addr: 0x0000F2D8, size 10680 bytes (not loaded, just info.) Only GR740 CORE#0 RAM file is loaded, starts from RAM Set CORE#0 %sp/%fp : 0x07FFFFFFF/0x08000000 Execution starts, 1000000 runs through Dhrystone Total execution time: 8.6 s Microseconds for one run through Dhrystone: 8.6 Dhrystones per Second: 116008.7 Dhrystones MIPS : CORE#0 halts because of Error Mode [434128415cycles/8.04961sec] 66.0 Performance statistics for CORE#0 Cycles : 434128415 Instructions : 343996263 Overall CPI : 1.26 CORE#0 performance (50MHz) : 39.62 MOPS (39.62 MIPS, 0.00 MFLOPS) Simulated time : 8.68 s Processor utilization : 100.00 % Performance of laysim-gr740 Real-time performance : 107.86 % laysim-gr740 performance : 42.73 MIPS Wall clock : 8.05 s PS C:\laysim-gr740\BCC-2.2.2> laysim-gr740-cli -r -core0 dhry.elf -freq 100 laysim for GR740 Quad Core Processor CLI v3.0 by layright CORE#0 RAM file is loaded - dhry.elf - .text, addr: 0x00000000, size 58544 bytes - .rodata, addr: 0x0000E4B0, size 2048 bytes - .data, addr: 0x0000ECB0, size 1576 bytes - .bss, addr: 0x0000F2D8, size 10680 bytes (not loaded, just info.) Only GR740 CORE#0 RAM file is loaded, starts from RAM Set CORE#0 %sp/%fp : 0x07FFFFFFF/0x08000000 Execution starts, 1000000 runs through Dhrystone Total execution time: 4.3 s Microseconds for one run through Dhrystone: 4.3 Dhrystones per Second: 232017.3 Dhrystones MIPS : CORE#0 halts because of Error Mode [437186565cycles/7.81561sec] 132.1 Performance statistics for CORE#0 Cycles : 437186565 Instructions : 344969338 Overall CPI : 1.27 CORE#0 performance (100MHz) : 78.91 MOPS (78.91 MIPS, 0.00 MFLOPS) Simulated time : 4.37 s Processor utilization : 100.00 % Performance of laysim-gr740 Real-time performance : 55.94 % laysim-gr740 performance : 44.14 MIPS Wall clock : 7.82 s PS C:\laysim-gr740\BCC-2.2.2></pre>

- **-baud** : When RAM code is loaded, laysim-GR740 sets the default baudrate of APBUART#0 to 38400bps. The baudrate of APBUART#0 can be set using the **-baud** option (the **fast_uart** option is used separately).

Figure 3–17. Using '-baud' start-up option

laysim-gr740-cli in Linux	laysim-gr740-cli in Windows
<pre>[layright@CentOS BCC-2.2.2]\$ laysim-gr740-cli -r -core0 dhry.elf -baud 9600 laysim for GR740 Quad Core Processor CLI v3.0 by layright CORE#0 RAM file is loaded - dhry.elf - .text, addr: 0x00000000, size 58544 bytes - .rodata, addr: 0x0000E4B0, size 2048 bytes - .data, addr: 0x0000ECB0, size 1576 bytes - .bss, addr: 0x0000F2D8, size 10680 bytes (not loaded, just info.) Only GR740 CORE#0 RAM file is loaded, starts from RAM Set CORE#0 %sp/%fp : 0x07FFFFFF0/0x08000000 Execution starts, 1000000 runs through Dhrystone Total execution time: 1.7 s Microseconds for one run through Dhrystone: 1.7 Dhrystones per Second: 580043.4 Dhrystones MIPS : CORE#0 halts because of Error Mode [492234825cycles/7.47392sec] 330.1 Performance statistics for CORE#0 Cycles : 492234825 Instructions : 362484622 Overall CPI : 1.36 CORE#0 performance (250MHz) : 184.10 MOPS (184.10 MIPS, 0.00 MFLOPS) Simulated time : 1.97 s Processor utilization : 100.00 % Performance of laysim-gr740 Real-time performance : 26.34 % laysim-gr740 performance : 48.50 MIPS Wall clock : 7.47 s [layright@CentOS BCC-2.2.2]\$ laysim-gr740-cli -r -core0 dhry.elf -baud 115200 laysim for GR740 Quad Core Processor CLI v3.0 by layright CORE#0 RAM file is loaded - dhry.elf - .text, addr: 0x00000000, size 58544 bytes - .rodata, addr: 0x0000E4B0, size 2048 bytes - .data, addr: 0x0000ECB0, size 1576 bytes - .bss, addr: 0x0000F2D8, size 10680 bytes (not loaded, just info.) Only GR740 CORE#0 RAM file is loaded, starts from RAM Set CORE#0 %sp/%fp : 0x07FFFFFF0/0x08000000 Execution starts, 1000000 runs through Dhrystone Total execution time: 1.7 s Microseconds for one run through Dhrystone: 1.7 Dhrystones per Second: 580043.4 Dhrystones MIPS : CORE#0 halts because of Error Mode [436167101cycles/7.1424sec] 330.1 Performance statistics for CORE#0 Cycles : 436167101 Instructions : 344644891 Overall CPI : 1.27 CORE#0 performance (250MHz) : 197.54 MOPS (197.54 MIPS, 0.00 MFLOPS) Simulated time : 1.74 s Processor utilization : 100.00 % Performance of laysim-gr740 Real-time performance : 24.43 % laysim-gr740 performance : 48.25 MIPS Wall clock : 7.14 s [layright@CentOS BCC-2.2.2]\$</pre>	<pre>PS C:\laysim-gr740\BCC-2.2.2> laysim-gr740-cli -r -core0 dhry.elf -baud 9600 laysim for GR740 Quad Core Processor CLI v3.0 by layright CORE#0 RAM file is loaded - dhry.elf - .text, addr: 0x00000000, size 58544 bytes - .rodata, addr: 0x0000E4B0, size 2048 bytes - .data, addr: 0x0000ECB0, size 1576 bytes - .bss, addr: 0x0000F2D8, size 10680 bytes (not loaded, just info.) Only GR740 CORE#0 RAM file is loaded, starts from RAM Set CORE#0 %sp/%fp : 0x07FFFFFF0/0x08000000 Execution starts, 1000000 runs through Dhrystone Total execution time: 1.7 s Microseconds for one run through Dhrystone: 1.7 Dhrystones per Second: 580043.4 Dhrystones MIPS : CORE#0 halts because of Error Mode [492234825cycles/8.31481sec] 330.1 Performance statistics for CORE#0 Cycles : 492234825 Instructions : 362484622 Overall CPI : 1.36 CORE#0 performance (250MHz) : 184.10 MOPS (184.10 MIPS, 0.00 MFLOPS) Simulated time : 1.97 s Processor utilization : 100.00 % Performance of laysim-gr740 Real-time performance : 23.68 % laysim-gr740 performance : 43.60 MIPS Wall clock : 8.31 s PS C:\laysim-gr740\BCC-2.2.2> laysim-gr740-cli -r -core0 dhry.elf -baud 115200 laysim for GR740 Quad Core Processor CLI v3.0 by layright CORE#0 RAM file is loaded - dhry.elf - .text, addr: 0x00000000, size 58544 bytes - .rodata, addr: 0x0000E4B0, size 2048 bytes - .data, addr: 0x0000ECB0, size 1576 bytes - .bss, addr: 0x0000F2D8, size 10680 bytes (not loaded, just info.) Only GR740 CORE#0 RAM file is loaded, starts from RAM Set CORE#0 %sp/%fp : 0x07FFFFFF0/0x08000000 Execution starts, 1000000 runs through Dhrystone Total execution time: 1.7 s Microseconds for one run through Dhrystone: 1.7 Dhrystones per Second: 580043.4 Dhrystones MIPS : CORE#0 halts because of Error Mode [436167101cycles/7.73761sec] 330.1 Performance statistics for CORE#0 Cycles : 436167101 Instructions : 344644891 Overall CPI : 1.27 CORE#0 performance (250MHz) : 197.54 MOPS (197.54 MIPS, 0.00 MFLOPS) Simulated time : 1.74 s Processor utilization : 100.00 % Performance of laysim-gr740 Real-time performance : 22.55 % laysim-gr740 performance : 44.54 MIPS Wall clock : 7.74 s PS C:\laysim-gr740\BCC-2.2.2></pre>

- quantum : laysim-gr740-dbt and laysim-gr740-mdbt use the time quantum (TQ) method for multicore scheduling and synchronization. The default TQ is 500 (500 x 4nsec = 2usec), and it can be changed by '-quantum VALUE' on start. Increasing TQ improves RTP but may violate core synchronization because synchronization occurs at every quantum boundary. When the time quantum is set 0 in laysim-gr740-dbt, it changes the multicore scheduling to the basic block scheduling.

Figure 3–18. Using '-quantum' start-up option

laysim-gr740-dbt-cli in Linux	laysim-gr740-dbt-cli in Windows
<pre>[layright@CentOS smptests]\$ laysim-gr740-dbt-cli -r -core0 smpatomic01.elf -quantum 0 laysim for GR740 Quad Core Processor (DBT) CLI v3.0 by layright Multicore control : Basic block CORE#0 RAM file is loaded - smpatomic01.elf - .text, addr: 0x00000000, size 112384 bytes - .rtmsroset, addr: 0x0001B700, size 144 bytes - .rtmsstack, addr: 0x0001B7C0/0x0001B790, size 262144 bytes (not loaded, just info.) - .data, addr: 0x0005B7C0, size 1328 bytes - .bss, addr: 0x0005BD00, size 109088 bytes (not loaded, just info.) - Detect RTEMS v5.1(SMP) with thread information Only GR740 CORE#0 RAM file is loaded, starts from RAM Set CORE#0 %sp/%fp : 0x07FFFFFF0/0x08000000 *** BEGIN OF TEST SMPATOMIC 1 *** *** TEST VERSION: 5.1.0 *** TEST STATE: EXPECTED_PASS *** TEST BUILD: RTEMS_NETWORKING RTEMS_POSIX_API RTEMS_SMP *** TEST TOOLS: 7.5.0 20191114 (RTEMS 5, RSB 5.1, Newlib 7947581)</pre>	<pre>PS C:\laysim-gr740\RTSM55.1\smptests> laysim-gr740-dbt-cli -r -core0 smpatomic01.elf -q uantum 0 laysim for GR740 Quad Core Processor (DBT) CLI v3.0 by layright Multicore control : Basic block CORE#0 RAM file is loaded - smpatomic01.elf - .text, addr: 0x00000000, size 112384 bytes - .rtmsroset, addr: 0x0001B700, size 144 bytes - .rtmsstack, addr: 0x0001B7C0/0x0001B790, size 262144 bytes (not loaded, just info.) - .data, addr: 0x0005B7C0, size 1328 bytes - .bss, addr: 0x0005BD00, size 109088 bytes (not loaded, just info.) - Detect RTEMS v5.1(SMP) with thread information Only GR740 CORE#0 RAM file is loaded, starts from RAM Set CORE#0 %sp/%fp : 0x07FFFFFF0/0x08000000 *** BEGIN OF TEST SMPATOMIC 1 *** *** TEST VERSION: 5.1.0 *** TEST STATE: EXPECTED_PASS *** TEST BUILD: RTEMS_NETWORKING RTEMS_POSIX_API RTEMS_SMP *** TEST TOOLS: 7.5.0 20191114 (RTEMS 5, RSB 5.1, Newlib 7947581)</pre>


```

=== atomic add test case ===
worker 0 value: 8535834
worker 1 value: 8537689
worker 2 value: 8537689
worker 3 value: 8537621
atomic value: expected = 34148833, actual = 34148833
=== atomic flag test case ===
worker 0 value: 1
worker 1 value: 3
worker 2 value: 8255402
worker 3 value: 2
atomic value: expected = 8255408, actual = 8255408
=== atomic sub test case ===
worker 0 value: 4286386459
worker 1 value: 4286384604
worker 2 value: 4286384604
worker 3 value: 4286384676
atomic value: expected = 4260638455, actual = 4260638455
=== atomic compare exchange test case ===
worker 0 value: 1
worker 1 value: 3
worker 2 value: 2
worker 3 value: 8036103
atomic value: expected = 8036109, actual = 8036109
=== atomic or/and test case ===
worker 0 value: 0
worker 1 value: 2
worker 2 value: 4
worker 3 value: 0
atomic value: expected = 6, actual = 6
=== atomic fence test case ===
normal value = 31235609, second value = 31235609
=== atomic store release and load acquire test case ===
processor 0 delta 676ns, load count 0
processor 1 delta 712ns, load count 6
processor 2 delta 696ns, load count 5
processor 3 delta 692ns, load count 5
=== atomic read-modify-write test case ===
processor 0 delta 488ns, read-modify-write count 0
processor 1 delta 873ns, read-modify-write count 0
processor 2 delta 857ns, read-modify-write count 0
processor 3 delta 853ns, read-modify-write count 0
=== single writer seqlock test case ===
processor 0 count 16521098
processor 1 count 6355648
processor 2 count 6355648
processor 3 count 6355594
=== multi writer seqlock test case ===
processor 0 count 7572264
processor 1 count 1082358
processor 2 count 7572986
processor 3 count 1082244

*** END OF TEST SMPATOMIC 1 ***

CORE#3 halts because of Error Mode [7.04083sec/2127925182, 2127925198, 2127925186, 2127925092cycles]

Performance statistics for CORE#0
Cycles : 2127925182
Instructions : 1016348682
CORE#0 performance (250MHz) : 119.41 MIPS
Simulated time : 8.51 s

Performance statistics for CORE#1
Cycles : 2127925198
Instructions : 926280330
CORE#1 performance (250MHz) : 108.82 MIPS
Simulated time : 8.51 s

Performance statistics for CORE#2
Cycles : 2127925186
Instructions : 964699816
CORE#2 performance (250MHz) : 113.34 MIPS
Simulated time : 8.51 s

Performance statistics for CORE#3
Cycles : 2127925092
Instructions : 979470330
CORE#3 performance (250MHz) : 115.07 MIPS
Simulated time : 8.51 s

Performance of laysim-gr740
Real-time performance : 120.89 %
laysim-gr740 performance : 552.04 MIPS
Wall clock : 7.04 s

[layright@CentOS smptests]$ laysim-gr740-dbt-cli -r -core0 smpatomic01.elf -quantum 2000
laysim for GR740 Quad Core Processor (DBT) CLI v3.0 by layright

Multicore control : Time quantum (2000)
CORE#0 RAM file is loaded - smpatomic01.elf
- .text, addr: 0x00000000, size 112384 bytes
- .rtmsroset, addr: 0x0001B700, size 144 bytes
- .rtmsstack, addr: 0x0001B7C0/0x0001B790, size 262144 bytes (not loaded, just info.)
- .data, addr: 0x0005B7C0, size 1328 bytes
- .bss, addr: 0x0005BD00, size 109088 bytes (not loaded, just info.)
- Detect RTEMS v5.1(SMP) with thread information
Only GR740 CORE#0 RAM file is loaded, starts from RAM
Set CORE#0 %sp/%fp : 0x07FFFFFFF/0x08000000

*** BEGIN OF TEST SMPATOMIC 1 ***
*** TEST VERSION: 5.1.0
*** TEST STATE: EXPECTED_PASS
*** TEST BUILD: RTEMS_NETWORKING RTEMS_POSIX_API RTEMS_SMP
*** TEST TOOLS: 7.5.0 20191114 (RTEMS 5, RSB 5.1, Newlib 7947581)
=== atomic add test case ===
worker 0 value: 8535637
worker 1 value: 8537510
worker 2 value: 8537577
worker 3 value: 8537457
atomic value: expected = 34148181, actual = 34148181
=== atomic flag test case ===
worker 0 value: 8253390
worker 1 value: 8255201
worker 2 value: 8255265

```

```

=== atomic add test case ===
worker 0 value: 8535834
worker 1 value: 8537689
worker 2 value: 8537689
worker 3 value: 8537621
atomic value: expected = 34148833, actual = 34148833
=== atomic flag test case ===
worker 0 value: 1
worker 1 value: 3
worker 2 value: 8255402
worker 3 value: 2
atomic value: expected = 8255408, actual = 8255408
=== atomic sub test case ===
worker 0 value: 4286386459
worker 1 value: 4286384604
worker 2 value: 4286384604
worker 3 value: 4286384676
atomic value: expected = 4260638455, actual = 4260638455
=== atomic compare exchange test case ===
worker 0 value: 1
worker 1 value: 3
worker 2 value: 2
worker 3 value: 8036103
atomic value: expected = 8036109, actual = 8036109
=== atomic or/and test case ===
worker 0 value: 0
worker 1 value: 2
worker 2 value: 4
worker 3 value: 0
atomic value: expected = 6, actual = 6
=== atomic fence test case ===
normal value = 31235609, second value = 31235609
=== atomic store release and load acquire test case ===
processor 0 delta 676ns, load count 0
processor 1 delta 712ns, load count 6
processor 2 delta 696ns, load count 5
processor 3 delta 692ns, load count 5
=== atomic read-modify-write test case ===
processor 0 delta 488ns, read-modify-write count 0
processor 1 delta 873ns, read-modify-write count 0
processor 2 delta 857ns, read-modify-write count 0
processor 3 delta 853ns, read-modify-write count 0
=== single writer seqlock test case ===
processor 0 count 16521098
processor 1 count 6355648
processor 2 count 6355648
processor 3 count 6355594
=== multi writer seqlock test case ===
processor 0 count 7572264
processor 1 count 1082358
processor 2 count 7572986
processor 3 count 1082244

*** END OF TEST SMPATOMIC 1 ***

CORE#3 halts because of Error Mode [6.41161sec/2127925182, 2127925198, 2127925186, 2127925092cycles]

Performance statistics for CORE#0
Cycles : 2127925182
Instructions : 1016348682
CORE#0 performance (250MHz) : 119.41 MIPS
Simulated time : 8.51 s

Performance statistics for CORE#1
Cycles : 2127925198
Instructions : 926280330
CORE#1 performance (250MHz) : 108.82 MIPS
Simulated time : 8.51 s

Performance statistics for CORE#2
Cycles : 2127925186
Instructions : 964699816
CORE#2 performance (250MHz) : 113.34 MIPS
Simulated time : 8.51 s

Performance statistics for CORE#3
Cycles : 2127925092
Instructions : 979470330
CORE#3 performance (250MHz) : 115.07 MIPS
Simulated time : 8.51 s

Performance of laysim-gr740
Real-time performance : 132.75 %
laysim-gr740 performance : 606.21 MIPS
Wall clock : 6.41 s

PS C:\laysim-gr740\RTMS5.1\smptests> laysim-gr740-dbt-cli -r -core0 smpatomic01.elf -q
uantum 2000
laysim for GR740 Quad Core Processor (DBT) CLI v3.0 by layright

Multicore control : Time quantum (2000)
CORE#0 RAM file is loaded - smpatomic01.elf
- .text, addr: 0x00000000, size 112384 bytes
- .rtmsroset, addr: 0x0001B700, size 144 bytes
- .rtmsstack, addr: 0x0001B7C0/0x0001B790, size 262144 bytes (not loaded, just info.)
- .data, addr: 0x0005B7C0, size 1328 bytes
- .bss, addr: 0x0005BD00, size 109088 bytes (not loaded, just info.)
- Detect RTEMS v5.1(SMP) with thread information
Only GR740 CORE#0 RAM file is loaded, starts from RAM
Set CORE#0 %sp/%fp : 0x07FFFFFFF/0x08000000

*** BEGIN OF TEST SMPATOMIC 1 ***
*** TEST VERSION: 5.1.0
*** TEST STATE: EXPECTED_PASS
*** TEST BUILD: RTEMS_NETWORKING RTEMS_POSIX_API RTEMS_SMP
*** TEST TOOLS: 7.5.0 20191114 (RTEMS 5, RSB 5.1, Newlib 7947581)
=== atomic add test case ===
worker 0 value: 8535637
worker 1 value: 8537510
worker 2 value: 8537577
worker 3 value: 8537457
atomic value: expected = 34148181, actual = 34148181
=== atomic flag test case ===
worker 0 value: 8253390
worker 1 value: 8255201
worker 2 value: 8255265

```


<pre> worker 3 value: 8255146 atomic value: expected = 33019002, actual = 33019002 === atomic sub test case === worker 0 value: 4286431583 worker 1 value: 4286429710 worker 2 value: 4286429644 worker 3 value: 4286429767 atomic value: expected = 4260818816, actual = 4260818816 === atomic compare exchange test case === worker 0 value: 8034249 worker 1 value: 8036000 worker 2 value: 8036063 worker 3 value: 8035947 atomic value: expected = 32142259, actual = 32142259 === atomic or/and test case === worker 0 value: 0 worker 1 value: 0 worker 2 value: 4 worker 3 value: 8 atomic value: expected = 12, actual = 12 === atomic fence test case === normal value = 31218453, second value = 31218453 === atomic store release and load acquire test case === processor 0 delta 676ns, load count 0 processor 1 delta 9995876ns, load count 416397 processor 2 delta 9995868ns, load count 416397 processor 3 delta 9995884ns, load count 416397 === atomic read-modify-write test case === processor 0 delta 505ns, read-modify-write count 0 processor 1 delta 820ns, read-modify-write count 0 processor 2 delta 813ns, read-modify-write count 0 processor 3 delta 829ns, read-modify-write count 0 === single writer seqlock test case === processor 0 count 16570375 processor 1 count 6374613 processor 2 count 6374663 processor 3 count 6374571 === multi writer seqlock test case === processor 0 count 7572111 processor 1 count 6408562 processor 2 count 7573815 processor 3 count 6408520 *** END OF TEST SMPATOMIC 1 *** CORE#0 halts because of Error Mode [3.12638sec/2133715695, 2133714445, 2133714430, 2133714436cycles] Performance statistics for CORE#0 Cycles : 2133715695 Instructions : 1144269309 CORE#0 performance (250MHz) : 134.07 MIPS Simulated time : 8.53 s Performance statistics for CORE#1 Cycles : 2133714445 Instructions : 1057604360 CORE#1 performance (250MHz) : 123.92 MIPS Simulated time : 8.53 s Performance statistics for CORE#2 Cycles : 2133714430 Instructions : 1057016968 CORE#2 performance (250MHz) : 123.85 MIPS Simulated time : 8.53 s Performance statistics for CORE#3 Cycles : 2133714436 Instructions : 1063663881 CORE#3 performance (250MHz) : 124.63 MIPS Simulated time : 8.53 s Performance of laysim-gr740 Real-time performance : 272.99 % laysim-gr740 performance : 1382.57 MIPS Wall clock : 3.13 s [layright@CentOS smptests]\$ </pre>	<pre> worker 3 value: 8255146 atomic value: expected = 33019002, actual = 33019002 === atomic sub test case === worker 0 value: 4286431583 worker 1 value: 4286429710 worker 2 value: 4286429644 worker 3 value: 4286429767 atomic value: expected = 4260818816, actual = 4260818816 === atomic compare exchange test case === worker 0 value: 8034249 worker 1 value: 8036000 worker 2 value: 8036063 worker 3 value: 8035947 atomic value: expected = 32142259, actual = 32142259 === atomic or/and test case === worker 0 value: 0 worker 1 value: 0 worker 2 value: 4 worker 3 value: 8 atomic value: expected = 12, actual = 12 === atomic fence test case === normal value = 31218453, second value = 31218453 === atomic store release and load acquire test case === processor 0 delta 676ns, load count 0 processor 1 delta 9995876ns, load count 416397 processor 2 delta 9995868ns, load count 416397 processor 3 delta 9995884ns, load count 416397 === atomic read-modify-write test case === processor 0 delta 505ns, read-modify-write count 0 processor 1 delta 820ns, read-modify-write count 0 processor 2 delta 813ns, read-modify-write count 0 processor 3 delta 829ns, read-modify-write count 0 === single writer seqlock test case === processor 0 count 16570375 processor 1 count 6374613 processor 2 count 6374663 processor 3 count 6374571 === multi writer seqlock test case === processor 0 count 7572111 processor 1 count 6408562 processor 2 count 7573815 processor 3 count 6408520 *** END OF TEST SMPATOMIC 1 *** CORE#0 halts because of Error Mode [3.32281sec/2133715695, 2133714445, 2133714430, 2133714436cycles] Performance statistics for CORE#0 Cycles : 2133715695 Instructions : 1144269309 CORE#0 performance (250MHz) : 134.07 MIPS Simulated time : 8.53 s Performance statistics for CORE#1 Cycles : 2133714445 Instructions : 1057604360 CORE#1 performance (250MHz) : 123.92 MIPS Simulated time : 8.53 s Performance statistics for CORE#2 Cycles : 2133714430 Instructions : 1057016968 CORE#2 performance (250MHz) : 123.85 MIPS Simulated time : 8.53 s Performance statistics for CORE#3 Cycles : 2133714436 Instructions : 1063663881 CORE#3 performance (250MHz) : 124.63 MIPS Simulated time : 8.53 s Performance of laysim-gr740 Real-time performance : 256.86 % laysim-gr740 performance : 1300.87 MIPS Wall clock : 3.32 s PS C:\laysim-gr740\RTM55.1\smptests> </pre>
laysim-gr740-mdbt-cli in Linux <pre> [layright@CentOS smptests]\$ laysim-gr740-mdbt-cli -r -core0 smpatomic01.elf -quantum 50 laysim for GR740 Quad Core Processor (MDBT) CLI v3.0 by layright Multicore control : Time quantum (50) CORE#0 RAM file is loaded - smpatomic01.elf - .text, addr: 0x00000000, size 112384 bytes - .rtmsroset, addr: 0x0001B700, size 144 bytes - .rtmsstack, addr: 0x0001B7C0/0x0001B790, size 262144 bytes (not loaded, just info.) - .data, addr: 0x0005B7C0, size 1328 bytes - .bss, addr: 0x0005BD00, size 109088 bytes (not loaded, just info.) - Detect RTEMS v5.1(SMP) with thread information Only GR740 CORE#0 RAM file is loaded, starts from RAM Set CORE#0 %sp/%fp : 0x07FFFFFF0/0x08000000 *** BEGIN OF TEST SMPATOMIC 1 *** *** TEST VERSION: 5.1.0 *** TEST STATE: EXPECTED_PASS *** TEST BUILD: RTEMS_NETWORKING RTEMS_POSIX_API RTEMS_SMP *** TEST TOOLS: 7.5.0 20191114 (RTEMS 5, RSB 5.1, Newlib 7947581) === atomic add test case === worker 0 value: 4303216 worker 1 value: 4388135 worker 2 value: 4387714 worker 3 value: 4419437 atomic value: expected = 17498502, actual = 17498502 === atomic flag test case === worker 0 value: 3497305 worker 1 value: 3320602 worker 2 value: 2040679 worker 3 value: 2050422 atomic value: expected = 10909008, actual = 10909008 === atomic sub test case === worker 0 value: 4290680675 worker 1 value: 4290638482 worker 2 value: 4290829373 worker 3 value: 4290560663 atomic value: expected = 4277807305, actual = 4277807305 </pre>	laysim-gr740-mdbt-cli in Windows <pre> PS C:\laysim-gr740\RTM55.1\smptests> laysim-gr740-mdbt-cli -r -core0 smpatomic01.elf -quantum 50 laysim for GR740 Quad Core Processor (MDBT) CLI v3.0 by layright Multicore control : Time quantum (50) CORE#0 RAM file is loaded - smpatomic01.elf - .text, addr: 0x00000000, size 112384 bytes - .rtmsroset, addr: 0x0001B700, size 144 bytes - .rtmsstack, addr: 0x0001B7C0/0x0001B790, size 262144 bytes (not loaded, just info.) - .data, addr: 0x0005B7C0, size 1328 bytes - .bss, addr: 0x0005BD00, size 109088 bytes (not loaded, just info.) - Detect RTEMS v5.1(SMP) with thread information Only GR740 CORE#0 RAM file is loaded, starts from RAM Set CORE#0 %sp/%fp : 0x07FFFFFF0/0x08000000 *** BEGIN OF TEST SMPATOMIC 1 *** *** TEST VERSION: 5.1.0 *** TEST STATE: EXPECTED_PASS *** TEST BUILD: RTEMS_NETWORKING RTEMS_POSIX_API RTEMS_SMP *** TEST TOOLS: 7.5.0 20191114 (RTEMS 5, RSB 5.1, Newlib 7947581) === atomic add test case === worker 0 value: 4441759 worker 1 value: 4746770 worker 2 value: 4455658 worker 3 value: 4468611 atomic value: expected = 18112798, actual = 18112798 === atomic flag test case === worker 0 value: 3133045 worker 1 value: 2699219 worker 2 value: 2643254 worker 3 value: 1741763 atomic value: expected = 10217281, actual = 10217281 === atomic sub test case === worker 0 value: 4290574134 worker 1 value: 4290506787 worker 2 value: 4290477159 worker 3 value: 4290640176 atomic value: expected = 4277296368, actual = 4277296368 </pre>

```

=== atomic compare exchange test case ===
worker 0 value: 2605135
worker 1 value: 1555992
worker 2 value: 2591492
worker 3 value: 2963268
atomic value: expected = 9715887, actual = 9715887
=== atomic or/and test case ===
worker 0 value: 1
worker 1 value: 0
worker 2 value: 0
worker 3 value: 8
atomic value: expected = 9, actual = 9
=== atomic fence test case ===
normal value = 31235603, second value = 31235603
=== atomic store release and load acquire test case ===
processor 0 delta 676ns, load count 0
processor 1 delta 736ns, load count 5
processor 2 delta 748ns, load count 6
processor 3 delta 748ns, load count 6
=== atomic read-modify-write test case ===
processor 0 delta 505ns, read-modify-write count 0
processor 1 delta 897ns, read-modify-write count 0
processor 2 delta 909ns, read-modify-write count 0
processor 3 delta 909ns, read-modify-write count 0
=== single writer seqlock test case ===
processor 0 count 16521087
processor 1 count 1525873
processor 2 count 1418992
processor 3 count 1585057
=== multi writer seqlock test case ===
processor 0 count 2042805
processor 1 count 612646
processor 2 count 6679148
processor 3 count 622320

*** END OF TEST SMPATOMIC 1 ***

CORE#0 halts because of Error Mode [25.2135sec/2130297745, 2130297877, 2130297803, 2130
297764cycles]

Performance statistics for CORE#0
Cycles : 2130297745
Instructions : 1056769653
CORE#0 performance (250MHz) : 124.02 MIPS
Simulated time : 8.52 s

Performance statistics for CORE#1
Cycles : 2130297877
Instructions : 855865271
CORE#1 performance (250MHz) : 100.44 MIPS
Simulated time : 8.52 s

Performance statistics for CORE#2
Cycles : 2130297803
Instructions : 887158012
CORE#2 performance (250MHz) : 104.11 MIPS
Simulated time : 8.52 s

Performance statistics for CORE#3
Cycles : 2130297764
Instructions : 864141267
CORE#3 performance (250MHz) : 101.41 MIPS
Simulated time : 8.52 s

Performance of laysim-gr740
Real-time performance : 33.80 %
laysim-gr740 performance : 145.32 MIPS
Wall clock : 25.21 s

[layright@CentOS smptests]$ laysim-gr740-mdbt-cli -r -core0 smpatomic01.elf -quantum 20
00
laysim for GR740 Quad Core Processor (MDBT) CLI v3.0 by layright
Multicore control : Time quantum (2000)
CORE#0 RAM file is loaded - smpatomic01.elf
- .text, addr: 0x00000000, size 112384 bytes
- .rtmsroset, addr: 0x00018700, size 144 bytes
- .rtmsstack, addr: 0x000187C0/0x00018790, size 262144 bytes (not loaded, just inf
o.)
- .data, addr: 0x0005B7C0, size 1328 bytes
- .bss, addr: 0x0005BD00, size 109088 bytes (not loaded, just info.)
- Detect RTEMS v5.1(SMP) with thread information
Only GR740 CORE#0 RAM file is loaded, starts from RAM
Set CORE#0 %sp/%fp : 0x07FFFFFFF/0x08000000

*** BEGIN OF TEST SMPATOMIC 1 ***
*** TEST VERSION: 5.1.0
*** TEST STATE: EXPECTED_PASS
*** TEST BUILD: RTEMS_NETWORKING RTEMS_POSIX_API RTEMS_SMP
*** TEST TOOLS: 7.5.0 20191114 (RTEMS 5, RSB 5.1, Newlib 7947581)
=== atomic add test case ===
worker 0 value: 6099821
worker 1 value: 5931819
worker 2 value: 6741881
worker 3 value: 6714967
atomic value: expected = 25488488, actual = 25488488
=== atomic flag test case ===
worker 0 value: 2559015
worker 1 value: 3255293
worker 2 value: 3183237
worker 3 value: 2274633
atomic value: expected = 11272178, actual = 11272178
=== atomic sub test case ===
worker 0 value: 4288048232
worker 1 value: 4288329305
worker 2 value: 4289000955
worker 3 value: 4288290254
atomic value: expected = 4268766858, actual = 4268766858
=== atomic compare exchange test case ===
worker 0 value: 1414645
worker 1 value: 3276653
worker 2 value: 1718130
worker 3 value: 3297975
atomic value: expected = 9707403, actual = 9707403
=== atomic or/and test case ===
worker 0 value: 1
worker 1 value: 0
worker 2 value: 4
worker 3 value: 8

```

```

=== atomic compare exchange test case ===
worker 0 value: 1565829
worker 1 value: 2536530
worker 2 value: 3057991
worker 3 value: 2572203
atomic value: expected = 9732553, actual = 9732553
=== atomic or/and test case ===
worker 0 value: 1
worker 1 value: 2
worker 2 value: 4
worker 3 value: 8
atomic value: expected = 15, actual = 15
=== atomic fence test case ===
normal value = 31219341, second value = 31219341
=== atomic store release and load acquire test case ===
processor 0 delta 676ns, load count 0
processor 1 delta 736ns, load count 5
processor 2 delta 732ns, load count 5
processor 3 delta 732ns, load count 5
=== atomic read-modify-write test case ===
processor 0 delta 505ns, read-modify-write count 0
processor 1 delta 897ns, read-modify-write count 0
processor 2 delta 893ns, read-modify-write count 0
processor 3 delta 977ns, read-modify-write count 0
=== single writer seqlock test case ===
processor 0 count 16521086
processor 1 count 1696608
processor 2 count 1586495
processor 3 count 1689889
=== multi writer seqlock test case ===
processor 0 count 2917494
processor 1 count 986193
processor 2 count 5847625
processor 3 count 951132

*** END OF TEST SMPATOMIC 1 ***

CORE#0 halts because of Error Mode [25.3656sec/2130297243, 2130297287, 2130297303, 2130
297267cycles]

Performance statistics for CORE#0
Cycles : 2130297243
Instructions : 1044770857
CORE#0 performance (250MHz) : 122.61 MIPS
Simulated time : 8.52 s

Performance statistics for CORE#1
Cycles : 2130297287
Instructions : 865330630
CORE#1 performance (250MHz) : 101.55 MIPS
Simulated time : 8.52 s

Performance statistics for CORE#2
Cycles : 2130297303
Instructions : 895851399
CORE#2 performance (250MHz) : 105.13 MIPS
Simulated time : 8.52 s

Performance statistics for CORE#3
Cycles : 2130297267
Instructions : 862121773
CORE#3 performance (250MHz) : 101.17 MIPS
Simulated time : 8.52 s

Performance of laysim-gr740
Real-time performance : 33.59 %
laysim-gr740 performance : 144.61 MIPS
Wall clock : 25.37 s

PS C:\laysim-gr740\RTEMS5.1\smptests> laysim-gr740-mdbt-cli -r -core0 smpatomic01.elf -
quantum 2000
laysim for GR740 Quad Core Processor (MDBT) CLI v3.0 by layright
Multicore control : Time quantum (2000)
CORE#0 RAM file is loaded - smpatomic01.elf
- .text, addr: 0x00000000, size 112384 bytes
- .rtmsroset, addr: 0x00018700, size 144 bytes
- .rtmsstack, addr: 0x000187C0/0x00018790, size 262144 bytes (not loaded, just inf
o.)
- .data, addr: 0x0005B7C0, size 1328 bytes
- .bss, addr: 0x0005BD00, size 109088 bytes (not loaded, just info.)
- Detect RTEMS v5.1(SMP) with thread information
Only GR740 CORE#0 RAM file is loaded, starts from RAM
Set CORE#0 %sp/%fp : 0x07FFFFFFF/0x08000000

*** BEGIN OF TEST SMPATOMIC 1 ***
*** TEST VERSION: 5.1.0
*** TEST STATE: EXPECTED_PASS
*** TEST BUILD: RTEMS_NETWORKING RTEMS_POSIX_API RTEMS_SMP
*** TEST TOOLS: 7.5.0 20191114 (RTEMS 5, RSB 5.1, Newlib 7947581)
=== atomic add test case ===
worker 0 value: 6694120
worker 1 value: 6321273
worker 2 value: 5314766
worker 3 value: 5898270
atomic value: expected = 24228429, actual = 24228429
=== atomic flag test case ===
worker 0 value: 1878514
worker 1 value: 3083224
worker 2 value: 2024119
worker 3 value: 3251553
atomic value: expected = 10237410, actual = 10237410
=== atomic sub test case ===
worker 0 value: 4290031331
worker 1 value: 4287686111
worker 2 value: 428858283
worker 3 value: 4289265185
atomic value: expected = 4270939022, actual = 4270939022
=== atomic compare exchange test case ===
worker 0 value: 1321672
worker 1 value: 2931470
worker 2 value: 2646907
worker 3 value: 3079525
atomic value: expected = 9979574, actual = 9979574
=== atomic or/and test case ===
worker 0 value: 0
worker 1 value: 2
worker 2 value: 0
worker 3 value: 8

```

```

atomic value: expected = 13, actual = 13
=== atomic fence test case ===
normal value = 31218534, second value = 31218534
=== atomic store release and load acquire test case ===
processor 0 delta 676ns, load count 0
processor 1 delta 2884ns, load count 100
processor 2 delta 508ns, load count 1
processor 3 delta 3392ns, load count 121
=== atomic read-modify-write test case ===
processor 0 delta 505ns, read-modify-write count 0
processor 1 delta 645ns, read-modify-write count 0
processor 2 delta 669ns, read-modify-write count 0
processor 3 delta 753ns, read-modify-write count 0
=== single writer seqlock test case ===
processor 0 count 16494772
processor 1 count 437855
processor 2 count 350428
processor 3 count 289337
=== multi writer seqlock test case ===
processor 0 count 5005033
processor 1 count 1378885
processor 2 count 5365026
processor 3 count 1354624

*** END OF TEST SMPATOMIC 1 ***

CORE#0 halts because of Error Mode [8.12371sec/2127937693, 2127938429, 2127938376, 2127938469cycles]

Performance statistics for CORE#0
Cycles : 2127937693
Instructions : 1057817443
CORE#0 performance (250MHz) : 124.28 MIPS
Simulated time : 8.51 s

Performance statistics for CORE#1
Cycles : 2127938429
Instructions : 90068590
CORE#1 performance (250MHz) : 105.74 MIPS
Simulated time : 8.51 s

Performance statistics for CORE#2
Cycles : 2127938376
Instructions : 936708622
CORE#2 performance (250MHz) : 110.05 MIPS
Simulated time : 8.51 s

Performance statistics for CORE#3
Cycles : 2127938469
Instructions : 894725097
CORE#3 performance (250MHz) : 105.12 MIPS
Simulated time : 8.51 s

Performance of laysim-gr740
Real-time performance : 104.78 %
laysim-gr740 performance : 466.45 MIPS
Wall clock : 8.12 s
[layright@CentOS smptests]$

atomic value: expected = 10, actual = 10
=== atomic fence test case ===
normal value = 31218451, second value = 31218451
=== atomic store release and load acquire test case ===
processor 0 delta 676ns, load count 0
processor 1 delta 1668ns, load count 42
processor 2 delta 10003788ns, load count 416730
processor 3 delta 10003784ns, load count 416730
=== atomic read-modify-write test case ===
processor 0 delta 488ns, read-modify-write count 0
processor 1 delta 628ns, read-modify-write count 0
processor 2 delta 4732ns, read-modify-write count 0
processor 3 delta 4729ns, read-modify-write count 0
=== single writer seqlock test case ===
processor 0 count 16583317
processor 1 count 417577
processor 2 count 347567
processor 3 count 414856
=== multi writer seqlock test case ===
processor 0 count 4636006
processor 1 count 1729055
processor 2 count 5443582
processor 3 count 1838102

*** END OF TEST SMPATOMIC 1 ***

CORE#0 halts because of Error Mode [8.56442sec/2130443696, 2130444426, 2130444377, 213044470cycles]

Performance statistics for CORE#0
Cycles : 2130443696
Instructions : 1042916479
CORE#0 performance (250MHz) : 122.38 MIPS
Simulated time : 8.52 s

Performance statistics for CORE#1
Cycles : 2130444426
Instructions : 905802376
CORE#1 performance (250MHz) : 106.29 MIPS
Simulated time : 8.52 s

Performance statistics for CORE#2
Cycles : 2130444377
Instructions : 929371052
CORE#2 performance (250MHz) : 109.06 MIPS
Simulated time : 8.52 s

Performance statistics for CORE#3
Cycles : 2130444470
Instructions : 908082668
CORE#3 performance (250MHz) : 106.56 MIPS
Simulated time : 8.52 s

Performance of laysim-gr740
Real-time performance : 99.32 %
laysim-gr740 performance : 441.28 MIPS
Wall clock : 8.58 s
PS C:\laysim-gr740\RTMS5.1\smptests>

```

- mod : Load loadable module (SPI, UART, GPIO, PCI, I/O, 1553B RT, SpW Node and CAN Node modules). A loadable module should be dynamic shared objects (.so file on Linux and .dll file on Windows) and should follow the loadable interfaces of laysim-GR740. Where its interfaces and source codes are only available for the professional version.

Figure 3–19. Using '-mod' start-up option

laysim-gr740-dbt-cli in Linux	laysim-gr740-cli in Windows
<pre> [layright@CentOS Samples]\$ laysim-gr740-dbt-cli -r -core0 rtems-rmap-logical-addressing.elf -mod ../../Loadable/loadable-SpW-Nodes/laysim_user_spw_node.so laysim for GR740 Quad Core Processor (DBT) CLI v3.0 by layright Loaded user loadable module ../../Loadable/loadable-SpW-Nodes/laysim_user_spw_node.so Multicore control : Time quantum (500) CORE#0 RAM file is loaded - rtems-rmap-logical-addressing.elf - .text, addr: 0x00000000, size 230576 bytes - .rtemsreset, addr: 0x00038400, size 176 bytes - .rtemsstack, addr: 0x00038580, size 8192 bytes (not loaded, just info.) - .data, addr: 0x0003A580, size 8272 bytes - .bss, addr: 0x0003C600, size 171024 bytes (not loaded, just info.) - Detect RTEMS v5.1(SMP) with thread information Only GR740 CORE#0 RAM file is loaded, starts from RAM Set CORE#0 %sp/%fp : 0x07FFFFFF0/0x08000000 #### To change configuration of this example, you can set a breakpoint #### on label <rmap_conf_label> and modify the following global variables: ## remote_base_address: 0x40000000 /* Base address on Remote. This is the address to which the data is copied. */ ## remote_dst_address: 0x42 /* SpW Destination address. */ ## remote_dst_key: 0x00 /* SpW Destination Key */ ## source_src_address: 0x20 /* SpW Source address */ ## rmap_dev_idx: 0 /* This parameter chooses which initiator GRSPW2 core is used */ ## route_entry { nr; /* Number of addresses in dstadr array */; dstadr[16]; /* Path Addresses */} ## routetab[ROUTE_TO]={1,{3,0,0, ...}} ## routetab[ROUTE_FROM]={1,{9,0,0, ...}} ## path_addressing: 0 /* Use path addressing (!=0) or not (0) */ ## rmap_pkt_size: 992 ## rmap_data_bytes: 4096 /* Data bytes to send over RMAP */ #### RTEMS RMAP example Setting up SpaceWire router Configuring Router Hardware Configuration of SpaceWire Router: Number of SpW ports: 8 Number of AMBA ports: 4 </pre>	<pre> PS C:\laysim-gr740\RCC-1.3.1\Samples> laysim-gr740-dbt-cli -r -core0 rtems-rmap-logical-addressing.elf -mod ../../Loadable/loadable-SpW-Nodes\laysim_user_spw_node-dbt-cli.dll laysim for GR740 Quad Core Processor (DBT) CLI v3.0 by layright Loaded user loadable module ../../Loadable/loadable-SpW-Nodes\laysim_user_spw_node-dbt-cli.dll Multicore control : Time quantum (500) CORE#0 RAM file is loaded - rtems-rmap-logical-addressing.elf - .text, addr: 0x00000000, size 230576 bytes - .rtemsreset, addr: 0x00038400, size 176 bytes - .rtemsstack, addr: 0x00038580, size 8192 bytes (not loaded, just info.) - .data, addr: 0x0003A580, size 8272 bytes - .bss, addr: 0x0003C600, size 171024 bytes (not loaded, just info.) - Detect RTEMS v5.1(SMP) with thread information Only GR740 CORE#0 RAM file is loaded, starts from RAM Set CORE#0 %sp/%fp : 0x07FFFFFF0/0x08000000 #### To change configuration of this example, you can set a breakpoint #### on label <rmap_conf_label> and modify the following global variables: ## remote_base_address: 0x40000000 /* Base address on Remote. This is the address to which the data is copied. */ ## remote_dst_address: 0x42 /* SpW Destination address. */ ## remote_dst_key: 0x00 /* SpW Destination Key */ ## source_src_address: 0x20 /* SpW Source address */ ## rmap_dev_idx: 0 /* This parameter chooses which initiator GRSPW2 core is used */ ## route_entry { nr; /* Number of addresses in dstadr array */; dstadr[16]; /* Path Addresses */} ## routetab[ROUTE_TO]={1,{3,0,0, ...}} ## routetab[ROUTE_FROM]={1,{9,0,0, ...}} ## path_addressing: 0 /* Use path addressing (!=0) or not (0) */ ## rmap_pkt_size: 992 ## rmap_data_bytes: 4096 /* Data bytes to send over RMAP */ #### RTEMS RMAP example Setting up SpaceWire router Configuring Router Hardware Configuration of SpaceWire Router: Number of SpW ports: 8 Number of AMBA ports: 4 </pre>

```

Number of FIFO ports:      0
Timers available:         YES
Plug and Play available:  YES
MAJOR Version:           1
MINOR Version:           2
PATCH Version:          0
Current Instance ID:      19
ROUTER CFG/STS: 0x4100c013
PORT[00]: TYPE=CFG CTRL=0x01000210 STATUS=0x01000210 LINKSTATE=N/A
PORT[01]: TYPE=SpW CTRL=0x2700022c STATUS=0x0000a000 LINKSTATE=Ready
PORT[02]: TYPE=SpW CTRL=0x2700022c STATUS=0x0000a000 LINKSTATE=Ready
PORT[03]: TYPE=SpW CTRL=0x2700022c STATUS=0x0000a000 LINKSTATE=Ready
PORT[04]: TYPE=SpW CTRL=0x2700022c STATUS=0x0000a000 LINKSTATE=Ready
PORT[05]: TYPE=SpW CTRL=0x2700022c STATUS=0x0000a000 LINKSTATE=Ready
PORT[06]: TYPE=SpW CTRL=0x2700022c STATUS=0x0000a000 LINKSTATE=Ready
PORT[07]: TYPE=SpW CTRL=0x2700022c STATUS=0x0000a000 LINKSTATE=Ready
PORT[08]: TYPE=SpW CTRL=0x2700022c STATUS=0x0000a000 LINKSTATE=Ready
PORT[09]: TYPE=AMBA CTRL=0x00000228 STATUS=0x40000000 LINKSTATE=N/A
PORT[10]: TYPE=AMBA CTRL=0x00000228 STATUS=0x40000000 LINKSTATE=N/A
PORT[11]: TYPE=AMBA CTRL=0x00000228 STATUS=0x40000000 LINKSTATE=N/A
PORT[12]: TYPE=AMBA CTRL=0x00000228 STATUS=0x40000000 LINKSTATE=N/A
Activating all Links/Ports
PORT[01]: TYPE=SpW CTRL=0x0300002e STATUS=0x0000a000 LINK-STATE=Ready
PORT[02]: TYPE=SpW CTRL=0x0300002e STATUS=0x0000a000 LINK-STATE=Ready
PORT[03]: TYPE=SpW CTRL=0x0300002e STATUS=0x0000d000 LINK-STATE=Run
PORT[04]: TYPE=SpW CTRL=0x0300002e STATUS=0x0000a000 LINK-STATE=Ready
PORT[05]: TYPE=SpW CTRL=0x0300002e STATUS=0x0000a000 LINK-STATE=Ready
PORT[06]: TYPE=SpW CTRL=0x0300002e STATUS=0x0000a000 LINK-STATE=Ready
PORT[07]: TYPE=SpW CTRL=0x0300002e STATUS=0x0000a000 LINK-STATE=Ready
PORT[08]: TYPE=SpW CTRL=0x0300002e STATUS=0x0000a000 LINK-STATE=Ready
Configuring Router
Failed to open SpW Router
Initializing SpaceWire device 0
NOTE: running on SPW-ROUTER DMA SpaceWire link (no link-state available)
Initializing SpaceWire device 1
NOTE: running on SPW-ROUTER DMA SpaceWire link (no link-state available)
Initializing SpaceWire device 2
NOTE: running on SPW-ROUTER DMA SpaceWire link (no link-state available)
Initializing SpaceWire device 3
NOTE: running on SPW-ROUTER DMA SpaceWire link (no link-state available)

Test built for at least 4 AMBA ports.
Note that the SpW-links will not be used.
There are 4 GRSPW cores in the system.
System clock: 10000 us / tick

Starting GRSPW0: DMA Started Successfully
NOTE: running on SPW-ROUTER DMA SpaceWire link (no link-state available)
RMAP connection is working
Initializing data array

Starting transmission type 1: one packet at a time

RMAP write cycle completed in 0.000409 seconds
RMAP read cycle completed in 0.000416 seconds
Data bytes sent: 4096, Packet size: 992, Packet count: 5
Wrote 4096 bytes to target in 0.000409 seconds. WRITE RATE: 9782.232179 Kib/s
Read 4096 bytes from target in 0.000416 seconds. READ RATE: 9610.276912 Kib/s
Wrote 5 packets to target in 0.000409 seconds. WRITE RATE: 12227.790224 pkt/s
Read 5 packets from target in 0.000416 seconds. READ RATE: 12012.846140 pkt/s

EXAMPLE 1 SUCCESSFULLY COMPLETED.

Starting transmission type 2: all packets at a time

RMAP write cycle completed in 0.000373 seconds
RMAP read cycle completed in 0.000127 seconds
Data bytes sent: 4096, Packet size: 992, Packet count: 5
Wrote 4096 bytes to target in 0.000373 seconds. WRITE RATE: 10735.271186 Kib/s
Read 4096 bytes from target in 0.000127 seconds. READ RATE: 31536.120301 Kib/s
Wrote 5 packets to target in 0.000373 seconds. WRITE RATE: 13419.088982 pkt/s
Read 5 packets from target in 0.000127 seconds. READ RATE: 39420.150376 pkt/s

EXAMPLE 2 SUCCESSFULLY COMPLETED.

CORE#0 halts because of Error Mode [0.297153sec/333129798, 0, 0, 0cycles]

Performance statistics for CORE#0
Cycles : 333129798
Instructions : 211294714
CORE#0 performance (250MHz) : 158.57 MIPS
Simulated time : 1.33 s

Performance of laysim-gr740
Real-time performance : 448.43 %
laysim-gr740 performance : 711.06 MIPS
Wall clock : 0.30 s
[laysim-gr740@CentOS Samples]$

```

```

Number of FIFO ports:      0
Timers available:         YES
Plug and Play available:  YES
MAJOR Version:           1
MINOR Version:           2
PATCH Version:          0
Current Instance ID:      19
ROUTER CFG/STS: 0x4100c013
PORT[00]: TYPE=CFG CTRL=0x01000210 STATUS=0x01000210 LINKSTATE=N/A
PORT[01]: TYPE=SpW CTRL=0x2700022c STATUS=0x0000a000 LINKSTATE=Ready
PORT[02]: TYPE=SpW CTRL=0x2700022c STATUS=0x0000a000 LINKSTATE=Ready
PORT[03]: TYPE=SpW CTRL=0x2700022c STATUS=0x0000a000 LINKSTATE=Ready
PORT[04]: TYPE=SpW CTRL=0x2700022c STATUS=0x0000a000 LINKSTATE=Ready
PORT[05]: TYPE=SpW CTRL=0x2700022c STATUS=0x0000a000 LINKSTATE=Ready
PORT[06]: TYPE=SpW CTRL=0x2700022c STATUS=0x0000a000 LINKSTATE=Ready
PORT[07]: TYPE=SpW CTRL=0x2700022c STATUS=0x0000a000 LINKSTATE=Ready
PORT[08]: TYPE=SpW CTRL=0x2700022c STATUS=0x0000a000 LINKSTATE=Ready
PORT[09]: TYPE=AMBA CTRL=0x00000228 STATUS=0x40000000 LINKSTATE=N/A
PORT[10]: TYPE=AMBA CTRL=0x00000228 STATUS=0x40000000 LINKSTATE=N/A
PORT[11]: TYPE=AMBA CTRL=0x00000228 STATUS=0x40000000 LINKSTATE=N/A
PORT[12]: TYPE=AMBA CTRL=0x00000228 STATUS=0x40000000 LINKSTATE=N/A
Activating all Links/Ports
PORT[01]: TYPE=SpW CTRL=0x0300002e STATUS=0x0000a000 LINK-STATE=Ready
PORT[02]: TYPE=SpW CTRL=0x0300002e STATUS=0x0000a000 LINK-STATE=Ready
PORT[03]: TYPE=SpW CTRL=0x0300002e STATUS=0x0000d000 LINK-STATE=Run
PORT[04]: TYPE=SpW CTRL=0x0300002e STATUS=0x0000a000 LINK-STATE=Ready
PORT[05]: TYPE=SpW CTRL=0x0300002e STATUS=0x0000a000 LINK-STATE=Ready
PORT[06]: TYPE=SpW CTRL=0x0300002e STATUS=0x0000a000 LINK-STATE=Ready
PORT[07]: TYPE=SpW CTRL=0x0300002e STATUS=0x0000a000 LINK-STATE=Ready
PORT[08]: TYPE=SpW CTRL=0x0300002e STATUS=0x0000a000 LINK-STATE=Ready
Configuring Router
Failed to open SpW Router
Initializing SpaceWire device 0
NOTE: running on SPW-ROUTER DMA SpaceWire link (no link-state available)
Initializing SpaceWire device 1
NOTE: running on SPW-ROUTER DMA SpaceWire link (no link-state available)
Initializing SpaceWire device 2
NOTE: running on SPW-ROUTER DMA SpaceWire link (no link-state available)
Initializing SpaceWire device 3
NOTE: running on SPW-ROUTER DMA SpaceWire link (no link-state available)

Test built for at least 4 AMBA ports.
Note that the SpW-links will not be used.
There are 4 GRSPW cores in the system.
System clock: 10000 us / tick

Starting GRSPW0: DMA Started Successfully
NOTE: running on SPW-ROUTER DMA SpaceWire link (no link-state available)
RMAP connection is working
Initializing data array

Starting transmission type 1: one packet at a time

RMAP write cycle completed in 0.000409 seconds
RMAP read cycle completed in 0.000416 seconds
Data bytes sent: 4096, Packet size: 992, Packet count: 5
Wrote 4096 bytes to target in 0.000409 seconds. WRITE RATE: 9782.232179 Kib/s
Read 4096 bytes from target in 0.000416 seconds. READ RATE: 9610.276912 Kib/s
Wrote 5 packets to target in 0.000409 seconds. WRITE RATE: 12227.790224 pkt/s
Read 5 packets from target in 0.000416 seconds. READ RATE: 12012.846140 pkt/s

EXAMPLE 1 SUCCESSFULLY COMPLETED.

Starting transmission type 2: all packets at a time

RMAP write cycle completed in 0.000373 seconds
RMAP read cycle completed in 0.000127 seconds
Data bytes sent: 4096, Packet size: 992, Packet count: 5
Wrote 4096 bytes to target in 0.000373 seconds. WRITE RATE: 10735.271186 Kib/s
Read 4096 bytes from target in 0.000127 seconds. READ RATE: 31536.120301 Kib/s
Wrote 5 packets to target in 0.000373 seconds. WRITE RATE: 13419.088982 pkt/s
Read 5 packets from target in 0.000127 seconds. READ RATE: 39420.150376 pkt/s

EXAMPLE 2 SUCCESSFULLY COMPLETED.

CORE#0 halts because of Error Mode [0.904802sec/333129798, 0, 0, 0cycles]

Performance statistics for CORE#0
Cycles : 333129798
Instructions : 211294714
CORE#0 performance (250MHz) : 158.57 MIPS
Simulated time : 1.33 s

Performance of laysim-gr740
Real-time performance : 147.27 %
laysim-gr740 performance : 233.53 MIPS
Wall clock : 0.90 s
PS C:\laysim-gr740\RCC-1.3.1\Samples>

```

- **-air** : Configuration of GRETH and GRCAN for AIR hypervisor test suites.
- **-cfs** : Configuration of GRETH for NASA cFS with IP 10.10.10.22 and UDP port 1234/1235 for telecommand/telemetry.
- **-gdb** : Listen for GDB connection directly at start-up. This option is only available in the CLI version of laysim-GR740.
- **-port** : Set the port number to be used for GDB communication. The default port number is 1234. This option is only available in the

CLI version of laysim-GR740.

3.3 Control Console of laysim-GR740

The control console of laysim-GR740 provides a variety of command line interfaces for controlling laysim-GR740 described below.

- **help** : The help command shows available commands in the control console as shown in Figure 3-20.

Figure 3-20. help command

```

help command in laysim-gr740-cli

laysim> help

Execute help
dis/display [addr]/[sym] [count] : Disassemble [count] instructions at [addr]/[sym]
mem/memory [addr]/[sym] [count] : Show memory at [addr]/[sym] with [count]
wmem [addr]/[sym] [value] : Write a word to memory at [addr]/[sym] with [value]
memdump [addr]/[sym] [wcnt] [file] : Save memory at [addr]/[sym] with [wcnt] to [file]
load [file] : Load GR740 RAM [file] to CORE#0
- Only a file can be loaded once
batch [file] : Execute a batch [file] of laysim commands
save [file] : Save laysim-gr740 state to [file]
- It also saves the state of loadable modules
restore [file] : Restore laysim-gr740 state from [file]
- It also restores the state of loadable modules
reg/register : Show GR740 SPARC IU registers
float : Show GR740 SPARC FPU registers
s/step : Single step
n/next : Single step with showing IU registers
t/trace [count] : Trace with [count]
r/run : Run or continue
rundelta [dcycles] : Run till current cycles + [dcycles]
reset : Reset laysim-GR740 state on stop mode
b/break [addr]/[sym] : Add a breakpoint at [addr]/[sym]
d/del [addr]/[sym] : Delete a breakpoint at [addr]/[sym]
watch [addr]/[sym] : Add a watchpoint at [addr]/[sym]
dw/dwatch [addr]/[sym] : Delete a watchpoint at [addr]/[sym]
icache [cmd] : GR740 instruction cache operation
: icache [core] disable - Disable CORE# icache
: icache [core] enable - Enable CORE# icache
: icache [core] frozen - Freeze CORE# icache
: icache [core] flush - Flush CORE# icache
: icache [core] ite - Set CORE# ite in icache
: icache [core] ide - Set CORE# ide in icache
: icache [core] dump - Dump CORE#0/#1 icache
dcache [cmd] : GR740 data cache operation
: dcache [core] disable - Disable CORE# dcache
: dcache [core] enable - Enable CORE# dcache
: dcache [core] frozen - Freeze CORE# dcache
: dcache [core] flush - Flush CORE# dcache
: dcache [core] dte - Set CORE# dte in dcache
: dcache [core] dde - Set CORE# dde in dcache
: dcache [core] dump - Dump CORE# dcache
l2cache [cmd] : GR740 L2 cache operation
: l2cache [way] [count] [index] - Dump L2 cache
: l2cache disable - Disable L2 cache
: l2cache enable - Enable L2 cache
: l2cache flush - Flush L2 cache
core [cmd] : CORE status and debug selection
: core - Show status of cores
: core 0 - CORE#0 is selected for debugging
: core 1 - CORE#1 is selected for debugging
: core 2 - CORE#2 is selected for debugging
: core 3 - CORE#3 is selected for debugging
: core 4 - All COREs are selected for debugging
history [core] : Show executed instructions (Max 1000)
wakeup [core] : Wake-up CORE#n with [core]
perf : Show performance statistics
: perf reset - Reset performance statistics
fastuart : Set UART baudrate
: fastuart on - Set UART baudrate as infinite
: fastuart off - Set UART baudrate as correct
rtems : Show RTEMS v4.8/4.10/5.1(SMP only) information
: rtems task - Show RTEMS task information
: rtems task [id] - Show RTEMS selected task information
: rtems trap - Show RTEMS trap information
: rtems interrupt - Show RTEMS BSP interrupt information (v4.10/5.1)
: rtems core [num] - Show RTEMS-SMP core [num] information
mmu [core] : Show MMU registers for selected core
: mmu 0 - Show MMU registers for CORE#0
: mmu 1 - Show MMU registers for CORE#1
walk [core] vaddr : Printout a MMU table walk for the given vaddr
: walk 0 0xFC000000 - Show vaddr to paddr of CORE#0

```

```

: walk 1 0xFC000100          - Show vaddr to paddr of CORE#1
vdis [core] [vaddr] [count]  : Disassemble [count] instructions at [vaddr]
: vdis 0 0xFC009350 0x10     - Disassemble instructions of CORE#0 at virtual address
air                          : Show AIR/GMV hypervisor information
: air                       - Show AIR/GMV information
: air break                 - Show AIR partition breakpoints
: air break [part] [addr]/[sym] - Add a breakpoint at AIR partition
: air del [part] [addr]/[sym] - Delete a breakpoint at AIR partition
: air pload [part] file      - Load the debugging information from AIR partition file
xng-setup [addr]            : Set up XNG RAM execution with XCF address
event                       : Show events in the event queue
info sys                    : Show GR740 system information
info reg [core_name]        : Show all registers of [core_name]
iommu lookup [base] [ioaddr] : Lookup specified IO address in page table starting at base
gdb                         : Start GDB server
quit                       : Quit laysim CLI
laysim>

```

help command in laysim-gr740-dbt-cli

laysim> help

```

Execute help
dis/display [addr]/[sym] [count] : Disassemble [count] instructions at [addr]/[sym]
mem/memory [addr]/[sym] [count] : Show memory at [addr]/[sym] with [count]
wmem [addr]/[sym] [value]       : Write a word to memory at [addr]/[sym] with [value]
memdump [addr]/[sym] [wcnt] [file] : Save memory at [addr]/[sym] with [wcnt] to [file]
load [file]                     : Load GR740 RAM [file] to CORE#0 on stop/reset mode
                                - Only a file can be loaded once
batch [file]                    : Execute a batch [file] of laysim commands
save [file]                     : Save laysim-gr740 state to [file]
                                - It also saves the state of loadable modules
restore [file]                  : Restore laysim-gr740 state from [file]
                                - It also restores the state of loadable modules
reg/register                    : Show GR740 SPARC IU registers
float                          : Show GR740 SPARC FPU registers
s/step                         : Single step
n/next                         : Single step with showing IU registers
t/trace [count]                : Trace with [count]
r/run                          : Run or continue
reset                          : Reset laysim-GR740 state on stop mode
b/break [addr]/[sym]           : Add a breakpoint at [addr]/[sym]
: break                        - Show all breakpoints
: break Init                  - Set a breakpoint at Init
d/del [addr]/[sym]             : Delete a breakpoint at [addr]/[sym]
watch [addr]/[sym]             : Add a watchpoint at [addr]/[sym]
: watch                       - Show all watchpoints
: watch 0xFF900000            - Set a write-watchpoint at 0xFF900000
dw/dwatch [addr]/[sym]         : Delete a watchpoint at [addr]/[sym]
core [cmd]                     : CORE status and debug selection
: core                       - Show status of cores
: core 0                     - CORE#0 is selected for debugging
: core 1                     - CORE#1 is selected for debugging
: core 2                     - CORE#2 is selected for debugging
: core 3                     - CORE#3 is selected for debugging
: core 4                     - All COREs are selected for debugging
wakeUp [core]                  : Wake-up CORE#n with [core]
perf                           : Show performance statistics
fastuart                       : Set UART baudrate
: fastuart on                 - Set UART baudrate as infinite
: fastuart off                - Set UART baudrate as correct
rtms                            : Show RTEMS v4.8/4.10/5.1(SMP only) information
: rtms task                   - Show RTEMS task information
: rtms task [id]              - Show RTEMS selected task information
: rtms trap                   - Show RTEMS trap information
: rtms interrupt              - Show RTEMS BSP interrupt information (v4.10/5.1)
: rtms core [num]             - Show RTEMS-SMP core [num] information
mmu [core]                     : Show MMU registers for selected core
: mmu 0                       - Show MMU registers for CORE#0
: mmu 1                       - Show MMU registers for CORE#1
walk [core] vaddr              : Printout a MMU table walk for the given vaddr
: walk 0 0xFC000000           - Show vaddr to paddr of CORE#0
: walk 1 0xFC000100           - Show vaddr to paddr of CORE#1
vdis [core] [vaddr] [count]    : Disassemble [count] instructions at [vaddr]
: vdis 0 0xFC009350 0x10     - Disassemble instructions of CORE#0 at virtual address
air                            : Show AIR/GMV hypervisor information
xng-setup [addr]               : Set up XNG RAM execution with XCF address
event                         : Show events in the event queue
info sys                       : Show GR740 system information
info reg [core_name]           : Show all registers of [core_name]
iommu lookup [base] [ioaddr]    : Lookup specified IO address in page table starting at base
gdb                           : Start GDB server
quit                           : Quit laysim CLI
laysim>

```

help command in laysim-gr740-mdbt-cli

laysim> help

```

Execute help
dis/display [addr]/[sym] [count] : Disassemble [count] instructions at [addr]/[sym]
mem/memory [addr]/[sym] [count] : Show memory at [addr]/[sym] with [count]
wmem [addr]/[sym] [value]       : Write a word to memory at [addr]/[sym] with [value]
memdump [addr]/[sym] [wcnt] [file] : Save memory at [addr]/[sym] with [wcnt] to [file]
load [file]                     : Load GR740 RAM [file] to CORE#0 on stop/reset mode

```

```

batch [file]           : Execute a batch [file] of laysim commands
save [file]           : Save laysim-gr740 state to [file]
                       - It also saves the state of loadable modules
restore [file]         : Restore laysim-gr740 state from [file]
                       - It also restores the state of loadable modules
reg/register          : Show GR740 SPARC IU registers
float                : Show GR740 SPARC FPU registers
s/step               : Single step
n/next              : Single step with showing IU registers
t/trace [count]      : Trace with [count]
r/run               : Run or continue
reset               : Reset laysim-GR740 state on stop mode
b/break [addr]/[sym] : Add a breakpoint at [addr]/[sym]
                     - Show all breakpoints
                     - Set a breakpoint at Init
d/del [addr]/[sym]   : Delete a breakpoint at [addr]/[sym]
watch [addr]/[sym]   : Add a watchpoint at [addr]/[sym]
                     - Show all watchpoints
                     - Set a write-watchpoint at 0xFF900000
dw/dwatch [addr]/[sym] : Delete a watchpoint at [addr]/[sym]
core [cmd]           : CORE status and debug selection
                     - Show status of cores
                     - CORE#0 is selected for debugging
                     - CORE#1 is selected for debugging
                     - CORE#2 is selected for debugging
                     - CORE#3 is selected for debugging
                     - All COREs are selected for debugging
wakeupt [core]       : Wake-up CORE#n with [core]
perf                : Show performance statistics
fastuart            : Set UART baudrate
                     - Set UART baudrate as infinite
                     - Set UART baudrate as correct
rtms                : Show RTEMS v4.8/4.10/5.1(SMP only) information
                     - Show RTEMS task information
                     - Show RTEMS selected task information
                     - Show RTEMS trap information
                     - Show RTEMS BSP interrupt information (v4.10/5.1)
                     - Show RTEMS5-SMP core [num] information
mmu [core]           : Show MMU registers for selected core
                     - Show MMU registers for CORE#0
                     - Show MMU registers for CORE#1
walk [core] vaddr     : Printout a MMU table walk for the given vaddr
                     - Show vaddr to paddr of CORE#0
                     - Show vaddr to paddr of CORE#1
vdis [core] [vaddr] [count] : Disassemble [count] instructions at [vaddr]
                     - Disassemble instructions of CORE#0 at virtual address
air                 : Show AIR/GMV hypervisor information
xng-setup [addr]     : Set up XNG RAM execution with XCF address
event               : Show events in the event queue
info sys            : Show GR740 system information
info reg [core_name] : Show all registers of [core_name]
iommu lookup [base] [ioaddr] : Lookup specified IO address in page table starting at base
gdb                 : Start GDB server
quit                : Quit laysim CLI
laysim>

```

- **display/dis** : The display command disassembles [count] instructions at [addr]/[sym] with [count] based on selected core as shown in Figure 3-21. Only SDRAM/MRAM memory can be displayed . Default value for count is 10.

Figure 3–21. display command

```

laysim> dis main

[CORE#0] Address of main is 0x000013AC
000013AC : 9DE3BF48  save    %sp, -184, %sp
000013B0 : 400005D0  call    0x00002af0
000013B4 : 90102030  or      %g0, 0x30, %o0
000013B8 : 03000046  sethi   %hi(0x11800), %g1
000013BC : BA100008  or      %g0, %o0, %i5
000013C0 : 90102030  or      %g0, 0x30, %o0
000013C4 : 400005CB  call    0x00002af0
000013C8 : FA206308  st      %i5, [%g1 + 0x308]
000013CC : 82102002  or      %g0, 0x2, %g1
000013D0 : FA220000  st      %i5, [%o0]
laysim>

```

- **memory/mem** : The memory command shows the contents of AMBA memory at [addr]/[sym] with [count] based on selected core as shown in Figure 3-22. Default value for count is 10.

Figure 3–22. memory command

```
laysim> memory printf 40

[CORE#0] Address of printf is 0x00003648
00003640 : 81C7E008 91E80008 9DE3BF90 F227A048 .....'.H
00003650 : 0300003B F427A04C F627A050 F827A054 ...;'.L'.P'.T
00003660 : FA27A058 D00060F4 80A22000 22800007 '..X...'."...
00003670 : D2022008 C2022018 80A06000 02800009 .....'.....
00003680 : 01000000 D2022008 9607A048 94100018 .....'.H....
00003690 : 400001D1 D627BFFC 81C7E008 91E80008 @.....'.....
000036A0 : 7FFFBD4 D027BFF4 D007BFF4 D2022008 .....'.H....@.....
000036B0 : 9607A048 94100018 400001C7 D627BFFC .....'.H....@.....
000036C0 : 81C7E008 91E80008 9DE3BF90 80A62000 .....'.H....@.....
000036D0 : 22800007 F4062008 C2062018 80A06000 .....'.H....@.....

laysim> mem 0xff900000
FF900000 : 00000000 00000086 80000003 0000032D .....-
FF900010 : 00000000 00000000 00000000 00000000 .....
FF900020 : 00000000 00000000 00000000 00000000 .....
laysim>
```

- **register/reg** : The register command shows the current status of selected core GR740 IU registers as shown in Figure 3-23. There is no modification function of IU registers.

Figure 3–23. register command

```
laysim> core 4
GR740 all COREs are selected for debugging

laysim> reg

----- CORE#0 -----
INS      LOCALS      OUTS      GLOBALS
0: 00071520 00000004 00071520 00000000
1: 0001AD94 00018A78 00000000 000716C8
2: 00000003 0001AE34 00000004 000000FF
3: 00071524 00076180 00000000 0002956F
4: 00000000 00000100 00000000 00000003
5: 00000004 00000000 000000D1 00000000
6: 0007A5B8 00000000 0007A548 00071E80
7: 00018C44 00000000 00018B14 00000000

psr:F3000E5 wim:00000040 tbr:000008B0 y:00000000
[n:0 z:0 v:0 c:0 ef:0 pil:0 s:1 ps:1 et:1 cwp:5]

pc: 000018A4 : C4684000 ldstub [%g1], %g2
npc: 000018A8 : 8088A001 andcc %g2, 0x1, %g0

----- CORE#1 -----
INS      LOCALS      OUTS      GLOBALS
0: 00071520 00000004 00071520 00000000
1: 0001AD94 00018A78 00000000 000716C8
2: 00000003 0001AE34 00000004 000000FF
3: 00071524 00076180 00000001 00030234
4: 00000001 00000100 00067890 00000002
5: 00000004 00000000 015B3474 00000000
6: 000795B0 00000000 00079540 00072080
7: 00018C44 00000000 00018C28 00000000

psr:F3000E5 wim:00000040 tbr:000008B0 y:00000000
[n:0 z:0 v:0 c:0 ef:0 pil:0 s:1 ps:1 et:1 cwp:5]

pc: 000018A4 : C4684000 ldstub [%g1], %g2
npc: 000018A8 : 8088A001 andcc %g2, 0x1, %g0

----- CORE#2 -----
INS      LOCALS      OUTS      GLOBALS
0: 00071520 00000004 00071520 00000000
1: 0001AD94 00018A78 00000000 000716C8
2: 00000003 0001AE34 00000004 000000FF
3: 00071524 00076180 00000002 0002F788
4: 00000002 00000100 00067D80 00000000
5: 00000004 00000000 015B3472 00000000
6: 000785A8 00000000 00078538 00072280
7: 00018C44 00000000 00018C28 00000000

psr:F3000E5 wim:00000040 tbr:000008B0 y:00000000
[n:0 z:0 v:0 c:0 ef:0 pil:0 s:1 ps:1 et:1 cwp:5]

pc: 000018A4 : C4684000 ldstub [%g1], %g2
npc: 000018A8 : 8088A001 andcc %g2, 0x1, %g0

laysim>

----- CORE#3 -----
INS      LOCALS      OUTS      GLOBALS
0: 00071520 00000004 00071520 00000000
1: 0001AD94 00018A78 00000000 000716C8
2: 00000003 0001AE34 00000004 00000000
3: 00071524 00076180 00000003 00055679
4: 00000003 00000008 00072480 00000001
5: 00000004 00000008 015B3475 00000000
6: 00077538 000000D7 000774C8 00072480
7: 00018D58 0001B27F 00018C28 00000000

psr:F3400E0 wim:00000002 tbr:000008B0 y:0006804A
[n:0 z:1 v:0 c:0 ef:0 pil:0 s:1 ps:1 et:1 cwp:0]

pc: 000018B4 : C4022120 ld [%o0 + 0x120], %g2
npc: 000018B8 : 8400A001 add %g2, 0x1, %g2
```

- **wmem** : The wmem command writes a 32bit word to AMBA memory at [addr]/[sym] with [value] as shown in Figure 3-24.

Figure 3–24. wmem command

```
laysim> wmem 0xff90000c 0x123

laysim> mem 0xff900000
FF900000 : 00000000 00000086 80000003 00000123 .....#
FF900010 : 00000000 00000000 00000000 00000000 .....
```

```
FF900020 : 00000000 00000000 00000000 00000000 .....
laysim>
```

- **float** : The float command shows the current status of selected core of GR740 FPU registers as shown in Figure 3-25. There is no modification function of FPU registers.

Figure 3-25. float command

```
laysim> core 4

GR740 all COREs are selected for debugging

laysim> float

----- CORE#0 -----
fsr: 00040421
[nvm:0 ofm:0 ufm:0 dzm:0 nxm:0 ftt:0 fcc:1]
[nva:0 ofa:0 ufa:0 dza:0 nxa:1 nvc:0 ofc:0 ufc:0 dzc:0 nxc:1]
f00 3FEFFFFF 1.875000e+00 1.000000e+00
f01 FFFFFFF9 -nan
f02 00000000 0.000000e+00 0.000000e+00
f03 00000000 0.000000e+00
f04 00000000 0.000000e+00 0.000000e+00
f05 00000000 0.000000e+00
f06 00000000 0.000000e+00 0.000000e+00
f07 00000000 0.000000e+00
f08 3D09EDA2 3.367389e-02 1.151446e-14
f09 D3151708 -6.403365e+11
f10 3FE14EEB 1.760221e+00 5.408837e-01
f11 488D2E2B 2.891373e+05
f12 00000000 0.000000e+00 0.000000e+00
f13 00000000 0.000000e+00
f14 00000000 0.000000e+00 0.000000e+00
f15 00000000 0.000000e+00
f16 3C7CB3B3 1.542370e-02 2.489498e-17
f17 99E00000 -2.316106e-23
f18 3C8CB3B3 1.717553e-02 4.978996e-17
f19 99E00000 -2.316106e-23
f20 3FD55555 1.666667e+00 3.333333e-01
f21 55555500 1.466008e+13
f22 3FF00000 1.875000e+00 1.000000e+00
f23 00000000 0.000000e+00
f24 3FD93836 1.697028e+00 3.940559e-01
f25 0F69844B 1.151327e-29
f26 00000000 0.000000e+00 0.000000e+00
f27 00000000 0.000000e+00
f28 00000000 0.000000e+00 0.000000e+00
f29 00000000 0.000000e+00
f30 00000000 0.000000e+00 0.000000e+00
f31 00000000 0.000000e+00

----- CORE#1 -----
fsr: 00040421
[nvm:0 ofm:0 ufm:0 dzm:0 nxm:0 ftt:0 fcc:1]
[nva:0 ofa:0 ufa:0 dza:0 nxa:1 nvc:0 ofc:0 ufc:0 dzc:0 nxc:1]
f00 3FEFFA60 1.874828e+00 9.993135e-01
f01 66C4A627 4.643244e+23
f02 00000000 0.000000e+00 0.000000e+00
f03 00000000 0.000000e+00
f04 00000000 0.000000e+00 0.000000e+00
f05 00000000 0.000000e+00
f06 00000000 0.000000e+00 0.000000e+00
f07 00000000 0.000000e+00
f08 3DFFFA5F 1.749828e+00 4.996565e-01
f09 5A845856 1.862591e+16
f10 3DFFFA5F 1.749828e+00 4.996567e-01
f11 FD76136F -2.044319e+37
f12 3DFFFFFE 1.750000e+00 4.999998e-01
f13 F39085F5 -2.290063e+31
f14 3C700788 1.465023e-02 1.390331e-17
f15 7AF0CBBD 6.251418e+35
f16 3F31E369 6.948763e-01 2.729542e-04
f17 5C923A3B 3.292750e+17
f18 BE4FFFD0 -2.031242e-01 -1.490080e-08
f19 1E502D1C 1.102076e-20
f20 3FD55555 1.666667e+00 3.333333e-01
f21 55555500 1.466008e+13
f22 3FF00000 1.875000e+00 1.000000e+00
f23 00000000 0.000000e+00
f24 3FD93836 1.697028e+00 3.940559e-01
f25 0F69844B 1.151327e-29
f26 00000000 0.000000e+00 0.000000e+00
f27 00000000 0.000000e+00
f28 00000000 0.000000e+00 0.000000e+00
f29 00000000 0.000000e+00
f30 00000000 0.000000e+00 0.000000e+00
f31 00000000 0.000000e+00

----- CORE#2 -----
fsr: 00040421
[nvm:0 ofm:0 ufm:0 dzm:0 nxm:0 ftt:0 fcc:1]
[nva:0 ofa:0 ufa:0 dza:0 nxa:1 nvc:0 ofc:0 ufc:0 dzc:0 nxc:1]
f00 3FE14EE4 1.760220e+00 5.408804e-01
f01 52E0E270 4.829358e+11
f02 00000000 0.000000e+00 0.000000e+00
f03 00000000 0.000000e+00
f04 00000000 0.000000e+00 0.000000e+00
f05 00000000 0.000000e+00
f06 00000000 0.000000e+00 0.000000e+00
f07 00000000 0.000000e+00
f08 3FF8E183 1.944382e+00 1.555057e+00
f09 2229B7B7 2.300103e-18
f10 3FF8E183 1.944382e+00 1.555057e+00
f11 2229B7B7 2.300103e-18
f12 3FEC1681 1.844437e+00 8.777472e-01
f13 41F27645 3.030775e+01
f14 00000000 0.000000e+00 0.000000e+00
f15 00000000 0.000000e+00
f16 3C7CB3B3 1.542370e-02 2.489498e-17
f17 99E00000 -2.316106e-23
f18 3C8CB3B3 1.717553e-02 4.978996e-17
f19 99E00000 -2.316106e-23
f20 3FD55555 1.666667e+00 3.333333e-01
f21 55555500 1.466008e+13
f22 3FF00000 1.875000e+00 1.000000e+00
f23 00000000 0.000000e+00
f24 3FD93836 1.697028e+00 3.940559e-01
f25 0F69844B 1.151327e-29
f26 00000000 0.000000e+00 0.000000e+00
f27 00000000 0.000000e+00
f28 00000000 0.000000e+00 0.000000e+00
f29 00000000 0.000000e+00

----- CORE#3 -----
fsr: 00040421
[nvm:0 ofm:0 ufm:0 dzm:0 nxm:0 ftt:0 fcc:1]
[nva:0 ofa:0 ufa:0 dza:0 nxa:1 nvc:0 ofc:0 ufc:0 dzc:0 nxc:1]
f00 3FE14EE3 1.760220e+00 5.408801e-01
f01 C9D37D54 -1.732522e+06
f02 00000000 0.000000e+00 0.000000e+00
f03 00000000 0.000000e+00
f04 00000000 0.000000e+00 0.000000e+00
f05 00000000 0.000000e+00
f06 00000000 0.000000e+00 0.000000e+00
f07 00000000 0.000000e+00
f08 BDA8FAE9 -8.250982e-02 -1.135965e-11
f09 BE8838D4 -2.660586e-01
f10 3D19EDB5 3.758021e-02 2.302918e-14
f11 435C73E2 2.204527e+02
f12 3E845E37 2.585313e-01 1.517537e-07
f13 63200000 2.951479e+21
f14 BF9FDA87 -1.248856e+00 -3.110706e-02
f15 931145FA -1.833607e-27
f16 3C7CB3B3 1.542370e-02 2.489498e-17
f17 99E00000 -2.316106e-23
f18 3C8CB3B3 1.717553e-02 4.978996e-17
f19 99E00000 -2.316106e-23
f20 3FD55555 1.666667e+00 3.333333e-01
f21 55555500 1.466008e+13
f22 3FF00000 1.875000e+00 1.000000e+00
f23 00000000 0.000000e+00
f24 3FD93836 1.697028e+00 3.940559e-01
f25 0F69844B 1.151327e-29
f26 00000000 0.000000e+00 0.000000e+00
f27 00000000 0.000000e+00
f28 00000000 0.000000e+00 0.000000e+00
f29 00000000 0.000000e+00
```

```
f30 00000000 0.000000e+00 0.000000e+00 f30 00000000 0.000000e+00 0.000000e+00
f31 00000000 0.000000e+00 f31 00000000 0.000000e+00
laysim>
```

- **run/r** and **stop** : The run command is to run or continue execution. This command is different from TSIM run/go/cont commands. The stop command is to stop execution. **laysim-GR740** supports the power-down mode of GR740 only on run mode, so stop command will break the power-down mode of the processor.

Figure 3–26. run/stop command

```
laysim for GR740 Quad Core Processor v3.0 by layright

laysim>
CORE#0 RAM file is loaded - /home/layright/laysim-gr740/Cycle_Accuracy/BCC-2.2.2/dhry.elf
- .text, addr: 0x00000000, size 57920 bytes
- .rodata, addr: 0x0000E240, size 1952 bytes
- .data, addr: 0x0000E9E0, size 1576 bytes
- .bss, addr: 0x0000F008, size 10680 bytes (not loaded, just info.)
- found DWARF debug information
Only GR740 CORE#0 RAM file is loaded, starts from RAM
Set CORE#0 %sp/%fp : 0x07FFFFFF/0x08000000

laysim> run
laysim> stop
laysim-gr740 emulator is stopped by user

laysim>
```

- **step/s** and **next/n** : The step and next commands are to single step of execution based on selected core as shown in Figure 3-27. The step command executes and disassembles one instruction, whereas the next command executes and disassembles one instruction with showing current status of the GR740 IU registers. Theses commands are only effective in stop mode. The step over command is not supported in the control console, it is only available in the source code window. **laysim-GR740** supports the power-down mode of GR740 only on run mode, so step/next/trace command would not enter the power-down mode of the processor.

Figure 3–27. step & next command

```
laysim> core 4
GR740 all COREs are selected for debugging
laysim> s

59525269 (59525274) -E CORE#0 0000158C : 80A34004 subcc %o5, %g4, %g0
59525269 (59525270) -E CORE#1 00001594 : 8210000D or %g0, %o5, %g1
59525269 (59525282) -E CORE#2 0000158C : 80A34004 subcc %o5, %g4, %g0
59525269 (59525299) -E CORE#3 0000158C : 80A34004 subcc %o5, %g4, %g0
laysim> n

59525271 (59525274) -E CORE#0 0000158C : 80A34004 subcc %o5, %g4, %g0
59525271 (59525271) -E CORE#1 00001580 : 88100001 or %g0, %g1, %g4
59525271 (59525282) -E CORE#2 0000158C : 80A34004 subcc %o5, %g4, %g0
59525271 (59525299) -E CORE#3 0000158C : 80A34004 subcc %o5, %g4, %g0

----- CORE#0 -----
INS LOCALS OUTS GLOBALS
0: 00071520 00000004 00071520 00000000
1: 0001AD80 00018A78 00000000 0013A79C
2: 00000003 0001AE34 00000004 000009B3
3: 00071524 00076180 00000000 000715BC
4: 00000000 00000100 00000000 0013A79C
5: 00000004 00000000 0013A79D 00000000
6: 0007A5B8 00000000 0007A548 00071E80
7: 00018C44 00000000 00018B14 00000000
psr:F30000E5 wim:00000040 tbr:000008B0 y:FFFFFFFE
[n:0 z:0 v:0 c:0 ef:0 pil:0 s:1 ps:1 et:1 cwp:5]
pc: 0000158C : 80A34004 subcc %o5, %g4, %g0
npc: 00001590 : 12BFFFFC bne 0x00001580

----- CORE#1 -----
INS LOCALS OUTS GLOBALS
0: 00071520 00000004 00071520 00000000
1: 0001AD80 00018A78 00000000 0013A79D
2: 00000003 0001AE34 00000004 00000001
3: 00071524 00076180 00000001 000715BC
4: 00000001 00000100 00067890 0013A79D
5: 00000004 00000000 0013A79D 00000000
6: 000795B0 00000000 00079540 00072080
7: 00018C44 00000000 00018C28 00000000
psr:F30000E5 wim:00000040 tbr:000008B0 y:00000000
[n:0 z:0 v:0 c:0 ef:0 pil:0 s:1 ps:1 et:1 cwp:5]
pc: 00001584 : 9A006001 add %g1, 0x1, %o5
npc: 00001588 : DBE0C141 casa [%g3] 0xa, %g1, %o5

----- CORE#2 -----
----- CORE#3 -----
```

INS	LOCALS	OUTS	GLOBALS	INS	LOCALS	OUTS	GLOBALS
0: 00071520	00000004	00071520	00000000	0: 00071520	00000004	00071520	00000000
1: 0001AD80	00018A78	00000000	0013A79D	1: 0001AD80	00018A78	00000000	0013A79D
2: 00000003	0001AE34	00000004	00139DEC	2: 00000003	0001AE34	00000004	00000001
3: 00071524	00076180	00000002	000715BC	3: 00071524	00076180	00000003	000715BC
4: 00000002	00000100	00067D80	0013A79D	4: 00000003	00000008	00072480	0013A79D
5: 00000004	00000000	0013A79D	00000000	5: 00000004	00000008	0013A79E	00000000
6: 000785A8	00000000	00078538	00072280	6: 00077538	000000D7	000774C8	00072480
7: 00018C44	00000000	00018C28	00000000	7: 00018D58	0001B27F	00018C28	00000000
psr:F34000E5 wim:00000040 tbr:000008B0 y:00000000				psr:F30000E0 wim:00000002 tbr:000008B0 y:00069D22			
[n:0 z:1 v:0 c:0 ef:0 pil:0 s:1 ps:1 et:1 cwp:5]				[n:0 z:0 v:0 c:0 ef:0 pil:0 s:1 ps:1 et:1 cwp:0]			
pc: 0000158C : 80A34004	subcc	%o5, %g4, %g0		pc: 0000158C : 80A34004	subcc	%o5, %g4, %g0	
npc: 00001590 : 12BFFFFC	bne	0x00001580		npc: 00001590 : 12BFFFFC	bne	0x00001580	

- **trace/t** : The trace command executes and disassembles multiple instructions as indicated count based on selected core as shown in Figure 3-28. This command is effective only in stop mode.

Figure 3-28. trace command

laysim> t 10							
82885459 (82885459) -E CORE#0	0000158C : 80A34004	subcc	%o5, %g4, %g0				
82885460 (82885460) -E CORE#0	00001590 : 12BFFFFC	bne	0x00001580				
82885461 (82885461) -E CORE#0	00001594 : 8210000D	or	%g0, %o5, %g1				
82885462 (82885462) -E CORE#0	00001580 : 88100001	or	%g0, %g1, %g4				
82885463 (82885463) -E CORE#0	00001584 : 9A006001	add	%g1, 0x1, %o5				
82885464 (82885464) -E CORE#0	00001588 : DBE0C141	casa	[%g3] 0xa, %g1, %o5				
82885494 (82885494) -E CORE#0	0000158C : 80A34004	subcc	%o5, %g4, %g0				
82885495 (82885495) -E CORE#0	00001590 : 12BFFFFC	bne	0x00001580				
82885496 (82885496) -E CORE#0	00001594 : 8210000D	or	%g0, %o5, %g1				
82885497 (82885497) -E CORE#0	00001580 : 88100001	or	%g0, %g1, %g4				

- **break/b** and **del/d** : The break command adds a breakpoint at [addr]/[sym] with supporting virtual address based on selected core as shown in Figure 3-29. The status of breakpoints can be checked in the breakpoint window. The del command deletes a breakpoint with [addr]/[sym].

Figure 3-29. break command

```
laysim> break sin

[CORE#0] Set breakpoint at sin(0x00004DE4)
laysim> run

#####
Single Precision C/C++ Whetstone Benchmark

Calibrate
[CORE#0] Breakpoint hit at 0x00004DE4(sin)
laysim> reg

----- CORE#0 -----
      INS      LOCALS      OUTS      GLOBALS
0: 00000001    00015C00    3FDFFFFF    00000000
1: 00000001    000160DC    00000000    00014C00
2: 00014C00    00000C7E    00000002    3FDFFFFF
3: 00016000    00014C00    00016040    00000000
4: 00015C00    3FDFFFFF    00000001    20706F69
5: 00014E34    00000000    69786564    00000000
6: 07FFFA00    00000000    07FFFA00    00000000
7: 00004494    00000000    00001CD8    00000000

psr:F30010E6 wim:00000001 tbr:00000060 y:00000000
[n:0 z:0 v:0 c:0 ef:1 pil:0 s:1 ps:1 et:1 cwp:6]

pc: 00004DE4 : 9DE3BF88 save %sp, -120, %sp
npc: 00004DE8 : 03200000 sethi %hi(0x80000000), %g1
laysim> b
[CORE#0] Breakpoint at 0x00004DE4
laysim> del sin

[CORE#0] Delete the breakpoint at sin(0x00004DE4)
laysim>
```

- **watch** and **dwatch/dw** : The watch command adds a watchpoint at [addr]/[sym] with supporting virtual address based on selected core as shown in Figure 3-30. The status of watchpoints can be checked in the watchpoint window in GUI. The dwatch command deletes a watchpoint with [addr]/[sym]. **laysim-gr740** supports both read and write watchpoints, whereas **laysim-gr740-dbt/mdbt** support only write watchpoints

Figure 3–30. break command

```
laysim> core 4

GR740 all COREs are selected for debugging
laysim> watch 0xFF900000

[CORE#0] Set watchpoint at 0xFF900000
[CORE#1] Set watchpoint at 0xFF900000
[CORE#2] Set watchpoint at 0xFF900000
[CORE#3] Set watchpoint at 0xFF900000

laysim> run
[CORE#3] Watchpoint hit for write at 0xFF900000, new data : 0x0000000D

laysim> reg

----- CORE#0 -----
      INS      LOCALS      OUTS      GLOBALS
0: 0002EEA8 00020CC4 00000000 00000000
1: 0000E1EC 00022EA0 00022EA0 00001D08
2: 00000000 00020C58 0002A1FC 09010001
3: 0002A1FC 0002A210 00000001 0A010001
4: 0002A214 0002A000 00000000 0002A208
5: 0002EEA8 0002A1BC 00000000 00000000
6: 00022EB8 0002AF00 00022E58 00030840
7: 0000A34C 00020B90 000195D0 00000000

psr:F3000E3 wim:00000040 tbr:000008A0 y:FFFFFFFE
[n:0 z:0 v:0 c:0 ef:0 pil:0 s:1 ps:1 et:1 cwp:3]

pc: 00001D0C : C0838020 lda [%sp] 0x1, %g0
npc: 00001D10 : 30BFFFFE ba,a 0x00001d08

----- CORE#1 -----
      INS      LOCALS      OUTS      GLOBALS
0: 0002F398 00020CC4 00000000 00000000
1: 0000E1EC 00023EA0 00023EA0 00001D08
2: 00000000 00020C58 0002A1FC 09010002
3: 0002A1FC 0002A210 00000001 0A010001
4: 0002A214 0002A000 00000000 0002A208
5: 0002F398 0002A1BC 00000000 00000000
6: 00023EB8 00000000 00023E58 00030A40
7: 0000A34C 00000000 000195D0 00000000

psr:F3000E3 wim:00000040 tbr:000008A0 y:00000000
[n:0 z:0 v:0 c:0 ef:0 pil:0 s:1 ps:1 et:1 cwp:3]

pc: 00001D0C : C0838020 lda [%sp] 0x1, %g0
npc: 00001D10 : 30BFFFFE ba,a 0x00001d08

----- CORE#2 -----
      INS      LOCALS      OUTS      GLOBALS
0: 0002F888 00020CC4 00000000 00000000
1: 0000E1EC 00024EA0 00024EA0 00001D08
2: 00000000 00020C58 0002A1FC 09010003
3: 0002A1FC 0002A210 00000001 0A010001
4: 0002A214 0002A000 00000000 0002A208
5: 0002F888 0002A1BC 00000000 00000000
6: 00024EB8 00000000 00024E58 00030C40
7: 0000A34C 00000000 000195D0 00000000

psr:F3000E3 wim:00000040 tbr:000008A0 y:00000000
[n:0 z:0 v:0 c:0 ef:0 pil:0 s:1 ps:1 et:1 cwp:3]

pc: 00001D0C : C0838020 lda [%sp] 0x1, %g0
npc: 00001D10 : 30BFFFFE ba,a 0x00001d08

laysim> watch
[CORE#0] Watchpoint at 0xFF900000
[CORE#1] Watchpoint at 0xFF900000
[CORE#2] Watchpoint at 0xFF900000
[CORE#3] Watchpoint at 0xFF900000

laysim> dw 0xFF900000

[CORE#0] Delete the watchpoint at 0xFF900000
[CORE#1] Delete the watchpoint at 0xFF900000
[CORE#2] Delete the watchpoint at 0xFF900000
[CORE#3] Delete the watchpoint at 0xFF900000
laysim>
```

- **icache** : The icache command controls GR740 instruction cache operation. This command is only effective in stop mode. There are 6 sub-commands for the icache command. This command is only available in **laysim-gr740/laysimgr740-cli**.

Figure 3–31. icache command

```
laysim> icache 0 dump
```

```

CORE#0 icache set 0
00017000 : 00 : c2060000 b416a040 ba102000 b207bef8 10800005 b406be00 80a6c01c 02800014
00008020 : 00 : ba100008 40001cf8 90100018 c2062014 80a06000 22800002 fa262018 81c7e008
00017040 : 00 : 92100001 912f2009 7fffffa8 90068008 9010001d 7fffff5f 92102002 c2060000
00000060 : 00 : a7500000 2900005e 81c521fc a92ce001 a1480000 2900005d 81c52118 a6102107
00005080 : 00 : c406600c c2166010 c427bfc8 84102060 c227bfcc 9007bfc8 7ffffc75 c427bfd8
000050a0 : 00 : c207bfd0 d406600c 80a28001 0280000b 80a06000 02800012 9007bfc8 7ffffc24
000050c0 : 00 : 92102016 b0103fff 7ffffce8 9007bfc8 81c7e008 81e80000 d6166010 d2066038
000170e0 : 00 : 7fffffc0 9e104000 9de3bf98 9207bff8 7fffff46 90062040 c2062070 80a06000
...

```

- **dcache** : The dcache command controls GR740 data cache operation. This command is only effective in stop mode. There are 4 sub-commands for the dcache command. This command is only available in `haysim-gr740/haysimgr740-cli`.

Figure 3–32. dcache command

```

haysim> icache 0 dump

CORE#0 icache set 0
00017000 : 00 : c2060000 b416a040 ba102000 b207bef8 10800005 b406be00 80a6c01c 02800014
00008020 : 00 : ba100008 40001cf8 90100018 c2062014 80a06000 22800002 fa262018 81c7e008
00017040 : 00 : 92100001 912f2009 7fffffa8 90068008 9010001d 7fffff5f 92102002 c2060000
00000060 : 00 : a7500000 2900005e 81c521fc a92ce001 a1480000 2900005d 81c52118 a6102107
00005080 : 00 : c406600c c2166010 c427bfc8 84102060 c227bfcc 9007bfc8 7ffffc75 c427bfd8
000050a0 : 00 : c207bfd0 d406600c 80a28001 0280000b 80a06000 02800012 9007bfc8 7ffffc24
000050c0 : 00 : 92102016 b0103fff 7ffffce8 9007bfc8 81c7e008 81e80000 d6166010 d2066038
000170e0 : 00 : 7fffffc0 9e104000 9de3bf98 9207bff8 7fffff46 90062040 c2062070 80a06000
...

```

- **l2cache** : The l2cache command controls GR740 L2 cache operation. This command is only effective in stop mode. There are 4 sub-commands for the l2cache command. This command is only available in `haysim-gr740/haysimgr740-cli`.

Figure 3–33. l2cache command

```

haysim> l2cache 1 10 1024

L2 cache data in way-1
1024 00008000 [0000030A] : C401A020 82184002 80A00001 81C3E008 90603FFF 9DE3BFA0 40000B5D 90102001
1025 00008020 [0000030A] : BA100008 40001CF8 90100018 C2062014 80A06000 22800002 FA262018 81C7E008
1026 00008040 [0000030A] : 81E80000 9DE3BFA0 FA062014 90100018 40001D1C F0062018 80A76000 02800004
1027 00008060 [0000030A] : 01000000 81C7E008 81E80000 40000B48 81E80000 9DE3BFA0 80A6E000 0280000E
1028 00008080 [00000000] : 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
1029 000080A0 [00000000] : 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
1030 000080C0 [00000000] : 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
1031 000080E0 [0000030A] : 9DE3BFA0 C8062014 FA066004 82013FF8 DE062010 80A0401C 1A800003 860F7FFE
1032 00008100 [0000030A] : 8210001C 82006008 81802000 01000000 01000000 01000000 8470400F 8458800F
1033 00008120 [0000030A] : 84A04002 02800004 B8100001 9E03C001 B823C002 8400E004 88012004 82208001
haysim>

```

- **core** : The core command shows the status of each core and selects the core for debugging as shown in Figure 3-23. The selected core for debugging is used for all operation.

Figure 3–34. core command

```

haysim> core 4

GR740 all COREs are selected for debugging
haysim> core

GR740 CORE#0 is execution mode
GR740 CORE#1 is execution mode
GR740 CORE#2 is execution mode
GR740 CORE#3 is execution mode
GR740 all COREs are selected for debugging
haysim> reg

----- CORE#0 -----
INS    LOCALS    OUTS    GLOBALS
0: 0002EEA8 00020CC4 00000000 00000000

----- CORE#1 -----
INS    LOCALS    OUTS    GLOBALS
0: 0002F398 00020CC4 00000000 00000000

```

1: 0000E1EC 00022EA0 00022EA0 00001D08	1: 0000E1EC 00023EA0 00023EA0 00001D08
2: 00000000 00020C58 0002A1FC 09010001	2: 00000000 00020C58 0002A1FC 09010002
3: 0002A1FC 0002A210 00000001 0A010001	3: 0002A1FC 0002A210 00000001 0A010001
4: 0002A214 0002A000 00000000 0002A208	4: 0002A214 0002A000 00000000 0002A208
5: 0002EEA8 0002A1BC 00000000 00000000	5: 0002F398 0002A1BC 00000000 00000000
6: 00022EB8 0002AF00 00022E58 00030840	6: 00023EB8 00000000 00023E58 00030A40
7: 0000A34C 00020B90 000195D0 00000000	7: 0000A34C 00000000 000195D0 00000000
psr:F3000E3 wim:00000040 tbr:000008A0 y:FFFFFFFE	psr:F3000E3 wim:00000040 tbr:000008A0 y:00000000
[n:0 z:0 v:0 c:0 ef:0 pil:0 s:1 ps:1 et:1 cwp:3]	[n:0 z:0 v:0 c:0 ef:0 pil:0 s:1 ps:1 et:1 cwp:3]
pc: 00001D0C : C0838020 lda [%sp] 0x1, %g0	pc: 00001D0C : C0838020 lda [%sp] 0x1, %g0
npc: 00001D10 : 30BFFFFE ba,a 0x00001d08	npc: 00001D10 : 30BFFFFE ba,a 0x00001d08
----- CORE#2 -----	----- CORE#3 -----
INS LOCALS OUTS GLOBALS	INS LOCALS OUTS GLOBALS
0: 0002F888 00020CC4 00000000 00000000	0: 0000000A 00000000 FF900000 00000000
1: 0000E1EC 00024EA0 00024EA0 00001D08	1: 00000000 00000000 0000000D 00000086
2: 00000000 00020C58 0002A1FC 09010003	2: 00000000 00000000 00000001 00000000
3: 0002A1FC 0002A210 00000001 0A010001	3: 00000000 00000000 00000001 0000000D
4: 0002A214 0002A000 00000000 0002A208	4: 00000000 00000000 00000000 0002A208
5: 0002F888 0002A1BC 00000000 00000000	5: 0002A400 00000000 00000000 00000000
6: 00024EB8 00000000 00024E58 00030C40	6: 00032620 00000000 000325C0 00030E40
7: 0000A34C 00000000 000195D0 00000000	7: 0000E890 00000000 000040D4 00000000
psr:F3000E3 wim:00000040 tbr:000008A0 y:00000000	psr:F34010E7 wim:00000040 tbr:000008A0 y:00000000
[n:0 z:0 v:0 c:0 ef:0 pil:0 s:1 ps:1 et:1 cwp:3]	[n:0 z:1 v:0 c:0 ef:1 pil:0 s:1 ps:1 et:1 cwp:7]
pc: 00001D0C : C0838020 lda [%sp] 0x1, %g0	pc: 0001AB04 : D2220000 st %o1, [%o]
npc: 00001D10 : 30BFFFFE ba,a 0x00001d08	npc: 0001AB08 : 80A2E000 subcc %o3, 0x0, %g0
laysim>	

- **history/hist** : The history [core] command shows the maximum 1000 executed instructions of selected core as shown in Figure. 3-35.

This command is only available in `laysim-gr740/laysimgr740-cli`.

Figure 3–35. history command

laysim> hist 0				
46606539	-E CORE#0	00017908 : 9145C000	rd	%asr23, %o0
46606540	-E CORE#0	0000FD30 : 8222001D	sub	%o0, %i5, %g1
46606541	-E CORE#0	0000FD34 : 80A60001	subcc	%i0, %g1, %g0
46606542	-E CORE#0	0000FD38 : 18BFFFFC	bgu	0x0000fd28
46606543	-E CORE#0	0000FD3C : BA100008	or	%g0, %o0, %i5
46606544	-E CORE#0	0000FD28 : 9FC70000	jmp1	%i4 + %g0, %o7
46606547	-E CORE#0	0000FD2C : B0260001	sub	%i0, %g1, %i0
46606548	-E CORE#0	00017904 : 81C3E008	jmp1	%o7 + 0x8, %g0 ! _SPARC_Counter_read_asr23
46606549	-E CORE#0	00017908 : 9145C000	rd	%asr23, %o0
46606550	-E CORE#0	0000FD30 : 8222001D	sub	%o0, %i5, %g1
46606551	-E CORE#0	0000FD34 : 80A60001	subcc	%i0, %g1, %g0
46606552	-E CORE#0	0000FD38 : 18BFFFFC	bgu	0x0000fd28
46606553	-E CORE#0	0000FD3C : BA100008	or	%g0, %o0, %i5
46606554	-E CORE#0	0000FD28 : 9FC70000	jmp1	%i4 + %g0, %o7
46606557	-E CORE#0	0000FD2C : B0260001	sub	%i0, %g1, %i0
46606558	-E CORE#0	00017904 : 81C3E008	jmp1	%o7 + 0x8, %g0 ! _SPARC_Counter_read_asr23
46606559	-E CORE#0	00017908 : 9145C000	rd	%asr23, %o0
46606560	-E CORE#0	0000FD30 : 8222001D	sub	%o0, %i5, %g1
46606561	-E CORE#0	0000FD34 : 80A60001	subcc	%i0, %g1, %g0
46606562	-E CORE#0	0000FD38 : 18BFFFFC	bgu	0x0000fd28
46606563	-E CORE#0	0000FD3C : BA100008	or	%g0, %o0, %i5
46606564	-E CORE#0	0000FD28 : 9FC70000	jmp1	%i4 + %g0, %o7
46606567	-E CORE#0	0000FD2C : B0260001	sub	%i0, %g1, %i0
46606568	-E CORE#0	00017904 : 81C3E008	jmp1	%o7 + 0x8, %g0 ! _SPARC_Counter_read_asr23
46606569	-E CORE#0	00017908 : 9145C000	rd	%asr23, %o0
...				
...				

- **perf** : The perf command shows the performance statistics of `laysim-GR740` as shown in Figure 3-36. This command is only effective in stop mode.

Figure 3–36. perf command

laysim> perf	
Performance statistics for CORE#0	
Cycles	: 8790298027
Instructions	: 55004242


```

Overall CPI : 1.45
CORE#0 performance (250MHz) : 171.84 MOPS (171.84 MIPS, 0.00 MFLOPS)
Simulated time : 35.16 s
Processor utilization : 0.91 %

Performance statistics for CORE#1
Cycles : 8790298034
Instructions : 90760399
Overall CPI : 1.56
CORE#1 performance (250MHz) : 160.12 MOPS (160.12 MIPS, 0.00 MFLOPS)
Simulated time : 35.16 s
Processor utilization : 1.61 %

Performance statistics for CORE#2
Cycles : 8790298044
Instructions : 18225405
Overall CPI : 2.44
CORE#2 performance (250MHz) : 102.56 MOPS (102.56 MIPS, 0.00 MFLOPS)
Simulated time : 35.16 s
Processor utilization : 0.51 %

Performance statistics for CORE#3
Cycles : 8790298045
Instructions : 48215245
Overall CPI : 1.51
CORE#3 performance (250MHz) : 165.61 MOPS (165.61 MIPS, 0.00 MFLOPS)
Simulated time : 35.16 s
Processor utilization : 0.83 %

Performance of haysim-gr740
Real-time performance : 383.84 %
haysim-gr740 performance : 23.17 MIPS
Wall clock : 9.16 s
haysim>

```

- **fastuart** : The fastuart command controls the baudrate of APBUARTs. When fastuart is enabled, the baudrate of APBUART works as infinite.
- **mmu** : The mmu [core] command shows the current status of GR740 MMU.

Figure 3–37. mmu command

```

haysim> mmu 0
CORE#0 - MMU: 0x00904001, ctx: 0x36, ctxptr: 0x00740800, fsr: 0x00000384, far: 0x50023000
haysim> mmu 1
CORE#1 - MMU: 0x00904001, ctx: 0x37, ctxptr: 0x00740800, fsr: 0x00000304, far: 0x5018E000
haysim> mmu 2
CORE#2 - MMU: 0x00904001, ctx: 0x38, ctxptr: 0x00740800, fsr: 0x00000384, far: 0x50023000
haysim> mmu 3
CORE#3 - MMU: 0x00904001, ctx: 0x34, ctxptr: 0x00740800, fsr: 0x000003A4, far: 0xEFF4E000
haysim>

```

- **walk** : The walk [core] [vaddr] command shows a MMU table walk for the given virtual address as shown in Figure 3-38.

Figure 3–38. walk command

```

haysim> walk 0 0xF000B754
CORE#0 - vaddr : 0xF000B754 (Index1: 0xF0, Index2: 0x00, Index3: 0x0B)
- Context 54 0x007408D8 PTD: 0x00075301
- Region 240 0x007533C0 PTE: 0x0000009E
--> physical address : 0x0000B754
haysim> walk 1 0xF00133C0
CORE#1 - vaddr : 0xF00133C0 (Index1: 0xF0, Index2: 0x00, Index3: 0x13)
- Context 18 0x00740848 PTD: 0x000759C1
- Region 240 0x00759FC0 PTE: 0x0000009E
--> physical address : 0x000133C0
haysim>

```

- **vdis** : The vdis command disassembles [count] instructions at [vaddr] with [count] as shown in Figure 3-39.

Figure 3–39. vdis command

```
laysim> vdis 0 0xF00B754
0000B754 -> 0F00B754 : A935201C srl    %l4, 0x1c, %l4
0000B758 -> 0F00B758 : A92D2002 sll    %l4, 0x2, %l4
0000B75C -> 0F00B75C : 0D3C1C63 sethi   %hi(0xf0718c00), %g6
0000B760 -> 0F00B760 : 8C11A020 or     %g6, 0x20, %g6
0000B764 -> 0F00B764 : CC050006 ld     [%l4 + %g6], %g6
0000B768 -> 0F00B768 : 808C2040 andcc  %l0, 0x40, %g0
0000B76C -> 0F00B76C : 2280001B be,a   0xf00b7d8
0000B770 -> 0F00B770 : 81E00000 save   [%g6 + 0x0], %l4
0000B774 -> 0F00B774 : E801A000 ld     [%g0, %l4, %g0
0000B778 -> 0F00B778 : 80900014 orcc

laysim> vdis 1 0xF00133C0
000133C0 -> F00133C0 : 81C7E008 jmpl    %i7 + 0x8, %g0
000133C4 -> F00133C4 : 81E80000 restore
000133C8 -> F00133C8 : 9DE3BFA0 save   %sp, -96, %sp
000133CC -> F00133CC : 033C1C64 sethi   %hi(0xf0719000), %g1
000133D0 -> F00133D0 : 7FFFE554 call   0xf000c920
000133D4 -> F00133D4 : FA00610C ld     [%g1 + 0x10c], %i5
000133D8 -> F00133D8 : A7800000 wr     %g0, %g0, %asr19
000133DC -> F00133DC : C0874380 lda    [%i5] 0x1c, %g0
000133E0 -> F00133E0 : 81C7E008 jmpl    %i7 + 0x8, %g0
000133E4 -> F00133E4 : 81E80000 restore
laysim>
```

- **rtems** : The rtems command shows the RTEMS information based on RTEMS awareness of laysim-GR740.

Figure 3–40. rtems command

```
laysim> rtems
Detect RTEMS v5.1(SMP) with thread information
laysim> rtems task
```

Name	CORE#	Type	ID	TCB	Pri	Time (h:m:s)	Entry point	State
IDLE	core0	internal	0x09010001	0x0002EEA8	255	0:00:01.0330119174	(0x00001D08) leon3_power_down_loop	READY
IDLE	core1	internal	0x09010002	0x0002F398	255	0:00:01.0329932515	(0x00001D08) leon3_power_down_loop	READY
IDLE	core2	internal	0x09010003	0x0002F888	255	0:00:01.0329657981	(0x00001D08) leon3_power_down_loop	READY
IDLE	core3	internal	0x09010004	0x0002FD78	255	0:00:01.0000204784	(0x00001D08) leon3_power_down_loop	READY
TA01	core2	classic	0x0A010002	0x0002B1B0	2	0:00:00.0080276375	(0x00001624) Test_task	WAITING_FOR_TIME
TA02	core1	classic	0x0A010003	0x0002B6A0	2	0:00:00.0000000000	(0x00001624) Test_task	READY
TA03	core0	classic	0x0A010004	0x0002BB90	2	0:00:00.0000000000	(0x00001624) Test_task	READY
TA04	core3	classic	0x0A010005	0x0002C080	2	0:00:00.0000000000	(0x00001624) Test_task	READY
TA05	core2	classic	0x0A010006	0x0002C570	2	0:00:00.0080453051	(0x00001624) Test_task	WAITING_FOR_TIME
TA06	core2	classic	0x0A010007	0x0002CA60	2	0:00:00.0080464477	(0x00001624) Test_task	WAITING_FOR_TIME
TA07	core2	classic	0x0A010008	0x0002CF50	2	0:00:00.0080463104	(0x00001624) Test_task	WAITING_FOR_TIME
TA08	core2	classic	0x0A010009	0x0002D440	2	0:00:00.0080465026	(0x00001624) Test_task	WAITING_FOR_TIME
TA09	core2	classic	0x0A01000A	0x0002D930	2	0:00:00.0000000000	(0x00001624) Test_task	READY
TA10	core0	classic	0x0A01000B	0x0002DE20	2	0:00:00.0000000000	(0x00001624) Test_task	READY
TA11	core0	classic	0x0A01000C	0x0002E310	2	0:00:00.0000000000	(0x00001624) Test_task	READY
TA12	core0	classic	0x0A01000D	0x0002E800	2	0:00:00.0000000000	(0x00001624) Test_task	READY

```
laysim> rtems core 0

RTEMS5.1 is configured as MAX 4 cores

##### RTEMS5 CORE#0 Information #####
- cpu_per_cpu.fsr : 0x00000000
- interrupt_stack_low : 0x00025F80
- interrupt_stack_high : 0x00026F80
- isr_nest_level : 0x00000000
- isr_dispatch_disable : 0x00000000
- thread_dispatch_disable_level : 0x00000000
- dispatch_necessary : 0
- executing : 0x0002BB90 (TASK-TA03)
- heir : 0x0002BB90 (TASK-TA03)
- Interrupt_frame
  - Stack_frame.i0 : 0x00000000
  - Stack_frame.i1 : 0x00000000
  - Stack_frame.i2 : 0x00000000
  - Stack_frame.i3 : 0x00000000
  - Stack_frame.i4 : 0x00000000
  - Stack_frame.i5 : 0x00000000
  - Stack_frame.i6 : 0x00000000
  - Stack_frame.i7 : 0x00000000
  - Stack_frame.i8 : 0x00000000
  - Stack_frame.i9 : 0x00000000
  - Stack_frame.i10 : 0x00000000
  - Stack_frame.i11 : 0x00000000
  - Stack_frame.i12 : 0x00000000
  - Stack_frame.i13 : 0x00000000
  - Stack_frame.i14 : 0x00000000
```

```

- Stack_frame.i5          : 0x00000000
- Stack_frame.i6/fp       : 0x00000000
- Stack_frame.i7          : 0x00000000
- Stack_frame.return_address : 0x00000000
- Stack_frame.saved_arg0   : 0x00000000
- Stack_frame.saved_arg1   : 0x00000000
- Stack_frame.saved_arg2   : 0x00000000
- Stack_frame.saved_arg3   : 0x00000000
- Stack_frame.saved_arg4   : 0x00000000
- Stack_frame.saved_arg5   : 0x00000000
- Stack_frame.pad0        : 0x00000000
- psr                     : 0xF3000C4
- pc                      : 0x0017490
- npc                     : 0x0017494
- g1                      : 0xF3000E5
- g2                      : 0x00000006
- g3                      : 0x00000005
- g4                      : 0x00000040
- g5                      : 0x00000080
- reserved                : 0x00000000
- g7                      : 0x00000000
- i0                      : 0x00026DF0
- i1                      : 0x0002F040
- i2                      : 0xF3401FE5
- i3                      : 0x00030640
- i4                      : 0x00000000
- i5                      : 0x00000004
- i6/fp                   : 0x00030918
- i7                      : 0x000195BC
- y                       : 0xFFFFFFFF
- tpc                     : 0x0002FFB8
- cpu_usage_timestamp     : 0x000000113AD3806 (0:00:01.330119174)
- Watchdog
- Lock.next_ticket        : 0x00000014
- Lock.now_serving        : 0x00000014
- ticks                   : 0x0000000000000012
- Header[0].Watchdogs     : 0x00000000
- Header[0].first         : 0x00000000
- Header[1].Watchdogs     : 0x00000000
- Header[1].first         : 0x00000000
- Header[2].Watchdogs     : 0x00000000
- Header[2].first         : 0x00000000
- Lock.next_ticket        : 0x0000000D
- Lock.now_serving        : 0x0000000D
- Lock_context            : 0x00000000
- Threads_in_need_for_help
- Head.Node.next          : 0x00030958
- Head.Node.previous      : 0x00000000
- Head.fill               : 0x00030954
- message                 : 0x00000000
- Scheduler.control        : 0x00020B20 (_Scheduler_Table)
- Scheduler.context        : 0x00030310 (_Configuration_Scheduler_EDF_SMP_dflt)
- Scheduler.idle_if_       : 0x00000000
- ancestor                : 0x0002EEA8
- data                    : 0x00000000
- state                   : 0x00000003 (PER_CPU_STATE_UP)
- Jobs
- Lock.next_ticket        : 0x000001D3
- Lock.now_serving        : 0x000001D3
- head                    : 0x00000000
- tail                    : 0x00029C90
- online/boot             : 0x0101
- record                  : 0x00000000
laysim>

```

■ **info sys** : The info sys command shows the GR740 system information as shown in Figure 3-41.

■ **info reg** : The info reg command shows all the registers of [core_name] as shown in Figure 3-41.

Figure 3-41. info sys & reg command

```
laysim> info sys
```

Address	Description	Core name
0xFF900000	Generic APBUART #0	apbuart0
0xFF901000	Generic APBUART #1	apbuart1
0xFF902000	General Purpose I/O Port #0	grgpio0
0xFF903000	FT 8/16-bit PROM/IO Memory Interface	ftmctrl
0xFF904000	Multiprocessor Interrupt Controller	irqamp
0xFF908000	Modular Timer Unit #0	gptimer0
0xFF909000	Modular Timer Unit #1	gptimer1
0xFF90A000	Modular Timer Unit #2	gptimer2

```

0xFF90B000 Modular Timer Unit #3          gptimer3
0xFF90C000 Modular Timer Unit #4          gptimer4
0xFF90D000 GRSPW2 Spacewire Link #1 (AMBA Port 1) grspw1
0xFF90E000 GRSPW2 Spacewire Link #2 (AMBA Port 2) grspw2
0xFF90F000 GRSPW2 Spacewire Link #3 (AMBA Port 3) grspw3
0xFF910000 GRSPW2 Spacewire Link #4 (AMBA Port 4) grspw4
0xFF940000 Gigabit Ethernet MAC 0          greth0
0xFF980000 Gigabit Ethernet MAC 1          greth1

0xFFA00000 GRCPI2 with Configuration Space Header grpci2
0xFFA01000 CAN 2.0 Controller with DMA #0      grcan0
0xFFA03000 SPI Controller Supporting Master/Slave spictrl
0xFFA04000 Clock Gating Unit                  grclkgate
0xFFA05000 MIL-STD-1553B GR1553B              gr1553b
0xFFA06000 AHB Status Register #0             ahbstat0
0xFFA07000 AHB Status Register #1             ahbstat1
0xFFA08000 General Purpose I/O Port #0         grgpio1
0xFFA09000 General Purpose Register           grgpreg
0xFFA0A000 Temperature Sensore Controller     thsens
0xFFA0B000 General Purpose Register Bank       gprbank

0xFF880000 SpaceWire Router                   spwrouter
0xFF840000 IO Memory Management Unit          iommu

laysim> info reg gptimer0

Modular Timer Unit #0
0xFF908000 GPTIMER#0 Scaler Value Register    0x00000081
0xFF908004 GPTIMER#0 Scaler Reload Register   0x000000F9
0xFF908008 GPTIMER#0 Configuration Register   0x0000010D
0xFF90800C GPTIMER#0 Latch Config. Register   0x00000000
0xFF908010 GPTIMER#0 Timer#0 Counter Register 0x000025C4
0xFF908014 GPTIMER#0 Timer#0 Counter Reload   0x0000270F
0xFF908018 GPTIMER#0 Timer#0 Control Register 0x0000001F
0xFF90801C GPTIMER#0 Timer#0 Latch Register   0x00000000
0xFF908020 GPTIMER#0 Timer#1 Counter Register 0xFFFFFFFF
0xFF908024 GPTIMER#0 Timer#1 Counter Reload   0xFFFFFFFF
0xFF908028 GPTIMER#0 Timer#1 Control Register 0x00000000
0xFF90802C GPTIMER#0 Timer#1 Latch Register   0x00000000
0xFF908030 GPTIMER#0 Timer#2 Counter Register 0xFFFFFFFF
0xFF908034 GPTIMER#0 Timer#2 Counter Reload   0xFFFFFFFF
0xFF908038 GPTIMER#0 Timer#2 Control Register 0x00000000
0xFF90803C GPTIMER#0 Timer#2 Latch Register   0x00000000
0xFF908040 GPTIMER#0 Timer#3 Counter Register 0xFFFFFFFF
0xFF908044 GPTIMER#0 Timer#3 Counter Reload   0xFFFFFFFF
0xFF908048 GPTIMER#0 Timer#3 Control Register 0x00000000
0xFF90804C GPTIMER#0 Timer#3 Latch Register   0x00000000
0xFF908050 GPTIMER#0 Timer#4 Counter Register 0xFFFFFFFF
0xFF908054 GPTIMER#0 Timer#4 Counter Reload   0xFFFFFFFF
0xFF908058 GPTIMER#0 Timer#4 Control Register 0x00000000
0xFF90805C GPTIMER#0 Timer#4 Latch Register   0x00000000

laysim>

```

- **iommu lookup** : The iommu lookup command lookups the specified [IO address] in page table starting at [base]

Figure 3-42. iommu lookup command

```

laysim> iommu lookup 0x0018000 0xFF011000

Using ITR = 0 and PGSZ = 0 from IOMMU Control Register

Read IOPTe from address 0x00018044

IOPTe for IO address 0xFF011000:
- PPAGE : 0x000011
- W      : 1
- V      : 1
==> IOPTe mapping 0xFF011000 to 0x00011000

laysim>

```

- **air** : The air command shows AIR/GMV hypervisor information and provides 4 sub-commands. More information can be found in [laysim-AIR-001] Debugging AIR 5.6.0 on laysim-GR740.
- **event** : The event command shows the pending events in the event queue.
- **memdump** : Reads [wcnt] size from memory [addr]/[sym] and stores it into [file] in binary format. Note that the data is stored in little-endian format.

Figure 3–43. memdump command

```
laysim> mem main

[CORE#0] Address of main is 0x000056D8
000056D0 : 81C7E008 81E80000 9DE3BEC8 0300004B .....K
000056E0 : D100603C 310000AA 330000AA D12622F8 ..`<1...3....&".
000056F0 : D12662F4 1100004A 400006C5 90122338 .&b....J@.....#8

laysim> memdump main 10 output.bin

RAM from 0x000056D8 was saved to : output.bin
laysim>
```

- **rundelta** : This executes for [dcycles] based on the current cycle, and is only available in **laysim-gr740-cli**.

Figure 3–44. rundelta command

```
laysim> s

10946277 (10946278) -E CORE#0 00003488 : 16BFFFF7 bge 0x00003464

laysim> rundelta 1000
Current cycles : 10946279
Run till 10947279 cycles

laysim> s

10947280 (10947280) -E CORE#0 0000346C : 02800006 be 0x00003484
laysim>
```

- **reset** : This command resets all state information of the **laysim-GR740**.
- **load** : This loads a GR740 RAM executable in the CLI version. To load a new RAM executable, **laysim-GR740** must first be initialized using reset command. In the GUI version, executing the reset command activates the File Open toolbar again, allowing a new executable to be loaded.

Figure 3–45. load command

```
[layright@CentOS BCC-2.2.2]$ laysim-gr740-cli -core0 dhry.elf
laysim for GR740 Quad Core Processor CLI v3.0 by layright

CORE#0 RAM file is loaded - dhry.elf
- .text, addr: 0x00000000, size 58544 bytes
- .rodata, addr: 0x0000E4B0, size 2048 bytes
- .data, addr: 0x0000ECB0, size 1576 bytes
- .bss, addr: 0x0000F2D8, size 10680 bytes (not loaded, just info.)
Only GR740 CORE#0 RAM file is loaded, starts from RAM
Set CORE#0 %sp/%fp : 0x07FFFFFFF/0x08000000
laysim> run
Execution starts, 1000000 runs through Dhrystone
laysim> reset

Reset laysim-GR740 state
laysim> load stanford.elf
CORE#0 RAM file is loaded - stanford.elf
- .text, addr: 0x00000000, size 76336 bytes
- .rodata, addr: 0x00012A30, size 2288 bytes
- .data, addr: 0x00013320, size 1576 bytes
- .bss, addr: 0x00013948, size 95032 bytes (not loaded, just info.)
Only GR740 CORE#0 RAM file is loaded, starts from RAM
Set CORE#0 %sp/%fp : 0x07FFFFFFF/0x08000000
laysim> run
Starting
  Perm Towers Queens Intmm Mm Puzzle Quick Bubble Tree FFT
96967 72966 73767 64667 63083 530800 71867 97100 121017 75333

Nonfloating point composite is 143575

Floating point composite
CORE#0 halts because of Error Mode [36667740cycles/0.368329sec]
is 195443
laysim>
```

- **batch** : Reads the batch file [file] and executes it automatically. This command is only available in the CLI version.

Figure 3-46. batch command

```
[layright@CentOS Samples]$ laysim-gr740-dbt-cli
laysim for GR740 Quad Core Processor (DBT) CLI v3.0 by layright

Multicore control : Time quantum (500)
--> No CORE#0 RAM file is loaded
laysim> batch gr740.bat

Read batch file from gr740.bat

CORE#0 RAM file is loaded - rtems-hello.elf
- .text, addr: 0x00000000, size 165344 bytes
- .rtemsroset, addr: 0x000285E0, size 160 bytes
- .rtemsstack, addr: 0x00028680, size 8192 bytes (not loaded, just info.)
- .data, addr: 0x0002A680, size 4848 bytes
- .bss, addr: 0x0002B980, size 15792 bytes (not loaded, just info.)
- Detect RTEMS v5.1(SMP) with thread information
Only GR740 CORE#0 RAM file is loaded, starts from RAM
Set CORE#0 %sp/%fp : 0x07FFFFFFF/0x08000000

[CORE#0] Set breakpoint at Init(0x000012C8)

    0 (0) E CORE#0 00000000 : A0100000 or %g0, %g0, %l0 ! start
    1 (1) E CORE#0 00000004 : 29000004 sethi %hi(0x1000), %l4
    2 (2) E CORE#0 00000008 : 81C52000 jmp1 %l4, %g0
    5 (5) E CORE#0 0000000C : A6102000 or %g0, 0x0, %l3
    6 (6) E CORE#0 00001000 : 03000000 sethi %hi(0x0), %g1 ! hard_reset
    7 (7) E CORE#0 00001004 : 82106000 or %g1, 0x0, %g1
    8 (8) E CORE#0 00001008 : 81980001 wr %g0, %g1, %tbr
    9 (9) E CORE#0 0000100C : 83480000 rd %psr, %g1
   10 (10) E CORE#0 00001010 : 84006001 add %g1, 0x1, %g2
   11 (11) E CORE#0 00001014 : 8408A007 and %g2, 0x7, %g2

[CORE#0] Breakpoint hit at 0x000012C8(Init)

----- CORE#0 -----
      INS      LOCALS      OUTS      GLOBALS
0: 0002BB40 00030658 0002ECA0 00000000
1: 0001317C 0002658C 00030658 000012C8
2: 00000000 0002B460 0002B44C 0A010001
3: 0002B44C 0002B400 00000001 0A010001
4: 0002BB40 0002B464 0002A390 0003065C
5: 000265D4 0002B428 000263A0 00000000
6: 00030670 0002BE10 00030610 0002E700
7: 0000ED18 0002BB9C 000100EC 00000000

psr:F34000E3 wim:00000040 tbr:000008A0 y:FFFFFFFE
[n:0 z:1 v:0 c:0 ef:0 pil:0 s:1 ps:1 et:1 cwp:3]

pc: 000012C8 : 9DE3BFA0 save %sp, -96, %sp
npc: 000012CC : 11000098 sethi %hi(0x26000), %o0

Performance statistics for CORE#0
Cycles : 473414
Instructions : 327942
CORE#0 performance (250MHz) : 173.18 MIPS
Simulated time : 1.89 ms

Performance of laysim-gr740
Real-time performance : 22.55 %
laysim-gr740 performance : 39.05 MIPS
Wall clock : 0.01 s

[layright@CentOS Samples]$
```

- **save** : This command stores the state information of **laysim-GR740**, including loadable modules, into [file]. This command is supported in both CLI and GUI versions. The save() callback function of each loadable module is invoked to store its state information. Note that saved files are not compatible across different emulators.
- **restore** : This command restores the previously saved state of **laysim-GR740** from [file]. If loadable modules are used, their states are restored as well. This command is supported in both CLI and GUI versions. Note that saved files are not compatible across different emulators.

Figure 3-47. save & restore command

```
[layright@CentOS Samples]$ laysim-gr740-dbt-cli -core0 rtems-rmap-logical-addressing.elf -mod ../../Loadable/loadable-SpW-Nodes/laysim_user_spw_node.so
laysim for GR740 Quad Core Processor (DBT) CLI v3.0 by layright

Loaded user loadable module ../../Loadable/loadable-SpW-Nodes/laysim_user_spw_node.so

Multicore control : Time quantum (500)
CORE#0 RAM file is loaded - rtems-rmap-logical-addressing.elf
- .text, addr: 0x00000000, size 230576 bytes
- .rtemsroset, addr: 0x00038480, size 176 bytes
- .rtemsstack, addr: 0x00038580, size 8192 bytes (not loaded, just info.)
- .data, addr: 0x0003A580, size 8272 bytes
- .bss, addr: 0x0003C600, size 171024 bytes (not loaded, just info.)
- Detect RTEMS v5.1(SMP) with thread information
Only GR740 CORE#0 RAM file is loaded, starts from RAM
Set CORE#0 %sp/%fp : 0x07FFFFFFF/0x08000000
laysim> run

#### To change configuration of this example, you can set a breakpoint
#### on label <rmap_conf_label> and modify the following global variables:
## remote_base_address: 0x40000000 /* Base address on Remote. This is the address to which the data is copied. */
## remote_dst_address: 0x42 /* SpW Destination address. */
## remote_dst_key: 0x00 /* SpW Destination Key */
## source_src_address: 0x20 /* SpW Source address */
## rmap_dev_idx: 0 /* This parameter chooses which initiator GRSPW2 core is used */
## route_entry { nr; /* Number of addresses in dstadr array */, dstadr[16]; /* Path Addresses */}
## routetab[ROUTE_TO]={1,{3,0,0, ...}}
## routetab[ROUTE_FROM]={1,{9,0,0, ...}}
## path_addressing: 0 /* Use path addressing (!=0) or not (0) */
## rmap_pkt_size: 992
## rmap_data_bytes: 4096 /* Data bytes to send over RMAP */
#### RTEMS RMAP example

Setting up SpaceWire router
Configuring Router
Hardware Configuration of SpaceWire Router:
Number of SpW ports:      8
Number of AMBA ports:     4
Number of FIFO ports:     0
Timers available:         YES
Plug and Play available:  YES
MAJOR Version:            1
MINOR Version:            2
PATCH Version:           0
Current Instance ID:      19
ROUTER CFG/STS: 0x4100c013
PORT[00]:  TYPE=CFG      CTRL=0x01000210  STATUS=0x01000210  LINKSTATE=N/A
PORT[01]:  TYPE=SpW      CTRL=0x2700022c  STATUS=0x0000a000  LINKSTATE=Ready
PORT[02]:  TYPE=SpW      CTRL=0x2700022c  STATUS=0x0000a000  LINKSTATE=Ready
PORT[03]:  TYPE=SpW      CTRL=0x2700022c  STATUS=0x0000a000  LINKSTATE=Ready
PORT[04]:  TYPE=SpW      CTRL=0x2700022c  STATUS=0x0000a000  LINKSTATE=Ready
PORT[05]:  TYPE=SpW      CTRL=0x2700022c  STATUS=0x0000a000  LINKSTATE=Ready
PORT[06]:  TYPE=SpW      CTRL=0x2700022c  STATUS=0x0000a000  LINKSTATE=Ready
PORT[07]:  TYPE=SpW      CTRL=0x2700022c  STATUS=0x0000a000  LINKSTATE=Ready
PORT[08]:  TYPE=SpW      CTRL=0x2700022c  STATUS=0x0000a000  LINKSTATE=Ready
PORT[09]:  TYPE=AMBA      CTRL=0x00000228  STATUS=0x40000000  LINKSTATE=N/A
PORT[10]:  TYPE=AMBA      CTRL=0x00000228  STATUS=0x40000000  LINKSTATE=N/A
PORT[11]:  TYPE=AMBA      CTRL=0x00000228  STATUS=0x40000000  LINKSTATE=N/A
PORT[12]:  TYPE=AMBA      CTRL=0x00000228  STATUS=0x40000000  LINKSTATE=N/A
Activating all Links/Ports
PORT[01]:  TYPE=SpW      CTRL=0x0300002e  STATUS=0x0000a000  LINK-STATE=Ready
PORT[02]:  TYPE=SpW      CTRL=0x0300002e  STATUS=0x0000a000  LINK-STATE=Ready
PORT[03]:  TYPE=SpW      CTRL=0x0300002e  STATUS=0x0000d000  LINK-STATE=Run
PORT[04]:  TYPE=SpW      CTRL=0x0300002e  STATUS=0x0000a000  LINK-STATE=Ready
PORT[05]:  TYPE=SpW      CTRL=0x0300002e  STATUS=0x0000a000  LINK-STATE=Ready
PORT[06]:  TYPE=SpW      CTRL=0x0300002e  STATUS=0x0000a000  LINK-STATE=Ready
PORT[07]:  TYPE=SpW      CTRL=0x0300002e  STATUS=0x0000a000  LINK-STATE=Ready
PORT[08]:  TYPE=SpW      CTRL=0x0300002e  STATUS=0x0000a000  LINK-STATE=Ready
Configuring Router
Failed to open SpW Router
Initializing SpaceWire device 0
NOTE: running on SPW-ROUTER DMA SpaceWire link (no link-state available)
Initializing SpaceWire device 1
NOTE: running on SPW-ROUTER DMA SpaceWire link (no link-state available)
Initializing SpaceWire device 2
NOTE: running on SPW-ROUTER DMA SpaceWire link (no link-state available)
Initializing SpaceWire device 3
NOTE: running on SPW-ROUTER DMA SpaceWire link (no link-state available)

Test built for at least 4 AMBA ports.
Note that the SpW-links will not be used.
There are 4 GRSPW cores in the system.
System clock: 10000 us / tick

Starting GRSPW0: DMA Started Successfully
NOTE: running on SPW-ROUTER DMA SpaceWire link (no link-state available)
RMAP connection is working
Initializing data array
```



```

Starting transmission type 1: one packet at a time

RMAP write cycle completed in 0.000409 seconds
RMAP read cycle completed in 0.000416 seconds
Data bytes sent: 4096, Packet size: 992, Packet count: 5
Wrote 4096 bytes to target in 0.000409 seconds. WRITE RATE: 9782.232179 KiB/s
Read 4096 bytes from target in 0.000416 seconds. READ RATE: 9610.276912 KiB/s
Wrote 5 packets to target in 0.000409 seconds.
laysim> save check1

laysim-GR740 state saved to : check1.v3.0.cli.dbt
--> Total events : 1
0001  0x5a90f0  0x00000000  277927458

SpW Node state saved to : check1.v3.0.cli.dbt-spw

laysim> reset

Reset laysim-GR740 state

laysim> restore check1

laysim-GR740 state restored from : check1.v3.0.cli.dbt
--> Total events : 1
0001  0x5a90f0  0x00000000  277927458

- Detect RTEMS v5.1(SMP) with thread information

SpW Node state restored from : check1.v3.0.cli.dbt-spw

laysim> run

seconds. WRITE RATE: 12227.790224 pkt/s
Read 5 packets from target in 0.000416 seconds. READ RATE: 12012.846140 pkt/s

EXAMPLE 1 SUCCESSFULLY COMPLETED.

Starting transmission type 2: all packets at a time

RMAP write cycle completed in 0.000373 seconds
RMAP read cycle completed in 0.000127 seconds
Data bytes sent: 4096, Packet size: 992, Packet count: 5
Wrote 4096 bytes to target in 0.000373 seconds. WRITE RATE: 10735.271186 KiB/s
Read 4096 bytes from target in 0.000127 seconds. READ RATE: 31536.120301 KiB/s
Wrote 5 packets to target in 0.000373 seconds. WRITE RATE: 13419.088982 pkt/s
Read 5 packets from target in 0.000127 seconds. READ RATE: 39420.150376 pkt/s

EXAMPLE 2 SUCCESSFULLY COMPLETED.

CORE#0 halts because of Error Mode [0.271849sec/333129798, 0, 0, 0cycles]
laysim>

```

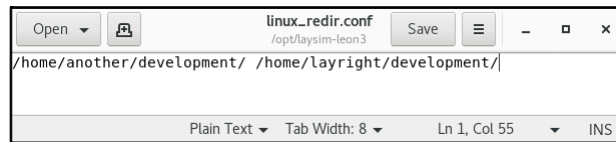
- **regupdate** : Enable or disable the real-time update of APB registers in the GUI version of laysim-GR740.
- **fastuart** : Configure APBUART#0/#1 to operate either in infinite mode or at the correct baudrate.
- **wake** : Forcibly wakes up the CORE#n specified by [core]. This command is used for XNG.
- **xng-setup** : Set up XNG RAM execution with XCF address.
- **gdb** : Start GDB server only for CLI version.
- **quit** : Quit laysim CLI version.

3.4 Directory Redirect for Debugging

If an executable which was built from the another platform or has separated source codes, the directory redirection of **laysim-gr740**

allows to debug it at the source code level. For example, RTEMS5 examples which were built by another users and their original base directory is '/home/another/development/'. After all source codes are copied to '/home/layright/development' in Linux, you can set the information of directory redirection in `linux_reconf.conf` as shown in Figure 3-48. It supports up to 10 directory redirection.

Figure 3-48. Set Directory Redirection



4. Loadable Modules

laysim-GR740 supports loadable modules for SPI, GPIO, UART, PCI, I/O, SpW Nodes, 1553B RTs and CAN Nodes with CAN bus simulation. The evaluation version of laysim-GR740 provides only the pre-built loadable modules. It is possible to build loadable modules with source codes, and the detailed information can be provided according to the separate license agreement (Professional version of laysim-GR740). Detailed information can be found in [laysim-GR740-009] laysim-GR740 Loadable Modules - User's Manual.

4.1 Loadable PCI Module

The overall architecture of loadable PCI module is shown in Figure 4-1 and the PCI address space is shown in Figure 4-2. The loadable PCI module is connected on PCI Bus [0:2:0] with IRQ #17 from GPIO#0 Port #4 of laysim-GR740. The loadable PCI module has GRPCI2, Actel 1553BRM, 2 x GRSpW2, MIRQMP (modified for PCI ACK.), FTAHBRAM (8KB), SRAM (2MB), GPTIMER, GPIO, AHBSTAT. Also laysim-GR740 supports GRPCI2 DMA of GR740.

Figure 4-1. Overall Architecture of Loadable PCI Module

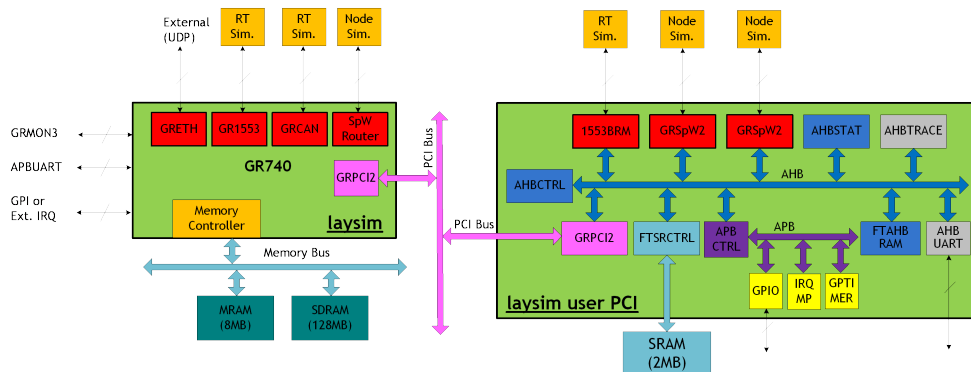
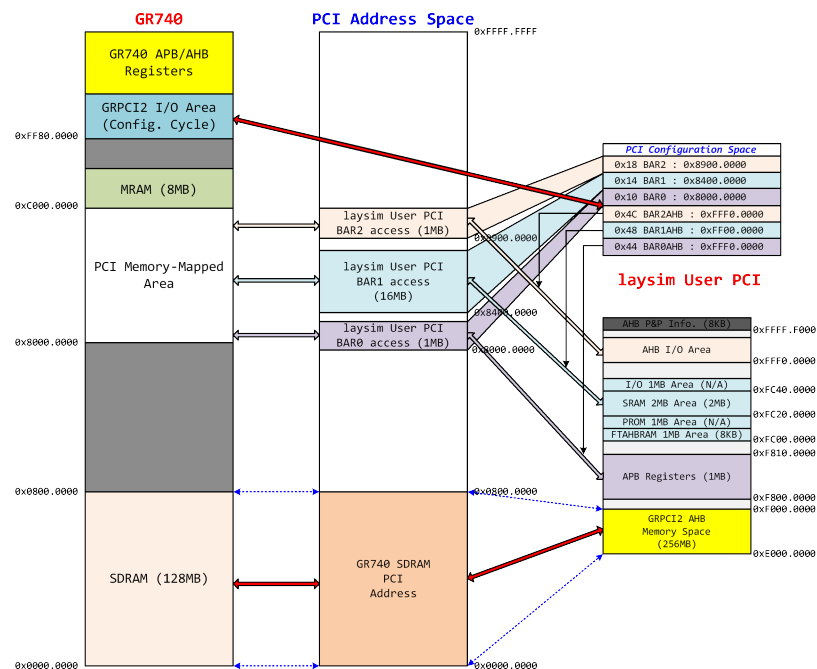


Figure 4-2. PCI Address Space for User PCI Module



4.1.1 RTEMS Shell for PCI Extension

Figure 4-3 shows the rtems-pci-shell with the loadable PCI module. The loadable PCI module can be loaded with '-mod ./Loadable/loadable-PCI/laysim_user_pci.so' on start-up.

Figure 4–3. rtems-pci-shell with loadable PCI module

laysim-gr740-cli in Linux	laysim-gr740-cli in Windows
<pre> [layright@CentOS user-pci]\$ laysim-gr740-cli -r -core0 rtems-pci-shell.elf -mod ./Loadable/loadable-PCI/laysim_user_pci.so laysim for GR740 Quad Core Processor CLI v3.0 by layright Loaded user loadable module ./Loadable/loadable-PCI/laysim_user_pci.so CORE#0 RAM file is loaded - rtems-pci-shell.elf - .text, addr: 0x00000000, size 645744 bytes - .rtemsroset, addr: 0x0009DA70, size 192 bytes - .rtemsstack, addr: 0x0009DB40, size 8192 bytes (not loaded, just info.) - .data, addr: 0x0009FB40, size 12896 bytes - .bss, addr: 0x000A2DC0, size 98880 bytes (not loaded, just info.) - Detect RTEMS v5.1(SMP) with thread information Only GR740 CORE#0 RAM file is loaded, starts from RAM Set CORE#0 %sp/%fp : 0x07FFFFFF0/0x08000000 --- BUS TOPOLOGY --- -> DEV 0xfce0f0 GAISLER_LEON4 -> DEV 0xfcf580 GAISLER_LEON4 -> DEV 0xfcf600 GAISLER_LEON4 -> DEV 0xfcd028 GAISLER_LEON4 -> DEV 0xfcd090 GAISLER_GRIOMMU -> DEV 0xfcd0f8 GAISLER_AHB2AHB -> DEV 0xfcd160 GAISLER_L2CACHE -> DEV 0xfcd1c8 GAISLER_APBMS -> DEV 0xfcd230 GAISLER_APBMS -> DEV 0xfcd298 GAISLER_APBUART -> DEV 0xfcd300 GAISLER_APBUART -> DEV 0xfcd368 GAISLER_GPIO -> DEV 0xfcd3d0 GAISLER_FTMCTRL -> DEV 0xfcd438 GAISLER_IRQMP -> DEV 0xfcd4a0 GAISLER_GPTIMER -> DEV 0xfcd508 GAISLER_GPTIMER -> DEV 0xfcd570 GAISLER_GPTIMER -> DEV 0xfcd5d8 GAISLER_GPTIMER -> DEV 0xfcd640 GAISLER_GPTIMER -> DEV 0xfcd6a8 GAISLER_SPW2_DMA -> DEV 0xfcd710 GAISLER_SPW2_DMA -> DEV 0xfcd778 GAISLER_SPW2_DMA -> DEV 0xfcd7e0 GAISLER_SPW2_DMA -> DEV 0xfcd848 GAISLER_ETHMAC -> DEV 0xfcd8b0 GAISLER_ETHMAC -> DEV 0xfcd918 GAISLER_GRCAN -> DEV 0xfcd980 GAISLER_GRCAN -> DEV 0xfcd9e8 GAISLER_SPICTRL -> DEV 0xfdda50 GAISLER_CLKGATE -> DEV 0xfddab8 GAISLER_GR1553B -> DEV 0xfddb20 GAISLER_AHBSTAT -> DEV 0xfddb88 GAISLER_AHBSTAT -> DEV 0xfdbf00 GAISLER_GPIO -> DEV 0xfdbf58 GAISLER_GPREG -> DEV 0xfdbf80 GAISLER_GR740THS -> DEV 0xfdd28 GAISLER_GPREGBANK -> DEV 0xfdd98 GAISLER_SPWTD -> DEV 0xfde00 GAISLER_L4STAT -> DEV 0xfde68 GAISLER_L2CACHE -> DEV 0xfde6d0 GAISLER_MEMSCRUB -> DEV 0xfdf38 GAISLER_GRIOMMU2 -> DEV 0xfdfa0 GAISLER_DDRSDMUX -> DEV 0xfe008 GAISLER_AHB2AHB -> DEV 0xfe070 GAISLER_FTMCTRL -> DEV 0xfe0d8 GAISLER_GRPIC2 -> DEV 0xff320 PCI_0:2:0_00ec:0001 -> DEV 0xffb48 GAISLER_GRPIC2 -> DEV 0xffbb0 GAISLER_AHBUART -> DEV 0xffc18 GAISLER_SPW2 -> DEV 0xffc80 GAISLER_SPW2 -> DEV 0xffce8 GAISLER_B1553BRM -> DEV 0xffd50 GAISLER_APBMS -> DEV 0xffdb8 GAISLER_FTAHBRAM -> DEV 0xffe20 GAISLER_AHBTTRACE -> DEV 0xffe88 GAISLER_FTSRCTRL -> DEV 0xffef0 GAISLER_IRQMP -> DEV 0xffff58 GAISLER_GPIO -> DEV 0xffffc0 GAISLER_GPTIMER -> DEV 0x100028 GAISLER_AHBSTAT -> DEV 0xfe140 GAISLER_SPWRROUTER You can use the shell commands drvmgr and pci to find out more about the system Creating /etc/passwd and group with three useable accounts root/pwd , test/pwd, rtems/NO PASSWORD RTEMS Shell on dev/console. Use 'help' to list commands. SHLL [/] # pci ls PCI devices found and configured: Bus 0 Slot 2 function: 0 [0x10] Vendor id: 0xec, device id: 0x1 IRQ INTA# LINE: 17 BAR 0: 80000000 [1MB] BAR 1: 84000000 [16MB] BAR 2: 89000000 [1MB] SHLL [/] # </pre>	<pre> PS C:\laysim-gr740\user-pci> laysim-gr740-cli -r -core0 rtems-pci-shell.elf -mod ./Loadable/loadable-PCI/laysim_user_pci-cli.dll laysim for GR740 Quad Core Processor CLI v3.0 by layright Loaded user loadable module ./Loadable/loadable-PCI/laysim_user_pci-cli.dll CORE#0 RAM file is loaded - rtems-pci-shell.elf - .text, addr: 0x00000000, size 645744 bytes - .rtemsroset, addr: 0x0009DA70, size 192 bytes - .rtemsstack, addr: 0x0009DB40, size 8192 bytes (not loaded, just info.) - .data, addr: 0x0009FB40, size 12896 bytes - .bss, addr: 0x000A2DC0, size 98880 bytes (not loaded, just info.) - Detect RTEMS v5.1(SMP) with thread information Only GR740 CORE#0 RAM file is loaded, starts from RAM Set CORE#0 %sp/%fp : 0x07FFFFFF0/0x08000000 --- BUS TOPOLOGY --- -> DEV 0xfce0f0 GAISLER_LEON4 -> DEV 0xfcf580 GAISLER_LEON4 -> DEV 0xfcf600 GAISLER_LEON4 -> DEV 0xfcd028 GAISLER_LEON4 -> DEV 0xfcd090 GAISLER_GRIOMMU -> DEV 0xfcd0f8 GAISLER_AHB2AHB -> DEV 0xfcd160 GAISLER_L2CACHE -> DEV 0xfcd1c8 GAISLER_APBMS -> DEV 0xfcd230 GAISLER_APBMS -> DEV 0xfcd298 GAISLER_APBUART -> DEV 0xfcd300 GAISLER_APBUART -> DEV 0xfcd368 GAISLER_GPIO -> DEV 0xfcd3d0 GAISLER_FTMCTRL -> DEV 0xfcd438 GAISLER_IRQMP -> DEV 0xfcd4a0 GAISLER_GPTIMER -> DEV 0xfcd508 GAISLER_GPTIMER -> DEV 0xfcd570 GAISLER_GPTIMER -> DEV 0xfcd5d8 GAISLER_GPTIMER -> DEV 0xfcd640 GAISLER_GPTIMER -> DEV 0xfcd6a8 GAISLER_SPW2_DMA -> DEV 0xfcd710 GAISLER_SPW2_DMA -> DEV 0xfcd778 GAISLER_SPW2_DMA -> DEV 0xfcd7e0 GAISLER_SPW2_DMA -> DEV 0xfcd848 GAISLER_ETHMAC -> DEV 0xfcd8b0 GAISLER_ETHMAC -> DEV 0xfcd918 GAISLER_GRCAN -> DEV 0xfcd980 GAISLER_GRCAN -> DEV 0xfcd9e8 GAISLER_SPICTRL -> DEV 0xfdda50 GAISLER_CLKGATE -> DEV 0xfddab8 GAISLER_GR1553B -> DEV 0xfddb20 GAISLER_AHBSTAT -> DEV 0xfddb88 GAISLER_AHBSTAT -> DEV 0xfdbf00 GAISLER_GPIO -> DEV 0xfdbf58 GAISLER_GPREG -> DEV 0xfdbf80 GAISLER_GR740THS -> DEV 0xfdd28 GAISLER_GPREGBANK -> DEV 0xfdd98 GAISLER_SPWTD -> DEV 0xfde00 GAISLER_L4STAT -> DEV 0xfde68 GAISLER_L2CACHE -> DEV 0xfde6d0 GAISLER_MEMSCRUB -> DEV 0xfdf38 GAISLER_GRIOMMU2 -> DEV 0xfdfa0 GAISLER_DDRSDMUX -> DEV 0xfe008 GAISLER_AHB2AHB -> DEV 0xfe070 GAISLER_FTMCTRL -> DEV 0xfe0d8 GAISLER_GRPIC2 -> DEV 0xff320 PCI_0:2:0_00ec:0001 -> DEV 0xffb48 GAISLER_GRPIC2 -> DEV 0xffbb0 GAISLER_AHBUART -> DEV 0xffc18 GAISLER_SPW2 -> DEV 0xffc80 GAISLER_SPW2 -> DEV 0xffce8 GAISLER_B1553BRM -> DEV 0xffd50 GAISLER_APBMS -> DEV 0xffdb8 GAISLER_FTAHBRAM -> DEV 0xffe20 GAISLER_AHBTTRACE -> DEV 0xffe88 GAISLER_FTSRCTRL -> DEV 0xffef0 GAISLER_IRQMP -> DEV 0xffff58 GAISLER_GPIO -> DEV 0xffffc0 GAISLER_GPTIMER -> DEV 0x100028 GAISLER_AHBSTAT -> DEV 0xfe140 GAISLER_SPWRROUTER You can use the shell commands drvmgr and pci to find out more about the system Creating /etc/passwd and group with three useable accounts root/pwd , test/pwd, rtems/NO PASSWORD RTEMS Shell on dev/console. Use 'help' to list commands. SHLL [/] # pci ls PCI devices found and configured: Bus 0 Slot 2 function: 0 [0x10] Vendor id: 0xec, device id: 0x1 IRQ INTA# LINE: 17 BAR 0: 80000000 [1MB] BAR 1: 84000000 [16MB] BAR 2: 89000000 [1MB] SHLL [/] # </pre>

4.1.2 RTEMS Actel 1553BRM for PCI Extension

Figure 4-4 shows the result of rtems-pci-brm with the loadable PCI module. The loadable PCI module can be loaded with '-mod ./Loadable/loadable-PCI/laysim_user_pci.so' on start-up. This example is to test the simple BRM interface on B1553BRM with RT simulation. The 2MB SRAM in the loadable PCI module is used for 1553B shared memory, and it also supports 1553B traffic monitor.

Figure 4-4. rtems-pci-brm with loadable PCI module

laysim-gr740-dbt-cli in Linux	laysim-gr740-dbt-cli in Windows
<pre>[layright@CentOS user-pci]\$ laysim-gr740-dbt-cli -r -core0 rtems-pci-brm.elf -mod ./Loadable/loadable-PCI/laysim_user_pci.so laysim for GR740 Quad Core Processor (DBT) CLI v3.0 by layright Loaded user loadable module ./Loadable/loadable-PCI/laysim_user_pci.so Multicore control : Time quantum (500) CORE#0 RAM file is loaded - rtems-pci-brm.elf - .text, addr: 0x00000000, size 289008 bytes - .rtemsroset, addr: 0x000468F0, size 176 bytes - .rtemsstack, addr: 0x000469C0, size 8192 bytes (not loaded, just info.) - .data, addr: 0x000489C0, size 8400 bytes - .bss, addr: 0x0004AAC0, size 45312 bytes (not loaded, just info.) - Detect RTEMS v5.1(SMP) with thread information Only GR740 CORE#0 RAM file is loaded, starts from RAM Set CORE#0 %sp/%fp : 0x07FFFFFFF/0x08000000 ***** Starting Gaisler BRM test ***** Starting task 1: BC mode brmlib_open: Opening driver /dev/urpcio0/b1553brm0 brmlib_open: allocating memory 20 Task1: Setting BC mode Task1: Setting TX/RX blocking mode Setting up command list. ----- BC: START LIST EXECUTION ----- Start CMD list processing. (BUS A) : BC -> RT Msg No : 0 Cmdnd: 0828 (01-R-01-08) RT1-SA01 Data: 0000 0047 0052 0007 0000 0000 0000 0000 Status: 0800 (BUS A) : BC -> RT Msg No : 1 Cmdnd: 0848 (01-R-02-08) RT1-SA02 Data: 0000 0047 0052 0008 0000 0000 0000 0000 Status: 0800 (BUS A) : BC -> RT Msg No : 2 Cmdnd: 0868 (01-R-03-08) RT1-SA03 Data: 0000 0047 0052 0009 0000 0000 0000 0000W Status: 0800 (BUS A) : BC -> RT Msg No : 3 Cmdnd: 0889 (01-R-04-09) RT1-SA04 Data: 0000 0047 0052 000A 0000 0000 0000 0000 0000a Status: 0800 iting until list processed List processed. Sleeping 20s ----- BC: START LIST EXECUTION ----- Start RESULT list processing. (BUS A) : RT -> BC Msg No : 4 Cmdnd: 0C28 (01-T-01-08) RT1-SA01 Status: 0800 Data: 0020 0021 0022 0023 0024 0025 0026 0027 (BUS A) : RT -> BC Msg No : 5 Cmdnd: 0C48 (01-T-02-08) RT1-SA02 Status: 0800 Data: 0040 0041 0042 0043 0044 0045 0046 0047 (BUS A) : RT -> BC Msg No : 6 Cmdnd: 0C68 (01-T-03-08) RT1-SA03W Status: 0800 Data: 0060 0061 0062 0063 0064 0065 0066 0067 (BUS A) : RT -> BC Msg No : 7 Cmdnd: 0C89 (01-T-04-09) RT1-SA04a Status: 0800 Data: 0080 0081 0082 0083 0084 0085 0086 0087 0088 iting until list processed List processed. Response to message 1: (len: 8, tsw1: 800, tsw2: 0) 0x20 (.)0x21 (.)0x22 (.)0x23 (.)0x24 (.)0x25 (.)0x26 (.)0x27 (.) Response to message 2: (len: 8, tsw1: 800, tsw2: 0) 0x40 (.)0x41 (A)0x42 (B)0x43 (C)0x44 (D)0x45 (E)0x46 (F)0x47 (G) Response to message 3: (len: 8, tsw1: 800, tsw2: 0) 0x60 (.)0x61 (a)0x62 (b)0x63 (c)0x64 (d)0x65 (e)0x66 (f)0x67 (g) Response to message 4: (len: 9, tsw1: 800, tsw2: 0) 0x80 (.)0x81 (.)0x82 (.)0x83 (.)0x84 (.)0x85 (.)0x86 (.)0x87 (.)0x88 (.) Sleeping 15s ----- BC: START LIST EXECUTION ----- Start CMD list processing. (BUS A) : BC -> RT Msg No : 8 Cmdnd: 0828 (01-R-01-08) RT1-SA01 Data: 0000 0047 0052 0007 0000 0000 0000 0000 Status: 0800</pre>	<pre>PS C:\laysim-gr740\user-pci> laysim-gr740-dbt-cli -r -core0 rtems-pci-brm.elf -mod ./Loadable/loadable-PCI\laysim_user_pci-dbt-cli.dll laysim for GR740 Quad Core Processor (DBT) CLI v3.0 by layright Loaded user loadable module ./Loadable/loadable-PCI\laysim_user_pci-dbt-cli.dll Multicore control : Time quantum (500) CORE#0 RAM file is loaded - rtems-pci-brm.elf - .text, addr: 0x00000000, size 289008 bytes - .rtemsroset, addr: 0x000468F0, size 176 bytes - .rtemsstack, addr: 0x000469C0, size 8192 bytes (not loaded, just info.) - .data, addr: 0x000489C0, size 8400 bytes - .bss, addr: 0x0004AAC0, size 45312 bytes (not loaded, just info.) - Detect RTEMS v5.1(SMP) with thread information Only GR740 CORE#0 RAM file is loaded, starts from RAM Set CORE#0 %sp/%fp : 0x07FFFFFFF/0x08000000 ***** Starting Gaisler BRM test ***** Starting task 1: BC mode brmlib_open: Opening driver /dev/urpcio0/b1553brm0 brmlib_open: allocating memory 20 Task1: Setting BC mode Task1: Setting TX/RX blocking mode Setting up command list. ----- BC: START LIST EXECUTION ----- Start CMD list processing. (BUS A) : BC -> RT Msg No : 0 Cmdnd: 0828 (01-R-01-08) RT1-SA01 Data: 0000 0047 0052 0007 0000 0000 0000 0000 Status: 0800 (BUS A) : BC -> RT Msg No : 1 Cmdnd: 0848 (01-R-02-08) RT1-SA02 Data: 0000 0047 0052 0008 0000 0000 0000 0000 Status: 0800 (BUS A) : BC -> RT Msg No : 2 Cmdnd: 0868 (01-R-03-08) RT1-SA03 Data: 0000 0047 0052 0009 0000 0000 0000 0000W Status: 0800 (BUS A) : BC -> RT Msg No : 3 Cmdnd: 0889 (01-R-04-09) RT1-SA04 Data: 0000 0047 0052 000A 0000 0000 0000 0000 0000a Status: 0800 iting until list processed List processed. Sleeping 20s ----- BC: START LIST EXECUTION ----- Start RESULT list processing. (BUS A) : RT -> BC Msg No : 4 Cmdnd: 0C28 (01-T-01-08) RT1-SA01 Status: 0800 Data: 0020 0021 0022 0023 0024 0025 0026 0027 (BUS A) : RT -> BC Msg No : 5 Cmdnd: 0C48 (01-T-02-08) RT1-SA02 Status: 0800 Data: 0040 0041 0042 0043 0044 0045 0046 0047 (BUS A) : RT -> BC Msg No : 6 Cmdnd: 0C68 (01-T-03-08) RT1-SA03W Status: 0800 Data: 0060 0061 0062 0063 0064 0065 0066 0067 (BUS A) : RT -> BC Msg No : 7 Cmdnd: 0C89 (01-T-04-09) RT1-SA04a Status: 0800 Data: 0080 0081 0082 0083 0084 0085 0086 0087 0088 iting until list processed List processed. Response to message 1: (len: 8, tsw1: 800, tsw2: 0) 0x20 (.)0x21 (.)0x22 (.)0x23 (.)0x24 (.)0x25 (.)0x26 (.)0x27 (.) Response to message 2: (len: 8, tsw1: 800, tsw2: 0) 0x40 (.)0x41 (A)0x42 (B)0x43 (C)0x44 (D)0x45 (E)0x46 (F)0x47 (G) Response to message 3: (len: 8, tsw1: 800, tsw2: 0) 0x60 (.)0x61 (a)0x62 (b)0x63 (c)0x64 (d)0x65 (e)0x66 (f)0x67 (g) Response to message 4: (len: 9, tsw1: 800, tsw2: 0) 0x80 (.)0x81 (.)0x82 (.)0x83 (.)0x84 (.)0x85 (.)0x86 (.)0x87 (.)0x88 (.) Sleeping 15s ----- BC: START LIST EXECUTION ----- Start CMD list processing. (BUS A) : BC -> RT Msg No : 8 Cmdnd: 0828 (01-R-01-08) RT1-SA01 Data: 0000 0047 0052 0007 0000 0000 0000 0000 Status: 0800</pre>

4.1.3 RTEMS RMAP for PCI Extension

Figure 4-6 shows the result of `rtems-pci-rmap` with the loadable PCI module. The loadable PCI module can be loaded with '-mod ./Loadable/loadable-PCI/laysim_user_pci.so' on start-up. This example is to test SpW RMAP on GRSpW2 with Node simulation. The GRSpW2 core in the user PCI can access the SDRAM of GR740 through GRPCI2 I/F as shown in Figure 4-5.

Figure 4-5. Operation of GRSpW2 in user PCI

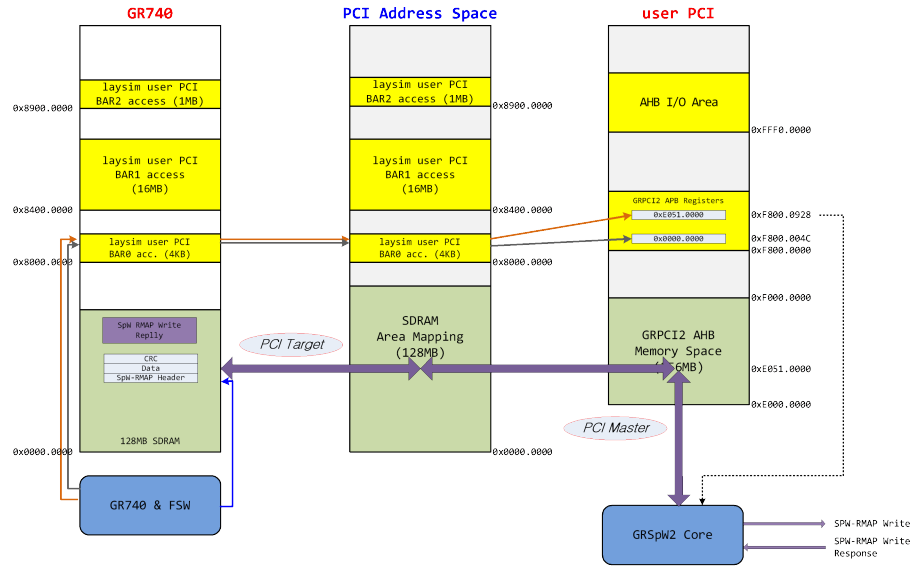


Figure 4-6. rtems-pci-rmap with loadable PCI module

laysim-gr740-dbt-cli in Linux	laysim-gr740-dbt-cli in Windows
<pre>[layright@CentOS user-pci]\$ laysim-gr740-dbt-cli -r -core0 rtems-pci-rmap.elf -mod ../Loadable/loadable-PCI/laysim_user_pci.so laysim for GR740 Quad Core Processor (DBT) CLI v3.0 by layright Loaded user loadable module ../Loadable/loadable-PCI/laysim_user_pci.so Multicore control : Time quantum (500) CORE#0 RAM file is loaded - rtems-pci-rmap.elf - .text, addr: 0x00000000, size 297888 bytes - .rtemsoset, addr: 0x00048BA0, size 176 bytes - .rtemsstack, addr: 0x00048C80, size 8192 bytes (not loaded, just info.) - .data, addr: 0x0004AC80, size 10512 bytes - .bss, addr: 0x0004D5C0, size 218528 bytes (not loaded, just info.) - Detect RTEMS v5.1(SMP) with thread information Only GR740 CORE#0 RAM file is loaded, starts from RAM Set CORE#0 %sp/%fp : 0x07FFFFFF/0x08000000 #### To change configuration of this example, you can set a breakpoint #### on label <rmap_conf_label> and modify the following global variables: ## remote_base_address: 0x40000000 /* Base address on Remote. This is the address to which the data is copied. */ ## remote_dst_address: 0xfe /* SpW Destination address. */ ## remote_dst_key: 0x00 /* SpW Destination Key */ ## source_src_address: 0x20 /* SpW Source address */ ## rmap_dev_idx: 4 /* This parameter chooses which initiator GRSPW2 core is used */ ## route_entry { nr; /* Number of addresses in dstadr array */; dstadr[16]; /* Path Addresses */} ## routetab[ROUTE_TO]={2,{1,5,0, ...}} ## routetab[ROUTE_FROM]={2,{1,9,0, ...}} ## path_addressing: 0 /* Use path addressing (!=0) or not (0) */ ## rmap_pkt_size: 992 ## rmap_data_bytes: 4096 /* Data bytes to send over RMAP */ #### RTEMS RMAP example Setting up SpaceWire router Configuring Router Hardware Configuration of SpaceWire Router: Number of SpW ports: 8 Number of AMBA ports: 4 Number of FIFO ports: 0 Timers available: YES Plug and Play available: YES MAJOR Version: 1 MINOR Version: 2 PATCH Version: 0 Current Instance ID: 19 ROUTER CFG/STS: 0x4100c013 PORT[00]: TYPE=CFG CTRL=0x01000210 STATUS=0x01000210 LINKSTATE=N/A PORT[01]: TYPE=SpW CTRL=0x2700022c STATUS=0x0000a000 LINKSTATE=Ready PORT[02]: TYPE=SpW CTRL=0x2700022c STATUS=0x0000a000 LINKSTATE=Ready PORT[03]: TYPE=SpW CTRL=0x2700022c STATUS=0x0000a000 LINKSTATE=Ready PORT[04]: TYPE=SpW CTRL=0x2700022c STATUS=0x0000a000 LINKSTATE=Ready PORT[05]: TYPE=SpW CTRL=0x2700022c STATUS=0x0000a000 LINKSTATE=Ready</pre>	<pre>PS C:\laysim-gr740\user-pci> laysim-gr740-dbt-cli -r -core0 rtems-pci-rmap.elf -mod ../Loadable/loadable-PCI/laysim_user_pci-dbt-cli.dll laysim for GR740 Quad Core Processor (DBT) CLI v3.0 by layright Loaded user loadable module ../Loadable/loadable-PCI/laysim_user_pci-dbt-cli.dll Multicore control : Time quantum (500) CORE#0 RAM file is loaded - rtems-pci-rmap.elf - .text, addr: 0x00000000, size 297888 bytes - .rtemsoset, addr: 0x00048BA0, size 176 bytes - .rtemsstack, addr: 0x00048C80, size 8192 bytes (not loaded, just info.) - .data, addr: 0x0004AC80, size 10512 bytes - .bss, addr: 0x0004D5C0, size 218528 bytes (not loaded, just info.) - Detect RTEMS v5.1(SMP) with thread information Only GR740 CORE#0 RAM file is loaded, starts from RAM Set CORE#0 %sp/%fp : 0x07FFFFFF/0x08000000 #### To change configuration of this example, you can set a breakpoint #### on label <rmap_conf_label> and modify the following global variables: ## remote_base_address: 0x40000000 /* Base address on Remote. This is the address to which the data is copied. */ ## remote_dst_address: 0xfe /* SpW Destination address. */ ## remote_dst_key: 0x00 /* SpW Destination Key */ ## source_src_address: 0x20 /* SpW Source address */ ## rmap_dev_idx: 4 /* This parameter chooses which initiator GRSPW2 core is used */ ## route_entry { nr; /* Number of addresses in dstadr array */; dstadr[16]; /* Path Addresses */} ## routetab[ROUTE_TO]={2,{1,5,0, ...}} ## routetab[ROUTE_FROM]={2,{1,9,0, ...}} ## path_addressing: 0 /* Use path addressing (!=0) or not (0) */ ## rmap_pkt_size: 992 ## rmap_data_bytes: 4096 /* Data bytes to send over RMAP */ #### RTEMS RMAP example Setting up SpaceWire router Configuring Router Hardware Configuration of SpaceWire Router: Number of SpW ports: 8 Number of AMBA ports: 4 Number of FIFO ports: 0 Timers available: YES Plug and Play available: YES MAJOR Version: 1 MINOR Version: 2 PATCH Version: 0 Current Instance ID: 19 ROUTER CFG/STS: 0x4100c013 PORT[00]: TYPE=CFG CTRL=0x01000210 STATUS=0x01000210 LINKSTATE=N/A PORT[01]: TYPE=SpW CTRL=0x2700022c STATUS=0x0000a000 LINKSTATE=Ready PORT[02]: TYPE=SpW CTRL=0x2700022c STATUS=0x0000a000 LINKSTATE=Ready PORT[03]: TYPE=SpW CTRL=0x2700022c STATUS=0x0000a000 LINKSTATE=Ready PORT[04]: TYPE=SpW CTRL=0x2700022c STATUS=0x0000a000 LINKSTATE=Ready PORT[05]: TYPE=SpW CTRL=0x2700022c STATUS=0x0000a000 LINKSTATE=Ready</pre>

<pre> PORT[06]: TYPE=Spw CTRL=0x2700022c STATUS=0x0000a000 LINKSTATE=Ready PORT[07]: TYPE=Spw CTRL=0x2700022c STATUS=0x0000a000 LINKSTATE=Ready PORT[08]: TYPE=Spw CTRL=0x2700022c STATUS=0x0000a000 LINKSTATE=Ready PORT[09]: TYPE=AMBA CTRL=0x00000228 STATUS=0x40000000 LINKSTATE=N/A PORT[10]: TYPE=AMBA CTRL=0x00000228 STATUS=0x40000000 LINKSTATE=N/A PORT[11]: TYPE=AMBA CTRL=0x00000228 STATUS=0x40000000 LINKSTATE=N/A PORT[12]: TYPE=AMBA CTRL=0x00000228 STATUS=0x40000000 LINKSTATE=N/A Activating all Links/Ports PORT[01]: TYPE=Spw CTRL=0x0300002e STATUS=0x0000a000 LINK-STATE=Ready PORT[02]: TYPE=Spw CTRL=0x0300002e STATUS=0x0000a000 LINK-STATE=Ready PORT[03]: TYPE=Spw CTRL=0x0300002e STATUS=0x0000a000 LINK-STATE=Ready PORT[04]: TYPE=Spw CTRL=0x0300002e STATUS=0x0000a000 LINK-STATE=Ready PORT[05]: TYPE=Spw CTRL=0x0300002e STATUS=0x0000a000 LINK-STATE=Ready PORT[06]: TYPE=Spw CTRL=0x0300002e STATUS=0x0000a000 LINK-STATE=Ready PORT[07]: TYPE=Spw CTRL=0x0300002e STATUS=0x0000a000 LINK-STATE=Ready PORT[08]: TYPE=Spw CTRL=0x0300002e STATUS=0x0000a000 LINK-STATE=Ready Configuring Router Failed to open Spw Router Initializing SpaceWire device 0 NOTE: running on SPW-ROUTER DMA SpaceWire link (no link-state available) Initializing SpaceWire device 1 NOTE: running on SPW-ROUTER DMA SpaceWire link (no link-state available) Initializing SpaceWire device 2 NOTE: running on SPW-ROUTER DMA SpaceWire link (no link-state available) Initializing SpaceWire device 3 NOTE: running on SPW-ROUTER DMA SpaceWire link (no link-state available) Initializing SpaceWire device 4 After Link Start: 5 Initializing SpaceWire device 5 After Link Start: 5 Test built for at least 6 AMBA ports. Note that the Spw-links will not be used. There are 6 GRSPW cores in the system. System clock: 10000 us / tick Starting GRSPW4: DMA Started Successfully RMAP connection is working Initializing data array Starting transmission type 1: one packet at a time RMAP write cycle completed in 0.000430 seconds RMAP read cycle completed in 0.000433 seconds Data bytes sent: 4096, Packet size: 992, Packet count: 5 Wrote 4096 bytes to target in 0.000430 seconds. WRITE RATE: 9309.079610 Kib/s Read 4096 bytes from target in 0.000433 seconds. READ RATE: 9233.788174 Kib/s Wrote 5 packets to target in 0.000430 seconds. WRITE RATE: 11636.349513 pkt/s Read 5 packets from target in 0.000433 seconds. READ RATE: 11542.235217 pkt/s EXAMPLE 1 SUCCESSFULLY COMPLETED. Starting transmission type 2: all packets at a time RMAP write cycle completed in 0.000394 seconds RMAP read cycle completed in 0.000380 seconds Data bytes sent: 4096, Packet size: 992, Packet count: 5 Wrote 4096 bytes to target in 0.000394 seconds. WRITE RATE: 10149.555959 Kib/s Read 4096 bytes from target in 0.000380 seconds. READ RATE: 10538.038551 Kib/s Wrote 5 packets to target in 0.000394 seconds. WRITE RATE: 12686.944949 pkt/s Read 5 packets from target in 0.000380 seconds. READ RATE: 13172.548188 pkt/s EXAMPLE 2 SUCCESSFULLY COMPLETED. CORE#0 halts because of Error Mode [0.30709sec/335922255, 0, 0, 0cycles] Performance statistics for CORE#0 Cycles : 335922255 Instructions : 213165440 CORE#0 performance (250MHz) : 158.64 MIPS Simulated time : 1.34 s Performance of laysim-gr740 Real-time performance : 437.56 % laysim-gr740 performance : 694.15 MIPS Wall clock : 0.31 s [layrigh@CentOS user-pci]\$ </pre>	<pre> PORT[06]: TYPE=Spw CTRL=0x2700022c STATUS=0x0000a000 LINKSTATE=Ready PORT[07]: TYPE=Spw CTRL=0x2700022c STATUS=0x0000a000 LINKSTATE=Ready PORT[08]: TYPE=Spw CTRL=0x2700022c STATUS=0x0000a000 LINKSTATE=Ready PORT[09]: TYPE=AMBA CTRL=0x00000228 STATUS=0x40000000 LINKSTATE=N/A PORT[10]: TYPE=AMBA CTRL=0x00000228 STATUS=0x40000000 LINKSTATE=N/A PORT[11]: TYPE=AMBA CTRL=0x00000228 STATUS=0x40000000 LINKSTATE=N/A PORT[12]: TYPE=AMBA CTRL=0x00000228 STATUS=0x40000000 LINKSTATE=N/A Activating all Links/Ports PORT[01]: TYPE=Spw CTRL=0x0300002e STATUS=0x0000a000 LINK-STATE=Ready PORT[02]: TYPE=Spw CTRL=0x0300002e STATUS=0x0000a000 LINK-STATE=Ready PORT[03]: TYPE=Spw CTRL=0x0300002e STATUS=0x0000a000 LINK-STATE=Ready PORT[04]: TYPE=Spw CTRL=0x0300002e STATUS=0x0000a000 LINK-STATE=Ready PORT[05]: TYPE=Spw CTRL=0x0300002e STATUS=0x0000a000 LINK-STATE=Ready PORT[06]: TYPE=Spw CTRL=0x0300002e STATUS=0x0000a000 LINK-STATE=Ready PORT[07]: TYPE=Spw CTRL=0x0300002e STATUS=0x0000a000 LINK-STATE=Ready PORT[08]: TYPE=Spw CTRL=0x0300002e STATUS=0x0000a000 LINK-STATE=Ready Configuring Router Failed to open Spw Router Initializing SpaceWire device 0 NOTE: running on SPW-ROUTER DMA SpaceWire link (no link-state available) Initializing SpaceWire device 1 NOTE: running on SPW-ROUTER DMA SpaceWire link (no link-state available) Initializing SpaceWire device 2 NOTE: running on SPW-ROUTER DMA SpaceWire link (no link-state available) Initializing SpaceWire device 3 NOTE: running on SPW-ROUTER DMA SpaceWire link (no link-state available) Initializing SpaceWire device 4 After Link Start: 5 Initializing SpaceWire device 5 After Link Start: 5 Test built for at least 6 AMBA ports. Note that the Spw-links will not be used. There are 6 GRSPW cores in the system. System clock: 10000 us / tick Starting GRSPW4: DMA Started Successfully RMAP connection is working Initializing data array Starting transmission type 1: one packet at a time RMAP write cycle completed in 0.000430 seconds RMAP read cycle completed in 0.000433 seconds Data bytes sent: 4096, Packet size: 992, Packet count: 5 Wrote 4096 bytes to target in 0.000430 seconds. WRITE RATE: 9309.079610 Kib/s Read 4096 bytes from target in 0.000433 seconds. READ RATE: 9233.788174 Kib/s Wrote 5 packets to target in 0.000430 seconds. WRITE RATE: 11636.349513 pkt/s Read 5 packets from target in 0.000433 seconds. READ RATE: 11542.235217 pkt/s EXAMPLE 1 SUCCESSFULLY COMPLETED. Starting transmission type 2: all packets at a time RMAP write cycle completed in 0.000394 seconds RMAP read cycle completed in 0.000380 seconds Data bytes sent: 4096, Packet size: 992, Packet count: 5 Wrote 4096 bytes to target in 0.000394 seconds. WRITE RATE: 10149.555959 Kib/s Read 4096 bytes from target in 0.000380 seconds. READ RATE: 10538.038551 Kib/s Wrote 5 packets to target in 0.000394 seconds. WRITE RATE: 12686.944949 pkt/s Read 5 packets from target in 0.000380 seconds. READ RATE: 13172.548188 pkt/s EXAMPLE 2 SUCCESSFULLY COMPLETED. CORE#0 halts because of Error Mode [0.904802sec/335922255, 0, 0, 0cycles] Performance statistics for CORE#0 Cycles : 335922255 Instructions : 213165440 CORE#0 performance (250MHz) : 158.64 MIPS Simulated time : 1.34 s Performance of laysim-gr740 Real-time performance : 148.51 % laysim-gr740 performance : 235.59 MIPS Wall clock : 0.90 s PS C:\laysim-gr740\user-pci> </pre>
--	---

4.1.4 RTEMS TLIB for PCI Extension

Figure 4-7 shows the result of rtems-pci-tlib with the loadable PCI module. The loadable PCI module can be loaded with '-mod ./Loadable/loadable-PCI/laysim_user_pci.so' on start-up. This example is to test all GPTIMERs with 3 GPTIMERs in the user PCI.

Figure 4-7. rtems-pci-tlib with loadable PCI module

laysim-gr740-mdbt-cli in Linux	laysim-gr740-mdbt-cli in Windows
<pre> [layrigh@CentOS user-pci]\$ laysim-gr740-mdbt-cli -r -core0 rtems-pci-tlib.elf -mod ./Loadable/loadable-PCI/laysim_user_pci.so laysim for GR740 Quad Core Processor (MDBT) CLI v3.0 by layright Loaded user loadable module ./Loadable/loadable-PCI/laysim_user_pci.so Multicore control : Time quantum (500) CORE#0 RAM file is loaded - rtems-pci-tlib.elf - .text, addr: 0x00000000, size 281920 bytes - .rtemsreset, addr: 0x00044040, size 176 bytes - .rtemsstack, addr: 0x00044E00, size 8192 bytes (not loaded, just info.) - .data, addr: 0x00046E00, size 8192 bytes </pre>	<pre> PS C:\laysim-gr740\user-pci> laysim-gr740-mdbt-cli -r -core0 rtems-pci-tlib.elf -mod ./Loadable/loadable-PCI/laysim_user_pci-mdbt-cli.dll laysim for GR740 Quad Core Processor (MDBT) CLI v3.0 by layright Loaded user loadable module ./Loadable/loadable-PCI/laysim_user_pci-mdbt-cli.dll Multicore control : Time quantum (500) CORE#0 RAM file is loaded - rtems-pci-tlib.elf - .text, addr: 0x00000000, size 281920 bytes - .rtemsreset, addr: 0x00044D00, size 176 bytes - .rtemsstack, addr: 0x00044E00, size 8192 bytes (not loaded, just info.) - .data, addr: 0x00046E00, size 8192 bytes </pre>

laysim-GR740-001 laysim-GR740 Quad Core Processor Emulator – User's Manual [Rev13]

```
- .bss, addr: 0x00048E00, size 20528 bytes (not loaded, just info.)
- Detect RTEMS v5.1(SMP) with thread information
Only GR740 CORE#0 RAM file is loaded, starts from RAM
Set CORE#0 %sp/%fp : 0x07FFFFFF0/0x08000000

--- BUS TOPOLOGY ---
|-> DEV 0x55ad0 GAISLER_LEON4
|-> DEV 0x55b38 GAISLER_LEON4
|-> DEV 0x55ba0 GAISLER_LEON4
|-> DEV 0x55c08 GAISLER_LEON4
|-> DEV 0x55c70 GAISLER_GRIOMMU
|-> DEV 0x55cd8 GAISLER_AHB2AHB
|-> DEV 0x55d40 GAISLER_L2CACHE
|-> DEV 0x55da8 GAISLER_APBMMST
|-> DEV 0x55e10 GAISLER_APBMMST
|-> DEV 0x55e78 GAISLER_APBMMST
|-> DEV 0x55ee0 GAISLER_APBMMST
|-> DEV 0x55f48 GAISLER_GPIO
|-> DEV 0x55fb0 GAISLER_FTMCTRL
|-> DEV 0x56018 GAISLER_IRQMP
|-> DEV 0x56080 GAISLER_GPTIMER
|-> DEV 0x560e8 GAISLER_GPTIMER

...
...
...
Waiting for one tick
Current counter value: 4294967295
Testing Interrupt routine over 500 System clock ticks
Timer is now stopped
Current counter value: 2900
Current tickrate: 100000
Number of interrupts (ticks): 49
Timer 4 test OK

24 Timers in system
Opened Timer3 for testing
Base frequency: 1000000Hz
Current tickrate: 0Hz
Current counter value: 4294967295

Resetting timer ...

Base frequency: 1000000Hz
Current tickrate: 0Hz
Current counter value: 4294967295
Set 100000 clock "clicks" per tick (10 ticks/sec)
Waiting for one tick
Current counter value: 4294967295
Testing Interrupt routine over 500 System clock ticks
Timer is now stopped
Current counter value: 2900
Current tickrate: 100000
Number of interrupts (ticks): 49
Timer 3 test OK

24 Timers in system
Opened Timer2 for testing
Base frequency: 1000000Hz
Current tickrate: 0Hz
Current counter value: 4294967295

Resetting timer ...

Base frequency: 1000000Hz
Current tickrate: 0Hz
Current counter value: 4294967295
Set 100000 clock "clicks" per tick (10 ticks/sec)
Waiting for one tick
Current counter value: 4294967295
Testing Interrupt routine over 500 System clock ticks
Timer is now stopped
Current counter value: 2900
Current tickrate: 100000
Number of interrupts (ticks): 49
Timer 2 test OK

24 Timers in system
Opened Timer1 for testing
Base frequency: 1000000Hz
Current tickrate: 0Hz
Current counter value: 4294967295

Resetting timer ...

Base frequency: 1000000Hz
Current tickrate: 0Hz
Current counter value: 4294967295
Set 100000 clock "clicks" per tick (10 ticks/sec)
Waiting for one tick
Current counter value: 4294967295
Testing Interrupt routine over 500 System clock ticks
Timer is now stopped
Current counter value: 2900
Current tickrate: 100000
Number of interrupts (ticks): 49
Timer 1 test OK

24 Timers in system
Failed to open timer0
Timer 0 test failed: -3

CORE#0 halts because of Error Mode [5.73848sec/35214173814, 0, 0, 0cycles]

Performance statistics for CORE#0
Cycles : 35214173814
Instructions : 939460206
CORE#0 performance (250MHz) : 6.67 MIPS
Simulated time : 140.86 s

Performance of laysim-gr740
Real-time performance : 2454.60 %
laysim-gr740 performance : 163.71 MIPS
Wall clock : 5.74 s
[lavright@CentOS user-pci]$
```

```
- .bss, addr: 0x00048E00, size 20528 bytes (not loaded, just info.)
- Detect RTEMS v5.1(SMP) with thread information
Only GR740 CORE#0 RAM file is loaded, starts from RAM
Set CORE#0 %sp/%fp : 0x07FFFFFF0/0x08000000

--- BUS TOPOLOGY ---
|-> DEV 0x55ad0 GAISLER_LEON4
|-> DEV 0x55b38 GAISLER_LEON4
|-> DEV 0x55ba0 GAISLER_LEON4
|-> DEV 0x55c08 GAISLER_LEON4
|-> DEV 0x55c70 GAISLER_GRIOMMU
|-> DEV 0x55cd8 GAISLER_AHB2AHB
|-> DEV 0x55d40 GAISLER_L2CACHE
|-> DEV 0x55da8 GAISLER_APBMMST
|-> DEV 0x55e10 GAISLER_APBMMST
|-> DEV 0x55e78 GAISLER_APBMMST
|-> DEV 0x55ee0 GAISLER_APBMMST
|-> DEV 0x55f48 GAISLER_GPIO
|-> DEV 0x55fb0 GAISLER_FTMCTRL
|-> DEV 0x56018 GAISLER_IRQMP
|-> DEV 0x56080 GAISLER_GPTIMER
|-> DEV 0x560e8 GAISLER_GPTIMER

...
...
...
Waiting for one tick
Current counter value: 4294967295
Testing Interrupt routine over 500 System clock ticks
Timer is now stopped
Current counter value: 2900
Current tickrate: 100000
Number of interrupts (ticks): 49
Timer 4 test OK

24 Timers in system
Opened Timer3 for testing
Base frequency: 1000000Hz
Current tickrate: 0Hz
Current counter value: 4294967295

Resetting timer ...

Base frequency: 1000000Hz
Current tickrate: 0Hz
Current counter value: 4294967295
Set 100000 clock "clicks" per tick (10 ticks/sec)
Waiting for one tick
Current counter value: 4294967295
Testing Interrupt routine over 500 System clock ticks
Timer is now stopped
Current counter value: 2900
Current tickrate: 100000
Number of interrupts (ticks): 49
Timer 3 test OK

24 Timers in system
Opened Timer2 for testing
Base frequency: 1000000Hz
Current tickrate: 0Hz
Current counter value: 4294967295

Resetting timer ...

Base frequency: 1000000Hz
Current tickrate: 0Hz
Current counter value: 4294967295
Set 100000 clock "clicks" per tick (10 ticks/sec)
Waiting for one tick
Current counter value: 4294967295
Testing Interrupt routine over 500 System clock ticks
Timer is now stopped
Current counter value: 2900
Current tickrate: 100000
Number of interrupts (ticks): 49
Timer 2 test OK

24 Timers in system
Opened Timer1 for testing
Base frequency: 1000000Hz
Current tickrate: 0Hz
Current counter value: 4294967295

Resetting timer ...

Base frequency: 1000000Hz
Current tickrate: 0Hz
Current counter value: 4294967295
Set 100000 clock "clicks" per tick (10 ticks/sec)
Waiting for one tick
Current counter value: 4294967295
Testing Interrupt routine over 500 System clock ticks
Timer is now stopped
Current counter value: 2900
Current tickrate: 100000
Number of interrupts (ticks): 49
Timer 1 test OK

24 Timers in system
Failed to open timer0
Timer 0 test failed: -3

CORE#0 halts because of Error Mode [7.89361sec/35214173814, 0, 0, 0cycles]

Performance statistics for CORE#0
Cycles : 35214173814
Instructions : 939460206
CORE#0 performance (250MHz) : 6.67 MIPS
Simulated time : 140.86 s

Performance of laysim-gr740
Real-time performance : 1784.44 %
laysim-gr740 performance : 119.02 MIPS
Wall clock : 7.89 s
PS C:\laysim-gr740\user-pci>
```

4.1.5 RTEMS PCIDMA for PCI Extension

Figure 4-8 shows the result of rtems-pci-pcidma with the loadable PCI module. The loadable PCI module can be loaded with '-mod ./Loadable/loadable-PCI/laysim_user_pci.so' on start-up. This example is to test the GRPCI2 PCIDMA with the loadable PCI module.

Figure 4-8. rtems-pci-pcidma with loadable PCI module

laysim-gr740-mdbt-cli in Linux	laysim-gr740-mdbt-cli in Windows
<pre>[layright@CentOS user-pci]\$ laysim-gr740-mdbt-cli -r -core0 rtems-pci-pcidma.elf -mod ./Loadable/loadable-PCI/laysim_user_pci.so laysim for GR740 Quad Core Processor (MDBT) CLI v3.0 by layright Loaded user loadable module ./Loadable/loadable-PCI/laysim_user_pci.so Multicore control : Time quantum (500) CORE#0 RAM file is loaded - rtems-pci-pcidma.elf - .text, addr: 0x00000000, size 290256 bytes - .rtemsroset, addr: 0x00046DD0, size 176 bytes - .rtemsstack, addr: 0x00046E80/0x00048E80, size 8192 bytes (not loaded, just info.) - .data, addr: 0x00048E80, size 8384 bytes - .bss, addr: 0x0004AF40, size 44576 bytes (not loaded, just info.) - Detect RTEMS v5.1(SMP) with thread information Only GR740 CORE#0 RAM file is loaded, starts from RAM Set CORE#0 %sp/%fp : 0x07FFFFFF0/0x08000000 Test built for PCI. System clock: 2000 us / tick Test will run for 3 s. Remote Memory (0xfc200000) mapped to local 0x84200000 Time: 3.966406 seconds Shutting down: 0 t0.tv_sec: 0 t0.tv_nsec: 37830763 t1.tv_sec: 4 t1.tv_nsec: 4237151 Time: 3.966406 seconds Bytes sent: 232783872 bytes Data desc: 128 desc Block size: 262144 bytes Transfers: 1 transfers Throughput: 58688860.342432 bytes/sec Throughput: 58688.861099 Kbytes/sec Throughput: 469510.888790 Kbits/sec CORE#0 halts because of Error Mode [2.04852sec/1029477545, 0, 0, 0cycles] Performance statistics for CORE#0 Cycles : 1029477545 Instructions : 835506006 CORE#0 performance (250MHz) : 202.90 MIPS Simulated time : 4.12 s Performance of laysim-gr740 Real-time performance : 201.02 % laysim-gr740 performance : 407.86 MIPS Wall clock : 2.05 s [layright@CentOS user-pci]\$</pre>	<pre>PS C:\laysim-gr740\user-pci> laysim-gr740-mdbt-cli -r -core0 rtems-pci-pcidma.elf -mod ./Loadable/loadable-PCI\laysim_user_pci-mdbt-cli.dll laysim for GR740 Quad Core Processor (MDBT) CLI v3.0 by layright Loaded user loadable module ./Loadable/loadable-PCI\laysim_user_pci-mdbt-cli.dll Multicore control : Time quantum (500) CORE#0 RAM file is loaded - rtems-pci-pcidma.elf - .text, addr: 0x00000000, size 290256 bytes - .rtemsroset, addr: 0x00046DD0, size 176 bytes - .rtemsstack, addr: 0x00046E80/0x00048E80, size 8192 bytes (not loaded, just info.) - .data, addr: 0x00048E80, size 8384 bytes - .bss, addr: 0x0004AF40, size 44576 bytes (not loaded, just info.) - Detect RTEMS v5.1(SMP) with thread information Only GR740 CORE#0 RAM file is loaded, starts from RAM Set CORE#0 %sp/%fp : 0x07FFFFFF0/0x08000000 Test built for PCI. System clock: 2000 us / tick Test will run for 3 s. Remote Memory (0xfc200000) mapped to local 0x84200000 Time: 3.966406 seconds Shutting down: 0 t0.tv_sec: 0 t0.tv_nsec: 37830763 t1.tv_sec: 4 t1.tv_nsec: 4237151 Time: 3.966406 seconds Bytes sent: 232783872 bytes Data desc: 128 desc Block size: 262144 bytes Transfers: 1 transfers Throughput: 58688860.342432 bytes/sec Throughput: 58688.861099 Kbytes/sec Throughput: 469510.888790 Kbits/sec CORE#0 halts because of Error Mode [2.0592sec/1029477545, 0, 0, 0cycles] Performance statistics for CORE#0 Cycles : 1029477545 Instructions : 835506006 CORE#0 performance (250MHz) : 202.90 MIPS Simulated time : 4.12 s Performance of laysim-gr740 Real-time performance : 199.98 % laysim-gr740 performance : 405.74 MIPS Wall clock : 2.06 s PS C:\laysim-gr740\user-pci></pre>

4.2 Loadable I/O Module

The loadable I/O module has 4 components

- 1) I/O DMA ENGINE (0xD0000000 ~ 0xD000000F)
 - Simple example for demonstrating DMA read/write operation
 - Raise GPIO#0 port 3 interrupt after completion
- 2) I/O C1553BRM MODEL (0xD1000000 ~ 0xD100007F)
 - Actel Core 1553BRM
 - Raise GPIO#0 port 4 interrupt
 - 1553B RT traffic monitoring & RT simulation
- 3) I/O Shared Memory (0xD2000000 ~ 0xD201FFFF, 128KB)
 - Shared memory for C1553BRM
- 4) Board Reset Register (0xD0000100)

4.2.1 Actel Core 1553B on I/O Area

Figure 4-9 shows the result of user-io-c1553brm with the loadable I/O module. The loadable I/O module can be loaded with '-mod ./Loadable/loadable-I0/laysim_user_io.so' on start-up. This example is to test the simple BRM interface on B1553BRM with RT simulation. The 128KB SRAM in laysim user I/O module is used for 1553B shared memory, and it also supports 1553B traffic monitor.

Figure 4-9. user-io-c1553brm with loadable I/O module

laysim-gr740-dbt-cli in Linux	laysim-gr740-dbt-cli in Windows
<pre>[layright@CentOS user-io]\$ laysim-gr740-dbt-cli -r -core0 user-io-c1553brm.elf -mod ./Loadable/loadable-I0/laysim_user_io.so laysim for GR740 Quad Core Processor (DBT) CLI v3.0 by layright Loaded user loadable module ../Loadable/loadable-I0/laysim_user_io.so Multicore control : Time quantum (500) CORE#0 RAM file is loaded - user-io-c1553brm.elf - .text, addr: 0x00000000, size 68000 bytes - .rodata, addr: 0x000109A0, size 4720 bytes - .data, addr: 0x00011C10, size 1568 bytes - .bss, addr: 0x00012230, size 424 bytes (not loaded, just info.) Only GR740 CORE#0 RAM file is loaded, starts from RAM Set CORE#0 %sp/%fp : 0x07FFFFFF0/0x08000000 Command Block #0 1) Control Word : 0x46FE - OPCODE : 0x4 -> Execute Block - Continue (1553B Op.) - 1 Retry - Channel A - No RT-to-RT - Condition 7 (No Response) is Enabled - Condition 6 (Message Error) is Enabled - Condition 5 (Busy bit) is Enabled - Condition 4 (Terminal Flag) is Enabled - Condition 3 (Subsystem Fail) is Enabled - Condition 2 (Instrumentation) is Enabled - Condition 1 (Service Request) is Enabled - BAME : 0 2) Command Word 1 : 0x0820 - (01-R-01-32) 3) Command Word 2 : 0x0000 4) Data Pointer : 0x0640 (0xD2000C80) - BC -> RT will be sent : 0x0000 0x0001 0x0002 0x0003 0x0004 0x0005 0x0006 0x0007 0x0008 0x0009 0x000A 0x000B 0x000C 0x000D 0x000E 0x000F 0x0010 0x0011 0x0012 0x0013 0x0014 0x0015 0x0016 0x0017 0x0018 0x0019 0x001A 0x001B 0x001C 0x001D 0x001E 0x001F 5) Status Word 1 : 0x0000 6) Status Word 2 : 0x0000 7) Branch Address : 0x0000 (0xD2000000) 8) Timer Value : 0x0000 Command Block #1 1) Control Word : 0x46FE - OPCODE : 0x4 -> Execute Block - Continue (1553B Op.) - 1 Retry - Channel A - No RT-to-RT - Condition 7 (No Response) is Enabled - Condition 6 (Message Error) is Enabled - Condition 5 (Busy bit) is Enabled - Condition 4 (Terminal Flag) is Enabled - Condition 3 (Subsystem Fail) is Enabled - Condition 2 (Instrumentation) is Enabled - Condition 1 (Service Request) is Enabled - BAME : 0 2) Command Word 1 : 0x0C40 - (01-T-02-32) 3) Command Word 2 : 0x0000 4) Data Pointer : 0x0660 (0xD2000CC0) 5) Status Word 1 : 0x0000 6) Status Word 2 : 0x0000 7) Branch Address : 0x0000 (0xD2000000) 8) Timer Value : 0x0000 Command Block #2 1) Control Word : 0x0000 - OPCODE : 0x0 -> EOL - 4 Retry - Channel B - No RT-to-RT - Condition 7 (No Response) is Disabled - Condition 6 (Message Error) is Disabled - Condition 5 (Busy bit) is Disabled - Condition 4 (Terminal Flag) is Disabled - Condition 3 (Subsystem Fail) is Disabled - Condition 2 (Instrumentation) is Disabled - Condition 1 (Service Request) is Disabled - BAME : 0 2) Command Word 1 : 0x0000 3) Command Word 2 : 0x0000 4) Data Pointer : 0x0680 (0xD2000D00) 5) Status Word 1 : 0x0000 6) Status Word 2 : 0x0000 7) Branch Address : 0x0000 (BUS A) : BC -> RT Msg No : 0 [150215039] Cmd: 0820 (01-R-01-32) RT1-SA01 Data: 0000 0001 0002 0003 0004 0005 0006 0007 0008 0009 000A 000B 000C 000D 000E 000F 0010 0011 0012 0013 0014 0015 0016 0017 0018 0019 001A 001B 001C 001D 001E 001F0 Status: 0800 ----- (BUS A) : RT -> BC Msg No : 1 [150386910] Cmd: 0C40 (01-T-02-32) RT1-SA02(0x</pre>	<pre>PS C:\laysim-gr740\user-io> laysim-gr740-dbt-cli -r -core0 user-io-c1553brm.elf -mod ..\Loadable\loadable-I0\laysim_user_io-dbt-cli.dll laysim for GR740 Quad Core Processor (DBT) CLI v3.0 by layright Loaded user loadable module ..\Loadable\loadable-I0\laysim_user_io-dbt-cli.dll Multicore control : Time quantum (500) CORE#0 RAM file is loaded - user-io-c1553brm.elf - .text, addr: 0x00000000, size 68000 bytes - .rodata, addr: 0x000109A0, size 4720 bytes - .data, addr: 0x00011C10, size 1568 bytes - .bss, addr: 0x00012230, size 424 bytes (not loaded, just info.) Only GR740 CORE#0 RAM file is loaded, starts from RAM Set CORE#0 %sp/%fp : 0x07FFFFFF0/0x08000000 Command Block #0 1) Control Word : 0x46FE - OPCODE : 0x4 -> Execute Block - Continue (1553B Op.) - 1 Retry - Channel A - No RT-to-RT - Condition 7 (No Response) is Enabled - Condition 6 (Message Error) is Enabled - Condition 5 (Busy bit) is Enabled - Condition 4 (Terminal Flag) is Enabled - Condition 3 (Subsystem Fail) is Enabled - Condition 2 (Instrumentation) is Enabled - Condition 1 (Service Request) is Enabled - BAME : 0 2) Command Word 1 : 0x0820 - (01-R-01-32) 3) Command Word 2 : 0x0000 4) Data Pointer : 0x0640 (0xD2000C80) - BC -> RT will be sent : 0x0000 0x0001 0x0002 0x0003 0x0004 0x0005 0x0006 0x0007 0x0008 0x0009 0x000A 0x000B 0x000C 0x000D 0x000E 0x000F 0x0010 0x0011 0x0012 0x0013 0x0014 0x0015 0x0016 0x0017 0x0018 0x0019 0x001A 0x001B 0x001C 0x001D 0x001E 0x001F 5) Status Word 1 : 0x0000 6) Status Word 2 : 0x0000 7) Branch Address : 0x0000 (0xD2000000) 8) Timer Value : 0x0000 Command Block #1 1) Control Word : 0x46FE - OPCODE : 0x4 -> Execute Block - Continue (1553B Op.) - 1 Retry - Channel A - No RT-to-RT - Condition 7 (No Response) is Enabled - Condition 6 (Message Error) is Enabled - Condition 5 (Busy bit) is Enabled - Condition 4 (Terminal Flag) is Enabled - Condition 3 (Subsystem Fail) is Enabled - Condition 2 (Instrumentation) is Enabled - Condition 1 (Service Request) is Enabled - BAME : 0 2) Command Word 1 : 0x0C40 - (01-T-02-32) 3) Command Word 2 : 0x0000 4) Data Pointer : 0x0660 (0xD2000CC0) 5) Status Word 1 : 0x0000 6) Status Word 2 : 0x0000 7) Branch Address : 0x0000 (0xD2000000) 8) Timer Value : 0x0000 Command Block #2 1) Control Word : 0x0000 - OPCODE : 0x0 -> EOL - 4 Retry - Channel B - No RT-to-RT - Condition 7 (No Response) is Disabled - Condition 6 (Message Error) is Disabled - Condition 5 (Busy bit) is Disabled - Condition 4 (Terminal Flag) is Disabled - Condition 3 (Subsystem Fail) is Disabled - Condition 2 (Instrumentation) is Disabled - Condition 1 (Service Request) is Disabled - BAME : 0 2) Command Word 1 : 0x0000 3) Command Word 2 : 0x0000 4) Data Pointer : 0x0680 (0xD2000D00) 5) Status Word 1 : 0x0000 6) Status Word 2 : 0x0000 7) Branch Address : 0x0000 (BUS A) : BC -> RT Msg No : 0 [150215039] Cmd: 0820 (01-R-01-32) RT1-SA01 Data: 0000 0001 0002 0003 0004 0005 0006 0007 0008 0009 000A 000B 000C 000D 000E 000F 0010 0011 0012 0013 0014 0015 0016 0017 0018 0019 001A 001B 001C 001D 001E 001F0 Status: 0800 ----- (BUS A) : RT -> BC Msg No : 1 [150386910] Cmd: 0C40 (01-T-02-32) RT1-SA02(0x</pre>

```
Status: 0000
Data: 0040 0041 0042 0043 0044 0045 0046 0047
      0048 0049 004A 004B 004C 004D 004E 004F
      0050 0051 0052 0053 0054 0055 0056 0057
      0058 0059 005A 005B 005C 005D 005E 005F
```

```
D2000000)
8) Timer Value : 0x0000
```

```
Register #4 (Pending Interrupt, 0x00000020) : 0x00000000
- bit 15 - DMAF (DMA Fail Interrupt) : 0 (Not Occurred)
- bit 14 - WRAPF (Wrap Fail Interrupt) : 0 (Not Occurred)
- bit 13 - TAPF (RT Only) : 0 (Not Occurred)
- bit 12 - Reserved : 0
- bit 11 - MERR (Message Error Int.) : 0 (Not Occurred)
- bit 10 - SUBAD (RT Only) : 0 (Not Occurred)
- bit 09 - BDRCV (RT Only) : 0 (Not Occurred)
- bit 08 - IXEQ0 (RT Only) : 0 (Not Occurred)
- bit 07 - ILLCMD (RT Only) : 0 (Not Occurred)
- bit 06 - Reserved : 0
- bit 05 - EOL (End of List Interrupt) : 1 (Occurred)
- bit 04 - ILLCMD (Illogical Com. Int.) : 0 (Not Occurred)
- bit 03 - ILL0P (Illogical Opc. Int.) : 0 (Not Occurred)
- bit 02 - RTF (Retry Fail. Interrupt) : 0 (Not Occurred)
- bit 01 - CBA (Command Blk Acc. Int.) : 0 (Not Occurred)
- bit 00 - MBC (BM Only) : 0 (Not Occurred)
```

```
Result Report - Command Block #0
1) Control Word : 0x46FE
- OPCODE : 0x4 -> Execute Block - Continue (1553B Op.)
- 1 Retry
- Channel A
- No RT-to-RT
- Condition 7 (No Response) is Enabled
- Condition 6 (Message Error) is Enabled
- Condition 5 (Busy Bit) is Enabled
- Condition 4 (Terminal Flag) is Enabled
- Condition 3 (Subsystem Fail) is Enabled
- Condition 2 (Instrumentation) is Enabled
- Condition 1 (Service Request) is Enabled
- BAME : 0 (No Error)
2) Command Word 1 : 0x0820
- (01-R-01-32)
3) Command Word 2 : 0x0000
4) Data Pointer : 0x0640 (0xD2000C80)
- BC -> RT was sent :
  0x0000 0x0001 0x0002 0x0003 0x0004 0x0005 0x0006 0x0007
  0x0008 0x0009 0x000A 0x000B 0x000C 0x000D 0x000E 0x000F
  0x0010 0x0011 0x0012 0x0013 0x0014 0x0015 0x0016 0x0017
  0x0018 0x0019 0x001A 0x001B 0x001C 0x001D 0x001E 0x001F
5) Received Status Word 1 : 0x0800
- bit 11-15 (RT ADDRESS) : 1
- bit 10 (MESSAGE ERROR) : 0 (Not Set)
- bit 09 (INSTRUMENTATION) : 0 (Not Set)
- bit 08 (SERVICE REQUEST) : 0 (Not Set)
- bit 05-07 (RESERVED) : 0
- bit 04 (BROADCAST CMD RCVED) : 0 (Not Set)
- bit 03 (BUSY) : 0 (Not Set)
- bit 02 (SUBSYS FLAG) : 0 (Not Set)
- bit 01 (DYNAMIC BUS CTRL) : 0 (Not Set)
- bit 00 (TERMINAL FLAG) : 0 (Not Set)
6) Received Status Word 2 : 0x0000
7) Branch Address : 0x0000 (0xD2000000)
8) Timer Value : 0x0000
```

```
Result Report - Command Block #1
1) Control Word : 0x46FE
- OPCODE : 0x4 -> Execute Block - Continue (1553B Op.)
- 1 Retry
- Channel A
- No RT-to-RT
- Condition 7 (No Response) is Enabled
- Condition 6 (Message Error) is Enabled
- Condition 5 (Busy Bit) is Enabled
- Condition 4 (Terminal Flag) is Enabled
- Condition 3 (Subsystem Fail) is Enabled
- Condition 2 (Instrumentation) is Enabled
- Condition 1 (Service Request) is Enabled
- BAME : 0 (No Error)
2) Command Word 1 : 0x0C40
- (01-T-02-32)
3) Command Word 2 : 0x0000
4) Data Pointer : 0x0660 (0xD2000CC0)
- BC -> RT was received :
  0x0040 0x0041 0x0042 0x0043 0x0044 0x0045 0x0046 0x0047
  0x0048 0x0049 0x004A 0x004B 0x004C 0x004D 0x004E 0x004F
  0x0050 0x0051 0x0052 0x0053 0x0054 0x0055 0x0056 0x0057
  0x0058 0x0059 0x005A 0x005B 0x005C 0x005D 0x005E 0x005F
5) Received Status Word 1 : 0x0800
- bit 11-15 (RT ADDRESS) : 1
- bit 10 (MESSAGE ERROR) : 0 (Not Set)
- bit 09 (INSTRUMENTATION) : 0 (Not Set)
- bit 08 (SERVICE REQUEST) : 0 (Not Set)
- bit 05-07 (RESERVED) : 0
- bit 04 (BROADCAST CMD RCVED) : 0 (Not Set)
- bit 03 (BUSY) : 0 (Not Set)
- bit 02 (SUBSYS FLAG) : 0 (Not Set)
- bit 01 (DYNAMIC BUS CTRL) : 0 (Not Set)
- bit 00 (TERMINAL FLAG) : 0 (Not Set)
6) Received Status Word 2 : 0x0000
7) Branch Address : 0x0000 (0xD2000000)
8) Timer Value : 0x0000
```

```
Result Report - Command Block #2
1) Control Word : 0x0000
- OPCODE : 0x0 -> EOL
- 4 Retry
- Channel B
- No RT-to-RT
- Condition 7 (No Response) is Disabled
- Condition 6 (Message Error) is Disabled
- Condition 5 (Busy Bit) is Disabled
- Condition 4 (Terminal Flag) is Disabled
- Condition 3 (Subsystem Fail) is Disabled
- Condition 2 (Instrumentation) is Disabled
- Condition 1 (Service Request) is Disabled
- BAME : 0 (No Error)
2) Command Word 1 : 0x0000
3) Command Word 2 : 0x0000
```

```
Status: 0000
Data: 0040 0041 0042 0043 0044 0045 0046 0047
      0048 0049 004A 004B 004C 004D 004E 004F
      0050 0051 0052 0053 0054 0055 0056 0057
      0058 0059 005A 005B 005C 005D 005E 005F
```

```
D2000000)
8) Timer Value : 0x0000
```

```
Register #4 (Pending Interrupt, 0x00000020) : 0x00000000
- bit 15 - DMAF (DMA Fail Interrupt) : 0 (Not Occurred)
- bit 14 - WRAPF (Wrap Fail Interrupt) : 0 (Not Occurred)
- bit 13 - TAPF (RT Only) : 0 (Not Occurred)
- bit 12 - Reserved : 0
- bit 11 - MERR (Message Error Int.) : 0 (Not Occurred)
- bit 10 - SUBAD (RT Only) : 0 (Not Occurred)
- bit 09 - BDRCV (RT Only) : 0 (Not Occurred)
- bit 08 - IXEQ0 (RT Only) : 0 (Not Occurred)
- bit 07 - ILLCMD (RT Only) : 0 (Not Occurred)
- bit 06 - Reserved : 0
- bit 05 - EOL (End of List Interrupt) : 1 (Occurred)
- bit 04 - ILLCMD (Illogical Com. Int.) : 0 (Not Occurred)
- bit 03 - ILL0P (Illogical Opc. Int.) : 0 (Not Occurred)
- bit 02 - RTF (Retry Fail. Interrupt) : 0 (Not Occurred)
- bit 01 - CBA (Command Blk Acc. Int.) : 0 (Not Occurred)
- bit 00 - MBC (BM Only) : 0 (Not Occurred)
```

```
Result Report - Command Block #0
1) Control Word : 0x46FE
- OPCODE : 0x4 -> Execute Block - Continue (1553B Op.)
- 1 Retry
- Channel A
- No RT-to-RT
- Condition 7 (No Response) is Enabled
- Condition 6 (Message Error) is Enabled
- Condition 5 (Busy Bit) is Enabled
- Condition 4 (Terminal Flag) is Enabled
- Condition 3 (Subsystem Fail) is Enabled
- Condition 2 (Instrumentation) is Enabled
- Condition 1 (Service Request) is Enabled
- BAME : 0 (No Error)
2) Command Word 1 : 0x0820
- (01-R-01-32)
3) Command Word 2 : 0x0000
4) Data Pointer : 0x0640 (0xD2000C80)
- BC -> RT was sent :
  0x0000 0x0001 0x0002 0x0003 0x0004 0x0005 0x0006 0x0007
  0x0008 0x0009 0x000A 0x000B 0x000C 0x000D 0x000E 0x000F
  0x0010 0x0011 0x0012 0x0013 0x0014 0x0015 0x0016 0x0017
  0x0018 0x0019 0x001A 0x001B 0x001C 0x001D 0x001E 0x001F
5) Received Status Word 1 : 0x0800
- bit 11-15 (RT ADDRESS) : 1
- bit 10 (MESSAGE ERROR) : 0 (Not Set)
- bit 09 (INSTRUMENTATION) : 0 (Not Set)
- bit 08 (SERVICE REQUEST) : 0 (Not Set)
- bit 05-07 (RESERVED) : 0
- bit 04 (BROADCAST CMD RCVED) : 0 (Not Set)
- bit 03 (BUSY) : 0 (Not Set)
- bit 02 (SUBSYS FLAG) : 0 (Not Set)
- bit 01 (DYNAMIC BUS CTRL) : 0 (Not Set)
- bit 00 (TERMINAL FLAG) : 0 (Not Set)
6) Received Status Word 2 : 0x0000
7) Branch Address : 0x0000 (0xD2000000)
8) Timer Value : 0x0000
```

```
Result Report - Command Block #1
1) Control Word : 0x46FE
- OPCODE : 0x4 -> Execute Block - Continue (1553B Op.)
- 1 Retry
- Channel A
- No RT-to-RT
- Condition 7 (No Response) is Enabled
- Condition 6 (Message Error) is Enabled
- Condition 5 (Busy Bit) is Enabled
- Condition 4 (Terminal Flag) is Enabled
- Condition 3 (Subsystem Fail) is Enabled
- Condition 2 (Instrumentation) is Enabled
- Condition 1 (Service Request) is Enabled
- BAME : 0 (No Error)
2) Command Word 1 : 0x0C40
- (01-T-02-32)
3) Command Word 2 : 0x0000
4) Data Pointer : 0x0660 (0xD2000CC0)
- BC -> RT was received :
  0x0040 0x0041 0x0042 0x0043 0x0044 0x0045 0x0046 0x0047
  0x0048 0x0049 0x004A 0x004B 0x004C 0x004D 0x004E 0x004F
  0x0050 0x0051 0x0052 0x0053 0x0054 0x0055 0x0056 0x0057
  0x0058 0x0059 0x005A 0x005B 0x005C 0x005D 0x005E 0x005F
5) Received Status Word 1 : 0x0800
- bit 11-15 (RT ADDRESS) : 1
- bit 10 (MESSAGE ERROR) : 0 (Not Set)
- bit 09 (INSTRUMENTATION) : 0 (Not Set)
- bit 08 (SERVICE REQUEST) : 0 (Not Set)
- bit 05-07 (RESERVED) : 0
- bit 04 (BROADCAST CMD RCVED) : 0 (Not Set)
- bit 03 (BUSY) : 0 (Not Set)
- bit 02 (SUBSYS FLAG) : 0 (Not Set)
- bit 01 (DYNAMIC BUS CTRL) : 0 (Not Set)
- bit 00 (TERMINAL FLAG) : 0 (Not Set)
6) Received Status Word 2 : 0x0000
7) Branch Address : 0x0000 (0xD2000000)
8) Timer Value : 0x0000
```

```
Result Report - Command Block #2
1) Control Word : 0x0000
- OPCODE : 0x0 -> EOL
- 4 Retry
- Channel B
- No RT-to-RT
- Condition 7 (No Response) is Disabled
- Condition 6 (Message Error) is Disabled
- Condition 5 (Busy Bit) is Disabled
- Condition 4 (Terminal Flag) is Disabled
- Condition 3 (Subsystem Fail) is Disabled
- Condition 2 (Instrumentation) is Disabled
- Condition 1 (Service Request) is Disabled
- BAME : 0 (No Error)
2) Command Word 1 : 0x0000
3) Command Word 2 : 0x0000
```

<pre> 4) Data Pointer : 0x0680 (0xD2000000) 5) Received Status Word 1 : 0x0000 6) Received Status Word 2 : 0x0000 7) Branch Address : 0x0000 (0xD2000000) 8) Timer Value : 0x0000 Test done! CORE#0 halts because of Error Mode [0.695869sec/462975151, 0, 0, 0cycles] Performance statistics for CORE#0 Cycles : 462975151 Instructions : 162769019 CORE#0 performance (250MHz) : 87.89 MIPS Simulated time : 1.85 s Performance of laysim-gr740 Real-time performance : 266.13 % laysim-gr740 performance : 233.91 MIPS Wall clock : 0.70 s [layright@CentOS user-io]\$ </pre>	<pre> 4) Data Pointer : 0x0680 (0xD2000000) 5) Received Status Word 1 : 0x0000 6) Received Status Word 2 : 0x0000 7) Branch Address : 0x0000 (0xD2000000) 8) Timer Value : 0x0000 Test done! CORE#0 halts because of Error Mode [1.716sec/462975151, 0, 0, 0cycles] Performance statistics for CORE#0 Cycles : 462975151 Instructions : 162769019 CORE#0 performance (250MHz) : 87.89 MIPS Simulated time : 1.85 s Performance of laysim-gr740 Real-time performance : 107.92 % laysim-gr740 performance : 94.85 MIPS Wall clock : 1.72 s PS C:\laysim-gr740\user-io> </pre>
---	--

4.2.2 DMA Operation on I/O Area

Figure 4-10 shows the result of user-io-dma with the loadable I/O module. The loadable I/O module can be loaded with '-mod ./Loadable/loadable-I0/laysim_user_io.so' on start-up. This example is to test the simple DMA read/write operation in I/O area. It copies the SDRAM data to I/O shared memory using DMA and it raises the GPIO#0 port 4 interrupt of the GR740.

Figure 4-10. user-io-dma with loadable I/O module

laysim-gr740-cli in Linux	laysim-gr740-cli in Windows
<pre> [layright@CentOS user-io]\$ laysim-gr740-cli -r -core0 user-io-dma.elf -mod ./Loadable/loadable-I0/laysim_user_io.so laysim for GR740 Quad Core Processor CLI v3.0 by layright Loaded user loadable module ./Loadable/loadable-I0/laysim_user_io.so CORE#0 RAM file is loaded - user-io-dma.elf - .text, addr: 0x00000000, size 57360 bytes - .rodata, addr: 0x0000E010, size 1664 bytes - .data, addr: 0x0000E690, size 1568 bytes - .bss, addr: 0x0000ECB0, size 456 bytes (not loaded, just info.) Only GR740 CORE#0 RAM file is loaded, starts from RAM Set CORE#0 %sp/%fp : 0x07FFFFFF0/0x080000000 D2000000 : DEAD0000 D2000004 : DEAD0001 D2000008 : DEAD0002 D200000C : DEAD0003 D2000010 : DEAD0004 D2000014 : DEAD0005 D2000018 : DEAD0006 D200001C : DEAD0007 D2000020 : DEAD0008 D2000024 : DEAD0009 CORE#0 halts because of Error Mode [14155524cycles/0.094112sec] 09 Test done! Performance statistics for CORE#0 Cycles : 14155524 Instructions : 4727793 Overall CPI : 2.99 CORE#0 performance (250MHz) : 83.50 MOPS (83.50 MIPS, 0.00 MFLOPS) Simulated time : 56.62 ms Processor utilization : 100.00 % Performance of laysim-gr740 Real-time performance : 60.16 % laysim-gr740 performance : 50.24 MIPS Wall clock : 0.09 s [layright@CentOS user-io]\$ </pre>	<pre> PS C:\laysim-gr740\user-io> laysim-gr740-cli -r -core0 user-io-dma.elf -mod ./Loadable/loadable-I0\laysim_user_io-cli.dll laysim for GR740 Quad Core Processor CLI v3.0 by layright Loaded user loadable module ..\Loadable\loadable-I0\laysim_user_io-cli.dll CORE#0 RAM file is loaded - user-io-dma.elf - .text, addr: 0x00000000, size 57360 bytes - .rodata, addr: 0x0000E010, size 1664 bytes - .data, addr: 0x0000E690, size 1568 bytes - .bss, addr: 0x0000ECB0, size 456 bytes (not loaded, just info.) Only GR740 CORE#0 RAM file is loaded, starts from RAM Set CORE#0 %sp/%fp : 0x07FFFFFF0/0x080000000 D2000000 : DEAD0000 D2000004 : DEAD0001 D2000008 : DEAD0002 D200000C : DEAD0003 D2000010 : DEAD0004 D2000014 : DEAD0005 D2000018 : DEAD0006 D200001C : DEAD0007 D2000020 : DEAD0008 D2000024 : DEAD0009 CORE#0 halts because of Error Mode [14155524cycles/0.1092sec] 09 Test done! Performance statistics for CORE#0 Cycles : 14155524 Instructions : 4727793 Overall CPI : 2.99 CORE#0 performance (250MHz) : 83.50 MOPS (83.50 MIPS, 0.00 MFLOPS) Simulated time : 56.62 ms Processor utilization : 100.00 % Performance of laysim-gr740 Real-time performance : 51.85 % laysim-gr740 performance : 43.29 MIPS Wall clock : 0.11 s PS C:\laysim-gr740\user-io> </pre>

4.2.3 Board Reset on I/O Area

Figure 4-11 shows the result of smp08-reset.mram with the loadable I/O module. The loadable I/O module can be loaded with '-mod ./Loadable/loadable-I0/laysim_user_io.so' on start-up. This example is to test the board reset from the board reset register in I/O area.

Figure 4-11. smp08-reset with loadable I/O module

laysim-gr740-dbt-cli in Linux	laysim-gr740-cli in Windows
<pre> [layright@CentOS user-io]\$ laysim-gr740-dbt-cli -r -rom smp08-reset.mram -mod ./Loadable/loadable-I0/laysim_user_io.so laysim for GR740 Quad Core Processor (DBT) CLI v3.0 by layright Loaded user loadable module ./Loadable/loadable-I0/laysim_user_io.so Multicore control : Time quantum (500) ROM file is loaded - smp08-reset.mram - binary file size : 129200 bytes GR740 ROM file is loaded, starts from ROM </pre>	<pre> PS C:\laysim-gr740\user-io> laysim-gr740-dbt-cli -r -rom smp08-reset.mram -mod ..\Loadable\loadable-I0\laysim_user_io-dbt-cli.dll laysim for GR740 Quad Core Processor (DBT) CLI v3.0 by layright Loaded user loadable module ..\Loadable\loadable-I0\laysim_user_io-dbt-cli.dll Multicore control : Time quantum (500) ROM file is loaded - smp08-reset.mram - binary file size : 129200 bytes GR740 ROM file is loaded, starts from ROM </pre>

[illegible]


```
CPU 3 running task TA01 - rtems_clock_gettod 09:00:10 12/31/1988
CPU 3 running task TA02 - rtems_clock_gettod 09:00:10 12/31/1988
CPU 3 running task TA01 - rtems_clock_gettod 09:00:15 12/31/1988
CPU 3 running task TA03 - rtems_clock_gettod 09:00:15 12/31/1988
...
...
```

```
CPU 1 running task TA01 - rtems_clock_gettod 09:00:10 12/31/1988
CPU 1 running task TA02 - rtems_clock_gettod 09:00:10 12/31/1988
CPU 1 running task TA01 - rtems_clock_gettod 09:00:15 12/31/1988
CPU 1 running task TA03 - rtems_clock_gettod 09:00:15 12/31/1988
...
...
```

4.3 Loadable SpW Module

haysim-GR740 supports the loadable SpW module by using '-mod MODULE' on start-up. Figure 4-12 shows the result of rtems-rmap-path-addressing with the loadable SpW module. The loadable SpW module can be loaded with '-mod ./Loadable/loadable-SpW-Nodes/haysim_user_spw_node.so' on start-up.

Figure 4–12. SpW Node Simulation using loadable SpW module

haysim-gr740-dbt-cli in Linux	haysim-gr740-cli in Windows
<pre>[layright@CentOS Samples]\$ haysim-gr740-dbt-cli -r -core0 rtems-rmap-logical-addressing.elf -mod ./Loadable/loadable-SpW-Nodes/haysim_user_spw_node.so haysim for GR740 Quad Core Processor (DBT) CLI v3.0 by layright Loaded user loadable module ./Loadable/loadable-SpW-Nodes/haysim_user_spw_node.so Multicore control : Time quantum (500) CORE#0 RAM file is loaded - rtems-rmap-logical-addressing.elf - .text, addr: 0x00000000, size 230576 bytes - .rtemsroset, addr: 0x000384B0, size 176 bytes - .rtemsstack, addr: 0x00038580, size 8192 bytes (not loaded, just info.) - .data, addr: 0x0003A580, size 8272 bytes - .bss, addr: 0x0003C600, size 171024 bytes (not loaded, just info.) - Detect RTEMS v5.1(SMP) with thread information Only GR740 CORE#0 RAM file is loaded, starts from RAM Set CORE#0 %sp/%fp : 0x07FFFFF0/0x08000000 #### To change configuration of this example, you can set a breakpoint #### on label <rmap_conf_label> and modify the following global variables: ## remote_base_address: 0x40000000 /* Base address on Remote. This is the address to which the data is copied. */ ## remote_dst_address: 0x42 /* SpW Destination address. */ ## remote_dst_key: 0x00 /* SpW Destination Key */ ## source_src_address: 0x20 /* SpW Source address */ ## rmap_dev_idx: 0 /* This parameter chooses which initiator GRSPW2 core is used */ ## route_entry { nr; /* Number of addresses in dstadr array */, dstadr[16]; /* Path Addresses */} ## routetab[ROUTE_TO]={1,{3,0,0, ...}} ## routetab[ROUTE_FROM]={1,{9,0,0, ...}} ## path_addressing: 0 /* Use path addressing (!=0) or not (0) */ ## rmap_pkt_size: 992 ## rmap_data_bytes: 4096 /* Data bytes to send over RMAP */ #### RTEMS RMAP example Setting up SpaceWire router Configuring Router Hardware Configuration of SpaceWire Router: Number of SpW ports: 8 Number of AMBA ports: 4 Number of FIFO ports: 0 Timers available: YES Plug and Play available: YES MAJOR Version: 1 MINOR Version: 2 PATCH Version: 0 Current Instance ID: 19 ROUTER CFG/STS: 0x4100c013 PORT[00]: TYPE=CFG CTRL=0x010000210 STATUS=0x010000210 LINKSTATE=N/A PORT[01]: TYPE=SpW CTRL=0x2700022c STATUS=0x0000a000 LINKSTATE=Ready PORT[02]: TYPE=SpW CTRL=0x2700022c STATUS=0x0000a000 LINKSTATE=Ready PORT[03]: TYPE=SpW CTRL=0x2700022c STATUS=0x0000a000 LINKSTATE=Ready PORT[04]: TYPE=SpW CTRL=0x2700022c STATUS=0x0000a000 LINKSTATE=Ready PORT[05]: TYPE=SpW CTRL=0x2700022c STATUS=0x0000a000 LINKSTATE=Ready PORT[06]: TYPE=SpW CTRL=0x2700022c STATUS=0x0000a000 LINKSTATE=Ready PORT[07]: TYPE=SpW CTRL=0x2700022c STATUS=0x0000a000 LINKSTATE=Ready PORT[08]: TYPE=SpW CTRL=0x2700022c STATUS=0x0000a000 LINKSTATE=Ready PORT[09]: TYPE=AMBA CTRL=0x00000228 STATUS=0x40000000 LINKSTATE=N/A PORT[10]: TYPE=AMBA CTRL=0x00000228 STATUS=0x40000000 LINKSTATE=N/A PORT[11]: TYPE=AMBA CTRL=0x00000228 STATUS=0x40000000 LINKSTATE=N/A PORT[12]: TYPE=AMBA CTRL=0x00000228 STATUS=0x40000000 LINKSTATE=N/A Activating all Links/Ports PORT[01]: TYPE=SpW CTRL=0x0300002e STATUS=0x0000a000 LINK-STATE=Ready PORT[02]: TYPE=SpW CTRL=0x0300002e STATUS=0x0000a000 LINK-STATE=Ready PORT[03]: TYPE=SpW CTRL=0x0300002e STATUS=0x0000a000 LINK-STATE=Run PORT[04]: TYPE=SpW CTRL=0x0300002e STATUS=0x0000a000 LINK-STATE=Ready PORT[05]: TYPE=SpW CTRL=0x0300002e STATUS=0x0000a000 LINK-STATE=Ready PORT[06]: TYPE=SpW CTRL=0x0300002e STATUS=0x0000a000 LINK-STATE=Ready PORT[07]: TYPE=SpW CTRL=0x0300002e STATUS=0x0000a000 LINK-STATE=Ready PORT[08]: TYPE=SpW CTRL=0x0300002e STATUS=0x0000a000 LINK-STATE=Ready Configuring Router Failed to open SpW Router Initializing SpaceWire device 0 NOTE: running on SPW-ROUTER DMA SpaceWire link (no link-state available) Initializing SpaceWire device 1 NOTE: running on SPW-ROUTER DMA SpaceWire link (no link-state available) Initializing SpaceWire device 2 NOTE: running on SPW-ROUTER DMA SpaceWire link (no link-state available) Initializing SpaceWire device 3 NOTE: running on SPW-ROUTER DMA SpaceWire link (no link-state available) Test built for at least 4 AMBA ports. Note that the SpW-links will not be used. There are 4 GRSPW cores in the system. System clock: 10000 us / tick Starting GRSPW0: DMA Started Successfully</pre>	<pre>PS C:\haysim-gr740\RCC-1.3.1\Samples> haysim-gr740-dbt-cli -r -core0 rtems-rmap-logical-addressing.elf -mod ./Loadable/loadable-SpW-Nodes/haysim_user_spw_node-dbt-cli.dll haysim for GR740 Quad Core Processor (DBT) CLI v3.0 by layright Loaded user loadable module ./Loadable/loadable-SpW-Nodes/haysim_user_spw_node-dbt-cli.dll Multicore control : Time quantum (500) CORE#0 RAM file is loaded - rtems-rmap-logical-addressing.elf - .text, addr: 0x00000000, size 230576 bytes - .rtemsroset, addr: 0x000384B0, size 176 bytes - .rtemsstack, addr: 0x00038580, size 8192 bytes (not loaded, just info.) - .data, addr: 0x0003A580, size 8272 bytes - .bss, addr: 0x0003C600, size 171024 bytes (not loaded, just info.) - Detect RTEMS v5.1(SMP) with thread information Only GR740 CORE#0 RAM file is loaded, starts from RAM Set CORE#0 %sp/%fp : 0x07FFFFF0/0x08000000 #### To change configuration of this example, you can set a breakpoint #### on label <rmap_conf_label> and modify the following global variables: ## remote_base_address: 0x40000000 /* Base address on Remote. This is the address to which the data is copied. */ ## remote_dst_address: 0x42 /* SpW Destination address. */ ## remote_dst_key: 0x00 /* SpW Destination Key */ ## source_src_address: 0x20 /* SpW Source address */ ## rmap_dev_idx: 0 /* This parameter chooses which initiator GRSPW2 core is used */ ## route_entry { nr; /* Number of addresses in dstadr array */, dstadr[16]; /* Path Addresses */} ## routetab[ROUTE_TO]={1,{3,0,0, ...}} ## routetab[ROUTE_FROM]={1,{9,0,0, ...}} ## path_addressing: 0 /* Use path addressing (!=0) or not (0) */ ## rmap_pkt_size: 992 ## rmap_data_bytes: 4096 /* Data bytes to send over RMAP */ #### RTEMS RMAP example Setting up SpaceWire router Configuring Router Hardware Configuration of SpaceWire Router: Number of SpW ports: 8 Number of AMBA ports: 4 Number of FIFO ports: 0 Timers available: YES Plug and Play available: YES MAJOR Version: 1 MINOR Version: 2 PATCH Version: 0 Current Instance ID: 19 ROUTER CFG/STS: 0x4100c013 PORT[00]: TYPE=CFG CTRL=0x010000210 STATUS=0x010000210 LINKSTATE=N/A PORT[01]: TYPE=SpW CTRL=0x2700022c STATUS=0x0000a000 LINKSTATE=Ready PORT[02]: TYPE=SpW CTRL=0x2700022c STATUS=0x0000a000 LINKSTATE=Ready PORT[03]: TYPE=SpW CTRL=0x2700022c STATUS=0x0000a000 LINKSTATE=Ready PORT[04]: TYPE=SpW CTRL=0x2700022c STATUS=0x0000a000 LINKSTATE=Ready PORT[05]: TYPE=SpW CTRL=0x2700022c STATUS=0x0000a000 LINKSTATE=Ready PORT[06]: TYPE=SpW CTRL=0x2700022c STATUS=0x0000a000 LINKSTATE=Ready PORT[07]: TYPE=SpW CTRL=0x2700022c STATUS=0x0000a000 LINKSTATE=Ready PORT[08]: TYPE=SpW CTRL=0x2700022c STATUS=0x0000a000 LINKSTATE=Ready PORT[09]: TYPE=AMBA CTRL=0x00000228 STATUS=0x40000000 LINKSTATE=N/A PORT[10]: TYPE=AMBA CTRL=0x00000228 STATUS=0x40000000 LINKSTATE=N/A PORT[11]: TYPE=AMBA CTRL=0x00000228 STATUS=0x40000000 LINKSTATE=N/A PORT[12]: TYPE=AMBA CTRL=0x00000228 STATUS=0x40000000 LINKSTATE=N/A Activating all Links/Ports PORT[01]: TYPE=SpW CTRL=0x0300002e STATUS=0x0000a000 LINK-STATE=Ready PORT[02]: TYPE=SpW CTRL=0x0300002e STATUS=0x0000a000 LINK-STATE=Ready PORT[03]: TYPE=SpW CTRL=0x0300002e STATUS=0x0000a000 LINK-STATE=Run PORT[04]: TYPE=SpW CTRL=0x0300002e STATUS=0x0000a000 LINK-STATE=Ready PORT[05]: TYPE=SpW CTRL=0x0300002e STATUS=0x0000a000 LINK-STATE=Ready PORT[06]: TYPE=SpW CTRL=0x0300002e STATUS=0x0000a000 LINK-STATE=Ready PORT[07]: TYPE=SpW CTRL=0x0300002e STATUS=0x0000a000 LINK-STATE=Ready PORT[08]: TYPE=SpW CTRL=0x0300002e STATUS=0x0000a000 LINK-STATE=Ready Configuring Router Failed to open SpW Router Initializing SpaceWire device 0 NOTE: running on SPW-ROUTER DMA SpaceWire link (no link-state available) Initializing SpaceWire device 1 NOTE: running on SPW-ROUTER DMA SpaceWire link (no link-state available) Initializing SpaceWire device 2 NOTE: running on SPW-ROUTER DMA SpaceWire link (no link-state available) Initializing SpaceWire device 3 NOTE: running on SPW-ROUTER DMA SpaceWire link (no link-state available) Test built for at least 4 AMBA ports. Note that the SpW-links will not be used. There are 4 GRSPW cores in the system. System clock: 10000 us / tick Starting GRSPW0: DMA Started Successfully</pre>

NOTE: running on SPW-ROUTER DMA SpaceWire link (no link-state available) RMAP connection is working Initializing data array Starting transmission type 1: one packet at a time RMAP write cycle completed in 0.000409 seconds RMAP read cycle completed in 0.000416 seconds Data bytes sent: 4096, Packet size: 992, Packet count: 5 Wrote 4096 bytes to target in 0.000409 seconds. WRITE RATE: 9782.232179 Kib/s Read 4096 bytes from target in 0.000416 seconds. READ RATE: 9610.276912 Kib/s Wrote 5 packets to target in 0.000409 seconds. WRITE RATE: 12227.790224 pkt/s Read 5 packets from target in 0.000416 seconds. READ RATE: 12012.846140 pkt/s EXAMPLE 1 SUCCESSFULLY COMPLETED. Starting transmission type 2: all packets at a time RMAP write cycle completed in 0.000373 seconds RMAP read cycle completed in 0.000127 seconds Data bytes sent: 4096, Packet size: 992, Packet count: 5 Wrote 4096 bytes to target in 0.000373 seconds. WRITE RATE: 10735.271186 Kib/s Read 4096 bytes from target in 0.000127 seconds. READ RATE: 31536.120301 Kib/s Wrote 5 packets to target in 0.000373 seconds. WRITE RATE: 13419.088982 pkt/s Read 5 packets from target in 0.000127 seconds. READ RATE: 39420.150376 pkt/s EXAMPLE 2 SUCCESSFULLY COMPLETED. CORE#0 halts because of Error Mode [0.297153sec/333129798, 0, 0, 0cycles] Performance statistics for CORE#0 Cycles : 333129798 Instructions : 211294714 CORE#0 performance (250MHz) : 158.57 MIPS Simulated time : 1.33 s Performance of laysim-gr740 Real-time performance : 448.43 % laysim-gr740 performance : 711.06 MIPS Wall clock : 0.30 s [layright@CentOS Samples]\$	NOTE: running on SPW-ROUTER DMA SpaceWire link (no link-state available) RMAP connection is working Initializing data array Starting transmission type 1: one packet at a time RMAP write cycle completed in 0.000409 seconds RMAP read cycle completed in 0.000416 seconds Data bytes sent: 4096, Packet size: 992, Packet count: 5 Wrote 4096 bytes to target in 0.000409 seconds. WRITE RATE: 9782.232179 Kib/s Read 4096 bytes from target in 0.000416 seconds. READ RATE: 9610.276912 Kib/s Wrote 5 packets to target in 0.000409 seconds. WRITE RATE: 12227.790224 pkt/s Read 5 packets from target in 0.000416 seconds. READ RATE: 12012.846140 pkt/s EXAMPLE 1 SUCCESSFULLY COMPLETED. Starting transmission type 2: all packets at a time RMAP write cycle completed in 0.000373 seconds RMAP read cycle completed in 0.000127 seconds Data bytes sent: 4096, Packet size: 992, Packet count: 5 Wrote 4096 bytes to target in 0.000373 seconds. WRITE RATE: 10735.271186 Kib/s Read 4096 bytes from target in 0.000127 seconds. READ RATE: 31536.120301 Kib/s Wrote 5 packets to target in 0.000373 seconds. WRITE RATE: 13419.088982 pkt/s Read 5 packets from target in 0.000127 seconds. READ RATE: 39420.150376 pkt/s EXAMPLE 2 SUCCESSFULLY COMPLETED. CORE#0 halts because of Error Mode [0.904802sec/333129798, 0, 0, 0cycles] Performance statistics for CORE#0 Cycles : 333129798 Instructions : 211294714 CORE#0 performance (250MHz) : 158.57 MIPS Simulated time : 1.33 s Performance of laysim-gr740 Real-time performance : 147.27 % laysim-gr740 performance : 233.53 MIPS Wall clock : 0.90 s PS C:\laysim-gr740\RCC-1.3.1\Samples>
--	---

4.4 Loadable 1553B RT Module

laysim-GR740 supports the loadable 1553B RT module by using '-mod MODULE' on start-up. Figure 4-13 shows the result of rtems-gr1553bcm with the loadable 1553B RT module. The loadable 1553B RT module can be loaded with '-mod ./Loadable/loadable-1553RT/laysim_user_1553rt.so' on start-up. This example is to test the GR1553B with RT simulation.

Figure 4–13. 1553B RT Simulation using loadable 1553B RT module

laysim-gr740-dbt-cli in Linux	laysim-gr740-cli in Windows
<pre>[layright@CentOS Samples]\$ laysim-gr740-dbt-cli -r -core0 rtems-gr1553bcm_sw_trig.elf -mod ../Loadable/loadable-1553RT/laysim_user_1553rt.so laysim for GR740 Quad Core Processor (DBT) CLI v3.0 by layright Loaded user loadable module ../Loadable/loadable-1553RT/laysim_user_1553rt.so Multicore control : Time quantum (500) CORE#0 RAM file is loaded - rtems-gr1553bcm_sw_trig.elf - .text, addr: 0x00000000, size 432448 bytes - .rtmsroset, addr: 0x00069940, size 448 bytes - .rtmsstack, addr: 0x00069800/0x0006BB00, size 8192 bytes (not loaded, just info.) - .data, addr: 0x0006BB00, size 11328 bytes - .bss, addr: 0x0006E740, size 62496 bytes (not loaded, just info.) - Detect RTEMS v5.1(SMP) with thread information Only GR740 CORE#0 RAM file is loaded, starts from RAM Set CORE#0 %sp/%fp : 0x07FFFFF0/0x08000000 --- BUS TOPOLOGY --- -> DEV 0xc6840 GAISLER_LEON4 -> DEV 0xc68a8 GAISLER_LEON4 -> DEV 0xc6910 GAISLER_LEON4 -> DEV 0xc6978 GAISLER_LEON4 -> DEV 0xc69e0 GAISLER_GRIOMMU -> DEV 0xc6a48 GAISLER_AHB2AHB -> DEV 0xc6ab0 GAISLER_L2CACHE -> DEV 0xc6b18 GAISLER_APBMS -> DEV 0xc6b80 GAISLER_APBMS -> DEV 0xc6be8 GAISLER_APBMS -> DEV 0xc6c50 GAISLER_APBMS -> DEV 0xc6cb8 GAISLER_GPIO -> DEV 0xc6d20 GAISLER_FTMCTRL -> DEV 0xc6d88 GAISLER_IRQMP -> DEV 0xc6df0 GAISLER_GPTIMER -> DEV 0xc6e58 GAISLER_GPTIMER -> DEV 0xc6ec0 GAISLER_GPTIMER -> DEV 0xc6f28 GAISLER_GPTIMER -> DEV 0xc6f90 GAISLER_GPTIMER -> DEV 0xc6ff8 GAISLER_SPW2_DMA -> DEV 0xc7060 GAISLER_SPW2_DMA -> DEV 0xc70c8 GAISLER_SPW2_DMA -> DEV 0xc7130 GAISLER_SPW2_DMA -> DEV 0xc7198 GAISLER_ETHMAC -> DEV 0xc7208 GAISLER_ETHMAC -> DEV 0xc7268 GAISLER_GRCAN -> DEV 0xc72d0 GAISLER_GRCAN -> DEV 0xc7338 GAISLER_SPICTRL -> DEV 0xc73a0 GAISLER_CLKGATE</pre>	<pre>PS C:\laysim-gr740\RCC-1.3.1\Samples> laysim-gr740-dbt-cli -r -core0 rtems-gr1553bcm_sw_trig.elf -mod ../Loadable/loadable-1553RT/laysim_user_1553rt-dbt-cli.dll laysim for GR740 Quad Core Processor (DBT) CLI v3.0 by layright Loaded user loadable module ../Loadable/loadable-1553RT/laysim_user_1553rt-dbt-cli.dll Multicore control : Time quantum (500) CORE#0 RAM file is loaded - rtems-gr1553bcm_sw_trig.elf - .text, addr: 0x00000000, size 432448 bytes - .rtmsroset, addr: 0x00069940, size 448 bytes - .rtmsstack, addr: 0x00069800/0x0006BB00, size 8192 bytes (not loaded, just info.) - .data, addr: 0x0006BB00, size 11328 bytes - .bss, addr: 0x0006E740, size 62496 bytes (not loaded, just info.) - Detect RTEMS v5.1(SMP) with thread information Only GR740 CORE#0 RAM file is loaded, starts from RAM Set CORE#0 %sp/%fp : 0x07FFFFF0/0x08000000 --- BUS TOPOLOGY --- -> DEV 0xc6840 GAISLER_LEON4 -> DEV 0xc68a8 GAISLER_LEON4 -> DEV 0xc6910 GAISLER_LEON4 -> DEV 0xc6978 GAISLER_LEON4 -> DEV 0xc69e0 GAISLER_GRIOMMU -> DEV 0xc6a48 GAISLER_AHB2AHB -> DEV 0xc6ab0 GAISLER_L2CACHE -> DEV 0xc6b18 GAISLER_APBMS -> DEV 0xc6b80 GAISLER_APBMS -> DEV 0xc6be8 GAISLER_APBMS -> DEV 0xc6c50 GAISLER_APBMS -> DEV 0xc6cb8 GAISLER_GPIO -> DEV 0xc6d20 GAISLER_FTMCTRL -> DEV 0xc6d88 GAISLER_IRQMP -> DEV 0xc6df0 GAISLER_GPTIMER -> DEV 0xc6e58 GAISLER_GPTIMER -> DEV 0xc6ec0 GAISLER_GPTIMER -> DEV 0xc6f28 GAISLER_GPTIMER -> DEV 0xc6f90 GAISLER_GPTIMER -> DEV 0xc6ff8 GAISLER_SPW2_DMA -> DEV 0xc7060 GAISLER_SPW2_DMA -> DEV 0xc70c8 GAISLER_SPW2_DMA -> DEV 0xc7130 GAISLER_SPW2_DMA -> DEV 0xc7198 GAISLER_ETHMAC -> DEV 0xc7208 GAISLER_ETHMAC -> DEV 0xc7268 GAISLER_GRCAN -> DEV 0xc72d0 GAISLER_GRCAN -> DEV 0xc7338 GAISLER_SPICTRL -> DEV 0xc73a0 GAISLER_CLKGATE</pre>

```

-> DEV 0xc7408 GAISLER_GR1553B
-> DEV 0xc7470 GAISLER_AHBSTAT
-> DEV 0xc74d8 GAISLER_AHBSTAT
-> DEV 0xc7540 GAISLER_GPIO
-> DEV 0xc75a8 GAISLER_GPRG
-> DEV 0xc7610 GAISLER_GR740THS
-> DEV 0xc7678 GAISLER_GPREGBANK
-> DEV 0xc76e8 GAISLER_SPWTD
-> DEV 0xc7750 GAISLER_L4STAT
-> DEV 0xc77b8 GAISLER_L2CACHE
-> DEV 0xc7820 GAISLER_MEMSCRUB
-> DEV 0xc7888 GAISLER_GRIOMMU2
-> DEV 0xc78f0 GAISLER_DDRSDMUX
-> DEV 0xc7958 GAISLER_AHB2AHB
-> DEV 0xc79c0 GAISLER_FTMCTRL
-> DEV 0xc7a28 GAISLER_GRPIC2
-> DEV 0xc7a90 GAISLER_SPWRROUTER

```

Major Frame created successful

List created successful

LIST

```

major cnt: 11
RT[0] timeout: 18
RT[1] timeout: 18
RT[2] timeout: 18
RT[3] timeout: 18
RT[4] timeout: 18
RT[5] timeout: 18
RT[6] timeout: 18
RT[7] timeout: 18
RT[8] timeout: 18
RT[9] timeout: 18
RT[10] timeout: 18
RT[11] timeout: 18
RT[12] timeout: 18
RT[13] timeout: 18
RT[14] timeout: 18
RT[15] timeout: 18
RT[16] timeout: 18
RT[17] timeout: 18
RT[18] timeout: 18
RT[19] timeout: 18
RT[20] timeout: 18
RT[21] timeout: 18
RT[22] timeout: 18
RT[23] timeout: 18
RT[24] timeout: 18
RT[25] timeout: 18
RT[26] timeout: 18
RT[27] timeout: 18
RT[28] timeout: 18
RT[29] timeout: 18
RT[30] timeout: 18
RT[31] timeout: 30

```

MAJOR[0]

minor count: 3

MINOR[0]

```

bd: 0x000d27f0 (HW:0x000d27f0)
slot cnt: 32
timefree: 10000
timetotal: 50000

```

MINOR[1]

```

bd: 0x000d2a10 (HW:0x000d2a10)
slot cnt: 20
timefree: 32000
timetotal: 50000

```

MINOR[2]

```

bd: 0x000d2b70 (HW:0x000d2b70)
slot cnt: 10
timefree: 41000
timetotal: 50000

```

MAJOR[1]

minor count: 1

MINOR[0]

```

bd: 0x000d2c30 (HW:0x000d2c30)
slot cnt: 3
timefree: 60000
timetotal: 60000

```

MAJOR[2]

minor count: 9

MINOR[0]

```

bd: 0x000d2c80 (HW:0x000d2c80)
slot cnt: 4
timefree: 48
timetotal: 100

```

MINOR[1]

```

bd: 0x000d2ce0 (HW:0x000d2ce0)
slot cnt: 4
timefree: 6224
timetotal: 6225

```

MINOR[2]

```

bd: 0x000d2d40 (HW:0x000d2d40)
slot cnt: 4
timefree: 6224
timetotal: 6225

```

MINOR[3]

```

bd: 0x000d2da0 (HW:0x000d2da0)
slot cnt: 4
timefree: 6224
timetotal: 6225

```

MINOR[4]

```

bd: 0x000d2e00 (HW:0x000d2e00)
slot cnt: 4
timefree: 6224
timetotal: 6225

```

MINOR[5]

```

bd: 0x000d2e60 (HW:0x000d2e60)
slot cnt: 32
timefree: 12500
timetotal: 12500

```

MINOR[6]

```

bd: 0x000d3080 (HW:0x000d3080)
slot cnt: 32
timefree: 25000
timetotal: 25000

```

MINOR[7]

```

bd: 0x000d32a0 (HW:0x000d32a0)
slot cnt: 32
timefree: 12500

```

```

-> DEV 0xc7408 GAISLER_GR1553B
-> DEV 0xc7470 GAISLER_AHBSTAT
-> DEV 0xc74d8 GAISLER_AHBSTAT
-> DEV 0xc7540 GAISLER_GPIO
-> DEV 0xc75a8 GAISLER_GPRG
-> DEV 0xc7610 GAISLER_GR740THS
-> DEV 0xc7678 GAISLER_GPREGBANK
-> DEV 0xc76e8 GAISLER_SPWTD
-> DEV 0xc7750 GAISLER_L4STAT
-> DEV 0xc77b8 GAISLER_L2CACHE
-> DEV 0xc7820 GAISLER_MEMSCRUB
-> DEV 0xc7888 GAISLER_GRIOMMU2
-> DEV 0xc78f0 GAISLER_DDRSDMUX
-> DEV 0xc7958 GAISLER_AHB2AHB
-> DEV 0xc79c0 GAISLER_FTMCTRL
-> DEV 0xc7a28 GAISLER_GRPIC2
-> DEV 0xc7a90 GAISLER_SPWRROUTER

```

Major Frame created successful

List created successful

LIST

```

major cnt: 11
RT[0] timeout: 18
RT[1] timeout: 18
RT[2] timeout: 18
RT[3] timeout: 18
RT[4] timeout: 18
RT[5] timeout: 18
RT[6] timeout: 18
RT[7] timeout: 18
RT[8] timeout: 18
RT[9] timeout: 18
RT[10] timeout: 18
RT[11] timeout: 18
RT[12] timeout: 18
RT[13] timeout: 18
RT[14] timeout: 18
RT[15] timeout: 18
RT[16] timeout: 18
RT[17] timeout: 18
RT[18] timeout: 18
RT[19] timeout: 18
RT[20] timeout: 18
RT[21] timeout: 18
RT[22] timeout: 18
RT[23] timeout: 18
RT[24] timeout: 18
RT[25] timeout: 18
RT[26] timeout: 18
RT[27] timeout: 18
RT[28] timeout: 18
RT[29] timeout: 18
RT[30] timeout: 18
RT[31] timeout: 30

```

MAJOR[0]

minor count: 3

MINOR[0]

```

bd: 0x000d27f0 (HW:0x000d27f0)
slot cnt: 32
timefree: 10000
timetotal: 50000

```

MINOR[1]

```

bd: 0x000d2a10 (HW:0x000d2a10)
slot cnt: 20
timefree: 32000
timetotal: 50000

```

MINOR[2]

```

bd: 0x000d2b70 (HW:0x000d2b70)
slot cnt: 10
timefree: 41000
timetotal: 50000

```

MAJOR[1]

minor count: 1

MINOR[0]

```

bd: 0x000d2c30 (HW:0x000d2c30)
slot cnt: 3
timefree: 60000
timetotal: 60000

```

MAJOR[2]

minor count: 9

MINOR[0]

```

bd: 0x000d2c80 (HW:0x000d2c80)
slot cnt: 4
timefree: 48
timetotal: 100

```

MINOR[1]

```

bd: 0x000d2ce0 (HW:0x000d2ce0)
slot cnt: 4
timefree: 6224
timetotal: 6225

```

MINOR[2]

```

bd: 0x000d2d40 (HW:0x000d2d40)
slot cnt: 4
timefree: 6224
timetotal: 6225

```

MINOR[3]

```

bd: 0x000d2da0 (HW:0x000d2da0)
slot cnt: 4
timefree: 6224
timetotal: 6225

```

MINOR[4]

```

bd: 0x000d2e00 (HW:0x000d2e00)
slot cnt: 4
timefree: 6224
timetotal: 6225

```

MINOR[5]

```

bd: 0x000d2e60 (HW:0x000d2e60)
slot cnt: 32
timefree: 12500
timetotal: 12500

```

MINOR[6]

```

bd: 0x000d3080 (HW:0x000d3080)
slot cnt: 32
timefree: 25000
timetotal: 25000

```

MINOR[7]

```

bd: 0x000d32a0 (HW:0x000d32a0)
slot cnt: 32
timefree: 12500

```

```

timetotal: 12500
MINOR[8]
  bd: 0x000d34c0 (HW:0x000d34c0)
  slot cnt: 32
  timefree: 50000
  timetotal: 50000
MAJOR[3]
  minor count: 9
  MINOR[0]
    bd: 0x000d36e0 (HW:0x000d36e0)
    slot cnt: 4
    timefree: 48
    timetotal: 100
  MINOR[1]
    bd: 0x000d3740 (HW:0x000d3740)
    slot cnt: 4
    timefree: 6224
    timetotal: 6225
  MINOR[2]
    bd: 0x000d37a0 (HW:0x000d37a0)
    slot cnt: 4
    timefree: 6224
    timetotal: 6225
  MINOR[3]
    bd: 0x000d3800 (HW:0x000d3800)
    slot cnt: 4
    timefree: 6224
    timetotal: 6225
  MINOR[4]
    bd: 0x000d3860 (HW:0x000d3860)
    slot cnt: 4
    timefree: 6224
    timetotal: 6225
  MINOR[5]
    bd: 0x000d38c0 (HW:0x000d38c0)
    slot cnt: 32
    timefree: 12500
    timetotal: 12500
  MINOR[6]
    bd: 0x000d3ae0 (HW:0x000d3ae0)
    slot cnt: 32
    timefree: 25000
    timetotal: 25000
  MINOR[7]
    bd: 0x000d3d00 (HW:0x000d3d00)
    slot cnt: 32
    timefree: 12500
    timetotal: 12500
  MINOR[8]
    bd: 0x000d3f20 (HW:0x000d3f20)
    slot cnt: 32
    timefree: 50000
    timetotal: 50000
MAJOR[4]
  minor count: 9
  MINOR[0]
    bd: 0x000d4140 (HW:0x000d4140)
    slot cnt: 4
    timefree: 48
    timetotal: 100
  MINOR[1]
    bd: 0x000d41a0 (HW:0x000d41a0)
    slot cnt: 4
    timefree: 6224
    timetotal: 6225
  MINOR[2]
    bd: 0x000d4200 (HW:0x000d4200)
    slot cnt: 4
    timefree: 6224
    timetotal: 6225
  MINOR[3]
    bd: 0x000d4260 (HW:0x000d4260)
    slot cnt: 4
    timefree: 6224
    timetotal: 6225
  MINOR[4]
    bd: 0x000d42c0 (HW:0x000d42c0)
    slot cnt: 4
    timefree: 6224
    timetotal: 6225
  MINOR[5]
    bd: 0x000d4320 (HW:0x000d4320)
    slot cnt: 32
    timefree: 12500
    timetotal: 12500
  MINOR[6]
    bd: 0x000d4540 (HW:0x000d4540)
    slot cnt: 32
    timefree: 25000
    timetotal: 25000
  MINOR[7]
    bd: 0x000d4760 (HW:0x000d4760)
    slot cnt: 32
    timefree: 12500
    timetotal: 12500
  MINOR[8]
    bd: 0x000d4980 (HW:0x000d4980)
    slot cnt: 32
    timefree: 50000
    timetotal: 50000
MAJOR[5]
  minor count: 9
  MINOR[0]
    bd: 0x000d4ba0 (HW:0x000d4ba0)
    slot cnt: 4
    timefree: 48
    timetotal: 100
  MINOR[1]
    bd: 0x000d4c00 (HW:0x000d4c00)
    slot cnt: 4
    timefree: 6224
    timetotal: 6225
  MINOR[2]
    bd: 0x000d4c60 (HW:0x000d4c60)
    slot cnt: 4
    timefree: 6224
    timetotal: 6225
  MINOR[3]
    bd: 0x000d4cc0 (HW:0x000d4cc0)
    slot cnt: 4

```

```

timetotal: 12500
MINOR[8]
  bd: 0x000d34c0 (HW:0x000d34c0)
  slot cnt: 32
  timefree: 50000
  timetotal: 50000
MAJOR[3]
  minor count: 9
  MINOR[0]
    bd: 0x000d36e0 (HW:0x000d36e0)
    slot cnt: 4
    timefree: 48
    timetotal: 100
  MINOR[1]
    bd: 0x000d3740 (HW:0x000d3740)
    slot cnt: 4
    timefree: 6224
    timetotal: 6225
  MINOR[2]
    bd: 0x000d37a0 (HW:0x000d37a0)
    slot cnt: 4
    timefree: 6224
    timetotal: 6225
  MINOR[3]
    bd: 0x000d3800 (HW:0x000d3800)
    slot cnt: 4
    timefree: 6224
    timetotal: 6225
  MINOR[4]
    bd: 0x000d3860 (HW:0x000d3860)
    slot cnt: 4
    timefree: 6224
    timetotal: 6225
  MINOR[5]
    bd: 0x000d38c0 (HW:0x000d38c0)
    slot cnt: 32
    timefree: 12500
    timetotal: 12500
  MINOR[6]
    bd: 0x000d3ae0 (HW:0x000d3ae0)
    slot cnt: 32
    timefree: 25000
    timetotal: 25000
  MINOR[7]
    bd: 0x000d3d00 (HW:0x000d3d00)
    slot cnt: 32
    timefree: 12500
    timetotal: 12500
  MINOR[8]
    bd: 0x000d3f20 (HW:0x000d3f20)
    slot cnt: 32
    timefree: 50000
    timetotal: 50000
MAJOR[4]
  minor count: 9
  MINOR[0]
    bd: 0x000d4140 (HW:0x000d4140)
    slot cnt: 4
    timefree: 48
    timetotal: 100
  MINOR[1]
    bd: 0x000d41a0 (HW:0x000d41a0)
    slot cnt: 4
    timefree: 6224
    timetotal: 6225
  MINOR[2]
    bd: 0x000d4200 (HW:0x000d4200)
    slot cnt: 4
    timefree: 6224
    timetotal: 6225
  MINOR[3]
    bd: 0x000d4260 (HW:0x000d4260)
    slot cnt: 4
    timefree: 6224
    timetotal: 6225
  MINOR[4]
    bd: 0x000d42c0 (HW:0x000d42c0)
    slot cnt: 4
    timefree: 6224
    timetotal: 6225
  MINOR[5]
    bd: 0x000d4320 (HW:0x000d4320)
    slot cnt: 32
    timefree: 12500
    timetotal: 12500
  MINOR[6]
    bd: 0x000d4540 (HW:0x000d4540)
    slot cnt: 32
    timefree: 25000
    timetotal: 25000
  MINOR[7]
    bd: 0x000d4760 (HW:0x000d4760)
    slot cnt: 32
    timefree: 12500
    timetotal: 12500
  MINOR[8]
    bd: 0x000d4980 (HW:0x000d4980)
    slot cnt: 32
    timefree: 50000
    timetotal: 50000
MAJOR[5]
  minor count: 9
  MINOR[0]
    bd: 0x000d4ba0 (HW:0x000d4ba0)
    slot cnt: 4
    timefree: 48
    timetotal: 100
  MINOR[1]
    bd: 0x000d4c00 (HW:0x000d4c00)
    slot cnt: 4
    timefree: 6224
    timetotal: 6225
  MINOR[2]
    bd: 0x000d4c60 (HW:0x000d4c60)
    slot cnt: 4
    timefree: 6224
    timetotal: 6225
  MINOR[3]
    bd: 0x000d4cc0 (HW:0x000d4cc0)
    slot cnt: 4

```

```

timefree: 6224
timetotal: 6225
MINOR[4]
bd: 0x000d4d20 (HW:0x000d4d20)
slot cnt: 4
timefree: 6224
timetotal: 6225
MINOR[5]
bd: 0x000d4d80 (HW:0x000d4d80)
slot cnt: 32
timefree: 12500
timetotal: 12500
MINOR[6]
bd: 0x000d4fa0 (HW:0x000d4fa0)
slot cnt: 32
timefree: 25000
timetotal: 25000
MINOR[7]
bd: 0x000d51c0 (HW:0x000d51c0)
slot cnt: 32
timefree: 12500
timetotal: 12500
MINOR[8]
bd: 0x000d53e0 (HW:0x000d53e0)
slot cnt: 32
timefree: 50000
timetotal: 50000
MAJOR[6]
minor count: 9
MINOR[0]
bd: 0x000d5600 (HW:0x000d5600)
slot cnt: 4
timefree: 48
timetotal: 100
MINOR[1]
bd: 0x000d5660 (HW:0x000d5660)
slot cnt: 4
timefree: 6224
timetotal: 6225
MINOR[2]
bd: 0x000d56c0 (HW:0x000d56c0)
slot cnt: 4
timefree: 6224
timetotal: 6225
MINOR[3]
bd: 0x000d5720 (HW:0x000d5720)
slot cnt: 4
timefree: 6224
timetotal: 6225
MINOR[4]
bd: 0x000d5780 (HW:0x000d5780)
slot cnt: 4
timefree: 6224
timetotal: 6225
MINOR[5]
bd: 0x000d57e0 (HW:0x000d57e0)
slot cnt: 32
timefree: 12500
timetotal: 12500
MINOR[6]
bd: 0x000d5a00 (HW:0x000d5a00)
slot cnt: 32
timefree: 25000
timetotal: 25000
MINOR[7]
bd: 0x000d5c20 (HW:0x000d5c20)
slot cnt: 32
timefree: 12500
timetotal: 12500
MINOR[8]
bd: 0x000d5e40 (HW:0x000d5e40)
slot cnt: 32
timefree: 50000
timetotal: 50000
MAJOR[7]
minor count: 9
MINOR[0]
bd: 0x000d6060 (HW:0x000d6060)
slot cnt: 4
timefree: 48
timetotal: 100
MINOR[1]
bd: 0x000d60c0 (HW:0x000d60c0)
slot cnt: 4
timefree: 6224
timetotal: 6225
MINOR[2]
bd: 0x000d6120 (HW:0x000d6120)
slot cnt: 4
timefree: 6224
timetotal: 6225
MINOR[3]
bd: 0x000d6180 (HW:0x000d6180)
slot cnt: 4
timefree: 6224
timetotal: 6225
MINOR[4]
bd: 0x000d61e0 (HW:0x000d61e0)
slot cnt: 4
timefree: 6224
timetotal: 6225
MINOR[5]
bd: 0x000d6240 (HW:0x000d6240)
slot cnt: 32
timefree: 12500
timetotal: 12500
MINOR[6]
bd: 0x000d6460 (HW:0x000d6460)
slot cnt: 32
timefree: 25000
timetotal: 25000
MINOR[7]
bd: 0x000d6680 (HW:0x000d6680)
slot cnt: 32
timefree: 12500
timetotal: 12500
MINOR[8]
bd: 0x000d68a0 (HW:0x000d68a0)
slot cnt: 32
timefree: 50000
timetotal: 50000
timefree: 6224
timetotal: 6225
MINOR[4]
bd: 0x000d4d20 (HW:0x000d4d20)
slot cnt: 4
timefree: 6224
timetotal: 6225
MINOR[5]
bd: 0x000d4d80 (HW:0x000d4d80)
slot cnt: 32
timefree: 12500
timetotal: 12500
MINOR[6]
bd: 0x000d4fa0 (HW:0x000d4fa0)
slot cnt: 32
timefree: 25000
timetotal: 25000
MINOR[7]
bd: 0x000d51c0 (HW:0x000d51c0)
slot cnt: 32
timefree: 12500
timetotal: 12500
MINOR[8]
bd: 0x000d53e0 (HW:0x000d53e0)
slot cnt: 32
timefree: 50000
timetotal: 50000
MAJOR[6]
minor count: 9
MINOR[0]
bd: 0x000d5600 (HW:0x000d5600)
slot cnt: 4
timefree: 48
timetotal: 100
MINOR[1]
bd: 0x000d5660 (HW:0x000d5660)
slot cnt: 4
timefree: 6224
timetotal: 6225
MINOR[2]
bd: 0x000d56c0 (HW:0x000d56c0)
slot cnt: 4
timefree: 6224
timetotal: 6225
MINOR[3]
bd: 0x000d5720 (HW:0x000d5720)
slot cnt: 4
timefree: 6224
timetotal: 6225
MINOR[4]
bd: 0x000d5780 (HW:0x000d5780)
slot cnt: 4
timefree: 6224
timetotal: 6225
MINOR[5]
bd: 0x000d57e0 (HW:0x000d57e0)
slot cnt: 32
timefree: 12500
timetotal: 12500
MINOR[6]
bd: 0x000d5a00 (HW:0x000d5a00)
slot cnt: 32
timefree: 25000
timetotal: 25000
MINOR[7]
bd: 0x000d5c20 (HW:0x000d5c20)
slot cnt: 32
timefree: 12500
timetotal: 12500
MINOR[8]
bd: 0x000d5e40 (HW:0x000d5e40)
slot cnt: 32
timefree: 50000
timetotal: 50000
MAJOR[7]
minor count: 9
MINOR[0]
bd: 0x000d6060 (HW:0x000d6060)
slot cnt: 4
timefree: 48
timetotal: 100
MINOR[1]
bd: 0x000d60c0 (HW:0x000d60c0)
slot cnt: 4
timefree: 6224
timetotal: 6225
MINOR[2]
bd: 0x000d6120 (HW:0x000d6120)
slot cnt: 4
timefree: 6224
timetotal: 6225
MINOR[3]
bd: 0x000d6180 (HW:0x000d6180)
slot cnt: 4
timefree: 6224
timetotal: 6225
MINOR[4]
bd: 0x000d61e0 (HW:0x000d61e0)
slot cnt: 4
timefree: 6224
timetotal: 6225
MINOR[5]
bd: 0x000d6240 (HW:0x000d6240)
slot cnt: 32
timefree: 12500
timetotal: 12500
MINOR[6]
bd: 0x000d6460 (HW:0x000d6460)
slot cnt: 32
timefree: 25000
timetotal: 25000
MINOR[7]
bd: 0x000d6680 (HW:0x000d6680)
slot cnt: 32
timefree: 12500
timetotal: 12500
MINOR[8]
bd: 0x000d68a0 (HW:0x000d68a0)
slot cnt: 32
timefree: 50000
timetotal: 50000

```

```

timetotal: 50000
MAJOR[8]
minor count: 9
MINOR[0]
bd: 0x000d6ac0 (HW:0x000d6ac0)
slot cnt: 4
timefree: 48
timetotal: 100
MINOR[1]
bd: 0x000d6b20 (HW:0x000d6b20)
slot cnt: 4
timefree: 6224
timetotal: 6225
MINOR[2]
bd: 0x000d6b80 (HW:0x000d6b80)
slot cnt: 4
timefree: 6224
timetotal: 6225
MINOR[3]
bd: 0x000d6be0 (HW:0x000d6be0)
slot cnt: 4
timefree: 6224
timetotal: 6225
MINOR[4]
bd: 0x000d6c40 (HW:0x000d6c40)
slot cnt: 4
timefree: 6224
timetotal: 6225
MINOR[5]
bd: 0x000d6ca0 (HW:0x000d6ca0)
slot cnt: 32
timefree: 12500
timetotal: 12500
MINOR[6]
bd: 0x000d6ec0 (HW:0x000d6ec0)
slot cnt: 32
timefree: 25000
timetotal: 25000
MINOR[7]
bd: 0x000d70e0 (HW:0x000d70e0)
slot cnt: 32
timefree: 12500
timetotal: 12500
MINOR[8]
bd: 0x000d7300 (HW:0x000d7300)
slot cnt: 32
timefree: 50000
timetotal: 50000
MAJOR[9]
minor count: 9
MINOR[0]
bd: 0x000d7520 (HW:0x000d7520)
slot cnt: 4
timefree: 48
timetotal: 100
MINOR[1]
bd: 0x000d7580 (HW:0x000d7580)
slot cnt: 4
timefree: 6224
timetotal: 6225
MINOR[2]
bd: 0x000d75e0 (HW:0x000d75e0)
slot cnt: 4
timefree: 6224
timetotal: 6225
MINOR[3]
bd: 0x000d7640 (HW:0x000d7640)
slot cnt: 4
timefree: 6224
timetotal: 6225
MINOR[4]
bd: 0x000d76a0 (HW:0x000d76a0)
slot cnt: 4
timefree: 6224
timetotal: 6225
MINOR[5]
bd: 0x000d7700 (HW:0x000d7700)
slot cnt: 32
timefree: 12500
timetotal: 12500
MINOR[6]
bd: 0x000d7920 (HW:0x000d7920)
slot cnt: 32
timefree: 24800
timetotal: 25000
MINOR[7]
bd: 0x000d7b40 (HW:0x000d7b40)
slot cnt: 32
timefree: 12500
timetotal: 12500
MINOR[8]
bd: 0x000d7d60 (HW:0x000d7d60)
slot cnt: 32
timefree: 50000
timetotal: 50000
MAJOR[10]
minor count: 1
MINOR[0]
bd: 0x000d7f80 (HW:0x000d7f80)
slot cnt: 16
timefree: 46000
timetotal: 50000
MAJOR MINOR SLOT
00 00 0b
00 02 05
RT1 BUSB state: 420
RT1 BUSB state: 420
RT2 BUSB state: 820
RT2 BUSB state: 820
RT3 BUSB state: c20
RT3 BUSB state: c20
RT4 BUSB state: 1020
RT4 BUSB state: 1020
RT5 BUSB state: 1420
RT5 BUSB state: 1420
RT6 BUSB state: 1820
RT6 BUSB state: 1820
RT7 BUSB state: 1c20
RT7 BUSB state: 1c20
RT8 BUSB state: 2020

```

```

timetotal: 50000
MAJOR[8]
minor count: 9
MINOR[0]
bd: 0x000d6ac0 (HW:0x000d6ac0)
slot cnt: 4
timefree: 48
timetotal: 100
MINOR[1]
bd: 0x000d6b20 (HW:0x000d6b20)
slot cnt: 4
timefree: 6224
timetotal: 6225
MINOR[2]
bd: 0x000d6b80 (HW:0x000d6b80)
slot cnt: 4
timefree: 6224
timetotal: 6225
MINOR[3]
bd: 0x000d6be0 (HW:0x000d6be0)
slot cnt: 4
timefree: 6224
timetotal: 6225
MINOR[4]
bd: 0x000d6c40 (HW:0x000d6c40)
slot cnt: 4
timefree: 6224
timetotal: 6225
MINOR[5]
bd: 0x000d6ca0 (HW:0x000d6ca0)
slot cnt: 32
timefree: 12500
timetotal: 12500
MINOR[6]
bd: 0x000d6ec0 (HW:0x000d6ec0)
slot cnt: 32
timefree: 25000
timetotal: 25000
MINOR[7]
bd: 0x000d70e0 (HW:0x000d70e0)
slot cnt: 32
timefree: 12500
timetotal: 12500
MINOR[8]
bd: 0x000d7300 (HW:0x000d7300)
slot cnt: 32
timefree: 50000
timetotal: 50000
MAJOR[9]
minor count: 9
MINOR[0]
bd: 0x000d7520 (HW:0x000d7520)
slot cnt: 4
timefree: 48
timetotal: 100
MINOR[1]
bd: 0x000d7580 (HW:0x000d7580)
slot cnt: 4
timefree: 6224
timetotal: 6225
MINOR[2]
bd: 0x000d75e0 (HW:0x000d75e0)
slot cnt: 4
timefree: 6224
timetotal: 6225
MINOR[3]
bd: 0x000d7640 (HW:0x000d7640)
slot cnt: 4
timefree: 6224
timetotal: 6225
MINOR[4]
bd: 0x000d76a0 (HW:0x000d76a0)
slot cnt: 4
timefree: 6224
timetotal: 6225
MINOR[5]
bd: 0x000d7700 (HW:0x000d7700)
slot cnt: 32
timefree: 12500
timetotal: 12500
MINOR[6]
bd: 0x000d7920 (HW:0x000d7920)
slot cnt: 32
timefree: 24800
timetotal: 25000
MINOR[7]
bd: 0x000d7b40 (HW:0x000d7b40)
slot cnt: 32
timefree: 12500
timetotal: 12500
MINOR[8]
bd: 0x000d7d60 (HW:0x000d7d60)
slot cnt: 32
timefree: 50000
timetotal: 50000
MAJOR[10]
minor count: 1
MINOR[0]
bd: 0x000d7f80 (HW:0x000d7f80)
slot cnt: 16
timefree: 46000
timetotal: 50000
MAJOR MINOR SLOT
00 00 0b
00 02 05
RT1 BUSB state: 420
RT1 BUSB state: 420
RT2 BUSB state: 820
RT2 BUSB state: 820
RT3 BUSB state: c20
RT3 BUSB state: c20
RT4 BUSB state: 1020
RT4 BUSB state: 1020
RT5 BUSB state: 1420
RT5 BUSB state: 1420
RT6 BUSB state: 1820
RT6 BUSB state: 1820
RT7 BUSB state: 1c20
RT7 BUSB state: 1c20
RT8 BUSB state: 2020

```


<pre> 03 08 21 CF2:0 04 06 21 CF3:0 05 05 0d CF4:0 06 01 05 06 08 21 CF5:0 07 07 21 CF6:0 08 06 21 CF7:0 09 04 05 TSYNC 09 08 21 CF0:0 02 08 21 CF1:0 03 07 00 CF2:0 04 05 10 CF3:0 05 02 05 05 08 21 CF4:0 06 08 21 CF5:0 07 06 21 CF6:0 08 04 05 08 08 21 TSYNC CF7:0 09 08 21 CF0:0 02 07 03 CF1:0 03 05 21 CF2:0 04 02 05 04 08 21 CF3:0 05 08 21 CF4:0 06 06 21 CF5:0 07 05 00 07 08 21 CF6:0 08 08 21 TSYNC CF7:0 09 07 06 CF0:0 02 06 21 CF1:0 03 02 05 03 08 21 CF2:0 04 08 21 CF3:0 05 06 21 CF4:0 06 05 03 06 08 CF5:0 21 Switching to Shutdown State 07 08 21 CF6:0 08 08 21 CF7:0 0a 00 02 Quit! BM Entries: 0 CORE#0 halts because of Error Mode [1.09662sec/3421353536, 0, 0, 0cycles] Performance statistics for CORE#0 Cycles : 3421353536 Instructions : 616931098 CORE#0 performance (250MHz) : 45.08 MIPS Simulated time : 13.69 s Performance of laysim-gr740 Real-time performance : 1247.96 % laysim-gr740 performance : 562.58 MIPS Wall clock : 1.10 s [layright@CentOS Samples]\$ </pre>	<pre> 03 08 21 CF2:0 04 06 21 CF3:0 05 05 0d CF4:0 06 01 05 06 08 21 CF5:0 07 07 21 CF6:0 08 06 21 CF7:0 09 04 05 TSYNC 09 08 21 CF0:0 02 08 21 CF1:0 03 07 00 CF2:0 04 05 10 CF3:0 05 02 05 05 08 21 CF4:0 06 08 21 CF5:0 07 06 21 CF6:0 08 04 05 08 08 21 TSYNC CF7:0 09 08 21 CF0:0 02 07 03 CF1:0 03 05 21 CF2:0 04 02 05 04 08 21 CF3:0 05 08 21 CF4:0 06 06 21 CF5:0 07 05 00 07 08 21 CF6:0 08 08 21 TSYNC CF7:0 09 07 06 CF0:0 02 06 21 CF1:0 03 02 05 03 08 21 CF2:0 04 08 21 CF3:0 05 06 21 CF4:0 06 05 03 06 08 CF5:0 21 Switching to Shutdown State 07 08 21 CF6:0 08 08 21 CF7:0 0a 00 02 Quit! BM Entries: 0 CORE#0 halts because of Error Mode [2.91721sec/3421353536, 0, 0, 0cycles] Performance statistics for CORE#0 Cycles : 3421353536 Instructions : 616931098 CORE#0 performance (250MHz) : 45.08 MIPS Simulated time : 13.69 s Performance of laysim-gr740 Real-time performance : 469.13 % laysim-gr740 performance : 211.48 MIPS Wall clock : 2.92 s PS C:\laysim-gr740\RCC-1.3.1\Samples> </pre>
---	---

4.5 Loadable CAN Module

laysim-GR740 supports the loadable CAN module by using '-mod MODULE' on start-up. Figure 4-14 shows the result of rtems-grcan with the loadable CAN module. The loadable CAN Node module can be loaded with '-mod ../Loadable/loadable-CAN-Nodes/laysim_user_can_node1.so' on start-up.

Figure 4–14. CAN Node Simulation using loadable CAN module

laysim-gr740-dbt-cli in Linux	laysim-gr740-cli in Windows
<pre> [layright@CentOS Samples]\$ laysim-gr740-dbt-cli -r -core0 rtems-grcan.elf -mod ../Loadable/loadable-CAN-Nodes/laysim_user_can_node1.so laysim for GR740 Quad Core Processor (DBT) CLI v3.0 by layright Loaded user loadable module ../Loadable/loadable-CAN-Nodes/laysim_user_can_node1.so </pre>	<pre> PS C:\laysim-gr740\RCC-1.3.1\Samples> laysim-gr740-dbt-cli -r -core0 rtems-grcan.elf -mod ../Loadable/loadable-CAN-Nodes\laysim_user_can_node1-dbt-cli.dll laysim for GR740 Quad Core Processor (DBT) CLI v3.0 by layright Loaded user loadable module ../Loadable/loadable-CAN-Nodes\laysim_user_can_node1-dbt-cli.dll </pre>


```

Multicore control : Time quantum (500)
CORE#0 RAM file is loaded - rtems-grcan.elf
- .text, addr: 0x00000000, size 195984 bytes
- .rtemsreset, addr: 0x0002FD00, size 160 bytes
- .rtemsstack, addr: 0x0002FE40, size 8192 bytes (not loaded, just info.)
- .data, addr: 0x00031E40, size 5488 bytes
- .bss, addr: 0x000333C0, size 34656 bytes (not loaded, just info.)
- Detect RTEMS v5.1(SMP) with thread information
Only GR740 CORE#0 RAM file is loaded, starts from RAM
Set CORE#0 %sp/%fp : 0x07FFFFF0/0x08000000

```

```

***** Initializing CAN test *****
***** Initializing GRCAN test *****
Opening GRCAN device 0
***** Initializing GRCAN loopback *****
Opening GRCAN device 1
loop_task1: Entering CAN Loop-Back loop
***** MESSAGES THAT WILL BE TRANSMITTED *****
MSG[1]: STD length: 4, id: 0xa
MSGDATA[1]: 0x02 0xc4 0x4f 0xf2 ..0.
MSG[2]: EXT length: 5, id: 0xa
MSGDATA[2]: 0x02 0xc4 0x4f 0xf2 0x23 ..0..
MSG[3]: EXT length: 8, id: 0x37a
MSGDATA[3]: 0xaa 0xbb 0x11 0x22 0x55 0x12 0xff 0x00 ....U...
MSG[4]: EXT length: 7, id: 0x20ff
MSGDATA[4]: 0x47 0x61 0x69 0x73 0x6c 0x65 0x72 Gaisler
***** Start of transmission *****
Starting status task1
CAN INTS: 6
CAN RXPPTS: 46
CAN TXPKTS: 152 (TX-RX: 64)
CAN INTS: 2842
CAN RXPPTS: 2318
CAN TXPKTS: 2424 (TX-RX: 64)
CAN INTS: 5680
CAN RXPPTS: 4590
CAN TXPKTS: 4696 (TX-RX: 64)
CAN INTS: 8518
CAN RXPPTS: 6862
CAN TXPKTS: 6968 (TX-RX: 64)
CAN INTS: 11356
CAN RXPPTS: 9134
CAN TXPKTS: 9240 (TX-RX: 64)
CAN INTS: 14194
CAN RXPPTS: 11406
CAN TXPKTS: 11512 (TX-RX: 64)
CAN INTS: 17032
CAN RXPPTS: 13678
CAN TXPKTS: 13784 (TX-RX: 64)
CAN INTS: 19870
CAN RXPPTS: 15950
CAN TXPKTS: 16056 (TX-RX: 64)
CAN INTS: 22708
CAN RXPPTS: 18222
CAN TXPKTS: 18328 (TX-RX: 64)
CAN INTS: 25546
CAN RXPPTS: 20494
CAN TXPKTS: 20600 (TX-RX: 64)
CAN INTS: 28384
CAN RXPPTS: 22766
CAN RXSYNCS: 0
CAN TXPKTS: 22872 (TX-RX: 58)
CAN INTS: 31234
CAN RXPPTS: 25048
CAN TXPKTS: 25156 (TX-RX: 65)
CAN INTS: 34071
CAN RXPPTS: 27320
CAN TXPKTS: 27428 (TX-RX: 65)
CAN INTS: 36908
CAN RXPPTS: 29592
CAN TXPKTS: 29700 (TX-RX: 65)
CAN INTS: 39745
CAN RXPPTS: 31864
CAN TXPKTS: 31972 (TX-RX: 65)
CAN INTS: 42582
CAN RXPPTS: 34136
CAN TXPKTS: 34244 (TX-RX: 65)
CAN INTS: 45419
CAN RXPPTS: 36408
CAN TXPKTS: 36516 (TX-RX: 65)
CAN INTS: 48256
CAN RXPPTS: 38680
CAN TXPKTS: 38788 (TX-RX: 65)
CAN INTS: 51093
CAN RXPPTS: 40952
CAN TXPKTS: 41060 (TX-RX: 65)
CAN INTS: 53930
CAN RXPPTS: 43224
CAN TXPKTS: 43332 (TX-RX: 65)
CAN INTS: 56767
CAN RXPPTS: 45496
CAN TXPKTS: 45604 (TX-RX: 65)
CAN INTS: 59604
CAN RXPPTS: 47768
CAN RXSYNCS: 0
CAN TXPKTS: 47876 (TX-RX: 60)
CAN INTS: 62452
CAN RXPPTS: 50051
CAN TXPKTS: 50160 (TX-RX: 66)
CAN INTS: 65290
CAN RXPPTS: 52323
CAN TXPKTS: 52432 (TX-RX: 66)
CAN INTS: 68128
CAN RXPPTS: 54595
CAN TXPKTS: 54704 (TX-RX: 66)
CAN INTS: 70966
CAN RXPPTS: 56867
CAN TXPKTS: 56976 (TX-RX: 66)
CAN INTS: 73804
CAN RXPPTS: 59139
CAN TXPKTS: 59248 (TX-RX: 66)
CAN INTS: 76642
CAN RXPPTS: 61411
CAN TXPKTS: 61520 (TX-RX: 66)
CAN INTS: 79480
CAN RXPPTS: 63683
...

```

```

Multicore control : Time quantum (500)
CORE#0 RAM file is loaded - rtems-grcan.elf
- .text, addr: 0x00000000, size 195984 bytes
- .rtemsreset, addr: 0x0002FD00, size 160 bytes
- .rtemsstack, addr: 0x0002FE40, size 8192 bytes (not loaded, just info.)
- .data, addr: 0x00031E40, size 5488 bytes
- .bss, addr: 0x000333C0, size 34656 bytes (not loaded, just info.)
- Detect RTEMS v5.1(SMP) with thread information
Only GR740 CORE#0 RAM file is loaded, starts from RAM
Set CORE#0 %sp/%fp : 0x07FFFFF0/0x08000000

```

```

***** Initializing CAN test *****
***** Initializing GRCAN test *****
Opening GRCAN device 0
***** Initializing GRCAN loopback *****
Opening GRCAN device 1
loop_task1: Entering CAN Loop-Back loop
***** MESSAGES THAT WILL BE TRANSMITTED *****
MSG[1]: STD length: 4, id: 0xa
MSGDATA[1]: 0x02 0xc4 0x4f 0xf2 ..0.
MSG[2]: EXT length: 5, id: 0xa
MSGDATA[2]: 0x02 0xc4 0x4f 0xf2 0x23 ..0..
MSG[3]: EXT length: 8, id: 0x37a
MSGDATA[3]: 0xaa 0xbb 0x11 0x22 0x55 0x12 0xff 0x00 ....U...
MSG[4]: EXT length: 7, id: 0x20ff
MSGDATA[4]: 0x47 0x61 0x69 0x73 0x6c 0x65 0x72 Gaisler
***** Start of transmission *****
Starting status task1
CAN INTS: 6
CAN RXPPTS: 46
CAN TXPKTS: 152 (TX-RX: 64)
CAN INTS: 2842
CAN RXPPTS: 2318
CAN TXPKTS: 2424 (TX-RX: 64)
CAN INTS: 5680
CAN RXPPTS: 4590
CAN TXPKTS: 4696 (TX-RX: 64)
CAN INTS: 8518
CAN RXPPTS: 6862
CAN TXPKTS: 6968 (TX-RX: 64)
CAN INTS: 11356
CAN RXPPTS: 9134
CAN TXPKTS: 9240 (TX-RX: 64)
CAN INTS: 14194
CAN RXPPTS: 11406
CAN TXPKTS: 11512 (TX-RX: 64)
CAN INTS: 17032
CAN RXPPTS: 13678
CAN TXPKTS: 13784 (TX-RX: 64)
CAN INTS: 19870
CAN RXPPTS: 15950
CAN TXPKTS: 16056 (TX-RX: 64)
CAN INTS: 22708
CAN RXPPTS: 18222
CAN TXPKTS: 18328 (TX-RX: 64)
CAN INTS: 25546
CAN RXPPTS: 20494
CAN TXPKTS: 20600 (TX-RX: 64)
CAN INTS: 28384
CAN RXPPTS: 22766
CAN RXSYNCS: 0
CAN TXPKTS: 22872 (TX-RX: 58)
CAN INTS: 31234
CAN RXPPTS: 25048
CAN TXPKTS: 25156 (TX-RX: 65)
CAN INTS: 34071
CAN RXPPTS: 27320
CAN TXPKTS: 27428 (TX-RX: 65)
CAN INTS: 36908
CAN RXPPTS: 29592
CAN TXPKTS: 29700 (TX-RX: 65)
CAN INTS: 39745
CAN RXPPTS: 31864
CAN TXPKTS: 31972 (TX-RX: 65)
CAN INTS: 42582
CAN RXPPTS: 34136
CAN TXPKTS: 34244 (TX-RX: 65)
CAN INTS: 45419
CAN RXPPTS: 36408
CAN TXPKTS: 36516 (TX-RX: 65)
CAN INTS: 48256
CAN RXPPTS: 38680
CAN TXPKTS: 38788 (TX-RX: 65)
CAN INTS: 51093
CAN RXPPTS: 40952
CAN TXPKTS: 41060 (TX-RX: 65)
CAN INTS: 53930
CAN RXPPTS: 43224
CAN TXPKTS: 43332 (TX-RX: 65)
CAN INTS: 56767
CAN RXPPTS: 45496
CAN TXPKTS: 45604 (TX-RX: 65)
CAN INTS: 59604
CAN RXPPTS: 47768
CAN RXSYNCS: 0
CAN TXPKTS: 47876 (TX-RX: 60)
CAN INTS: 62452
CAN RXPPTS: 50051
CAN TXPKTS: 50160 (TX-RX: 66)
CAN INTS: 65290
CAN RXPPTS: 52323
CAN TXPKTS: 52432 (TX-RX: 66)
CAN INTS: 68128
CAN RXPPTS: 54595
CAN TXPKTS: 54704 (TX-RX: 66)
CAN INTS: 70966
CAN RXPPTS: 56867
CAN TXPKTS: 56976 (TX-RX: 66)
CAN INTS: 73804
CAN RXPPTS: 59139
CAN TXPKTS: 59248 (TX-RX: 66)
CAN INTS: 76642
CAN RXPPTS: 61411
CAN TXPKTS: 61520 (TX-RX: 66)
CAN INTS: 79480
CAN RXPPTS: 63683
...

```

4.6 Loadable SPI Module

laysim-GR740 supports the loadable SPI module by using '-mod MODULE' on start-up. Figure 4-15 shows the result of rtems-spi with the loadable SPI module. The loadable SPI module can be loaded with '-mod ./Loadable/loadable-SPI-Module/laysim_user_spi_module.so' on start-up.

Figure 4–15. SPI Simulation using loadable SPI module

laysim-gr740-dbt-cli in Linux	laysim-gr740-cli in Windows
<pre>[layright@CentOS Samples]\$ laysim-gr740-dbt-cli -r -core0 rtems-spi.elf -mod ../Loadable/loadable-SPI-Modules/laysim_user_spi_module.so laysim for GR740 Quad Core Processor (DBT) CLI v3.0 by layright Loaded user loadable module ../Loadable/loadable-SPI-Modules/laysim_user_spi_module.so Multicore control : Time quantum (500) CORE#0 RAM file is loaded - rtems-spi.elf - .text, addr: 0x00000000, size 202704 bytes - .rtemsroset, addr: 0x000317D0, size 176 bytes - .rtemsstack, addr: 0x00031880, size 8192 bytes (not loaded, just info.) - .data, addr: 0x00033880, size 5520 bytes - .bss, addr: 0x00034E40, size 34576 bytes (not loaded, just info.) - Detect RTEMS v5.1(SMP) with thread information Only GR740 CORE#0 RAM file is loaded, starts from RAM Set CORE#0 %sp/%fp : 0x07FFFFFF0/0x08000000 ***** Initializing SPICTRL test ***** Registering SPI FLASH driver: driver registered successfully READ 0x0 Read, bytes: 15 [0x0, 0x0, 0x0, 0x0] 0x00: 0 0 0 0 0x04: 1 0 0 0 0x08: 2 0 0 0 0x0C: 3 0 0 0 READ 0xf Read, bytes: 7 [0x0, 0x4] 0x0F: 0 4 0 0 0x13: 0 5 0 0 READ 0x10000 Read, bytes: 4 [0x0, 0x40] 0x10000: 0 40 0 0 READ 0x1ffffa Read, bytes: 10 [0x0, 0x0] 0x1FFFA: 0 0 0 ff 7f 0x1FFFE: 0 0 0 0 80 0x20002: 0 0 0 0 WRITE 0x1ffffc WRITE 0x03 CORE#0 halts because of Error Mode [0.065503sec/44157302, 0, 0, 0cycles] Performance statistics for CORE#0 Cycles : 44157302 Instructions : 29456443 CORE#0 performance (250MHz) : 166.77 MIPS Simulated time : 176.63 ms Performance of laysim-gr740 Real-time performance : 269.65 % laysim-gr740 performance : 449.70 MIPS Wall clock : 0.07 s [layright@CentOS Samples]\$</pre>	<pre>PS C:\laysim-gr740\RCC-1.3.1\Samples> laysim-gr740-dbt-cli -r -core0 rtems-spi.elf -mod ../Loadable/loadable-SPI-Modules\laysim_user_spi_module-dbt-cli.dll laysim for GR740 Quad Core Processor (DBT) CLI v3.0 by layright Loaded user loadable module ../Loadable/loadable-SPI-Modules\laysim_user_spi_module-dbt-cli.dll Multicore control : Time quantum (500) CORE#0 RAM file is loaded - rtems-spi.elf - .text, addr: 0x00000000, size 202704 bytes - .rtemsroset, addr: 0x000317D0, size 176 bytes - .rtemsstack, addr: 0x00031880, size 8192 bytes (not loaded, just info.) - .data, addr: 0x00033880, size 5520 bytes - .bss, addr: 0x00034E40, size 34576 bytes (not loaded, just info.) - Detect RTEMS v5.1(SMP) with thread information Only GR740 CORE#0 RAM file is loaded, starts from RAM Set CORE#0 %sp/%fp : 0x07FFFFFF0/0x08000000 ***** Initializing SPICTRL test ***** Registering SPI FLASH driver: driver registered successfully READ 0x0 Read, bytes: 15 [0x0, 0x0, 0x0, 0x0] 0x00: 0 0 0 0 0x04: 1 0 0 0 0x08: 2 0 0 0 0x0C: 3 0 0 0 READ 0xf Read, bytes: 7 [0x0, 0x4] 0x0F: 0 4 0 0 0x13: 0 5 0 0 READ 0x10000 Read, bytes: 4 [0x0, 0x40] 0x10000: 0 40 0 0 READ 0x1ffffa Read, bytes: 10 [0x0, 0x0] 0x1FFFA: 0 0 0 ff 7f 0x1FFFE: 0 0 0 0 80 0x20002: 0 0 0 0 WRITE 0x1ffffc WRITE 0x03 CORE#0 halts because of Error Mode [0.1092sec/44157302, 0, 0, 0cycles] Performance statistics for CORE#0 Cycles : 44157302 Instructions : 29456443 CORE#0 performance (250MHz) : 166.77 MIPS Simulated time : 176.63 ms Performance of laysim-gr740 Real-time performance : 161.75 % laysim-gr740 performance : 269.75 MIPS Wall clock : 0.11 s PS C:\laysim-gr740\RCC-1.3.1\Samples></pre>

4.7 Loadable GPIO Module

laysim-GR740 supports the loadable GPIO module by using '-mod MODULE' on start-up. Figure 4-16 shows the result of rtems-gpio with the loadable GPIO module. The loadable GPIO module can be loaded with '-mod ./Loadable/loadable-GPIO-Module/laysim_user_gpio_module.so' on start-up.

Figure 4–16. GPIO Simulation using loadable GPIO module

laysim-gr740-dbt-cli in Linux	laysim-gr740-cli in Windows
<pre>[layright@CentOS RCC-1.2.25]\$ laysim-gr740-dbt-cli -r -core0 rtems-gpio.elf -mod ../Loadable/loadable-GPIO-Modules/laysim_user_gpio_module.so laysim for GR740 Quad Core Processor (DBT) CLI v3.0 by layright Loaded user loadable module ../Loadable/loadable-GPIO-Modules/laysim_user_gpio_module.so Multicore control : Time quantum (500) CORE#0 RAM file is loaded - rtems-gpio.elf - .text, addr: 0x00000000, size 174896 bytes - .data, addr: 0x0002AB30, size 4864 bytes - .jcr, addr: 0x0002BE30, size 4 bytes</pre>	<pre>PS C:\laysim-gr740\RCC-1.2.25> laysim-gr740-dbt-cli -r -core0 rtems-gpio.elf -mod ../Loadable/loadable-GPIO-Modules\laysim_user_gpio_module-dbt-cli.dll laysim for GR740 Quad Core Processor (DBT) CLI v3.0 by layright Loaded user loadable module ../Loadable/loadable-GPIO-Modules\laysim_user_gpio_module-dbt-cli.dll Multicore control : Time quantum (500) CORE#0 RAM file is loaded - rtems-gpio.elf - .text, addr: 0x00000000, size 174896 bytes - .data, addr: 0x0002AB30, size 4864 bytes - .jcr, addr: 0x0002BE30, size 4 bytes</pre>

laysim-GR740-001 laysim-GR740 Quad Core Processor Emulator – User's Manual (Rev13)

```
- .bss, addr: 0x002BE40, size 5172 bytes (not loaded, just info.)
- Detect RTEMS v4.10 with thread information
Only GR740 CORE#0 RAM file is loaded, starts from RAM
Set CORE#0 %sp/%fp : 0x07FFFFF0/0x08000000
```

--- BUS TOPOLOGY ---

```
-> DEV 0x3fd28 GAISLER_LEON4
-> DEV 0x3fd90 GAISLER_LEON4
-> DEV 0x3fd88 GAISLER_LEON4
-> DEV 0x3fe60 GAISLER_LEON4
-> DEV 0x3fec8 GAISLER_GRIOMMU
-> DEV 0x3ff30 GAISLER_AHB2AHB
-> DEV 0x3ff98 GAISLER_L2CACHE
-> DEV 0x40000 GAISLER_APBMS
-> DEV 0x40068 GAISLER_APBMS
-> DEV 0x40080 GAISLER_APBMS
-> DEV 0x40138 GAISLER_APBMS
-> DEV 0x401a0 GAISLER_GPIO
-> DEV 0x40208 GAISLER_FTMCTRL
-> DEV 0x40270 GAISLER_IRQMP
-> DEV 0x402d8 GAISLER_GPTIMER
-> DEV 0x40340 GAISLER_GPTIMER
-> DEV 0x403a8 GAISLER_GPTIMER
-> DEV 0x40410 GAISLER_GPTIMER
-> DEV 0x40478 GAISLER_GPTIMER
-> DEV 0x404e0 GAISLER_SPW2_DMA
-> DEV 0x40548 GAISLER_SPW2_DMA
-> DEV 0x405b0 GAISLER_SPW2_DMA
-> DEV 0x40618 GAISLER_SPW2_DMA
-> DEV 0x40680 GAISLER_ETHMAC
-> DEV 0x406e8 GAISLER_ETHMAC
-> DEV 0x40750 GAISLER_GRPIC2
-> DEV 0x407b8 GAISLER_GRCAN
-> DEV 0x40820 GAISLER_GRCAN
-> DEV 0x40888 GAISLER_SPICTRL
-> DEV 0x408f0 GAISLER_CLKGATE
-> DEV 0x40958 GAISLER_GR1553B
-> DEV 0x409c0 GAISLER_AHBSTAT
-> DEV 0x40a28 GAISLER_AHBSTAT
-> DEV 0x40a90 GAISLER_GPIO
-> DEV 0x40af8 GAISLER_GPREG
-> DEV 0x40b50 NO NAME
-> DEV 0x40bb8 GAISLER_GPREGBANK
-> DEV 0x40c28 GAISLER_SPWTD
-> DEV 0x40c90 GAISLER_LASTAT
-> DEV 0x40cf8 GAISLER_L2CACHE
-> DEV 0x40d60 GAISLER_MEMSCRUB
-> DEV 0x40dc8 GAISLER_GRIOMMU2
-> DEV 0x40e30 GAISLER_DDRSDMUX
-> DEV 0x40e98 GAISLER_AHB2AHB
-> DEV 0x40f00 GAISLER_FTMCTRL
-> DEV 0x40f68 GAISLER_GRPIC2
-> DEV 0x40fd0 GAISLER_SWRROUTER
```

```
GRGPIO[0xffa08000] PORT[21]: IN/OUT/DIR: [1,0,0], MASK/POL/EDGE: [0,0,0], BYPASS: 0
GRGPIO[0xffa08000] PORT[20]: IN/OUT/DIR: [0,0,0], MASK/POL/EDGE: [0,0,0], BYPASS: 0
GRGPIO[0xffa08000] PORT[19]: IN/OUT/DIR: [1,0,0], MASK/POL/EDGE: [0,0,0], BYPASS: 0
GRGPIO[0xffa08000] PORT[18]: IN/OUT/DIR: [1,0,0], MASK/POL/EDGE: [0,0,0], BYPASS: 0
GRGPIO[0xffa08000] PORT[17]: IN/OUT/DIR: [1,0,0], MASK/POL/EDGE: [0,0,0], BYPASS: 0
GRGPIO[0xffa08000] PORT[16]: IN/OUT/DIR: [1,0,0], MASK/POL/EDGE: [0,0,0], BYPASS: 0
GRGPIO[0xffa08000] PORT[15]: IN/OUT/DIR: [1,0,0], MASK/POL/EDGE: [0,1,1], BYPASS: 0
GRGPIO[0xffa08000] PORT[14]: IN/OUT/DIR: [1,0,0], MASK/POL/EDGE: [0,1,1], BYPASS: 0
GRGPIO[0xffa08000] PORT[13]: IN/OUT/DIR: [1,0,0], MASK/POL/EDGE: [0,1,1], BYPASS: 0
GRGPIO[0xffa08000] PORT[12]: IN/OUT/DIR: [0,0,0], MASK/POL/EDGE: [0,1,1], BYPASS: 0
GRGPIO[0xffa08000] PORT[11]: IN/OUT/DIR: [0,0,0], MASK/POL/EDGE: [0,1,1], BYPASS: 0
GRGPIO[0xffa08000] PORT[10]: IN/OUT/DIR: [0,0,0], MASK/POL/EDGE: [0,1,1], BYPASS: 0
GRGPIO[0xffa08000] PORT[9]: IN/OUT/DIR: [1,0,0], MASK/POL/EDGE: [0,1,1], BYPASS: 0
GRGPIO[0xffa08000] PORT[8]: IN/OUT/DIR: [1,0,0], MASK/POL/EDGE: [0,1,1], BYPASS: 0
GRGPIO[0xffa08000] PORT[7]: IN/OUT/DIR: [0,0,0], MASK/POL/EDGE: [0,1,1], BYPASS: 0
GRGPIO[0xffa08000] PORT[6]: IN/OUT/DIR: [0,0,0], MASK/POL/EDGE: [0,1,1], BYPASS: 0
GRGPIO[0xffa08000] PORT[5]: IN/OUT/DIR: [0,0,0], MASK/POL/EDGE: [0,1,1], BYPASS: 0
GRGPIO[0xffa08000] PORT[4]: IN/OUT/DIR: [0,0,0], MASK/POL/EDGE: [0,1,1], BYPASS: 0
GRGPIO[0xffa08000] PORT[3]: IN/OUT/DIR: [0,0,0], MASK/POL/EDGE: [0,1,1], BYPASS: 0
GRGPIO[0xffa08000] PORT[2]: IN/OUT/DIR: [0,0,0], MASK/POL/EDGE: [0,1,1], BYPASS: 0
GRGPIO[0xffa08000] PORT[1]: IN/OUT/DIR: [1,0,0], MASK/POL/EDGE: [0,1,1], BYPASS: 0
GRGPIO[0xffa08000] PORT[0]: IN/OUT/DIR: [1,0,0], MASK/POL/EDGE: [0,0,0], BYPASS: 0
GRGPIO[0xff902000] PORT[15]: IN/OUT/DIR: [0,0,0], MASK/POL/EDGE: [0,1,1], BYPASS: 0
GRGPIO[0xff902000] PORT[14]: IN/OUT/DIR: [0,0,0], MASK/POL/EDGE: [0,1,1], BYPASS: 0
GRGPIO[0xff902000] PORT[13]: IN/OUT/DIR: [0,0,0], MASK/POL/EDGE: [0,1,1], BYPASS: 0
GRGPIO[0xff902000] PORT[12]: IN/OUT/DIR: [0,0,0], MASK/POL/EDGE: [0,1,1], BYPASS: 0
GRGPIO[0xff902000] PORT[11]: IN/OUT/DIR: [0,0,0], MASK/POL/EDGE: [0,1,1], BYPASS: 0
GRGPIO[0xff902000] PORT[10]: IN/OUT/DIR: [0,0,0], MASK/POL/EDGE: [0,1,1], BYPASS: 0
GRGPIO[0xff902000] PORT[9]: IN/OUT/DIR: [0,0,0], MASK/POL/EDGE: [0,1,1], BYPASS: 0
GRGPIO[0xff902000] PORT[8]: IN/OUT/DIR: [0,0,0], MASK/POL/EDGE: [0,1,1], BYPASS: 0
GRGPIO[0xff902000] PORT[7]: IN/OUT/DIR: [0,0,0], MASK/POL/EDGE: [0,1,1], BYPASS: 0
GRGPIO[0xff902000] PORT[6]: IN/OUT/DIR: [0,0,0], MASK/POL/EDGE: [0,1,1], BYPASS: 0
GRGPIO[0xff902000] PORT[5]: IN/OUT/DIR: [0,0,0], MASK/POL/EDGE: [0,1,1], BYPASS: 0
GRGPIO[0xff902000] PORT[4]: IN/OUT/DIR: [0,0,0], MASK/POL/EDGE: [0,1,1], BYPASS: 0
GRGPIO[0xff902000] PORT[3]: IN/OUT/DIR: [1,0,0], MASK/POL/EDGE: [0,1,1], BYPASS: 0
GRGPIO[0xff902000] PORT[2]: IN/OUT/DIR: [1,0,0], MASK/POL/EDGE: [0,1,1], BYPASS: 0
GRGPIO[0xff902000] PORT[1]: IN/OUT/DIR: [1,0,0], MASK/POL/EDGE: [0,1,1], BYPASS: 0
GRGPIO[0xff902000] PORT[0]: IN/OUT/DIR: [1,0,0], MASK/POL/EDGE: [0,0,0], BYPASS: 0
Failed to disable IRQ on gpio port 4, disabling port while not enabled should fail. OK
Current Value on GPIO: 1
GRGPIO[0xff902000] PORT[4]: IN/OUT/DIR: [1,0,0], MASK/POL/EDGE: [1,0,0], BYPASS: 0
GRGPIO[0xff902000] PORT[4]: IN/OUT/DIR: [1,0,0], MASK/POL/EDGE: [1,0,0], BYPASS: 0
TEST END
CORE#0 halts because of Error Mode [0.291649sec/340946551, 0, 0, 0cycles]
```

Performance statistics for CORE#0
Cycles : 340946551
Instructions : 227455650
CORE#0 performance (250MHz) : 166.78 MIPS
Simulated time : 1.36 s

Performance of laysim-gr740
Real-time performance : 467.54 %
laysim-gr740 performance : 779.77 MIPS
Wall clock : 0.29 s
[layright@CentOS RCC-1.2.25]\$

```
- .bss, addr: 0x002BE60, size 5172 bytes (not loaded, just info.)
- Detect RTEMS v4.10 with thread information
Only GR740 CORE#0 RAM file is loaded, starts from RAM
Set CORE#0 %sp/%fp : 0x07FFFFF0/0x08000000
```

--- BUS TOPOLOGY ---

```
-> DEV 0x3fd48 GAISLER_LEON4
-> DEV 0x3fdb0 GAISLER_LEON4
-> DEV 0x3fe18 GAISLER_LEON4
-> DEV 0x3fe80 GAISLER_LEON4
-> DEV 0x3fee8 GAISLER_GRIOMMU
-> DEV 0x3ff50 GAISLER_AHB2AHB
-> DEV 0x3ffb8 GAISLER_L2CACHE
-> DEV 0x40020 GAISLER_APBMS
-> DEV 0x40088 GAISLER_APBMS
-> DEV 0x400f0 GAISLER_APBMS
-> DEV 0x40158 GAISLER_APBMS
-> DEV 0x401c0 GAISLER_GPIO
-> DEV 0x40228 GAISLER_FTMCTRL
-> DEV 0x40290 GAISLER_IRQMP
-> DEV 0x402f8 GAISLER_GPTIMER
-> DEV 0x40360 GAISLER_GPTIMER
-> DEV 0x403c8 GAISLER_GPTIMER
-> DEV 0x40430 GAISLER_GPTIMER
-> DEV 0x40498 GAISLER_GPTIMER
-> DEV 0x40500 GAISLER_SPW2_DMA
-> DEV 0x40568 GAISLER_SPW2_DMA
-> DEV 0x405d0 GAISLER_SPW2_DMA
-> DEV 0x40638 GAISLER_SPW2_DMA
-> DEV 0x406a0 GAISLER_ETHMAC
-> DEV 0x40708 GAISLER_ETHMAC
-> DEV 0x40770 GAISLER_GRPIC2
-> DEV 0x407d8 GAISLER_GRCAN
-> DEV 0x40840 GAISLER_GRCAN
-> DEV 0x408a8 GAISLER_SPICTRL
-> DEV 0x40910 GAISLER_CLKGATE
-> DEV 0x40978 GAISLER_GR1553B
-> DEV 0x409e0 GAISLER_AHBSTAT
-> DEV 0x40a48 GAISLER_AHBSTAT
-> DEV 0x40ab0 GAISLER_GPIO
-> DEV 0x40b18 GAISLER_GPREG
-> DEV 0x40b90 NO NAME
-> DEV 0x40bd8 GAISLER_GPREGBANK
-> DEV 0x40c48 GAISLER_SPWTD
-> DEV 0x40cb0 GAISLER_LASTAT
-> DEV 0x40d18 GAISLER_L2CACHE
-> DEV 0x40d80 GAISLER_MEMSCRUB
-> DEV 0x40de8 GAISLER_GRIOMMU2
-> DEV 0x40e50 GAISLER_DDRSDMUX
-> DEV 0x40eb8 GAISLER_AHB2AHB
-> DEV 0x40f20 GAISLER_FTMCTRL
-> DEV 0x40f88 GAISLER_GRPIC2
-> DEV 0x40ff0 GAISLER_SWRROUTER
```

```
GRGPIO[0xffa08000] PORT[21]: IN/OUT/DIR: [1,0,0], MASK/POL/EDGE: [0,0,0], BYPASS: 0
GRGPIO[0xffa08000] PORT[20]: IN/OUT/DIR: [0,0,0], MASK/POL/EDGE: [0,0,0], BYPASS: 0
GRGPIO[0xffa08000] PORT[19]: IN/OUT/DIR: [1,0,0], MASK/POL/EDGE: [0,0,0], BYPASS: 0
GRGPIO[0xffa08000] PORT[18]: IN/OUT/DIR: [1,0,0], MASK/POL/EDGE: [0,0,0], BYPASS: 0
GRGPIO[0xffa08000] PORT[17]: IN/OUT/DIR: [1,0,0], MASK/POL/EDGE: [0,0,0], BYPASS: 0
GRGPIO[0xffa08000] PORT[16]: IN/OUT/DIR: [1,0,0], MASK/POL/EDGE: [0,0,0], BYPASS: 0
GRGPIO[0xffa08000] PORT[15]: IN/OUT/DIR: [1,0,0], MASK/POL/EDGE: [0,1,1], BYPASS: 0
GRGPIO[0xffa08000] PORT[14]: IN/OUT/DIR: [1,0,0], MASK/POL/EDGE: [0,1,1], BYPASS: 0
GRGPIO[0xffa08000] PORT[13]: IN/OUT/DIR: [1,0,0], MASK/POL/EDGE: [0,1,1], BYPASS: 0
GRGPIO[0xffa08000] PORT[12]: IN/OUT/DIR: [0,0,0], MASK/POL/EDGE: [0,1,1], BYPASS: 0
GRGPIO[0xffa08000] PORT[11]: IN/OUT/DIR: [0,0,0], MASK/POL/EDGE: [0,1,1], BYPASS: 0
GRGPIO[0xffa08000] PORT[10]: IN/OUT/DIR: [0,0,0], MASK/POL/EDGE: [0,1,1], BYPASS: 0
GRGPIO[0xffa08000] PORT[9]: IN/OUT/DIR: [1,0,0], MASK/POL/EDGE: [0,1,1], BYPASS: 0
GRGPIO[0xffa08000] PORT[8]: IN/OUT/DIR: [1,0,0], MASK/POL/EDGE: [0,1,1], BYPASS: 0
GRGPIO[0xffa08000] PORT[7]: IN/OUT/DIR: [0,0,0], MASK/POL/EDGE: [0,1,1], BYPASS: 0
GRGPIO[0xffa08000] PORT[6]: IN/OUT/DIR: [0,0,0], MASK/POL/EDGE: [0,1,1], BYPASS: 0
GRGPIO[0xffa08000] PORT[5]: IN/OUT/DIR: [0,0,0], MASK/POL/EDGE: [0,1,1], BYPASS: 0
GRGPIO[0xffa08000] PORT[4]: IN/OUT/DIR: [0,0,0], MASK/POL/EDGE: [0,1,1], BYPASS: 0
GRGPIO[0xffa08000] PORT[3]: IN/OUT/DIR: [0,0,0], MASK/POL/EDGE: [0,1,1], BYPASS: 0
GRGPIO[0xffa08000] PORT[2]: IN/OUT/DIR: [0,0,0], MASK/POL/EDGE: [0,1,1], BYPASS: 0
GRGPIO[0xffa08000] PORT[1]: IN/OUT/DIR: [1,0,0], MASK/POL/EDGE: [0,1,1], BYPASS: 0
GRGPIO[0xffa08000] PORT[0]: IN/OUT/DIR: [1,0,0], MASK/POL/EDGE: [0,0,0], BYPASS: 0
GRGPIO[0xff902000] PORT[15]: IN/OUT/DIR: [0,0,0], MASK/POL/EDGE: [0,1,1], BYPASS: 0
GRGPIO[0xff902000] PORT[14]: IN/OUT/DIR: [0,0,0], MASK/POL/EDGE: [0,1,1], BYPASS: 0
GRGPIO[0xff902000] PORT[13]: IN/OUT/DIR: [0,0,0], MASK/POL/EDGE: [0,1,1], BYPASS: 0
GRGPIO[0xff902000] PORT[12]: IN/OUT/DIR: [0,0,0], MASK/POL/EDGE: [0,1,1], BYPASS: 0
GRGPIO[0xff902000] PORT[11]: IN/OUT/DIR: [0,0,0], MASK/POL/EDGE: [0,1,1], BYPASS: 0
GRGPIO[0xff902000] PORT[10]: IN/OUT/DIR: [0,0,0], MASK/POL/EDGE: [0,1,1], BYPASS: 0
GRGPIO[0xff902000] PORT[9]: IN/OUT/DIR: [0,0,0], MASK/POL/EDGE: [0,1,1], BYPASS: 0
GRGPIO[0xff902000] PORT[8]: IN/OUT/DIR: [0,0,0], MASK/POL/EDGE: [0,1,1], BYPASS: 0
GRGPIO[0xff902000] PORT[7]: IN/OUT/DIR: [0,0,0], MASK/POL/EDGE: [0,1,1], BYPASS: 0
GRGPIO[0xff902000] PORT[6]: IN/OUT/DIR: [0,0,0], MASK/POL/EDGE: [0,1,1], BYPASS: 0
GRGPIO[0xff902000] PORT[5]: IN/OUT/DIR: [0,0,0], MASK/POL/EDGE: [0,1,1], BYPASS: 0
GRGPIO[0xff902000] PORT[4]: IN/OUT/DIR: [0,0,0], MASK/POL/EDGE: [0,1,1], BYPASS: 0
GRGPIO[0xff902000] PORT[3]: IN/OUT/DIR: [1,0,0], MASK/POL/EDGE: [0,1,1], BYPASS: 0
GRGPIO[0xff902000] PORT[2]: IN/OUT/DIR: [1,0,0], MASK/POL/EDGE: [0,1,1], BYPASS: 0
GRGPIO[0xff902000] PORT[1]: IN/OUT/DIR: [1,0,0], MASK/POL/EDGE: [0,1,1], BYPASS: 0
GRGPIO[0xff902000] PORT[0]: IN/OUT/DIR: [1,0,0], MASK/POL/EDGE: [0,0,0], BYPASS: 0
Failed to disable IRQ on gpio port 4, disabling port while not enabled should fail. OK
Current Value on GPIO: 1
GRGPIO[0xff902000] PORT[4]: IN/OUT/DIR: [1,0,0], MASK/POL/EDGE: [1,0,0], BYPASS: 0
GRGPIO[0xff902000] PORT[4]: IN/OUT/DIR: [1,0,0], MASK/POL/EDGE: [1,0,0], BYPASS: 0
TEST END
CORE#0 halts because of Error Mode [0.982802sec/340946551, 0, 0, 0cycles]
```

Performance statistics for CORE#0
Cycles : 340946551
Instructions : 227455650
CORE#0 performance (250MHz) : 166.78 MIPS
Simulated time : 1.36 s

Performance of laysim-gr740
Real-time performance : 138.77 %
laysim-gr740 performance : 231.44 MIPS
Wall clock : 0.98 s
PS C:\laysim-gr740\RCC-1.2.25>

4.8 Loadable UART Module

laysim-GR740 supports the loadable UART module by using '-mod MODULE' on start-up. Figure 4-17 shows the result of dhry with the

loadable UART module. The loadable UART module can be loaded with '-mod ./Loadable/loadable-UART-Module/laysim_user_uart_module.so' on start-up.

Figure 4-17. UART Simulation using loadable UART module

laysim-gr740-dbt-cli in Linux	laysim-gr740-cli in Windows
<pre>[layright@CentOS BCC-2.2.2]\$ laysim-gr740-dbt-cli -r -core0 dhry.elf -mod ./Loadable/loadable-UART-Modules/laysim_user_uart_module.so laysim for GR740 Quad Core Processor (DBT) CLI v3.0 by layright Loaded user loadable module ./Loadable/loadable-UART-Modules/laysim_user_uart_module.so Multicore control : Time quantum (500) CORE#0 RAM file is loaded - dhry.elf - .text, addr: 0x00000000, size 58544 bytes - .rodata, addr: 0x0000E4B0, size 2048 bytes - .data, addr: 0x0000ECB0, size 1576 bytes - .bss, addr: 0x0000F2D8, size 10680 bytes (not loaded, just info.) Only GR740 CORE#0 RAM file is loaded, starts from RAM Set CORE#0 %sp/%fp : 0x07FFFFFFF/0x08000000 Execution starts, 1000000 runs through Dhrystone Total execution time: 1.8 s Microseconds for one run through Dhrystone: 1.8 Dhrystones per Second: 570776.2 Dhrystones MIPS : 324.9 CORE#0 halts because of Error Mode [0.468214sec/453314025, 0, 0, 0cycles] Performance statistics for CORE#0 Cycles : 453314025 Instructions : 348377344 CORE#0 performance (250MHz) : 192.13 MIPS Simulated time : 1.81 s Performance of laysim-gr740 Real-time performance : 387.27 % laysim-gr740 performance : 744.06 MIPS Wall clock : 0.47 s [layright@CentOS BCC-2.2.2]\$</pre>	<pre>PS C:\laysim-gr740\BCC-2.2.2> laysim-gr740-dbt-cli -r -core0 dhry.elf -mod ./Loadable/loadable-UART-Modules\laysim_user_uart_module-dbt-cli.dll laysim for GR740 Quad Core Processor (DBT) CLI v3.0 by layright Loaded user loadable module ./Loadable/loadable-UART-Modules\laysim_user_uart_module-dbt-cli.dll Multicore control : Time quantum (500) CORE#0 RAM file is loaded - dhry.elf - .text, addr: 0x00000000, size 58544 bytes - .rodata, addr: 0x0000E4B0, size 2048 bytes - .data, addr: 0x0000ECB0, size 1576 bytes - .bss, addr: 0x0000F2D8, size 10680 bytes (not loaded, just info.) Only GR740 CORE#0 RAM file is loaded, starts from RAM Set CORE#0 %sp/%fp : 0x07FFFFFFF/0x08000000 Execution starts, 1000000 runs through Dhrystone Total execution time: 1.8 s Microseconds for one run through Dhrystone: 1.8 Dhrystones per Second: 570776.2 Dhrystones MIPS : 324.9 CORE#0 halts because of Error Mode [0.483601sec/453314025, 0, 0, 0cycles] Performance statistics for CORE#0 Cycles : 453314025 Instructions : 348377344 CORE#0 performance (250MHz) : 192.13 MIPS Simulated time : 1.81 s Performance of laysim-gr740 Real-time performance : 374.95 % laysim-gr740 performance : 720.38 MIPS Wall clock : 0.48 s PS C:\laysim-gr740\BCC-2.2.2></pre>

5. Shared Library

The professional version of laysim-GR740 provides as a stand-alone application as well as a set of libraries that can be integrated in an existing simulator. Detailed information can be found in [laysim-GR740-010] laysim-GR740 Shared Library - User's Manual.

5.1 Shared Library Example

app12-1.c loads rtems-rmap-path-addressing.elf to CORE#0 and loads the loadable SpW module (laysim_user_spw_node.so) and then it runs till 1056827640 nsec. After execution, it saves all states of laysim-GR740 and loadable SpW module to files. app12-2.c restores all states of laysim-GR740 and loadable SpW module from files. And then it continues the execution. Figure 5-2 shows the result of laysim-gr740-cli, laysim-gr740-dbt-cli, laysim-gr740-mdbt-cli.

Figure 5-1. Source Code – app12-1.c & app12-2.c

```

001 #include <stdio.h>
002 #include "laysim_gr740_lib_defs.h"
003 #include "laysim_gr740_lib_prot.h"
004
005 int main (void)
006 {
007     INT8 result;
008
009     /* Step 1 : Initialize laysim-GR740 environment */
010     result = laysim_gr740_lib_init (128, 8192, 250, 38400);
011
012     /* Step 2 : Load rtems-rmap-path-addressing.elf to CORE#0 */
013     result = laysim_gr740_lib_load_images ("../rtems-rmap-path-addressing.elf", NULL, NULL, NULL, NULL);
014     if (result != 0)
015     {
016         /* Step 2-1 : Exit laysim-GR740 */
017         laysim_gr740_lib_exit ();
018
019         /* Step 2-2 : Return */
020         return 0;
021     }
022
023     /* Step 3 : Load user SpW nodes */
024     #ifdef LINUX
025     result = laysim_gr740_lib_register_user_module ("../Loadable/loadable-SpW-Nodes/laysim_user_spw_node.so");
026     #else
027     #if defined DBT
028     result = laysim_gr740_lib_register_user_module ("../Loadable/loadable-SpW-Nodes/laysim_user_spw_node-dbt-cli-shared.dll");
029     #elif defined MDBT
030     result = laysim_gr740_lib_register_user_module ("../Loadable/loadable-SpW-Nodes/laysim_user_spw_node-mdbt-cli-shared.dll");
031     #else
032     result = laysim_gr740_lib_register_user_module ("../Loadable/loadable-SpW-Nodes/laysim_user_spw_node-cli-shared.dll");
033     #endif
034     #endif
035     if (result != 0)
036     {
037         /* Step 3-1 : Exit laysim-GR740 */
038         laysim_gr740_lib_exit ();
039
040         /* Step 3-2 : Return */
041         return 0;
042     }
043
044     /* Step 4 : Run with delta 264206910 cycles (1056827640nsec) */
045     result = laysim_gr740_lib_run_with_delta_cycles (264206910);
046
047     /* Step 5 : Save laysim-GR740 state */
048     result = laysim_gr740_lib_save ("app12-1");
049
050     /* Step 6 : Exit laysim-GR740 */
051     laysim_gr740_lib_exit ();
052
053     /* Step 7 : Return */
054     return 0;
055 }

```

```

001 #include <stdio.h>
002 #include "laysim_gr740_lib_defs.h"
003 #include "laysim_gr740_lib_prot.h"
004
005 int main (void)
006 {
007     INT8 result;
008
009     /* Step 1 : Initialize laysim-GR740 environment */
010     result = laysim_gr740_lib_init (128, 8192, 250, 38400);
011
012     /* Step 2 : Load user SpW nodes */
013     #ifdef LINUX
014     result = laysim_gr740_lib_register_user_module ("../Loadable/loadable-SpW-Nodes/laysim_user_spw_node.so");
015     #else
016     #if defined DBT
017     result = laysim_gr740_lib_register_user_module ("../Loadable/loadable-SpW-Nodes/laysim_user_spw_node-dbt-cli-shared.dll");
018     #elif defined MDBT
019     result = laysim_gr740_lib_register_user_module ("../Loadable/loadable-SpW-Nodes/laysim_user_spw_node-mdbt-cli-shared.dll");
020     #else
021     result = laysim_gr740_lib_register_user_module ("../Loadable/loadable-SpW-Nodes/laysim_user_spw_node-cli-shared.dll");
022     #endif

```

```

023     #endif
024     if (result != 0)
025     {
026         /* Step 2-1 : Exit laysim-GR740 */
027         laysim_gr740_lib_exit ();
028
029         /* Step 2-2 : Return */
030         return 0;
031     }
032
033     /* Step 3 : Restore laysim-GR740 state */
034     result = laysim_gr740_lib_restore ("app12-1");
035     if (result != 0)
036     {
037         /* Step 3-1 : Exit laysim-GR740 */
038         laysim_gr740_lib_exit ();
039
040         /* Step 3-2 : Return */
041         return 0;
042     }
043
044     /* Step 4 : Run */
045     result = laysim_gr740_lib_run ();
046
047     /* Step 5 : Exit laysim-GR740 */
048     laysim_gr740_lib_exit ();
049
050     /* Step 6 : Return */
051     return 0;
052 }

```

Figure 5-2. app12-1 and app12-2 Result

laysim-gr740-cli Result	laysim-gr740-dbt-cli Result
<pre> [layright@CentOS laysim-gr740-cli]\$./app12-1.exe CORE#0 RAM file is loaded - ../rtcms-rmap-path-addressing.elf - .text, addr: 0x00000000, size 230576 bytes - .rtcmsroset, addr: 0x00038480, size 176 bytes - .rtcmsstack, addr: 0x00038580, size 8192 bytes (not loaded, just info.) - .data, addr: 0x0003A580, size 8272 bytes - .bss, addr: 0x0003C600, size 171024 bytes (not loaded, just info.) - Detect RTEMS v5.1(SMP) with thread information Only GR740 CORE#0 RAM file is loaded, starts from RAM Set CORE#0 %sp/%fp : 0x07FFFFFF0/0x08000000 Loaded user loadable module ../Loadable/loadable-Spw-Nodes/laysim_user_spw_node.so #### To change configuration of this example, you can set a breakpoint #### on label <rmap_conf_label> and modify the following global variables: ## remote_base_address: 0x40000000 /* Base address on Remote. This is the address to which the data is copied. */ ## remote_dst_address: 0x42 /* SpW Destination address. */ ## remote_dst_key: 0x00/* SpW Destination Key */ ## source_src_address: 0x20 /* SpW Source address */ ## rmap_dev_idx: 0 /* This parameter chooses which initiator GRSPW2 core is used */ ## route entry { nr; /* Number of addresses in dstadr array */, dstadr[16];/* Path Addresses */} ## routetab[ROUTE_TO]={1,{3,0,0, ...}} ## routetab[ROUTE_FROM]={1,{9,0,0, ...}} ## path_addressing: 1 /* Use path addressing (!=0) or not (0) */ ## rmap_pkt_size: 992 ## rmap_data_bytes: 4096 /* Data bytes to send over RMAP */ #### RTEMS RMAP example Setting up SpaceWire router Configuring Router Hardware Configuration of SpaceWire Router: Number of SpW ports: 8 Number of AMBA ports: 4 Number of FIFO ports: 0 Timers available: YES Plug and Play available: YES MAJOR Version: 1 MINOR Version: 2 PATCH Version: 0 Current Instance ID: 19 ROUTER CFG/STS: 0x4100c013 PORT[00]: TYPE=CFG CTRL=0x01000210 STATUS=0x01000210 LINKSTATE=N/A PORT[01]: TYPE=SpW CTRL=0x2700022c STATUS=0x0000a000 LINKSTATE=Ready PORT[02]: TYPE=SpW CTRL=0x2700022c STATUS=0x0000a000 LINKSTATE=Ready PORT[03]: TYPE=SpW CTRL=0x2700022c STATUS=0x0000a000 LINKSTATE=Ready PORT[04]: TYPE=SpW CTRL=0x2700022c STATUS=0x0000a000 LINKSTATE=Ready PORT[05]: TYPE=SpW CTRL=0x2700022c STATUS=0x0000a000 LINKSTATE=Ready PORT[06]: TYPE=SpW CTRL=0x2700022c STATUS=0x0000a000 LINKSTATE=Ready PORT[07]: TYPE=SpW CTRL=0x2700022c STATUS=0x0000a000 LINKSTATE=Ready PORT[08]: TYPE=SpW CTRL=0x2700022c STATUS=0x0000a000 LINKSTATE=Ready PORT[09]: TYPE=AMBA CTRL=0x00000228 STATUS=0x40000000 LINKSTATE=N/A PORT[10]: TYPE=AMBA CTRL=0x00000228 STATUS=0x40000000 LINKSTATE=N/A PORT[11]: TYPE=AMBA CTRL=0x00000228 STATUS=0x40000000 LINKSTATE=N/A PORT[12]: TYPE=AMBA CTRL=0x00000228 STATUS=0x40000000 LINKSTATE=N/A Activating all Links/Ports PORT[01]: TYPE=SpW CTRL=0x0300002e STATUS=0x0000a000 LINK-STATE=Ready PORT[02]: TYPE=SpW CTRL=0x0300002e STATUS=0x0000a000 LINK-STATE=Ready PORT[03]: TYPE=SpW CTRL=0x0300002e STATUS=0x0000a000 LINK-STATE=Run PORT[04]: TYPE=SpW CTRL=0x0300002e STATUS=0x0000a000 LINK-STATE=Ready PORT[05]: TYPE=SpW CTRL=0x0300002e STATUS=0x0000a000 LINK-STATE=Ready PORT[06]: TYPE=SpW CTRL=0x0300002e STATUS=0x0000a000 LINK-STATE=Ready PORT[07]: TYPE=SpW CTRL=0x0300002e STATUS=0x0000a000 LINK-STATE=Ready PORT[08]: TYPE=SpW CTRL=0x0300002e STATUS=0x0000a000 LINK-STATE=Ready Configuring Router Failed to open SpW Router Initializing SpaceWire device 0 NOTE: running on SPW-ROUTER DMA SpaceWire link (no link-state available) Initializing SpaceWire device 1 NOTE: running on SPW-ROUTER DMA SpaceWire link (no link-state available) Initializing SpaceWire device 2 NOTE: running on SPW-ROUTER DMA SpaceWire link (no link-state available) Initializing SpaceWire device 3 NOTE: running on SPW-ROUTER DMA SpaceWire link (no link-state available) Test built for at least 4 AMBA ports. Note that the SpW-links will not be used. There are 4 GRSPW cores in the system. System clock: 10000 us / tick Starting GRSPW0: DMA Started Successfully NOTE: running on SPW-ROUTER DMA SpaceWire link (no link-state available) RMAP connection is working Initializing data array </pre>	<pre> [layright@CentOS laysim-gr740-dbt-cli]\$./app12-1.exe CORE#0 RAM file is loaded - ../rtcms-rmap-path-addressing.elf - .text, addr: 0x00000000, size 230576 bytes - .rtcmsroset, addr: 0x00038480, size 176 bytes - .rtcmsstack, addr: 0x00038580, size 8192 bytes (not loaded, just info.) - .data, addr: 0x0003A580, size 8272 bytes - .bss, addr: 0x0003C600, size 171024 bytes (not loaded, just info.) - Detect RTEMS v5.1(SMP) with thread information Only GR740 CORE#0 RAM file is loaded, starts from RAM Set CORE#0 %sp/%fp : 0x07FFFFFF0/0x08000000 Loaded user loadable module ../Loadable/loadable-Spw-Nodes/laysim_user_spw_node.so #### To change configuration of this example, you can set a breakpoint #### on label <rmap_conf_label> and modify the following global variables: ## remote_base_address: 0x40000000 /* Base address on Remote. This is the address to which the data is copied. */ ## remote_dst_address: 0x42 /* SpW Destination address. */ ## remote_dst_key: 0x00/* SpW Destination Key */ ## source_src_address: 0x20 /* SpW Source address */ ## rmap_dev_idx: 0 /* This parameter chooses which initiator GRSPW2 core is used */ ## route entry { nr; /* Number of addresses in dstadr array */, dstadr[16];/* Path Addresses */} ## routetab[ROUTE_TO]={1,{3,0,0, ...}} ## routetab[ROUTE_FROM]={1,{9,0,0, ...}} ## path_addressing: 1 /* Use path addressing (!=0) or not (0) */ ## rmap_pkt_size: 992 ## rmap_data_bytes: 4096 /* Data bytes to send over RMAP */ #### RTEMS RMAP example Setting up SpaceWire router Configuring Router Hardware Configuration of SpaceWire Router: Number of SpW ports: 8 Number of AMBA ports: 4 Number of FIFO ports: 0 Timers available: YES Plug and Play available: YES MAJOR Version: 1 MINOR Version: 2 PATCH Version: 0 Current Instance ID: 19 ROUTER CFG/STS: 0x4100c013 PORT[00]: TYPE=CFG CTRL=0x01000210 STATUS=0x01000210 LINKSTATE=N/A PORT[01]: TYPE=SpW CTRL=0x2700022c STATUS=0x0000a000 LINKSTATE=Ready PORT[02]: TYPE=SpW CTRL=0x2700022c STATUS=0x0000a000 LINKSTATE=Ready PORT[03]: TYPE=SpW CTRL=0x2700022c STATUS=0x0000a000 LINKSTATE=Ready PORT[04]: TYPE=SpW CTRL=0x2700022c STATUS=0x0000a000 LINKSTATE=Ready PORT[05]: TYPE=SpW CTRL=0x2700022c STATUS=0x0000a000 LINKSTATE=Ready PORT[06]: TYPE=SpW CTRL=0x2700022c STATUS=0x0000a000 LINKSTATE=Ready PORT[07]: TYPE=SpW CTRL=0x2700022c STATUS=0x0000a000 LINKSTATE=Ready PORT[08]: TYPE=SpW CTRL=0x2700022c STATUS=0x0000a000 LINKSTATE=Ready PORT[09]: TYPE=AMBA CTRL=0x00000228 STATUS=0x40000000 LINKSTATE=N/A PORT[10]: TYPE=AMBA CTRL=0x00000228 STATUS=0x40000000 LINKSTATE=N/A PORT[11]: TYPE=AMBA CTRL=0x00000228 STATUS=0x40000000 LINKSTATE=N/A PORT[12]: TYPE=AMBA CTRL=0x00000228 STATUS=0x40000000 LINKSTATE=N/A Activating all Links/Ports PORT[01]: TYPE=SpW CTRL=0x0300002e STATUS=0x0000a000 LINK-STATE=Ready PORT[02]: TYPE=SpW CTRL=0x0300002e STATUS=0x0000a000 LINK-STATE=Ready PORT[03]: TYPE=SpW CTRL=0x0300002e STATUS=0x0000a000 LINK-STATE=Run PORT[04]: TYPE=SpW CTRL=0x0300002e STATUS=0x0000a000 LINK-STATE=Ready PORT[05]: TYPE=SpW CTRL=0x0300002e STATUS=0x0000a000 LINK-STATE=Ready PORT[06]: TYPE=SpW CTRL=0x0300002e STATUS=0x0000a000 LINK-STATE=Ready PORT[07]: TYPE=SpW CTRL=0x0300002e STATUS=0x0000a000 LINK-STATE=Ready PORT[08]: TYPE=SpW CTRL=0x0300002e STATUS=0x0000a000 LINK-STATE=Ready Configuring Router Failed to open SpW Router Initializing SpaceWire device 0 NOTE: running on SPW-ROUTER DMA SpaceWire link (no link-state available) Initializing SpaceWire device 1 NOTE: running on SPW-ROUTER DMA SpaceWire link (no link-state available) Initializing SpaceWire device 2 NOTE: running on SPW-ROUTER DMA SpaceWire link (no link-state available) Initializing SpaceWire device 3 NOTE: running on SPW-ROUTER DMA SpaceWire link (no link-state available) Test built for at least 4 AMBA ports. Note that the SpW-links will not be used. There are 4 GRSPW cores in the system. System clock: 10000 us / tick Starting GRSPW0: DMA Started Successfully NOTE: running on SPW-ROUTER DMA SpaceWire link (no link-state available) RMAP connection is working Initializing data array </pre>

lwaysim-GR740-001 laysim-GR740 Quad Core Processor Emulator – User's Manual (Rev13)

Starting transmission type 1: one packet at a time

```
RMAP write cycle completed in 0.000434 seconds
RMAP read cycle completed in 0.000435 seconds
Data bytes sent: 4096, Packet size:
laysim-GR740 state saved to : app12-1.v3.0.lay
--> Total events : 1
```

```
0001 0x7fa6057e93a0 0x00000000 264260755
Spw Node state saved to : app12-1.v3.0.lay-spw
[layright@CentOS laysim-gr740-cli]$ ./app12-2.exe
Loaded user loadable module ../../Loadable/loadable-Spw-Nodes/laysim_user_spw_node.so
```

```
laysim-GR740 state is restored from : app12-1.v3.0.lay
--> Total events : 1
```

```
0001 0x7fa6057e93a0 0x00000000 264260755
- Detect RTEMS v5.1(SMP) with thread information
```

```
Spw Node state restored from : app12-1.v3.0.lay-spw92, Packet count: 5
Wrote 4096 bytes to target in 0.000434 seconds. WRITE RATE: 9211.568050 KiB/s
Read 4096 bytes from target in 0.000435 seconds. READ RATE: 9190.870467 KiB/s
Wrote 5 packets to target in 0.000434 seconds. WRITE RATE: 11514.460063 pkt/s
Read 5 packets from target in 0.000435 seconds. READ RATE: 11488.588083 pkt/s
```

EXAMPLE 1 SUCCESSFULLY COMPLETED.

Starting transmission type 2: all packets at a time

```
RMAP write cycle completed in 0.000393 seconds
RMAP read cycle completed in 0.000145 seconds
Data bytes sent: 4096, Packet size: 992, Packet count: 5
Wrote 4096 bytes to target in 0.000393 seconds. WRITE RATE: 10179.963442 KiB/s
Read 4096 bytes from target in 0.000145 seconds. READ RATE: 27554.450421 KiB/s
Wrote 5 packets to target in 0.000393 seconds. WRITE RATE: 12724.954302 pkt/s
Read 5 packets from target in 0.000145 seconds. READ RATE: 34443.063026 pkt/s
```

EXAMPLE 2 SUCCESSFULLY COMPLETED.

```
CORE#0 halts because of Error Mode [333574363cycles/4.37002sec]
[layright@CentOS laysim-gr740-cli]$
```

laysim-gr740-mdbt-cli Result

```
[layright@CentOS laysim-gr740-mdbt-cli]$ ./app12-1.exe
CORE#0 RAM file is loaded - ../rtms-rmap-path-addressing.elf
- .text, addr: 0x00000000, size 230576 bytes
- .rtmsroset, addr: 0x00038480, size 176 bytes
- .rtmsstack, addr: 0x00038580, size 8192 bytes (not loaded, just info.)
- .data, addr: 0x0003A580, size 8272 bytes
- .bss, addr: 0x0003C600, size 171024 bytes (not loaded, just info.)
- Detect RTEMS v5.1(SMP) with thread information
Only GR740 CORE#0 RAM file is loaded, starts from RAM
Set CORE#0 %sp/%fp : 0x07FFFFF0/0x00000000
Loaded user loadable module ../../Loadable/loadable-Spw-Nodes/laysim_user_spw_node.so
#### To change configuration of this example, you can set a breakpoint
#### on label <rmap_conf label> and modify the following global variables:
## remote_base_address: 0x40000000 /* Base address on Remote. This is the address to wh
ich the data is copied. */
## remote_dst_address: 0x42 /* Spw Destination address. */
## remote_dst_key: 0x00 /* Spw Destination Key */
## source_src_address: 0x20 /* Spw Source address */
## rmap_dev_idx: 0 /* This parameter chooses which initiator GRSPW2 core is used */
## route_entry { nr; /* Number of addresses in dstadr array */, dstadr[16]; /* Path Addr
esses */}
## routetab[ROUTE_TO]=1,{3,0,0,...}
## routetab[ROUTE_FROM]=1,{9,0,0,...}
## path_addressing: 1 /* Use path addressing (!=0) or not (0) */
## rmap_pkt_size: 992
## rmap_data_bytes: 4096 /* Data bytes to send over RMAP */
#### RTEMS RMAP example
```

Setting up SpaceWire router

Configuring Router

Hardware Configuration of SpaceWire Router:

```
Number of Spw ports: 8
Number of AMBA ports: 4
Number of FIFO ports: 0
Timers available: YES
Plug and Play available: YES
MAJOR Version: 1
MINOR Version: 2
PATCH Version: 0
Current Instance ID: 19
```

```
ROUTER CFG/STS: 0x4100c013
PORT[00]: TYPE=CFG CTRL=0x01000210 STATUS=0x01000210 LINKSTATE=N/A
PORT[01]: TYPE=Spw CTRL=0x2700022c STATUS=0x0000a000 LINKSTATE=Ready
PORT[02]: TYPE=Spw CTRL=0x2700022c STATUS=0x0000a000 LINKSTATE=Ready
PORT[03]: TYPE=Spw CTRL=0x2700022c STATUS=0x0000a000 LINKSTATE=Ready
PORT[04]: TYPE=Spw CTRL=0x2700022c STATUS=0x0000a000 LINKSTATE=Ready
PORT[05]: TYPE=Spw CTRL=0x2700022c STATUS=0x0000a000 LINKSTATE=Ready
PORT[06]: TYPE=Spw CTRL=0x2700022c STATUS=0x0000a000 LINKSTATE=Ready
PORT[07]: TYPE=Spw CTRL=0x2700022c STATUS=0x0000a000 LINKSTATE=Ready
PORT[08]: TYPE=Spw CTRL=0x2700022c STATUS=0x0000a000 LINKSTATE=Ready
PORT[09]: TYPE=AMBA CTRL=0x00000228 STATUS=0x40000000 LINKSTATE=N/A
PORT[10]: TYPE=AMBA CTRL=0x00000228 STATUS=0x40000000 LINKSTATE=N/A
PORT[11]: TYPE=AMBA CTRL=0x00000228 STATUS=0x40000000 LINKSTATE=N/A
PORT[12]: TYPE=AMBA CTRL=0x00000228 STATUS=0x40000000 LINKSTATE=N/A
```

Activating all Links/Ports

```
PORT[01]: TYPE=Spw CTRL=0x0300002e STATUS=0x0000a000 LINK-STATE=Ready
PORT[02]: TYPE=Spw CTRL=0x0300002e STATUS=0x0000a000 LINK-STATE=Ready
PORT[03]: TYPE=Spw CTRL=0x0300002e STATUS=0x00000000 LINK-STATE=Run
PORT[04]: TYPE=Spw CTRL=0x0300002e STATUS=0x00000000 LINK-STATE=Ready
PORT[05]: TYPE=Spw CTRL=0x0300002e STATUS=0x0000a000 LINK-STATE=Ready
PORT[06]: TYPE=Spw CTRL=0x0300002e STATUS=0x0000a000 LINK-STATE=Ready
PORT[07]: TYPE=Spw CTRL=0x0300002e STATUS=0x0000a000 LINK-STATE=Ready
PORT[08]: TYPE=Spw CTRL=0x0300002e STATUS=0x0000a000 LINK-STATE=Ready
```

Configuring Router

Failed to open Spw Router

Initializing SpaceWire device 0

NOTE: running on SPW-ROUTER DMA SpaceWire link (no link-state available)

Initializing SpaceWire device 1

NOTE: running on SPW-ROUTER DMA SpaceWire link (no link-state available)

Initializing SpaceWire device 2

NOTE: running on SPW-ROUTER DMA SpaceWire link (no link-state available)

Initializing SpaceWire device 3

NOTE: running on SPW-ROUTER DMA SpaceWire link (no link-state available)

Test built for at least 4 AMBA ports.

Note that the Spw-links will not be used.

There are 4 GRSPW cores in the system.

System clock: 10000 us / tick

Starting GRSPW0: DMA Started Successfully

NOTE: running on SPW-ROUTER DMA SpaceWire link (no link-state available)

Starting transmission type 1: one packet at a time

```
RMAP write cycle completed in 0.000411 seconds
RMAP read cycle completed in 0.000419 seconds
Data bytes sent: 4096, Packet size: 992, P
laysim-GR740 state saved to : app12-1.v3.0.dbt
--> Total events : 1
0001 0x7fb55ddb4980 0x00000000 264215201
```

```
Spw Node state saved to : app12-1.v3.0.dbt-spw
[layright@CentOS laysim-gr740-dbt-cli]$ ./app12-2.exe
Loaded user loadable module ../../Loadable/loadable-Spw-Nodes/laysim_user_spw_node.so
laysim-GR740 state is restored from : app12-1.v3.0.dbt
--> Total events : 1
0001 0x7fb55ddb4980 0x00000000 264215201
```

- Detect RTEMS v5.1(SMP) with thread information

Spw Node state restored from : app12-1.v3.0.dbt-spw

```
Wrote 4096 bytes to target in 0.000411 seconds. WRITE RATE: 9725.525898 KiB/s
Read 4096 bytes from target in 0.000419 seconds. READ RATE: 9552.820708 KiB/s
Wrote 5 packets to target in 0.000411 seconds. WRITE RATE: 12156.907372 pkt/s
Read 5 packets from target in 0.000419 seconds. READ RATE: 11941.025885 pkt/s
```

EXAMPLE 1 SUCCESSFULLY COMPLETED.

Starting transmission type 2: all packets at a time

```
RMAP write cycle completed in 0.000377 seconds
RMAP read cycle completed in 0.000129 seconds
Data bytes sent: 4096, Packet size: 992, Packet count: 5
Wrote 4096 bytes to target in 0.000377 seconds. WRITE RATE: 10613.033488 KiB/s
Read 4096 bytes from target in 0.000129 seconds. READ RATE: 30979.279400 KiB/s
Wrote 5 packets to target in 0.000377 seconds. WRITE RATE: 13266.291859 pkt/s
Read 5 packets from target in 0.000129 seconds. READ RATE: 38724.099250 pkt/s
```

EXAMPLE 2 SUCCESSFULLY COMPLETED.

```
CORE#0 halts because of Error Mode [0.056883sec/333129796, 0, 0, 0cycles]
[layright@CentOS laysim-gr740-dbt-cli]$
```

laysim-gr740-mdbt-cli (512MB) Result

```
[layright@CentOS laysim-gr740-mdbt-512-cli]$ ./app12-1.exe
CORE#0 RAM file is loaded - ../rtms-rmap-path-addressing.elf
- .text, addr: 0x00000000, size 230576 bytes
- .rtmsroset, addr: 0x00038480, size 176 bytes
- .rtmsstack, addr: 0x00038580, size 8192 bytes (not loaded, just info.)
- .data, addr: 0x0003A580, size 8272 bytes
- .bss, addr: 0x0003C600, size 171024 bytes (not loaded, just info.)
- Detect RTEMS v5.1(SMP) with thread information
Only GR740 CORE#0 RAM file is loaded, starts from RAM
Set CORE#0 %sp/%fp : 0x1FFFFFF0/0x20000000
Loaded user loadable module ../../Loadable/loadable-Spw-Nodes/laysim_user_spw_node.so
#### To change configuration of this example, you can set a breakpoint
#### on label <rmap_conf label> and modify the following global variables:
## remote_base_address: 0x40000000 /* Base address on Remote. This is the address to wh
ich the data is copied. */
## remote_dst_address: 0x42 /* Spw Destination address. */
## remote_dst_key: 0x00 /* Spw Destination Key */
## source_src_address: 0x20 /* Spw Source address */
## rmap_dev_idx: 0 /* This parameter chooses which initiator GRSPW2 core is used */
## route_entry { nr; /* Number of addresses in dstadr array */, dstadr[16]; /* Path Addr
esses */}
## routetab[ROUTE_TO]=1,{3,0,0,...}
## routetab[ROUTE_FROM]=1,{9,0,0,...}
## path_addressing: 1 /* Use path addressing (!=0) or not (0) */
## rmap_pkt_size: 992
## rmap_data_bytes: 4096 /* Data bytes to send over RMAP */
#### RTEMS RMAP example
```

Setting up SpaceWire router

Configuring Router

Hardware Configuration of SpaceWire Router:

```
Number of Spw ports: 8
Number of AMBA ports: 4
Number of FIFO ports: 0
Timers available: YES
Plug and Play available: YES
MAJOR Version: 1
MINOR Version: 2
PATCH Version: 0
Current Instance ID: 19
```

```
ROUTER CFG/STS: 0x4100c013
PORT[00]: TYPE=CFG CTRL=0x01000210 STATUS=0x01000210 LINKSTATE=N/A
PORT[01]: TYPE=Spw CTRL=0x2700022c STATUS=0x0000a000 LINKSTATE=Ready
PORT[02]: TYPE=Spw CTRL=0x2700022c STATUS=0x0000a000 LINKSTATE=Ready
PORT[03]: TYPE=Spw CTRL=0x2700022c STATUS=0x0000a000 LINKSTATE=Ready
PORT[04]: TYPE=Spw CTRL=0x2700022c STATUS=0x0000a000 LINKSTATE=Ready
PORT[05]: TYPE=Spw CTRL=0x2700022c STATUS=0x0000a000 LINKSTATE=Ready
PORT[06]: TYPE=Spw CTRL=0x2700022c STATUS=0x0000a000 LINKSTATE=Ready
PORT[07]: TYPE=Spw CTRL=0x2700022c STATUS=0x0000a000 LINKSTATE=Ready
PORT[08]: TYPE=Spw CTRL=0x2700022c STATUS=0x0000a000 LINKSTATE=Ready
PORT[09]: TYPE=AMBA CTRL=0x00000228 STATUS=0x40000000 LINKSTATE=N/A
PORT[10]: TYPE=AMBA CTRL=0x00000228 STATUS=0x40000000 LINKSTATE=N/A
PORT[11]: TYPE=AMBA CTRL=0x00000228 STATUS=0x40000000 LINKSTATE=N/A
PORT[12]: TYPE=AMBA CTRL=0x00000228 STATUS=0x40000000 LINKSTATE=N/A
```

Activating all Links/Ports

```
PORT[01]: TYPE=Spw CTRL=0x0300002e STATUS=0x0000a000 LINK-STATE=Ready
PORT[02]: TYPE=Spw CTRL=0x0300002e STATUS=0x0000a000 LINK-STATE=Ready
PORT[03]: TYPE=Spw CTRL=0x0300002e STATUS=0x00000000 LINK-STATE=Run
PORT[04]: TYPE=Spw CTRL=0x0300002e STATUS=0x00000000 LINK-STATE=Ready
PORT[05]: TYPE=Spw CTRL=0x0300002e STATUS=0x0000a000 LINK-STATE=Ready
PORT[06]: TYPE=Spw CTRL=0x0300002e STATUS=0x0000a000 LINK-STATE=Ready
PORT[07]: TYPE=Spw CTRL=0x0300002e STATUS=0x0000a000 LINK-STATE=Ready
PORT[08]: TYPE=Spw CTRL=0x0300002e STATUS=0x0000a000 LINK-STATE=Ready
```

Configuring Router

Failed to open Spw Router

Initializing SpaceWire device 0

NOTE: running on SPW-ROUTER DMA SpaceWire link (no link-state available)

Initializing SpaceWire device 1

NOTE: running on SPW-ROUTER DMA SpaceWire link (no link-state available)

Initializing SpaceWire device 2

NOTE: running on SPW-ROUTER DMA SpaceWire link (no link-state available)

Initializing SpaceWire device 3

NOTE: running on SPW-ROUTER DMA SpaceWire link (no link-state available)

Test built for at least 4 AMBA ports.

Note that the Spw-links will not be used.

There are 4 GRSPW cores in the system.

System clock: 10000 us / tick

Starting GRSPW0: DMA Started Successfully

NOTE: running on SPW-ROUTER DMA SpaceWire link (no link-state available)

RMAP connection is working
Initializing data array

Starting transmission type 1: one packet at a time

RMAP write cycle completed in 0.000411 seconds
RMAP read cycle completed in 0.000419 seconds
Data bytes sent: 4096, Packet size: 992, P
laysim-GR740 state saved to : app12-1.v3.0.mdbt
--> Total events : 1
0001 0x7f2371b79870 0x00000000 264215201

SpW Node state saved to : app12-1.v3.0.mdbt-spw
[layright@CentOS laysim-gr740-mdbt-cli]\$./app12-2.exe
Loaded user loadable module ../../Loadable/loadable-SpW-Nodes/laysim_user_spw_node.so
laysim-GR740 state is restored from : app12-1.v3.0.mdbt
--> Total events : 1
0001 0x7f2371b79870 0x00000000 264215201
- Detect RTEMS v5.1(SMP) with thread information

SpW Node state restored from : app12-1.v3.0.mdbt-spwacket count: 5
Wrote 4096 bytes to target in 0.000411 seconds. WRITE RATE: 9725.525898 KiB/s
Read 4096 bytes from target in 0.000419 seconds. READ RATE: 9552.820708 KiB/s
Wrote 5 packets to target in 0.000411 seconds. WRITE RATE: 12156.907372 pkt/s
Read 5 packets from target in 0.000419 seconds. READ RATE: 11941.025885 pkt/s

EXAMPLE 1 SUCCESSFULLY COMPLETED.

Starting transmission type 2: all packets at a time

RMAP write cycle completed in 0.000377 seconds
RMAP read cycle completed in 0.000129 seconds
Data bytes sent: 4096, Packet size: 992, Packet count: 5
Wrote 4096 bytes to target in 0.000377 seconds. WRITE RATE: 10613.033488 KiB/s
Read 4096 bytes from target in 0.000129 seconds. READ RATE: 30979.279400 KiB/s
Wrote 5 packets to target in 0.000377 seconds. WRITE RATE: 13266.291859 pkt/s
Read 5 packets from target in 0.000129 seconds. READ RATE: 38724.099250 pkt/s

EXAMPLE 2 SUCCESSFULLY COMPLETED.

CORE#0 halts because of Error Mode [0.058724sec/333129796, 0, 0, 0cycles]
[layright@CentOS laysim-gr740-mdbt-cli]\$

RMAP connection is working
Initializing data array

Starting transmission type 1: one packet at a time

RMAP write cycle completed in 0.000411 seconds
RMAP read cycle completed in 0.000419 seconds
Data bytes sent: 4096, Packet size: 992, P
laysim-GR740 state saved to : app12-1.v3.0.mdbt
--> Total events : 1
0001 0x7f6ea1c17870 0x00000000 264215201

SpW Node state saved to : app12-1.v3.0.mdbt-spw
[layright@CentOS laysim-gr740-mdbt-512-cli]\$./app12-2.exe
Loaded user loadable module ../../Loadable/loadable-SpW-Nodes/laysim_user_spw_node.so
laysim-GR740 state is restored from : app12-1.v3.0.mdbt
--> Total events : 1
0001 0x7f6ea1c17870 0x00000000 264215201
- Detect RTEMS v5.1(SMP) with thread information

SpW Node state restored from : app12-1.v3.0.mdbt-spwacket count: 5
Wrote 4096 bytes to target in 0.000411 seconds. WRITE RATE: 9725.525898 KiB/s
Read 4096 bytes from target in 0.000419 seconds. READ RATE: 9552.820708 KiB/s
Wrote 5 packets to target in 0.000411 seconds. WRITE RATE: 12156.907372 pkt/s
Read 5 packets from target in 0.000419 seconds. READ RATE: 11941.025885 pkt/s

EXAMPLE 1 SUCCESSFULLY COMPLETED.

Starting transmission type 2: all packets at a time

RMAP write cycle completed in 0.000377 seconds
RMAP read cycle completed in 0.000129 seconds
Data bytes sent: 4096, Packet size: 992, Packet count: 5
Wrote 4096 bytes to target in 0.000377 seconds. WRITE RATE: 10613.033488 KiB/s
Read 4096 bytes from target in 0.000129 seconds. READ RATE: 30979.279400 KiB/s
Wrote 5 packets to target in 0.000377 seconds. WRITE RATE: 13266.291859 pkt/s
Read 5 packets from target in 0.000129 seconds. READ RATE: 38724.099250 pkt/s

EXAMPLE 2 SUCCESSFULLY COMPLETED.

CORE#0 halts because of Error Mode [0.057025sec/333129796, 0, 0, 0cycles]
[layright@CentOS laysim-gr740-mdbt-512-cli]\$

6. GDB Operation

The CLI version of laysim-GR740 supports debugging with the GDB. When attached to the GDB, laysim-GR740 acts as a remote GDB target. Applications are loaded and debugged through the GDB. To initiate GDB communications, start the CLI version of laysim-GR740 with the -gdb start-up option or use the gdb command in the control console. Detailed information can be found in [laysim-GR740-012] Debugging with GDB on laysim-GR740.

6.1 Using Extended-Remote Protocol

The CLI version of laysim-GR740 supports the extended-remote protocol of GDB, so it is possible to run RTEMS 5.1 SMP examples using 'run' command in GDB.

Figure 6-1. Debugging with GDB #1 on laysim-GR740

laysim-gr740-cli Result	
<pre>[layright@CentOS smptests]\$ laysim-gr740-cli -gdb laysim for GR740 Quad Core Processor CLI v3.0 by layright --> No CORE#0 RAM file is loaded GDB interface: using port 1234 Starting GDB server. Use Ctrl-C to stop waiting for connection GDB connected at IP: 127.0.0.1 and port: 36916 *** BEGIN OF TEST SMP 9 *** *** TEST VERSION: 5.1.0 *** TEST STATE: EXPECTED_PASS *** TEST BUILD: RTEMS_NETWORKING RTEMS_POSIX_API RTEMS_SMP *** TEST TOOLS: 7.5.0 20191114 (RTEMS 5, RSB 5.1, Newlib 7947581) CPU 3 start task TA1 CPU 3 start task TA2 CPU 3 start task TA3 kill 10 clock ticks ----- CPU USAGE BY THREAD ----- ID NAME SECONDS PERCENT ----- 0x09010001 IDLE 1.116508 91.433 0x09010002 IDLE 1.110264 89.380 0x09010003 IDLE 1.104028 87.395 0x09010004 IDLE 1.000046 77.865 0x0a010001 UI1 0.305362 23.392 0x0a010002 TA1 0.222456 16.770 0x0a010003 TA2 0.237297 17.609 0x0a010004 TA3 0.252129 18.421 ----- TIME SINCE LAST CPU USAGE RESET IN SECONDS: 1.368638 *** END OF TEST SMP 9 *** CORE#3 halts because of Error Mode [13471568cycles/1.50054sec]</pre>	<pre>[layright@CentOS smptests]\$ sparc-gaisler-elf-gdb smp09.exe GNU gdb (GDB) 8.2.1 Copyright (C) 2018 Free Software Foundation, Inc. License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html> This is free software: you are free to change and redistribute it. There is NO WARRANTY, to the extent permitted by law. Type "show copying" and "show warranty" for details. This GDB was configured as "--host=x86_64-pc-linux-gnu --target=sparc-gaisler-elf". Type "show configuration" for configuration details. For bug reporting instructions, please see: <http://www.gnu.org/software/gdb/bugs/>. Find the GDB manual and other documentation resources online at: <http://www.gnu.org/software/gdb/documentation/>. For help, type "help". Type "apropos word" to search for commands related to "word"... Reading symbols from smp09.exe...done. (gdb) tar extended-remote localhost:1234 Remote debugging using localhost:1234 0xc0000000 in ?? () (gdb) load Loading section .text, size 0x1bea0 lma 0x0 Loading section .rtmsroset, size 0x90 lma 0x1bea0 Loading section .data, size 0x530 lma 0x23f40 Start address 0x0, load size 115808 Transfer rate: 113093 KB/sec, 6812 bytes/write. (gdb) b Init Breakpoint 1 at 0x12b0: file ../../../../../../rtems/c/src/../../../../testsuites/smptests/smp09/init.c, line 42. (gdb) run The program being debugged has been started already. Start it from the beginning? (y or n) y Starting program: /home/layright/rtems/RTEMS-dev/RTEMS5-builder/b-rtems/sparc-rtems5/c/gr740/testsuites/smptests/smp09.exe [New Thread 4] [New Thread 2] [New Thread 3] [Switching to Thread 4] Thread 2 hit Breakpoint 1, Init (argument=164992) at ../../../../../../rtems/c/src/../../../../testsuites/smptests/smp09/init.c:42 42 { (gdb) cont Continuing. Thread 2 received signal SIGTERM, Terminated. syscall () at ../../../../../../rtems/c/src/../../../../cpukit/score/cpu/sparc/syscall.S:44 44 ta 0 ! syscall 1, halt with %g1,%g2,%g3 info (gdb) info thread Id Target Id Frame 1 Thread 1 (CORE#0 [running]) dont_do_the_window () at ../../../../../../rtems/c/src/../../../../cpukit/score/cpu/sparc/cpu_asm.S:480 * 2 Thread 4 (CORE#3 [error]) syscall () at ../../../../../../rtems/c/src/../../../../cpukit/score/cpu/sparc/syscall.S:44 3 Thread 2 (CORE#1 [running]) dont_do_the_window () at ../../../../../../rtems/c/src/../../../../cpukit/score/cpu/sparc/cpu_asm.S:448 4 Thread 3 (CORE#2 [running]) win_overflow () at ../../../../../../rtems/c/src/../../../../cpukit/score/cpu/sparc/cpu_asm.S:355 (gdb) thread 1 [Switching to thread 1 (Thread 1)] #0 dont_do_the_window () at ../../../../../../rtems/c/src/../../../../cpukit/score/cpu/sparc/cpu_asm.S:480 480 st %l7, [%g6 + PER_CPU_ISR_NEST_LEVEL] (gdb) bt #0 dont_do_the_window () at ../../../../../../rtems/c/src/../../../../cpukit/score/cpu/sparc/cpu_asm.S:480 #1 0x00016994 in pil_fixed () at ../../../../../../rtems/c/src/../../../../cpukit/score/cpu/sparc/cpu_asm.S:592 Backtrace stopped: previous frame identical to this frame (corrupt stack?) (gdb) thread 2 [Switching to thread 2 (Thread 4)] #0 syscall () at ../../../../../../rtems/c/src/../../../../cpukit/score/cpu/sparc/syscall.S:44 44 ta 0 ! syscall 1, halt with %g1,%g2,%g3 info (gdb) bt #0 syscall () at ../../../../../../rtems/c/src/../../../../cpukit/score/cpu/sparc/syscall.S:44 #1 0x0000d850 in _User_extensions_Fatal_visitor (executing=0x5 <trap_table+5>, arg=0x0 <trap_table>, callouts=0x241a0 <_User_extensions_List+4>)</pre>

<pre> [layright@CentOS smptests]\$ laysim-gr740-dbt-cli -gdb laysim for GR740 Quad Core Processor (DBT) CLI v3.0 by layright Multicore control : Time quantum (500) --> No CORE#0 RAM file is loaded GDB interface: using port 1234 Starting GDB server. Use Ctrl-C to stop waiting for connection GDB connected at IP: 127.0.0.1 and port: 36918 *** BEGIN OF TEST SMP 9 *** *** TEST VERSION: 5.1.0 *** TEST STATE: EXPECTED PASS *** TEST BUILD: RTEMS_NETWORKING RTEMS_POSIX_API RTEMS_SMP *** TEST TOOLS: 7.5.0 20191114 (RTEMS 5, RSB 5.1, Newlib 7947581) CPU 3 start task TA1 CPU 3 start task TA2 CPU 3 start task TA3 kill 10 clock ticks ----- CPU USAGE BY THREAD ----- ID NAME SECONDS PERCENT ----- 0x09010001 IDLE 1.112360 91.404 0x09010002 IDLE 1.106116 89.344 0x09010003 IDLE 1.099869 87.352 0x09010004 IDLE 1.000013 78.114 0x0a010001 UI1 0.301250 23.150 0x0a010002 TA1 0.222469 16.823 0x0a010003 TA2 0.237299 17.663 0x0a010004 TA3 0.252131 18.478 ----- TIME SINCE LAST CPU USAGE RESET IN SECONDS: 1.364491 ----- *** END OF TEST SMP 9 *** CORE#3 halts because of Error Mode [0.123641sec/130991071, 130991100, 130991095, 130990 626cycles] </pre>	<pre> at ../../../../../../rtems/c/src/../../../../cpukit/score/src/userextiterate.c:131 #2 0x0001a02c in _CPU_Fatal_halt (source=source@entry=5, error=error@entry=0) at ../../../../../../rtems/c/src/lib/libbsp/sparc/leon3/../../../../bsp s/sparc/leon3/start/bsp_fatal.halt.c:33 #3 0x000db30 in Terminate (the_source=the_source@entry=RTEMS_FATAL_SOURCE_EXIT, the_error=the_error@entry=0) at ../../../../../../rtems/c/src/../../../../cpukit/score/src/interr.c:45 #4 0x0001a1f4 in rtems_shutdown_executive (result=result@entry=0) at ../../../../../../rtems/c/src/../../../../cpukit/sapi/src/exshutdown.c:21 #5 0x00019734 in rtems_test_exit (status=status@entry=0) at ../../../../../../rtems/c/src/../../../../cpukit/libtest/testbeginend.c:86 #6 0x0001474 in Init (argument=<optimized out>) at ../../../../../../rtems/c/src/../../../../testsuites/smptests/smp09/init.c:83 #7 0x000a0a3c in Thread_Entry_adaptor_numeric (executing=0x24680 <_RTEMS_tasks_Objects>) at ../../../../../../rtems/c/src/../../../../cpukit/score/src/threadentryadaptnumeric.c:25 #8 0x0009b68 in Thread_Handler () at ../../../../../../rtems/c/src/../../../../cpukit/score/src/threadhandler.c:139 #9 0x0009a80 in Thread_Handler () at ../../../../../../rtems/c/src/../../../../cpukit/score/src/threadhandler.c:83 (gdb) thread 3 [Switching to thread 3 (Thread 2)] #0 dont_do_the_window () at ../../../../../../rtems/c/src/../../../../cpukit/score/cpu/sparc/cpu_asm.S:448 448 st %g7, [%sp + ISF_G7_OFFSET] ! save g7 (gdb) bt #0 dont_do_the_window () at ../../../../../../rtems/c/src/../../../../cpukit/score/cpu/sparc/cpu_asm.S:448 #1 0x00016994 in pil_fixed () at ../../../../../../rtems/c/src/../../../../cpukit/score/cpu/sparc/cpu_asm.S:592 Backtrace stopped: previous frame identical to this frame (corrupt stack?) (gdb) thread 4 [Switching to thread 4 (Thread 3)] #0 win_overflow () at ../../../../../../rtems/c/src/../../../../cpukit/score/cpu/sparc/cpu_asm.S:355 355 mov %g4, %l4 ! save the globals this block us es (gdb) bt #0 win_overflow () at ../../../../../../rtems/c/src/../../../../cpukit/score/cpu/sparc/cpu_asm.S:355 #1 0x00016994 in pil_fixed () at ../../../../../../rtems/c/src/../../../../cpukit/score/cpu/sparc/cpu_asm.S:592 Backtrace stopped: previous frame identical to this frame (corrupt stack?) (gdb) </pre>
laysim-gr740-dbt-cli Result <pre> [layright@CentOS smptests]\$ laysim-gr740-dbt-cli -gdb laysim for GR740 Quad Core Processor (DBT) CLI v3.0 by layright Multicore control : Time quantum (500) --> No CORE#0 RAM file is loaded GDB interface: using port 1234 Starting GDB server. Use Ctrl-C to stop waiting for connection GDB connected at IP: 127.0.0.1 and port: 36918 *** BEGIN OF TEST SMP 9 *** *** TEST VERSION: 5.1.0 *** TEST STATE: EXPECTED PASS *** TEST BUILD: RTEMS_NETWORKING RTEMS_POSIX_API RTEMS_SMP *** TEST TOOLS: 7.5.0 20191114 (RTEMS 5, RSB 5.1, Newlib 7947581) CPU 3 start task TA1 CPU 3 start task TA2 CPU 3 start task TA3 kill 10 clock ticks ----- CPU USAGE BY THREAD ----- ID NAME SECONDS PERCENT ----- 0x09010001 IDLE 1.112360 91.404 0x09010002 IDLE 1.106116 89.344 0x09010003 IDLE 1.099869 87.352 0x09010004 IDLE 1.000013 78.114 0x0a010001 UI1 0.301250 23.150 0x0a010002 TA1 0.222469 16.823 0x0a010003 TA2 0.237299 17.663 0x0a010004 TA3 0.252131 18.478 ----- TIME SINCE LAST CPU USAGE RESET IN SECONDS: 1.364491 ----- *** END OF TEST SMP 9 *** CORE#3 halts because of Error Mode [0.123641sec/130991071, 130991100, 130991095, 130990 626cycles] </pre>	<pre> [layright@CentOS smptests]\$ sparc-gaisler-elf-gdb smp09.exe GNU gdb (GDB) 8.2.1 Copyright (C) 2018 Free Software Foundation, Inc. License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html> This is free software: you are free to change and redistribute it. There is NO WARRANTY, to the extent permitted by law. Type "show copying" and "show warranty" for details. This GDB was configured as "--host=x86_64-pc-linux-gnu --target=sparc-gaisler-elf". Type "show configuration" for configuration details. For bug reporting instructions, please see: <http://www.gnu.org/software/gdb/bugs/>. Find the GDB manual and other documentation resources online at: <http://www.gnu.org/software/gdb/documentation/>. For help, type "help". Type "apropos word" to search for commands related to "word"... Reading symbols from smp09.exe...done. (gdb) tar extended-remote localhost:1234 Remote debugging using localhost:1234 0xc0000000 in ?? () (gdb) load loading section .text, size 0x1bea0 lma 0x0 Loading section .rtemsreset, size 0x90 lma 0x1bea0 Loading section .data, size 0x530 lma 0x23f40 Start address 0x0, load size 115808 Transfer rate: 113093 KB/sec, 6812 bytes/write. (gdb) b Init Breakpoint 1 at 0x12b0: file ../../../../../../rtems/c/src/../../../../testsuites/smptests/sm p09/init.c, line 42. (gdb) run The program being debugged has been started already. Start it from the beginning? (y or n) y Starting program: /home/layright/rtems/RTEMS-dev/RTEMS5-builder/b-rtems/sparc-rtems5/c/ gr740/testsuites/smptests/smp09.exe [New Thread 4] [New Thread 2] [New Thread 3] [Switching to Thread 4] Thread 2 hit Breakpoint 1, Init (argument=164992) at ../../../../../../rtems/c/src/../../../../testsuites/smptests/smp09/init.c:42 42 { (gdb) cont Continuing. Thread 2 received signal SIGTERM, Terminated. syscall () at ../../../../../../rtems/c/src/../../../../cpukit/score/cpu/sparc/syscall.S:44 44 ta 0 ! syscall 1, halt with %g1,%g2,%g3 info (gdb) info thread Id Target Id Frame 1 Thread 1 (CORE#0 [running]) leon3_power_down_loop () at ../../../../../../rtems/c/src/lib/libbsp/sparc/leon3/../../../../bsp s/sparc/leon3/start/bspidle.S:26 * 2 Thread 4 (CORE#3 [error]) syscall () at ../../../../../../rtems/c/src/../../../../cpukit/score/cpu/sparc/syscall.S:44 3 Thread 2 (CORE#1 [running]) leon3_power_down_loop () at ../../../../../../rtems/c/src/lib/libbsp/sparc/leon3/../../../../bsp s/sparc/leon3/start/bspidle.S:26 4 Thread 3 (CORE#2 [running]) leon3_power_down_loop () at ../../../../../../rtems/c/src/lib/libbsp/sparc/leon3/../../../../bsp s/sparc/leon3/start/bspidle.S:26 (gdb) thread 1 [Switching to thread 1 (Thread 1)] #0 leon3_power_down_loop () at ../../../../../../rtems/c/src/lib/libbsp/sparc/leon3/../../../../bsp s/sparc/leon3/start/bspidle.S:26 26 lda [%sp] 1, %g0 ! Needed for UT699 and GR712 (gdb) bt #0 leon3 power down loop () </pre>

```

    at ../../../../../../rtems/c/src/lib/libbsp/sparc/leon3/../../../../bsp
s/sparc/leon3/start/bspidle.S:26
#1 0x000018cc in bsp_fatal_extension (source=RTEMS_FATAL_SOURCE_SMP,
    always_set_to_false=<optimized out>, code=5)
    at ../../../../../../rtems/c/src/lib/libbsp/sparc/leon3/../../../../bsp
s/sparc/leon3/start/bspclean.c:41
#2 0x0000d850 in _User_extensions_Fatal_visitor (
    executing=0x25550 <_RTEMS_tasks_Objects+3792>,
    arg=0x20e78 <_ISR_Stack_area_begin+3896>,
    callouts=0x1af58 <_User_extensions_Initial_extensions+72>)
    at ../../../../../../rtems/c/src/../../../../cpukit/score/src/userextiterate.c:131
#3 0x0000d8c8 in _User_extensions_Iterate (
    arg=arg@entry=0x20e78 <_ISR_Stack_area_begin+3896>,
    visitor=0xd82c <_User_extensions_Fatal_visitor>,
    direction=direction@entry=CHAIN_ITERATOR_FORWARD)
    at ../../../../../../rtems/c/src/../../../../cpukit/score/src/userextiterate.c:173
#4 0x0000db04 in _User_extensions_Fatal (error=5,
    source=RTEMS_FATAL_SOURCE_SMP)
    at /home/layright/RTEMS-dev/RTEMS5-builder/rtems/cpukit/include/rtems/score/userext
impl.h:445
#5 _Terminate (the_source=the_source@entry=RTEMS_FATAL_SOURCE_SMP,
    the_error=the_error@entry=5)
    at ../../../../../../rtems/c/src/../../../../cpukit/score/src/interr.c:36
#6 0x00002364 in _SMP_Fatal (code=SMP_FATAL_SHUTDOWN_RESPONSE)
    at /home/layright/RTEMS-dev/RTEMS5-builder/rtems/cpukit/include/rtems/score/smpimp
l.h:74
#7 _SMP_Inter_processor_interrupt_handler (
    cpu_self=0x27a00 <_Per_CPU_Information>)
    at /home/layright/RTEMS-dev/RTEMS5-builder/rtems/cpukit/include/rtems/score/smpimp
l.h:158
#8 bsp_inter_processor_interrupt (vector=<optimized out>)
    at ../../../../../../rtems/c/src/lib/libbsp/sparc/leon3/../../../../bsp
s/sparc/leon3/start/bspssp.c:35
#9 0x00016994 in pil_fixed ()
    at ../../../../../../rtems/c/src/../../../../cpukit/score/cpu/sparc/cpu_asm.S:592
Backtrace stopped: previous frame identical to this frame (corrupt stack?)
(gdb) thread 2
[Switching to thread 2 (Thread 4)]
#0 syscall ()
    at ../../../../../../rtems/c/src/../../../../cpukit/score/cpu/sparc/syscall.S:44
44      ta      0
! syscall 1, halt with %g1,%g2,%g3 info
(gdb) bt
#0 syscall ()
    at ../../../../../../rtems/c/src/../../../../cpukit/score/cpu/sparc/syscall.S:44
#1 0x0000d850 in _User_extensions_Fatal_visitor (
    executing=0x5 <trap_table+5>, arg=0x0 <trap_table>,
    callouts=0x241a0 <_User_extensions_List+4>)
    at ../../../../../../rtems/c/src/../../../../cpukit/score/src/userextiterate.c:131
#2 0x0001a02c in _CPU_Fatal_halt (source=source@entry=5,
    error=error@entry=0)
    at ../../../../../../rtems/c/src/lib/libbsp/sparc/leon3/../../../../bsp
s/sparc/leon3/start/bsp_fatal_halt.c:33
#3 0x0000db30 in _Terminate (
    the_source=the_source@entry=RTEMS_FATAL_SOURCE_EXIT,
    the_error=the_error@entry=0)
    at ../../../../../../rtems/c/src/../../../../cpukit/score/src/interr.c:45
#4 0x0001a1f4 in rtems_shutdown_executive (result=result@entry=0)
    at ../../../../../../rtems/c/src/../../../../cpukit/sapi/src/exshutdown.c:21
#5 0x00019734 in rtems_test_exit (status=status@entry=0)
    at ../../../../../../rtems/c/src/../../../../cpukit/libtest/testbeginend.c:86
#6 0x00001474 in Init (argument=<optimized out>)
    at ../../../../../../rtems/c/src/../../../../testsuites/smptests/smp09/init.c:83
#7 0x0000a83c in _Thread_Entry_adaptor_numeric (
    executing=0x24600 <_RTEMS_tasks_Objects>)
    at ../../../../../../rtems/c/src/../../../../cpukit/score/src/threadentryadaptornumeric.c:25
#8 0x00009b68 in _Thread_Handler ()
    at ../../../../../../rtems/c/src/../../../../cpukit/score/src/threadhandler.c:139
#9 0x00009a80 in _Thread_Handler ()
    at ../../../../../../rtems/c/src/../../../../cpukit/score/src/threadhandler.c:83
(gdb) thread 3
[Switching to thread 3 (Thread 2)]
#0 leon3_power_down_loop ()
    at ../../../../../../rtems/c/src/lib/libbsp/sparc/leon3/../../../../bsp
s/sparc/leon3/start/bspidle.S:26
26      lda    [%sp] 1, %g0      ! Needed for UT699 and GR712
(gdb) bt
#0 leon3_power_down_loop ()
    at ../../../../../../rtems/c/src/lib/libbsp/sparc/leon3/../../../../bsp
s/sparc/leon3/start/bspidle.S:26
#1 0x000018cc in bsp_fatal_extension (source=RTEMS_FATAL_SOURCE_SMP,
    always_set_to_false=<optimized out>, code=5)
    at ../../../../../../rtems/c/src/lib/libbsp/sparc/leon3/../../../../bsp
s/sparc/leon3/start/bspclean.c:41
#2 0x0000d850 in _User_extensions_Fatal_visitor (
    executing=0x25060 <_RTEMS_tasks_Objects+2528>,
    arg=0x21e78 <_ISR_Stack_area_begin+7992>,
    callouts=0x1af58 <_User_extensions_Initial_extensions+72>)
    at ../../../../../../rtems/c/src/../../../../cpukit/score/src/userextiterate.c:131
#3 0x0000d8c8 in _User_extensions_Iterate (
    arg=arg@entry=0x21e78 <_ISR_Stack_area_begin+7992>,
    visitor=0xd82c <_User_extensions_Fatal_visitor>,
    direction=direction@entry=CHAIN_ITERATOR_FORWARD)
    at ../../../../../../rtems/c/src/../../../../cpukit/score/src/userextiterate.c:173
#4 0x0000db04 in _User_extensions_Fatal (error=5,
    source=RTEMS_FATAL_SOURCE_SMP)
    at /home/layright/RTEMS-dev/RTEMS5-builder/rtems/cpukit/include/rtems/score/userext
impl.h:445
#5 _Terminate (the_source=the_source@entry=RTEMS_FATAL_SOURCE_SMP,
    the_error=the_error@entry=5)
    at ../../../../../../rtems/c/src/../../../../cpukit/score/src/interr.c:36
#6 0x00002364 in _SMP_Fatal (code=SMP_FATAL_SHUTDOWN_RESPONSE)
    at /home/layright/RTEMS-dev/RTEMS5-builder/rtems/cpukit/include/rtems/score/smpimp
l.h:74
#7 _SMP_Inter_processor_interrupt_handler (
    cpu_self=0x27c00 <_Per_CPU_Information+512>)
    at /home/layright/RTEMS-dev/RTEMS5-builder/rtems/cpukit/include/rtems/score/smpimp
l.h:158
#8 bsp_inter_processor_interrupt (vector=<optimized out>)
    at ../../../../../../rtems/c/src/lib/libbsp/sparc/leon3/../../../../bsp
s/sparc/leon3/start/bspssp.c:35
#9 0x00016994 in pil_fixed ()
    at ../../../../../../rtems/c/src/../../../../cpukit/score/cpu/sparc/cpu_asm.S:592
Backtrace stopped: previous frame identical to this frame (corrupt stack?)
(gdb) thread 4
[Switching to thread 4 (Thread 3)]
#0 leon3_power_down_loop ()
    at ../../../../../../rtems/c/src/lib/libbsp/sparc/leon3/../../../../bsp
s/sparc/leon3/start/bspidle.S:26
26      lda    [%sp] 1, %g0      ! Needed for UT699 and GR712

```

<pre>(gdb) bt #0 leon3_power_down_loop () at ../../../../../../rtmcs/c/src/lib/libbsp/sparc/leon3/../../../../bsp s/sparc/leon3/start/bspidle.S:26 #1 0x000018cc in bsp_fatal_extension (source=RTMS_FATAL_SOURCE_SMP, always_set_to_false=optimized out, code=5) at ../../../../../../rtmcs/c/src/lib/libbsp/sparc/leon3/../../../../bsp s/sparc/leon3/start/bspclean.c:41 #2 0x0000d850 in _User_extensions_Fatal_visitor (executing=0x24b70 <RTMS_tasks_Objects+1264>, arg=0x22e78 <_ISR_Stack_area_begin+12088>, callouts=0x1af58 <_User_extensions_Initial_extensions+72>) at ../../../../../../rtmcs/c/src/../../../../cpukit/score/src/userextiterate.c:131 #3 0x0000db04 in _User_extensions_Iterate (arg=arg@entry=0x22e78 <_ISR_Stack_area_begin+12088>, visitor=0xd82c <_User_extensions_Fatal_visitor>, direction=direction@entry=CHAIN_ITERATOR_FORWARD) at ../../../../../../rtmcs/c/src/../../../../cpukit/score/src/userextiterate.c:173 #4 0x0000db04 in _User_extensions_Fatal (error=5, source=RTMS_FATAL_SOURCE_SMP) at /home/layright/RTMS-dev/RTMS5-builder/rtmcs/cpukit/include/rtmcs/score/userext impl.h:445 #5 _Terminate (the_source=the_source@entry=RTMS_FATAL_SOURCE_SMP, the_error=the_error@entry=5) at ../../../../../../rtmcs/c/src/../../../../cpukit/score/src/interr.c:36 #6 0x00002364 in _SMP_Fatal (code=SMP_FATAL_SHUTDOWN_RESPONSE) at /home/layright/RTMS-dev/RTMS5-builder/rtmcs/cpukit/include/rtmcs/score/smpimp l.h:74 #7 _SMP_Inter_processor_interrupt_handler (cpu_self=0x27e00 <Per_CPU_Information+1024>) at /home/layright/RTMS-dev/RTMS5-builder/rtmcs/cpukit/include/rtmcs/score/smpimp l.h:158 #8 bsp_inter_processor_interrupt (vector=<optimized out>) at ../../../../../../rtmcs/c/src/lib/libbsp/sparc/leon3/../../../../bsp s/sparc/leon3/start/bspsmp.c:35 #9 0x00016994 in pil_fixed () at ../../../../../../rtmcs/c/src/../../../../cpukit/score/cpu/sparc/cpu_asm.S:592 Backtrace stopped: previous frame identical to this frame (corrupt stack?) (gdb)</pre>	
laysim-gr740-mdbt-cli Result	
<pre>[layright@CentOS smptests]\$ laysim-gr740-mdbt-cli -gdb laysim for GR740 Quad Core Processor (MDBT) CLI v3.0 by layright Multicore control : Time quantum (500) --> No CORE#0 RAM file is loaded GDB interface: using port 1234 Starting GDB server. Use Ctrl-C to stop waiting for connection GDB connected at IP: 127.0.0.1 and port: 36920 *** BEGIN OF TEST SMP 9 *** *** TEST VERSION: 5.1.0 *** TEST STATE: EXPECTED_PASS *** TEST BUILD: RTMS_NETWORKING RTMS_POSIX_API RTMS_SMP *** TEST TOOLS: 7.5.0 20191114 (RTMS 5, RSB 5.1, Newlib 7947581) CPU 3 start task TA1 CPU 3 start task TA2 CPU 3 start task TA3 kill 10 clock ticks ----- CPU USAGE BY THREAD ----- ID NAME SECONDS PERCENT ----- 0x09010001 IDLE 1.112368 91.404 0x09010002 IDLE 1.106125 89.344 0x09010003 IDLE 1.099878 87.353 0x09010004 IDLE 1.080013 78.114 0x0a010001 UI1 0.301258 23.151 0x0a010002 TA1 0.222469 16.823 0x0a010003 TA2 0.237299 17.663 0x0a010004 TA3 0.252131 18.477 ----- TIME SINCE LAST CPU USAGE RESET IN SECONDS: 1.364500 ----- *** END OF TEST SMP 9 *** CORE#3 halts because of Error Mode [0.177772sec/130925010, 130924932, 130924962, 130924 623cycles]</pre>	<pre>[layright@CentOS smptests]\$ sparc-gaisler-elf-gdb smp09.exe GNU gdb (GDB) 8.2.1 Copyright (C) 2018 Free Software Foundation, Inc. License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html> This is free software: you are free to change and redistribute it. There is NO WARRANTY, to the extent permitted by law. Type "show copying" and "show warranty" for details. This GDB was configured as "--host=x86_64-pc-linux-gnu --target=sparc-gaisler-elf". Type "show configuration" for configuration details. For bug reporting instructions, please see: <http://www.gnu.org/software/gdb/bugs/>. Find the GDB manual and other documentation resources online at: <http://www.gnu.org/software/gdb/documentation/>. For help, type "help". Type "apropos word" to search for commands related to "word"... Reading symbols from smp09.exe...done. (gdb) tar extended-remote localhost:1234 Remote debugging using localhost:1234 0xc0000000 in ?? () (gdb) load Loading section .text, size 0x1bea0 lma 0x0 Loading section .rtmcsroset, size 0x90 lma 0x1bea0 Loading section .data, size 0x530 lma 0x23f40 Start address 0x0, load size 115808 Transfer rate: 3534 KB/sec, 6812 bytes/write. (gdb) b Init Breakpoint 1 at 0x12b0: file ../../../../../../rtmcs/c/src/../../../../testsuites/smptests/sm p09/init.c, line 42. (gdb) run The program being debugged has been started already. Start it from the beginning? (y or n) y Starting program: /home/layright/rtmcs/RTMS5-builder/b-rtmcs/sparc-rtmcs5/c/ gr740/testsuites/smptests/smp09.exe [New Thread 4] [New Thread 2] [New Thread 3] [Switching to Thread 4] Thread 2 hit Breakpoint 1, Init (argument=164992) at ../../../../../../rtmcs/c/src/../../../../testsuites/smptests/smp09/init.c:42 42 { (gdb) cont Continuing. Thread 2 received signal SIGTERM, Terminated. syscall () at ../../../../../../rtmcs/c/src/../../../../cpukit/score/cpu/sparc/syscall.S:44 44 ta 0 ! syscall 1, halt with %g1,%g2,%g3 info (gdb) info thread Id Target Id Frame 1 Thread 1 (CORE#0 [running]) leon3_power_down_loop () at ../../../../../../rtmcs/c/src/lib/libbsp/sparc/leon3/../../../../bsp s/sparc/leon3/start/bspidle.S:26 * 2 Thread 4 (CORE#3 [error]) syscall () at ../../../../../../rtmcs/c/src/../../../../cpukit/score/cpu/sparc/syscall.S:44 3 Thread 2 (CORE#1 [running]) leon3_power_down_loop () at ../../../../../../rtmcs/c/src/lib/libbsp/sparc/leon3/../../../../bsp s/sparc/leon3/start/bspidle.S:26 4 Thread 3 (CORE#2 [running]) leon3_power_down_loop () at ../../../../../../rtmcs/c/src/lib/libbsp/sparc/leon3/../../../../bsp s/sparc/leon3/start/bspidle.S:26 (gdb) thread 1 [Switching to thread 1 (Thread 1)] #0 leon3_power_down_loop () at ../../../../../../rtmcs/c/src/lib/libbsp/sparc/leon3/../../../../bsp s/sparc/leon3/start/bspidle.S:26 26 lda [%sp] 1, %g0 ! Needed for UT699 and GR712 (gdb) bt #0 leon3_power_down_loop () at ../../../../../../rtmcs/c/src/lib/libbsp/sparc/leon3/../../../../bsp s/sparc/leon3/start/bspidle.S:26 #1 0x000018cc in bsp_fatal_extension (source=RTMS_FATAL_SOURCE_SMP, always_set_to_false=optimized out, code=5) at ../../../../../../rtmcs/c/src/lib/libbsp/sparc/leon3/../../../../bsp s/sparc/leon3/start/bspclean.c:41 #2 0x0000d850 in _User_extensions_Fatal_visitor (</pre>

```

executing=0x25550 < RTEMS_tasks_Objects+3792>,
arg=0x20e78 < ISR_Stack_area_begin+3896>,
callouts=0x1af58 < User_extensions_Initial_extensions+72>)
at ../../../../../../rtems/c/src/../../../../cpukit/score/src/userextiterate.c:131
#3 0x0000d8c8 in User_extensions_Iterate (
arg=arg@entry=0x20e78 < ISR_Stack_area_begin+3896>,
visitor=0xd82c < User_extensions_Fatal_visitor>,
direction=direction@entry=CHAIN_ITERATOR_FORWARD)
at ../../../../../../rtems/c/src/../../../../cpukit/score/src/userextiterate.c:173
#4 0x0000db04 in User_extensions_Fatal (error=5,
source=RTEMS_FATAL_SOURCE_SMP)
at /home/layright/RTEMS-dev/RTEMS5-builder/rtems/cpukit/include/rtems/score/userext
impl.h:445
#5 _Terminate (the_source=the_source@entry=RTEMS_FATAL_SOURCE_SMP,
the_error=the_error@entry=5)
at ../../../../../../rtems/c/src/../../../../cpukit/score/src/interr.c:36
#6 0x00002364 in SMP_Fatal (code=SMP_FATAL_SHUTDOWN_RESPONSE)
at /home/layright/RTEMS-dev/RTEMS5-builder/rtems/cpukit/include/rtems/score/smpimp
1.h:74
#7 _SMP_Inter_processor_interrupt_handler (
cpu_self=0x27a00 < Per_CPU_Information>)
at /home/layright/RTEMS-dev/RTEMS5-builder/rtems/cpukit/include/rtems/score/smpimp
1.h:158
#8 bsp_inter_processor_interrupt (vector=<optimized out>)
at ../../../../../../rtems/c/src/lib/libbsp/sparc/leon3/../../../../bsp
s/sparc/leon3/start/bspsmp.c:35
#9 0x00016994 in p1l_fixed ()
at ../../../../../../rtems/c/src/../../../../cpukit/score/cpu/sparc/cpu_asm.S:592
Backtrace stopped: previous frame identical to this frame (corrupt stack?)
(gdb) thread 2
[Switching to thread 2 (Thread 4)]
#0 syscall ()
at ../../../../../../rtems/c/src/../../../../cpukit/score/cpu/sparc/syscall.S:44
44 ta 0
! syscall 1, halt with %g1,%g2,%g3 info
(gdb) bt
#0 syscall ()
at ../../../../../../rtems/c/src/../../../../cpukit/score/cpu/sparc/syscall.S:44
#1 0x0000d850 in User_extensions_Fatal_visitor (
executing=0x5 <trap_table+5>, arg=0x0 <trap_table>,
callouts=0x241a0 < User_extensions_list+4>)
at ../../../../../../rtems/c/src/../../../../cpukit/score/src/userextiterate.c:131
#2 0x0001a02c in CPU_Fatal_halt (source=source@entry=5,
error=error@entry=0)
at ../../../../../../rtems/c/src/lib/libbsp/sparc/leon3/../../../../bsp
s/sparc/leon3/start/bsp_fatal.halt.c:33
#3 0x0000db30 in _Terminate (
the_source=the_source@entry=RTEMS_FATAL_SOURCE_EXIT,
the_error=the_error@entry=0)
at ../../../../../../rtems/c/src/../../../../cpukit/score/src/interr.c:45
#4 0x0001a1f4 in rtems_shutdown_executive (result=result@entry=0)
at ../../../../../../rtems/c/src/../../../../cpukit/sapi/src/exshutdown.c:21
#5 0x00019734 in rtems_test_exit (status=status@entry=0)
at ../../../../../../rtems/c/src/../../../../cpukit/libtest/testbeginend.c:86
#6 0x00001474 in Init (arguments=<optimized out>)
at ../../../../../../rtems/c/src/../../../../testsuites/smptests/smp09/init.c:83
#7 0x0000a83c in Thread_Entry_adaptor_numeric (
executing=0x24680 < RTEMS_tasks_Objects>)
at ../../../../../../rtems/c/src/../../../../cpukit/score/src/threadentryadaptornumeric.c:25
#8 0x00009b68 in Thread_Handler ()
at ../../../../../../rtems/c/src/../../../../cpukit/score/src/threadhandler.c:139
#9 0x00009a80 in Thread_Handler ()
at ../../../../../../rtems/c/src/../../../../cpukit/score/src/threadhandler.c:83
(gdb) thread 3
[Switching to thread 3 (Thread 2)]
#0 leon3_power_down_loop ()
at ../../../../../../rtems/c/src/lib/libbsp/sparc/leon3/../../../../bsp
s/sparc/leon3/start/bspidle.S:26
26 lda [%sp] 1, %g0 ! Needed for UT699 and GR712
(gdb) bt
#0 leon3_power_down_loop ()
at ../../../../../../rtems/c/src/lib/libbsp/sparc/leon3/../../../../bsp
s/sparc/leon3/start/bspidle.S:26
#1 0x000018cc in bsp_fatal_extension (source=RTEMS_FATAL_SOURCE_SMP,
always_set_to_false=<optimized out>, code=5)
at ../../../../../../rtems/c/src/lib/libbsp/sparc/leon3/../../../../bsp
s/sparc/leon3/start/bspclean.c:41
#2 0x0000d850 in User_extensions_Fatal_visitor (
executing=0x25000 < RTEMS_tasks_Objects+2528>,
arg=0x21e78 < ISR_Stack_area_begin+7992>,
callouts=0x1af58 < User_extensions_Initial_extensions+72>)
at ../../../../../../rtems/c/src/../../../../cpukit/score/src/userextiterate.c:131
#3 0x0000d8c8 in User_extensions_Iterate (
arg=arg@entry=0x21e78 < ISR_Stack_area_begin+7992>,
visitor=0xd82c < User_extensions_Fatal_visitor>,
direction=direction@entry=CHAIN_ITERATOR_FORWARD)
at ../../../../../../rtems/c/src/../../../../cpukit/score/src/userextiterate.c:173
#4 0x0000db04 in User_extensions_Fatal (error=5,
source=RTEMS_FATAL_SOURCE_SMP)
at /home/layright/RTEMS-dev/RTEMS5-builder/rtems/cpukit/include/rtems/score/userext
impl.h:445
#5 _Terminate (the_source=the_source@entry=RTEMS_FATAL_SOURCE_SMP,
the_error=the_error@entry=5)
at ../../../../../../rtems/c/src/../../../../cpukit/score/src/interr.c:36
#6 0x00002364 in SMP_Fatal (code=SMP_FATAL_SHUTDOWN_RESPONSE)
at /home/layright/RTEMS-dev/RTEMS5-builder/rtems/cpukit/include/rtems/score/smpimp
1.h:74
#7 _SMP_Inter_processor_interrupt_handler (
cpu_self=0x27c00 < Per_CPU_Information+512>)
at /home/layright/RTEMS-dev/RTEMS5-builder/rtems/cpukit/include/rtems/score/smpimp
1.h:158
#8 bsp_inter_processor_interrupt (vector=<optimized out>)
at ../../../../../../rtems/c/src/lib/libbsp/sparc/leon3/../../../../bsp
s/sparc/leon3/start/bspsmp.c:35
#9 0x00016994 in p1l_fixed ()
at ../../../../../../rtems/c/src/../../../../cpukit/score/cpu/sparc/cpu_asm.S:592
Backtrace stopped: previous frame identical to this frame (corrupt stack?)
(gdb) thread 4
[Switching to thread 4 (Thread 3)]
#0 leon3_power_down_loop ()
at ../../../../../../rtems/c/src/lib/libbsp/sparc/leon3/../../../../bsp
s/sparc/leon3/start/bspidle.S:26
26 lda [%sp] 1, %g0 ! Needed for UT699 and GR712
(gdb) bt
#0 leon3_power_down_loop ()
at ../../../../../../rtems/c/src/lib/libbsp/sparc/leon3/../../../../bsp
s/sparc/leon3/start/bspidle.S:26
#1 0x000018cc in bsp_fatal_extension (source=RTEMS_FATAL_SOURCE_SMP,
always_set_to_false=<optimized out>, code=5)
at ../../../../../../rtems/c/src/lib/libbsp/sparc/leon3/../../../../bsp

```

```
s/sparc/leon3/start/bspclean.c:41
#2 0x000d850 in _User_extensions_Fatal_visitor (
    executing=0x24b70 <RTMS_tasks_Objects+1264>,
    arg=0x22e78 <ISR_Stack_area_begin+12088>,
    callouts=0x1af58 <_User_extensions_Initial_extensions+72>)
    at ../../../../../../rtms/c/src/../../../../cpukit/score/src/userextiterate.c:131
#3 0x000d8c8 in _User_extensions_Iterate (
    arg=arg@entry=0x22e78 <ISR_Stack_area_begin+12088>,
    visitor=0xd82c <_User_extensions_Fatal_visitor>,
    direction=direction@entry=CHAIN_ITERATOR_FORWARD)
    at ../../../../../../rtms/c/src/../../../../cpukit/score/src/userextiterate.c:173
#4 0x000db04 in _User_extensions_Fatal (error=5,
    source=RTMS_FATAL_SOURCE_SMP)
    at /home/layright/RTMS-dev/RTMS5-builder/rtms/cpukit/include/rtms/score/userext
    impl.h:445
#5 _Terminate (the_source=the_source@entry=RTMS_FATAL_SOURCE_SMP,
    the_error=the_error@entry=5)
    at ../../../../../../rtms/c/src/../../../../cpukit/score/src/interr.c:36
#6 0x0002364 in _SMP_Fatal (code=SMP_FATAL_SHUTDOWN_RESPONSE)
    at /home/layright/RTMS-dev/RTMS5-builder/rtms/Cpukit/include/rtms/score/smpimp
    l.h:74
#7 _SMP_Inter_processor_interrupt_handler (
    cpu_self=0x27e00 <_Per_CPU_Information+1024>)
    at /home/layright/RTMS-dev/RTMS5-builder/rtms/cpukit/include/rtms/score/smpimp
    l.h:158
#8 bsp_inter_processor_interrupt (vector=<optimized out>)
    at ../../../../../../rtms/c/src/lib/bsp/sparc/leon3/../../../../bsp
    s/sparc/leon3/start/bsp.smp.c:35
#9 0x0016994 in _ll_fixed ()
    at ../../../../../../rtms/c/src/../../../../cpukit/score/cpu/sparc/cpu_asm.S:592
Backtrace stopped: previous frame identical to this frame (corrupt stack?)
(gdb)
```

6.2 Using Remote Protocol

The CLI version of laysim-GR740 supports the remote protocol of GDB, so it is required to set the SMP configuration using 'monitor gdb postload' after loading RTEMS 5.1 SMP examples. The remote protocol of GDB doesn't support 'run' command, so 'cont' command should be used.

Figure 6-2. Debugging with GDB #2 on laysim-GR740

laysim-gr740-cli Result	
<pre>[layright@CentOS smptests]\$ laysim-gr740-cli -gdb laysim for GR740 Quad Core Processor CLI v3.0 by layright --> No CORE#0 RAM file is loaded GDB interface: using port 1234 Starting GDB server. Use Ctrl-C to stop waiting for connection GDB connected at IP: 127.0.0.1 and port: 36944 *** BEGIN OF TEST SMP 9 *** *** TEST VERSION: 5.1.0 *** TEST STATE: EXPECTED_PASS *** TEST BUILD: RTEMS_NETWORKING RTEMS_POSIX_API RTEMS_SMP *** TEST TOOLS: 7.5.0 20191114 (RTEMS 5, RSB 5.1, Newlib 7947581) CPU 3 start task TA1 CPU 3 start task TA2 CPU 3 start task TA3 Kill 10 clock ticks ----- CPU USAGE BY THREAD ----- ID NAME SECONDS PERCENT ----- 0x09010001 IDLE 1.116437 91.433 0x09010002 IDLE 1.110194 89.379 0x09010003 IDLE 1.103951 87.394 0x09010004 IDLE 1.000027 77.867 0x0a010001 UI1 0.305311 23.389 0x0a010002 TA1 0.222463 16.771 0x0a010003 TA2 0.237297 17.610 0x0a010004 TA3 0.252129 18.422 ----- TIME SINCE LAST CPU USAGE RESET IN SECONDS: 1.368567 *** END OF TEST SMP 9 *** CORE#3 halts because of Error Mode [134657448cycles/1.5366sec] GDB: disconnected GDB interface: using port 1234 Starting GDB server. Use Ctrl-C to stop waiting for connection</pre>	<pre>[layright@CentOS smptests]\$ sparc-gaisler-elf-gdb smp09.exe GNU gdb (GDB) 8.2.1 Copyright (C) 2018 Free Software Foundation, Inc. License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html> This is free software: you are free to change and redistribute it. There is NO WARRANTY, to the extent permitted by law. Type "show copying" and "show warranty" for details. This GDB was configured as "--host=x86_64-pc-linux-gnu --target=sparc-gaisler-elf". Type "show configuration" for configuration details. For bug reporting instructions, please see: <http://www.gnu.org/software/gdb/bugs/>. Find the GDB manual and other documentation resources online at: <http://www.gnu.org/software/gdb/documentation/>. For help, type "help". Type "apropos word" to search for commands related to "word"... Reading symbols from smp09.exe...done. (gdb) tar remote localhost:1234 Remote debugging using localhost:1234 0xc0000000 in ?? () (gdb) load Loading section .text, size 0x1bea0 lma 0x0 Loading section .rtmsreset, size 0x90 lma 0x1bea0 Loading section .data, size 0x530 lma 0x23f40 Start address 0x0, load size 115808 Transfer rate: 113093 KB/sec, 6812 bytes/write. (gdb) monitor gdb postload (gdb) b Init Breakpoint 1 at 0x12b0: file ../../../../../../rtms/c/src/../../../../testsuites/smptests/smp09/init.c, line 42. (gdb) cont Continuing. [Switching to Thread 4] Thread 4 hit Breakpoint 1, Init (argument=164992) at ../../../../../../rtms/c/src/../../../../testsuites/smptests/smp09/init.c:42 42 { 49 TEST_BEGIN(); (gdb) step rtms_test_begin (name=name@entry=0x1af80 <rtms_test_name> "SMP 9", state=state@entry=RTMS_TEST_STATE_PASS) at ../../../../../../rtms/c/src/../../../../cpukit/libtest/testbeginend.c:40 40 return rtms_printf((gdb) cont Continuing. [Inferior 1 (Remote target) exited normally] (gdb)</pre>
laysim-gr740-dbt-cli Result	
<pre>[layright@CentOS smptests]\$ laysim-gr740-dbt-cli -gdb laysim for GR740 Quad Core Processor (DBT) CLI v3.0 by layright Multicore control : Time quantum (500) --> No CORE#0 RAM file is loaded GDB interface: using port 1234 Starting GDB server. Use Ctrl-C to stop waiting for connection GDB connected at IP: 127.0.0.1 and port: 36946 *** BEGIN OF TEST SMP 9 ***</pre>	<pre>[layright@CentOS smptests]\$ sparc-gaisler-elf-gdb smp09.exe GNU gdb (GDB) 8.2.1 Copyright (C) 2018 Free Software Foundation, Inc. License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html> This is free software: you are free to change and redistribute it. There is NO WARRANTY, to the extent permitted by law. Type "show copying" and "show warranty" for details. This GDB was configured as "--host=x86_64-pc-linux-gnu --target=sparc-gaisler-elf". Type "show configuration" for configuration details. For bug reporting instructions, please see: <http://www.gnu.org/software/gdb/bugs/>. Find the GDB manual and other documentation resources online at:</pre>

*** TEST VERSION: 5.1.0

*** TEST STATE: EXPECTED_PASS

*** TEST BUILD: RTEMS_NETWORKING RTEMS_POSIX_API RTEMS_SMP

*** TEST TOOLS: 7.5.0 20191114 (RTEMS 5, RSB 5.1, Newlib 7947581)

CPU 3 start task TA1

CPU 3 start task TA2

CPU 3 start task TA3

kill 10 clock ticks

CPU USAGE BY THREAD

ID	NAME	SECONDS	PERCENT
0x09010001	IDLE	1.112360	91.404
0x09010002	IDLE	1.106116	89.344
0x09010003	IDLE	1.099869	87.352
0x09010004	IDLE	1.000013	78.114
0x0a010001	UI1	0.301250	23.150
0x0a010002	TA1	0.222469	16.823
0x0a010003	TA2	0.237299	17.663
0x0a010004	TA3	0.252131	18.478

TIME SINCE LAST CPU USAGE RESET IN SECONDS: 1.364491

*** END OF TEST SMP 9 ***

CORE#3 halts because of Error Mode [0.105454sec/130991071, 130991100, 130991095, 130990 626cycles]

GDB: disconnected

GDB interface: using port 1234

Starting GDB server. Use Ctrl-C to stop waiting for connection

<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".

Type "apropos word" to search for commands related to "word"...

Reading symbols from smp09.exe...done.

(gdb) tar remote localhost:1234

Remote debugging using localhost:1234

0xc0000000 in ?? ()

(gdb) load

Loading section .text, size 0x1bea0 lma 0x0

Loading section .rtemsroset, size 0x90 lma 0x1bea0

Loading section .data, size 0x530 lma 0x23f40

Start address 0x0, load size 115808

Transfer rate: 113093 KB/sec, 6812 bytes/write.

(gdb) monitor gdb postload

(gdb) b Init

Breakpoint 1 at 0x12b0: file ../../../../../../rtems/c/src/../../../../testsuites/smp09/init.c, line 42.

(gdb) cont

Continuing.

[Switching to Thread 4]

Thread 4 hit Breakpoint 1, Init (argument=164992)

at ../../../../../../rtems/c/src/../../../../testsuites/smp09/init.c:42

42 {

(gdb) step

49 TEST_BEGIN();

(gdb) step

rtems_test_begin (name=name@entry=0x1af80 <rtems_test_name> "SMP 9",

state=state@entry=RTEMS_TEST_STATE_PASS)

at ../../../../../../rtems/c/src/../../../../cpukit/libtest/testbeginend.c:40

40 return rtems_printf(

(gdb) cont

Continuing.

[Inferior 1 (Remote target) exited normally]

(gdb)

laysim-gr740-mdbt-cli Result

[layright@CentOS smp09]\$ laysim-gr740-mdbt-cli -gdb

laysim for GR740 Quad Core Processor (MDBT) CLI v3.0 by layright

Multicore control : Time quantum (500)

--> No CORE#0 RAM file is loaded

GDB interface: using port 1234

Starting GDB server. Use Ctrl-C to stop waiting for connection

GDB connected at IP: 127.0.0.1 and port: 36948

*** BEGIN OF TEST SMP 9 ***

*** TEST VERSION: 5.1.0

*** TEST STATE: EXPECTED_PASS

*** TEST BUILD: RTEMS_NETWORKING RTEMS_POSIX_API RTEMS_SMP

*** TEST TOOLS: 7.5.0 20191114 (RTEMS 5, RSB 5.1, Newlib 7947581)

CPU 3 start task TA1

CPU 3 start task TA2

CPU 3 start task TA3

kill 10 clock ticks

CPU USAGE BY THREAD

ID	NAME	SECONDS	PERCENT
0x09010001	IDLE	1.112369	91.405
0x09010002	IDLE	1.106125	89.344
0x09010003	IDLE	1.099879	87.353
0x09010004	IDLE	1.000013	78.113
0x0a010001	UI1	0.301259	23.151
0x0a010002	TA1	0.222469	16.823
0x0a010003	TA2	0.237299	17.663
0x0a010004	TA3	0.252131	18.477

TIME SINCE LAST CPU USAGE RESET IN SECONDS: 1.364501

*** END OF TEST SMP 9 ***

CORE#3 halts because of Error Mode [0.190294sec/130923939, 130923971, 130923929, 130923 626cycles]

GDB: disconnected

GDB interface: using port 1234

Starting GDB server. Use Ctrl-C to stop waiting for connection

[layright@CentOS smp09]\$ sparc-gaisler-elf-gdb smp09.exe

GNU gdb (GDB) 8.2.1

Copyright (C) 2018 Free Software Foundation, Inc.

License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>

This is free software: you are free to change and redistribute it.

There is NO WARRANTY, to the extent permitted by law.

Type "show copying" and "show warranty" for details.

This GDB was configured as "--host=x86_64-pc-linux-gnu --target=sparc-gaisler-elf".

Type "show configuration" for configuration details.

For bug reporting instructions, please see:

<http://www.gnu.org/software/gdb/bugs/>.

Find the GDB manual and other documentation resources online at:

<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".

Type "apropos word" to search for commands related to "word"...

Reading symbols from smp09.exe...done.

(gdb) tar remote localhost:1234

Remote debugging using localhost:1234

0xc0000000 in ?? ()

(gdb) load

Loading section .text, size 0x1bea0 lma 0x0

Loading section .rtemsroset, size 0x90 lma 0x1bea0

Loading section .data, size 0x530 lma 0x23f40

Start address 0x0, load size 115808

Transfer rate: 4349 KB/sec, 6812 bytes/write.

(gdb) monitor gdb postload

(gdb) b Init

Breakpoint 1 at 0x12b0: file ../../../../../../rtems/c/src/../../../../testsuites/smp09/init.c, line 42.

(gdb) cont

Continuing.

[Switching to Thread 4]

Thread 4 hit Breakpoint 1, Init (argument=164992)

at ../../../../../../rtems/c/src/../../../../testsuites/smp09/init.c:42

42 {

(gdb) step

49 TEST_BEGIN();

(gdb) step

rtems_test_begin (name=name@entry=0x1af80 <rtems_test_name> "SMP 9",

state=state@entry=RTEMS_TEST_STATE_PASS)

at ../../../../../../rtems/c/src/../../../../cpukit/libtest/testbeginend.c:40

40 return rtems_printf(

(gdb) cont

Continuing.

[Inferior 1 (Remote target) exited normally]

(gdb)

6.3 Using Eclipse

The CLI version of laysim-GR740 can be connected to a GDB front-end such as DDD or Eclipse. Eclipse supports only the remote protocol of GDB, 'monitor gdb reset' and 'monitor gdb postload' should be added in Debug Configurations of Eclipse as shown in Figure 6-3.

Figure 6–3. Debug Configurations in Eclipse

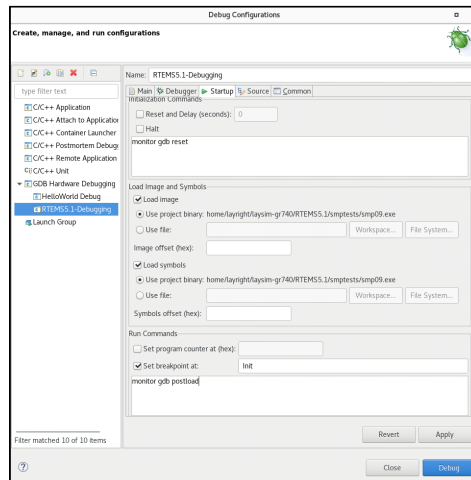
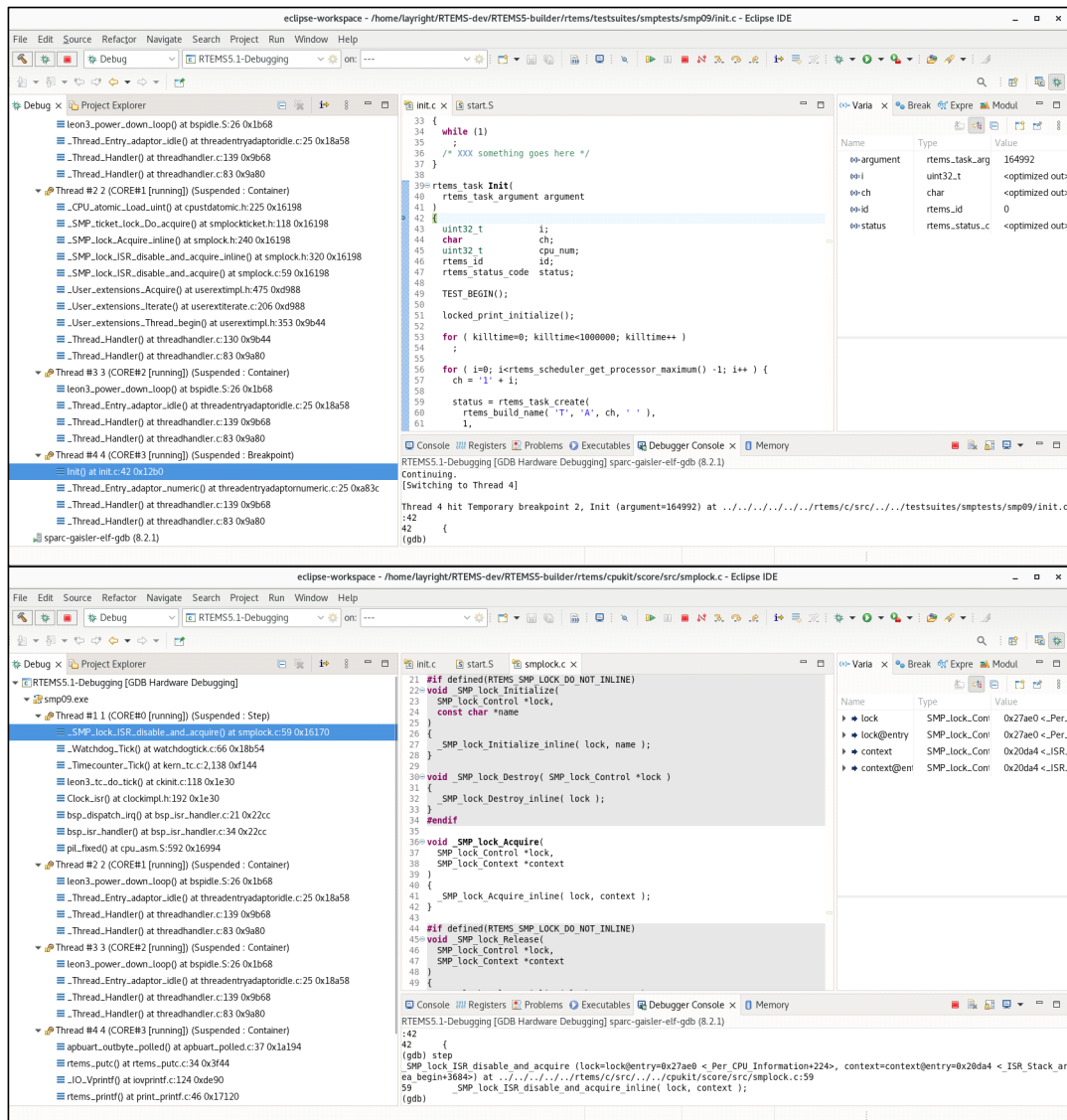


Figure 6–4. Debugging with Eclipse on laysim-GR740



7. Code Coverage

laysim-gr740-cli only supports the code coverage in the same format as TSIM3-LEON4. '-cov' start-up option should be enabled for the code coverage. Detailed information can be found in [laysim-GR740-013] Code coverage operation on laysim-GR740.

Figure 7-1 shows the result of `smp09.exe` with code coverage. After execution, `smp09.exe.cov` file is generated.

Figure 7-1. Code Coverage Test

laysim-gr740-cli in Linux	laysim-gr740-cli in Windows
<pre>[layright@CentOS smptests]\$ laysim-gr740-cli -r -core0 smp09.elf -cov laysim for GR740 Quad Core Processor CLI v3.0 by layright CORE#0 RAM file is loaded - smp09.elf - .text, addr: 0x00000000, size 114336 bytes - .rtmsroset, addr: 0x0001BEA0, size 144 bytes - .rtmsstack, addr: 0x0001BF40/0x0001BF30, size 32768 bytes (not loaded, just info.) - .data, addr: 0x00023F40, size 1328 bytes - .bss, addr: 0x00024480, size 17952 bytes (not loaded, just info.) - Detect RTEMS v5.1(SMP) with thread information Only GR740 CORE#0 RAM file is loaded, starts from RAM Set CORE#0 %sp/%fp : 0x07FFFFFFF/0x08000000 *** BEGIN OF TEST SMP 9 *** *** TEST VERSION: 5.1.0 *** TEST STATE: EXPECTED_PASS *** TEST BUILD: RTEMS_NETWORKING RTEMS_POSIX_API RTEMS_SMP *** TEST TOOLS: 7.5.0 20191114 (RTEMS 5, RSB 5.1, Newlib 7947581) CPU 3 start task TA1 CPU 3 start task TA2 CPU 3 start task TA3 kill 10 clock ticks ----- CPU USAGE BY THREAD ----- ID NAME SECONDS PERCENT ----- 0x09010001 IDLE 1.116508 91.433 0x09010002 IDLE 1.110264 89.380 0x09010003 IDLE 1.104028 87.395 0x09010004 IDLE 1.000046 77.865 0x0a010001 UI1 0.305362 23.392 0x0a010002 TA1 0.222456 16.770 0x0a010003 TA2 0.237297 17.609 0x0a010004 TA3 0.252129 18.421 ----- TIME SINCE LAST CPU USAGE RESET IN SECONDS: 1.368638 *** END OF TEST SMP 9 *** CORE#3 halts because of Error Mode [134715700cycles/1.53253sec] Performance statistics for CORE#0 Cycles : 134715707 Instructions : 259785 Overall CPI : 2.77 CORE#0 performance (250MHz) : 90.30 MOPS (90.30 MIPS, 0.00 MFLOPS) Simulated time : 538.86 ms Processor utilization : 0.53 % Performance statistics for CORE#1 Cycles : 134715700 Instructions : 260270 Overall CPI : 2.65 CORE#1 performance (250MHz) : 94.26 MOPS (94.26 MIPS, 0.00 MFLOPS) Simulated time : 538.86 ms Processor utilization : 0.51 % Performance statistics for CORE#2 Cycles : 134715708 Instructions : 261074 Overall CPI : 2.64 CORE#2 performance (250MHz) : 94.60 MOPS (94.60 MIPS, 0.00 MFLOPS) Simulated time : 538.86 ms Processor utilization : 0.51 % Performance statistics for CORE#3 Cycles : 134715700 Instructions : 58659970 Overall CPI : 2.30 CORE#3 performance (250MHz) : 108.86 MOPS (108.86 MIPS, 0.00 MFLOPS) Simulated time : 538.86 ms Processor utilization : 100.00 % Performance of laysim-gr740 Real-time performance : 35.16 % laysim-gr740 performance : 38.79 MIPS Wall clock : 1.53 s [layright@CentOS smptests]\$ ls *.cov smp09.elf.cov [layright@CentOS smptests]\$</pre>	<pre>PS C:\laysim-gr740\RTSM55.1\smptests> laysim-gr740-cli -r -core0 smp09.elf -cov laysim for GR740 Quad Core Processor CLI v3.0 by layright CORE#0 RAM file is loaded - smp09.elf - .text, addr: 0x00000000, size 114336 bytes - .rtmsroset, addr: 0x0001BEA0, size 144 bytes - .rtmsstack, addr: 0x0001BF40/0x0001BF30, size 32768 bytes (not loaded, just info.) - .data, addr: 0x00023F40, size 1328 bytes - .bss, addr: 0x00024480, size 17952 bytes (not loaded, just info.) - Detect RTEMS v5.1(SMP) with thread information Only GR740 CORE#0 RAM file is loaded, starts from RAM Set CORE#0 %sp/%fp : 0x07FFFFFFF/0x08000000 *** BEGIN OF TEST SMP 9 *** *** TEST VERSION: 5.1.0 *** TEST STATE: EXPECTED_PASS *** TEST BUILD: RTEMS_NETWORKING RTEMS_POSIX_API RTEMS_SMP *** TEST TOOLS: 7.5.0 20191114 (RTEMS 5, RSB 5.1, Newlib 7947581) CPU 3 start task TA1 CPU 3 start task TA2 CPU 3 start task TA3 kill 10 clock ticks ----- CPU USAGE BY THREAD ----- ID NAME SECONDS PERCENT ----- 0x09010001 IDLE 1.116508 91.433 0x09010002 IDLE 1.110264 89.380 0x09010003 IDLE 1.104028 87.395 0x09010004 IDLE 1.000046 77.865 0x0a010001 UI1 0.305362 23.392 0x0a010002 TA1 0.222456 16.770 0x0a010003 TA2 0.237297 17.609 0x0a010004 TA3 0.252129 18.421 ----- TIME SINCE LAST CPU USAGE RESET IN SECONDS: 1.368638 *** END OF TEST SMP 9 *** CORE#3 halts because of Error Mode [134715700cycles/1.6068sec] Performance statistics for CORE#0 Cycles : 134715707 Instructions : 259785 Overall CPI : 2.77 CORE#0 performance (250MHz) : 90.30 MOPS (90.30 MIPS, 0.00 MFLOPS) Simulated time : 538.86 ms Processor utilization : 0.53 % Performance statistics for CORE#1 Cycles : 134715700 Instructions : 260270 Overall CPI : 2.65 CORE#1 performance (250MHz) : 94.26 MOPS (94.26 MIPS, 0.00 MFLOPS) Simulated time : 538.86 ms Processor utilization : 0.51 % Performance statistics for CORE#2 Cycles : 134715708 Instructions : 261074 Overall CPI : 2.64 CORE#2 performance (250MHz) : 94.60 MOPS (94.60 MIPS, 0.00 MFLOPS) Simulated time : 538.86 ms Processor utilization : 0.51 % Performance statistics for CORE#3 Cycles : 134715700 Instructions : 58659970 Overall CPI : 2.30 CORE#3 performance (250MHz) : 108.86 MOPS (108.86 MIPS, 0.00 MFLOPS) Simulated time : 538.86 ms Processor utilization : 100.00 % Performance of laysim-gr740 Real-time performance : 33.54 % laysim-gr740 performance : 36.99 MIPS Wall clock : 1.61 s PS C:\laysim-gr740\RTSM55.1\smptests> ls *.cov Directory: C:\laysim-gr740\RTSM55.1\smptests Mode LastWriteTime Length Name ---- - -a--- 8/27/2025 3:09 PM 55473 smp09.elf.cov PS C:\laysim-gr740\RTSM55.1\smptests></pre>
<pre>00000000 : 7 3 3 3 6 2 2 2 6 2 2 2 6 2 2 2 6 2 2 2 19 19 0 1 1 1 11 6 2 2 2 00000080 : 6 2 2 2 6 2 2 2 6 2 2 2 6 2 2 2 6 2 2 2 6 2 2 2 6 2 2 2 6 2 2 2 00000100 : 6 6 6 2 7 1b 1b 1b 6 2 2 2 6 2 2 2 6 2 2 2 6 2 2 2 6 2 2 2 6 2 2 2</pre>	<pre>00000000 : 7 3 3 3 6 2 2 2 6 2 2 2 6 2 2 2 6 2 2 2 19 19 0 1 1 1 11 6 2 2 2 00000080 : 6 2 2 2 6 2 2 2 6 2 2 2 6 2 2 2 6 2 2 2 6 2 2 2 6 2 2 2 6 2 2 2 00000100 : 6 6 6 2 7 1b 1b 1b 6 2 2 2 6 2 2 2 6 2 2 2 6 2 2 2 6 2 2 2 6 2 2 2</pre>

[illegible]

8. Emulation Characteristics of laysim-GR740

This chapter describes which features of the GR740 are supported and not supported in laysim-GR740.

8.1 LEON4 Support

laysim-GR740 supports most of the features of the LEON4 in GR740, which can be summarized as follows.

- Integer Unit (IU) : laysim-GR740 supports the LEON4 integer unit with hardware multiply and divide instruction, and eight register windows.
 - ▶ Supported
 - Eight register windows
 - Hardware multiplier and Radix-2 divider
 - Processor configuration register (ASR17) as read-only register
 - Address space identifiers (ASI)
 - Power-down mode : laysim-GR740 supports the power-down mode of GR740, and laysim-GR740 can increase the simulation speed because it skips time until the next event on power-down mode. Also laysim-GR740 supports the idle-loop optimization, where it acts like the power-down mode.
 - Configuration registers : laysim-GR740 supports ASR17 and ASR22-23. ASR16 is always zero, and ASR24-31 are not supported because of the support breakpoints/watchpoint in emulator level.
 - ▶ Unsupported
 - Static branch prediction
 - Single-vector trapping for reduced code size
 - Partial WRPSR
 - Register file data protection : ASR16 is always zero
 - Hardware breakpoints : laysim-GR740 supports breakpoints and watchpoints at the emulator level.
 - Instruction trace buffer : laysim-gr740 supports history operations at the emulator level.
- Cache sub-system : Only laysim-gr740 supports L1 cache for instruction and data of the LEON4 with L1 cache hit/miss cycles and write buffer cycles. It also supports the data cache snooping, but it does not support the diagnostic access for the cache tag and data. laysim-gr740-dbt and laysim-gr740-mdbt do not model the L1/L2 cache of GR740 for speed-up.
 - ▶ Supported (Only laysim-gr740)
 - Enabling and disabling cache
 - Cache freeze and flushing
 - Data cache snooping
 - Operating with MMU
 - ▶ Unsupported (Only laysim-gr740)
 - Diagnostic access : The cache tag and data can be checked through the Cache Window of laysim-gr740.
- Memory Management Unit (MMU) : laysim-GR740 supports the MMU of GR740 including a three-level hardware table-walk, 16 TLBs, and MMU registers.

- Floating Point Unit (FPU) : l1aysim-GR740 supports the GRFPU with the simple FPU cycle model. It supports the handling denormalized numbers of the GRFPU, and supports only a single FPU trap queue.

8.2 L2 Cache Support

Only l1aysim-gr740 supports L2 cache of GR740. l1aysim-gr740 supports only a basic model of the GR740 L2 cache and includes L2 cache hit/miss cycle simulation. It also supports enabling, disabling and flushing the L2 cache. In the current version of l1aysim-gr740, the L2 cache behaves like a write-through cache, and other features such as MTTR, EDAC, and error injection are not supported. l1aysim-gr740-dbt and l1aysim-gr740-mdbt do not model the L1/L2 cache of GR740 for speed-up.

8.3 MMCTRL Support

MMCTRL does not provide actual functionality and only provides registers and P&P information in l1aysim-GR740. When the SDRAM size is set with -ramsz, it is reflected in MMCTRL.SDCFG1, and dual bank is used by default.

8.4 MEMSCRUB Support

l1aysim-GR740 does not support MMSCRUB and only provides dummy registers.

8.5 IOMMU Support

l1aysim-GR740 provides the access protection and address translation of IOMMU for masters on Master I/O AHB bus including PCI target, PCIDMA, GRETH, SpaceWire router AMBA interface, GR1553B and GRCAN. But it has some limitations for IOMMU. More information can be found in [l1aysim-GR740-014] GR740 IOMMU Operation on l1aysim-GR740.

- 1) Only Processor AHB bus selection is supported.
- 2) APV and TLB caches are not implemented.
- 3) No reflection of additional cycles which were generated by the IOMMU.
- 4) IOMMU ASMP features are not implemented.

8.6 SpaceWire Router Support

l1aysim-GR740 models the communication and routing functions of the GR740 SpaceWire Router, with the supported and unsupported features summarized as follows. More information can be found in [l1aysim-GR740-004] GR740 SpaceWire and Router Operation on l1aysim-GR740 and [l1aysim-GR740-009] l1aysim GR740 Loadable Modules - User's Manual.

- Supported
 - Four AMBA Ports and eight SpW Ports
 - Transmission and reception of SpaceWire packets : l1aysim-GR740 provides only 8K-bytes internal buffer for each SpW node communication, so the maximum packet size is 8192.
 - Routing Table
 - Port mapping
 - Address control
 - Group adaptive routing
 - Port disable
 - Static routing

- Self addressing
- Unsupported
 - Configuration Port 0 : **laysim-GR740** doesn't support the configuration port 0. However, since all configuration values are memory mapped that they can be configured by registers with AMBA access.
 - System time-distribution
 - SpaceWire distributed interrupt support
 - SpaceWire-D
 - SpaceWire Plug-and-Play
 - Spill-if-not-ready
 - Port data character timers
 - Packet length truncation

8.7 GRETH_GBIT Support

laysim-GR740 fully supports GRETH_GBIT with DMA. In addition, it provides UDP connectivity by default to facilitate integration with Tornado/Workbench. It can configure GRETH_GBIT for AIR and NASA cFS using '-air' or '-cfs' start-up option.

8.8 GRPCI2 Support

laysim-GR740 supports GRPCI2 with PCI initiator/target and DMA. The supported and unsupported features summarized as follows. More information can be found in [**laysim-GR740-009**] **laysim GR740 Loadable Modules - User's Manual**.

- Supported
 - PCI configuration space
 - PCI configuration cycles
 - PCI initiator interface
 - PCI target interface
 - PCI DMA controller
- Unsupported
 - I/O cycles
 - PCI trace buffer

8.9 GR1553B Support

The GR740 provides GR1553B (0x01:0x04D) for MIL-STD-1553B interface. This interface core connects the AMBA AHB/APB bus to a MIL-STD-1553B bus, and the core can operate as either Bus Controller, Remote Terminal or Bus Monitor. **laysim-GR740** only supports BC mode without supporting RT-to-RT transfer. More information can be found in [**laysim-GR740-005**] **GR740 GR1553B Operation on laysim-GR740** and [**laysim-GR740-009**] **laysim GR740 Loadable Modules - User's Manual**.

- Supported
 - Bus controller operation

- Secondary transfer list
- Timing control
- Unsupported
 - Remote terminal operation
 - Bus monitor operation

8.10 GRCAN Support

The GR740 provides two GRCANs (0x01:0x03D) for CAN interface. The GRCAN supports transmission and reception of set of messages by use of circular buffers located in memory external to the core. Separate transmit and receive buffers are assumed. Reception and transmission of sets of message can be ongoing simultaneously. Also it can detect a SYNC message and generate an interrupt. **laysim-GR740** supports the GRCAN operation including CAN protocol, message transmission, filtering and reception, SYNC message reception, and interrupt generation for Tx, TxEmpty, TxIRQ, TxAHBErr, TxSync, Rx, RxFull, RxIRQ, RxAHBErr, RxSync, Over-run and RxMiss. And it also provides error-passive, bus-off and interrupts for TxLoss, TxErrCntr, RxErrCntr, OFF, and PASS. More information can be found in [laysim-GR740-008] GR740 GRCAN Operation on laysim-GR740 and [laysim-GR740-009] laysim GR740 Loadable Modules - User's Manual.

8.11 FTMCTRL Support

FTMCTRL does not provide actual functionality and only provides registers and P&P information in **laysim-GR740**. When the MRAM size is set with -romsz, it is reflected in FTMCTRL.MCFG1. The memory mapped I/O is supported through a loadable module. More information can be found in [laysim-GR740-009] laysim GR740 Loadable Modules - User's Manual.

8.12 GPTIMER Support

laysim-GR740 supports five GPTIMERS of GR740 with latching. But it does not support system watchdog functionality. Only tick output of timer 3 on the GPTIMER0 is connected to external trigger input of GR1553B in **laysim-GR740**.

8.13 IRQ(A)MP Support

laysim-GR740 supports the IRQ(A)MP of GR740 with the support for asymmetric multiprocessing. It also supports interrupt prioritization, extended interrupts, processor status monitoring, interrupt broadcasting, interrupt remap, and interrupt timestamping. But, it does not support watchdog functionality. More information can be found in [laysim-GR740-006] GR740 IRQ(A)MP Operation on laysim-GR740.

8.14 GRGPIO Support

laysim-GR740 supports only the basic functionality of GRGPIO including I/O port, interrupt generation and interrupt map. But it doesn't support the extended functionality such as logical-AND, XOR, OR. More information can be found in [laysim-GR740-009] laysim GR740 Loadable Modules - User's Manual.

8.15 APBUART Support

laysim-GR740 supports APBUART with Tx/Rx FIFO and loadable UART Module. But it doesn't support the external clock, loop back, parity, break interrupt, and delayed interrupt. More information can be found in [laysim-GR740-009] laysim GR740 Loadable

Modules - User's Manual.

8.16 SPICTRL Support

laysim-GR740 supports SPICTRL with loadable SPI Module. But it doesn't support the three-wire mode, automatic slave select, and slave mode. More information can be found in [laysim-GR740-009] laysim GR740 Loadable Modules - User's Manual.

8.17 GRCLKGATE Support

In laysim-GR740, the GRCLKGATE is checked only when the corresponding core is enabled. If the GRCLKGATE is disabled, an error is logged. During a read operation, the register can be accessed normally, but a write operation is disabled. When a RAM executable is loaded, the GRCLKGATE of all cores is automatically enabled.

8.18 L4STAT Support

laysim-GR740 does not support L4STAT and only provides dummy registers.

8.19 AHBSTAT Support

laysim-GR740 supports two AHBSTATs for processor bus and slave IO bus with interrupts.

8.20 BOOTSTRAP Support

laysim-GR740 does not support BOOTSTRAP and only provides dummy registers.

8.21 GR740THSENS Support

laysim-GR740 only supports GR740THSENS registers.

8.22 GRGPRBANK Support

laysim-GR740 only supports FTMFUNC and ALTFUNC for checking the valid pinmux for APBUART#0/#1, GR1553B, GRCAN except PROMIO. If a core is accessed while the pinmux is configured invalidly, an error is logged. Other features of GRGPRBANK are not supported.

8.23 GRSPWTDTP Support

laysim-GR740 does not support GRSPWTDTP and registers.